
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

VIRTUAL SYSTEMS AND SERVICES

From Hypervisors to the Cloud-Native Stack

UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



One idea, six times over: put a layer underneath, and lie to what sits above it

A CPU that is not a CPU, a disk that is not a disk, a network that is not a network — and the cost of each lie.

What this volume covers. The HEC course *Virtual Systems and Services*, modernised. Parts I and II teach the 2017 outline close to its published specification — what a hypervisor is, why x86 resisted virtualization for twenty years, and what Intel and AMD added to hardware to end that. Parts III to VI carry the same ideas forward into containers, Kubernetes, storage and network virtualization, cloud service models, live migration, and confidential computing.

HEC published contents for this course once, in 2017. The 2025 revision drops it and divides its ground between *Cloud Computing* and *Microservices Architecture and Docker Containers*. Appendix F states exactly where the line falls, and why a course that goes one layer below both still earns its place.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

This book has one subject, met six times. A **virtualization layer** sits between something real and something that believes it has the real thing, and keeps the belief intact. Chapters 3 to 7 do it to a processor. Chapters 8 to 11 do it to an operating system. Chapter 12 does it to a disk, Chapter 13 to a network, Chapter 15 to a desktop, Chapter 20 to a graphics card. The machinery differs; the question never does — *what is the lie, who maintains it, and what does maintaining it cost?*

Read it in order. Each layer is built from the one before.

The six parts

Part I — Foundations of Virtualization (Chapters 1–3) fixes the vocabulary and then proves the hard part. Chapter 3 is the theoretical spine of the book: Popek and Goldberg’s conditions, why x86 failed them for two decades, and the two ways round that failure. Everything in Part II is a hardware answer to a problem stated in Chapter 3.

Part II — Hypervisors and Compute Virtualization (Chapters 4–7) is the engine room. What Intel and AMD put in silicon; the four platforms you will actually meet; how a CPU and a gigabyte of memory are divided among guests that each think they own everything; and how a device is shared, emulated or handed over outright.

Part III — Containers and Lightweight Virtualization (Chapters 8–11) changes what is virtualized. Not the hardware, but the operating system’s own view of itself: namespaces, control groups and layered filesystems. Then Docker, then Kubernetes, then the designs that put a hypervisor back underneath a container because isolation mattered more than density.

Part IV — Storage, Network and Cloud Virtualization (Chapters 12–15) applies the same idea to everything that is not a CPU. It ends at the cloud, which is this book’s six layers sold by the hour.

Part V — Management, Performance and Security (Chapters 16–19) is the operations half: what you run to manage a fleet, how a running machine moves between hosts without stopping, which numbers tell you a host is overloaded — and the attacks that exist only because the layer exists.

Part VI — Emerging Hardware and Practice (Chapters 20–23) closes with the parts still moving: accelerator partitioning, encrypted memory and attestation, four case studies, and your project.



Why the theory chapter comes third, and not later. Chapter 3 is the most demanding chapter in the book, and it is deliberately early. Without it, hardware-assisted virtualization is a list of acronyms to memorise; with it, every feature in Chapter 4 has a reason — each one exists because trap-and-emulate could not be made to work on x86. A reader who skips Chapter 3 can still pass the course and will never understand why any of it was built.

The furniture of every chapter

Every chapter has the same parts, in the same order. Learn to read them once.

Box	What it is for
🎯 Outcomes	What you will be able to <i>do</i> . Bloom verbs, not topics. Use it as a checklist before the examination.
🔗 Before you start	Which earlier chapters this one stands on. If these are shaky, go back; nothing here is improved by pressing on.
🔑 Key terms	The vocabulary introduced. Every one appears in the glossary of Appendix D.
📖 Definition	The canonical statement. Learn these verbatim; this field is full of terms used loosely in marketing and precisely in documentation.
💡 Concept	The judgement call — what an experienced administrator would decide here, and why.
⚠️ Warning	Something that will bite you, usually in production and usually at scale.
🧮 Worked example	Real arithmetic, every step shown. Consolidation ratios, page-walk costs, migration times, availability budgets. Do them with a pen.
❗ Common pitfalls	The mistakes that are actually made, with the symptom each produces.
🔧 Try it yourself	The laboratory. Each is a runnable file in <code>code/</code> , named for its chapter.
❓ Checkpoint	Three questions. Attempt them before reading the answers in Appendix C.
📋 Chapter summary	What to retain.
✍️ Review questions	Examination practice: multiple choice, short answer, applied.

How to study with it

1. Read the outcomes first. They tell you what the chapter is *for*.
2. Read the prose once without stopping, then again with a pen on the worked examples. Every number in this book was computed before it was typed; you should be able to reproduce each one.
3. Run the lab. This is not an optional enrichment: a hypervisor is a thing you develop intuition about by breaking it, and the labs are built to be broken. Overcommit memory until the guest swaps. Fill a thin pool. Migrate a machine while pinging it.
4. Do the checkpoint from memory. Then read Appendix C and argue with it.
5. Attempt the applied review questions in pairs. They are written to be discussed, not looked up.

⚠️ On the laboratories, and what they need

Every lab in this book runs on one student laptop with free software. Where the subject is an enterprise platform — vSphere, Hyper-V on a cluster, a Ceph estate, an NVIDIA vGPU — the chapter teaches the real thing and the lab uses a free equivalent that behaves the

same way, naming the substitution in its first line. KVM with `virsh` stands in for vSphere; `kind` stands in for a production Kubernetes cluster; LVM thin pools stand in for enterprise storage.

Three labs need hardware not every reader has: nested virtualization in Chapter 4, GPU partitioning in Chapter 20, and an SEV- or TDX-capable processor in Chapter 21. Each ships captured output in `code/` so the exercise can be done by reading. No lab in this book is an instruction to buy something.

Appendix A installs and verifies the whole stack in one sitting.

Course Specification

This page is the course file. It states what HEC requires of a course called *Virtual Systems and Services*, and shows where in this handout each requirement is met.

This course's position in the HEC curricula is unusual enough to state plainly before the tables begin. HEC published contents for it *once*, in the 2017 *Curriculum of BS & MS CS, SE & IT* (pp. 71–130). The 2023 revision keeps the title in the BSIT tables as a domain elective but publishes no contents for the Information Technology cluster at all. The 2025 revision removes the course, dividing its ground between *Cloud Computing*, a new major core, and *Microservices Architecture and Docker Containers*.

The 2017 outline is therefore the only authoritative description that exists, and this handout treats it as binding on Parts I and II and as a starting point for the rest.

Course particulars

Item	Value
Course title	Virtual Systems and Services
Programme	BS Information Technology. A domain elective under HEC 2017 and HEC 2023; absent from HEC 2025
Credit hours	3 (2–1): two credit hours of lectures and one of laboratory
Prerequisite	Programming Fundamentals (HEC 2017). In practice this course also assumes comfort at a Linux command line; the self-check two pages on says what that means concretely
Duration	Sixteen teaching weeks, with a midterm in Week 8 and a project due in Week 16 (Appendix G)
Course introduction	The HEC 2017 aim, verbatim: “This course will investigate the current state of virtualization in computing systems. Virtualization at both the hardware and software levels will be examined, with emphasis on the hypervisor configurations of systems such as Vmware, Zen and Hyper-V. The features and limitations of virtual environments will be considered, along with several case studies used to demonstrate the configuration and management of such systems. Para-virtualized software components will be analyzed and their pros and cons discussed. Processor and peripheral support for virtualization will also be examined, with a focus on emerging hardware features and the future of virtualization.”
Set texts	See the note below. The two titles HEC 2017 lists are not about this subject; Appendix E gives the references this handout actually uses

The set texts in the published outline are the wrong books

HEC 2017 lists two texts for this course: K. M. Stanney, *Handbook of Virtual Environments: Design, Implementation and Applications*, and G. Burdea, *Virtual Reality Technology*.

Both are virtual-reality references — head-mounted displays, haptics, presence and simulator sickness. Neither mentions a hypervisor. The outline's own description is unambiguously

about system virtualization, so the reading list appears to have been assembled from the course title rather than its contents. This handout does not use them. Appendix E lists the primary sources it does use, beginning with Popek and Goldberg’s 1974 paper, the Intel and AMD architecture manuals, and the Xen, KVM and Firecracker papers.

The outline also spells Xen as “Zen”. It is Xen, after the Xenoserver project at Cambridge.

Course learning outcomes

HEC 2017 publishes an outline but no Bloom-levelled outcomes, and the later editions publish neither. The six below are written against the 2017 outline and traced to the chapters that deliver them.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	Explain what a virtualization layer is, and classify a given system by the layer it virtualizes and the technique it uses	C2 Understand	1, 2, 8
CLO-2	State the Popek–Goldberg conditions and explain why x86 failed them, and how binary translation and hardware assistance each answer that failure	C3 Apply	3, 4
CLO-3	Configure and manage virtual machines and containers on at least three platforms, including storage and networking	C3 Apply	5–7, 9–13, 16
CLO-4	Analyze the performance of a virtualized workload, identify the bottleneck from hypervisor metrics, and size a host for a given set of guests	C4 Analyze	6, 17, 18
CLO-5	Evaluate the security and the cost of a virtualization or cloud design, and justify a deployment model against stated requirements	C5 Evaluate	14, 19, 21, 22
CLO-6	Build, deploy, measure and document a working virtualized service	C6 Create	23, and every lab

Coverage of the HEC 2017 course outline

The published outline is a single descriptive paragraph rather than a topic list. Each of its eleven clauses is traced below, in the order HEC writes them. Nothing is omitted.

HEC 2017 clause	Chapter	Where exactly
“The current state of virtualization in computing systems”	1	What the layer is, the four motives for inserting one, and the line from CP-67 to the cloud
“Virtualization at the hardware level”	4	VT-x and AMD-V, the VMCS, extended page tables, the IOMMU

HEC 2017 clause	Chapter	Where exactly
“... and software levels”	2, 3	Full virtualization, paravirtualization and binary translation; the virtual machine monitor itself
“Hypervisor configurations of systems such as VMware, Xen and Hyper-V”	5	All three, plus KVM, each as an architecture diagram and a comparison table
“The features ... of virtual environments”	6, 7, 17	Overcommit, ballooning, page sharing, NUMA; virtio and passthrough; snapshots and live migration
“... and limitations”	6, 18, 19	What overcommit costs, how a virtualized benchmark lies, and the attack surface the layer creates
“Several case studies ... configuration and management of such systems”	16, 22	The management platforms in Chapter 16; four full case studies in Chapter 22
“Para-virtualized software components will be analyzed”	7	The split-driver model, virtqueues, and the PV driver sets of each platform
“... and their pros and cons discussed”	7	Emulated against paravirtualized against passthrough, measured, with the cost of each
“Processor ... support for virtualization”	4	Chapter 4 entire
“... and peripheral support”	7, 20	SR-IOV and VFIO in Chapter 7; GPU and accelerator partitioning in Chapter 20
“Emerging hardware features and the future of virtualization”	20, 21	Accelerator virtualization; confidential computing, attestation, CXL and composable infrastructure

Topics taught here that the 2017 outline does not name

The 2017 outline predates Docker’s general adoption, Kubernetes entirely, and every confidential-computing platform in Chapter 21. Eight chapters go beyond its letter. They are listed here so that the departure is explicit.

Chapters	Why they are here	What they rest on
8–11 Containers	The dominant form of virtualization in industry since 2015, and the direct subject of the course HEC 2025 created to replace this one. Taught as what they are — operating-system-level virtualization, one layer above the hypervisor	HEC 2025, <i>Microservices Architecture and Docker Containers</i>
12–13 Storage and network	The 2017 clause “virtualization at both the hardware and software levels” is not restricted to the processor, and no hypervisor is useful without virtual disks and virtual switches	HEC 2017, “hardware and software levels”

Chapters	Why they are here	What they rest on
14 Cloud models	Where virtualization is actually consumed. Without it the course ends at the machine room door	HEC 2025, <i>Cloud Computing</i>
21 Confidential computing	The direct continuation of “emerging hardware features and the future of virtualization”, written sixteen years later	HEC 2017, “emerging hardware features”



The overlap with *Cloud Computing* is real, intended, and bounded. The curriculum map that traces these editions reaches a blunt conclusion — “virtualisation is one chapter of cloud computing, not a peer of it” — and where a department offers both courses, that is the right call.

Where this one is offered, it earns its three credit hours by going a layer *below* a cloud course rather than alongside it. A cloud course explains what an instance is and how to rent one. This course explains what happens on the host when you do: which instruction trapped, which page table was walked twice, which driver was split in half, and why the instance beside yours can make yours slow. Appendix F draws the line topic by topic, and names what this course deliberately does not teach.

The Course at a Glance

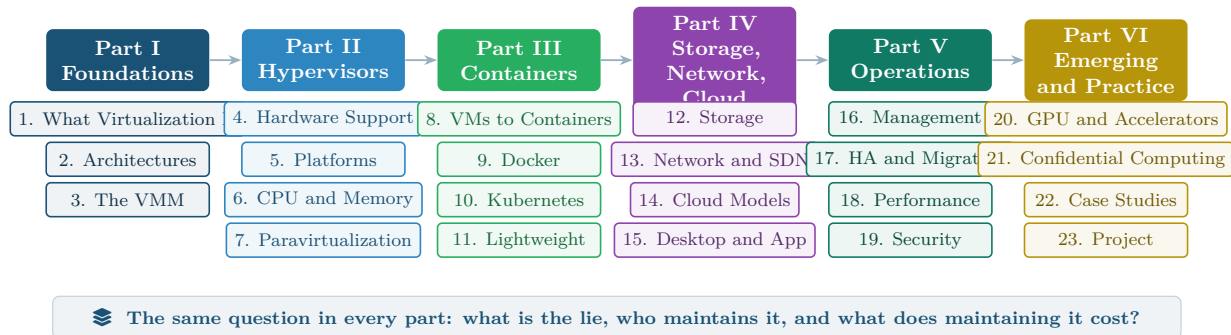


Figure F.1: The six parts and the chapters in each. Parts I and II teach the HEC 2017 outline close to specification; Parts III to VI carry it forward to the stack in production use today.

This course sits beside Cloud Computing, System and Network Administration and the CCNA track, and one layer below all three. Appendix F states what belongs to each.

The Laboratory Stack

This course is unusual among the BSIT electives in that the subject is infrastructure, and infrastructure is expensive. A faithful laboratory for the 2017 outline would need a vSphere licence, a Hyper-V cluster, a SAN and a GPU with vGPU entitlement — perhaps forty thousand dollars of software before any hardware.

That is not what this book asks for. Every lab runs on one laptop with free software, and the substitution is named each time it is made.

What the chapter teaches	What the lab uses	Why the substitution is honest
VMware ESXi and vSphere	KVM with <code>virsh</code> and <code>virt-manager</code>	Both are Type-1 hypervisors with the same primitives. The vocabulary differs; the mechanism does not
Microsoft Hyper-V	Hyper-V where Windows Pro is available, KVM otherwise	Hyper-V is free with the operating system most students already run
vMotion / Live Migration	<code>virsh migrate</code> between two nested hosts	Pre-copy migration with dirty-page tracking, which is the mechanism Chapter 17 teaches
Enterprise SAN, Ceph, vSAN	LVM thin pools; a single-node <code>cephadm</code> cluster	Thin provisioning, copy-on-write and the over-subscription failure are all reproducible on one disk
Production Kubernetes	<code>kind</code> (Kubernetes in Docker)	A real API server, scheduler and kubelet. Only the node count is a pretence
NVIDIA vGPU and MIG	<code>nvidia-smi</code> partitioning where a GPU exists; captured output otherwise	Partitioning policy is the lesson; the silicon is not available to rent by the hour
AWS, Azure and GCP	The three free tiers, with a teardown step in every lab	The instance you launch is the instance the chapter describes
Intel TDX, AMD SEV	Captured attestation output in <code>code/</code>	Confidential VMs need a specific processor generation and a cooperating host

⚠ One thing the free stack cannot teach you

Scale. Every pathology in Chapter 18 — the noisy neighbour, the oversubscribed cluster, the storage array that saturates at three in the afternoon — is a property of many machines sharing one resource, and a laptop running four guests will not show it to you honestly.

Where that is the case the chapter says so and gives measured numbers from real estates instead of asking you to produce them. Read those numbers as data, and treat the lab as the mechanism rather than the magnitude.

Appendix A installs the whole stack and verifies it. Appendix B is the command reference you will keep open beside the labs.

Notation and Conventions

Term or symbol	Meaning in this book
host	The physical machine, and the software running directly on it
guest	An operating system running inside a virtual machine
hypervisor, VMM	Used interchangeably, as the literature does. Where the distinction matters — KVM is a VMM inside a host kernel, not a free-standing hypervisor — the text says which it means
Ring 0, 1, 3	x86 privilege levels: kernel, the level Xen deprived guests into, and user
root, non-root	VMX operation modes, Chapter 4. Nothing to do with the <code>root</code> user
GVA, GPA, HPA	Guest virtual, guest physical and host physical address — the three address spaces a nested page walk crosses
vCPU	A virtual processor presented to a guest; scheduled onto a physical core by the hypervisor
pCPU	A physical core or hardware thread
R	Consolidation ratio: guests per host
σ	Overcommit ratio: allocated resource divided by physical resource
%RDY	Fraction of time a vCPU was runnable but not scheduled. The single most useful number in Chapter 18
RPO, RTO	Recovery point objective (how much data you may lose) and recovery time objective (how long you may be down)
p95, p99	The 95th and 99th percentile of a latency distribution
1 GiB, 1 GB	Binary and decimal gigabytes, 2^{30} and 10^9 bytes. Storage vendors mean the second; operating systems usually report the first. This book is explicit every time it matters



Two conventions worth stating because the industry is inconsistent about them.

Type-1 and Type-2 are used in this book as Goldberg defined them in 1973 — by whether the VMM runs on bare metal or on a host operating system — and Chapter 2 shows why the distinction is blurrier in practice than the textbook diagram suggests. KVM is the awkward case, and it is discussed rather than assigned.

Virtualization and emulation are not synonyms. Virtualization runs guest instructions on the real processor wherever it can; emulation interprets every one of them. QEMU does both, in different modes, which is why it appears on both sides of the comparison in Chapter 2.

Prerequisite Self-Check

The published prerequisite is *Programming Fundamentals*, which is not really what this course needs. What it needs is the twelve things below. If you can do ten of them, begin at Chapter 1. If you cannot, Appendix A and the first two sections of Appendix B will close most of the gap in an afternoon.

1. Explain what an operating system kernel does that a user program cannot. (Operating Systems)
2. Describe what happens on a system call, from the user instruction to the return. (Chapter 3)
3. Explain virtual memory: what a page table maps, and what the MMU does with it. (Chapter 4)
4. State what a process is, and what the kernel keeps for each one. (Chapter 8)
5. Navigate a Linux filesystem and read a file from the command line without a graphical tool. (Appendix B)
6. Edit a text file over SSH and explain what SSH is doing. (Appendix A)
7. Read a shell script containing a loop, a conditional and a pipeline. (Appendix B)
8. Explain the difference between an IP address, a subnet mask and a default gateway. (Chapter 13)
9. Say what a TCP port is and how two services share one host. (Chapter 13)
10. Convert between binary, decimal and hexadecimal, and read a size in bytes written either way. (Notation, above)
11. Explain what a file format is, given that a virtual disk is a file that contains a filesystem. (Chapter 12)
12. Describe what a compiler produces and in what sense a CPU “runs” it. (Programming Fundamentals)



Notice what is *not* on that list: no calculus, no statistics, no networking certification, and no prior exposure to any cloud platform. The arithmetic in this book is ratios, percentages and powers of two. What it genuinely demands instead is comfort with the idea that software can be *underneath* other software — which is uncomfortable the first time, and then never again.

Contents

How to Use This Handout	ii
Course Specification	v
The Course at a Glance	ix
The Laboratory Stack	x
Notation and Conventions	xi
Prerequisite Self-Check	xii
I Foundations of Virtualization	1
1 What Virtualization Is	3
1.1 A Room Full of Idle Servers	3
1.2 What It Is Not	4
1.3 Why Anyone Would Do This	5
1.4 Where It Came From	7
1.5 What Is Virtualized, and Where to Find It	8
2 Virtualization Architectures	12
2.1 Two Places to Stand	12
2.2 Three Ways to Keep the Promise	13
2.3 Where Containers Sit	15
2.4 Choosing	16
3 The Virtual Machine Monitor	20
3.1 What a Monitor Must Guarantee	20
3.2 Privileged, Sensitive, and the Difference	21
3.3 Why x86 Failed	22
3.4 Two Ways Round	23
II Hypervisors and Compute Virtualization	28
4 Hardware Support for Virtualization	30
4.1 What Intel and AMD Had to Fix	30
4.2 The VMCS, and What a Transition Costs	31
4.3 Two-Dimensional Paging	32
4.4 The IOMMU	34

5	Major Hypervisor Platforms	38
5.1	The Question That Separates Them	38
5.2	VMware ESXi and vSphere	39
5.3	Microsoft Hyper-V	39
5.4	KVM and QEMU	40
5.5	Xen	40
5.6	Choosing	41
6	CPU and Memory Virtualization	45
6.1	Scheduling a Processor That Is Not There	45
6.2	Why More vCPUs Can Be Slower	46
6.3	Memory, and Why It Is the Constraint	47
6.4	NUMA	49
7	Paravirtualization and Device Virtualization	53
7.1	Three Ways to Give a Guest a Device	53
7.2	What Passthrough Costs You	56
III	Containers and Lightweight Virtualization	60
8	From Virtual Machines to Containers	62
8.1	A Different Place to Cut	62
8.2	Namespaces	63
8.3	Control Groups	64
8.4	The Image, and the Union Filesystem	65
9	Docker and the Container Ecosystem	69
9.1	What Docker Actually Added	69
9.2	Images, Layers and Digests	70
9.3	The OCI Stack	70
9.4	The Build Cache, and the Mistake Everyone Makes	71
9.5	A Dockerfile That Holds Up	72
10	Container Orchestration with Kubernetes	77
10.1	The Problem Containers Created	77
10.2	What the Cluster Is Made Of	78
10.3	Pods, Deployments and Services	79
10.4	Requests, Limits and What the Scheduler Does With Them	79
11	Lightweight and Secure Virtualization	85
11.1	The Choice Nobody Wanted to Make	85
11.2	MicroVMs	86
11.3	gVisor: a Kernel in User Space	87
11.4	Kata Containers	87
11.5	Unikernels and WebAssembly	88

IV	Storage, Network and Cloud Virtualization	92
12	Storage Virtualization	94
12.1	Three Interfaces	94
12.2	The Virtual Disk	95
12.3	Thin Provisioning, and How It Fails	96
12.4	Snapshots and Copy-on-Write	97
12.5	Software-Defined Storage	98
13	Network Virtualization and SDN	101
13.1	The Switch That Is Not There	101
13.2	VLANs, and the Number 4094	102
13.3	Overlays	102
13.4	Separating the Planes	103
13.5	NFV and Container Networking	104
14	Cloud Service and Deployment Models	108
14.1	What Makes a Cloud a Cloud	108
14.2	The Service Models	109
14.3	Deployment Models, and What an Instance Is	110
14.4	Paying For It	110
14.5	Shared Responsibility	112
15	Desktop and Application Virtualization	116
15.1	Moving the Computer, Leaving the Screen	116
15.2	What Travels Over the Wire	117
15.3	Virtualizing the Application Instead	119
V	Management, Performance and Security	123
16	Virtualization Management Platforms	125
16.1	One Host Is Not a Small Fleet	125
16.2	The Platforms	126
16.3	Infrastructure as Code	127
16.4	The Image Pipeline	128
16.5	Drift	129
17	High Availability, Live Migration and Disaster Recovery	133
17.1	Moving a Running Machine	133
17.2	High Availability, Fault Tolerance and Quorum	135
17.3	RPO, RTO and the Cost of a Nine	136

18 Performance Monitoring and Capacity Planning	141
18.1 The Guest Is an Unreliable Witness	141
18.2 The Counters That Matter	142
18.3 Why the Mean Is the Wrong Number	142
18.4 Benchmarking Honestly	144
18.5 Capacity Planning	144
19 Security in Virtualized Environments	149
19.1 What the Layer Added	149
19.2 Where Escapes Come From	150
19.3 Blast Radius Is the Right Metric	151
19.4 Side Channels	152
19.5 Containers	152
19.6 Hardening, in Order	153
VI Emerging Hardware and Practice	157
20 GPU and Accelerator Virtualization	159
20.1 Why a GPU Is Not a CPU	159
20.2 The Three Models	160
20.3 DPUs: Moving the Infrastructure Off the Host	162
21 Confidential Computing and Emerging Hardware	165
21.1 The Trust Model Inverts	165
21.2 What the Hardware Does	166
21.3 Where the Platforms Stand	167
21.4 Composable Infrastructure and CXL	168
22 Case Studies in Enterprise and Cloud Virtualization	172
22.1 How to Read These	172
22.2 Case 1: The Consolidation That Nearly Failed	173
22.3 Case 2: The Cloud Migration That Cost More	174
22.4 Case 3: Edge Virtualization at Twenty-Three Sites	175
22.5 Case 4: A Licence Change, and What It Revealed	176
22.6 What the Four Have in Common	177
23 The Semester Project	180
23.1 What the Project Is	180
23.2 Choosing One	181
23.3 Measuring Honestly	182
23.4 The Report	182
23.5 The Presentation	183
23.6 Marking	183

Appendices	187
A The Laboratory Toolkit	188
A.1 What You Need	188
A.2 The Stack, in One Pass	188
A.3 Windows Hosts	189
A.4 Cloud Accounts	190
A.5 When a Lab Cannot Run Here	190
A.6 Tearing It All Down	191
B Command Reference	192
B.1 Is This Machine Capable?	192
B.2 Virtual Machines: <code>virsh</code>	192
B.3 Virtual Disks: <code>qemu-img</code>	193
B.4 Containers: <code>docker</code>	193
B.5 Kubernetes: <code>kubect1</code>	193
B.6 Networking	194
B.7 Storage	194
B.8 Measurement	194
B.9 Shell Idioms Used in the Labs	195
C Answers to the Checkpoints	196
D Glossary	205
E Resources and Further Reading	206
F The Boundary with the Neighbouring Courses	207
G The Sixteen-Week Schedule	208

List of Figures

F.1	The six parts and the chapters in each. Parts I and II teach the HEC 2017 outline close to specification; Parts III to VI carry it forward to the stack in production use today.	ix
P1.1	Reading path through Part I. Chapter 3 is the hinge: it states the conditions a correct hypervisor must meet, and every hardware feature in Part II exists because x86 did not meet them.	2
1.1	The layer, and the only claim it makes. Each guest is told a different machine exists, and none of the three was modified to be told. Every later chapter replaces one row of this diagram with something more specific.	4
1.2	The same work, before and after. Nothing was made faster; the idle capacity was simply stopped from being bought. The right-hand picture is also the risk picture: three failure domains where there were forty.	7
1.3	The field in ten dates, coloured by the part of this book that covers each. The gap in the middle is the subject of Chapter 3: the theory was settled in 1974, and the dominant architecture then spent a quarter of a century unable to satisfy it.	7
2.1	The only two places a hypervisor can stand. The right-hand stack is easier to install and slower at everything, because every device access the guest makes is translated twice.	13
2.2	The same instruction, three architectures. Only the middle one requires the guest's cooperation; only the right-hand one requires the processor's. The red step is where the work is done, and it is a different party each time.	14
2.3	Two questions, not one: <i>which layer</i> is virtualized, and <i>by what technique</i> . Products are placed by what they principally do; most occupy more than one cell in practice.	16
3.1	Trap-and-emulate. The design is sound and was old by 1974; the whole question is whether the processor traps everything the monitor needs it to trap.	21
3.2	The condition, and the architecture that failed it. The seventeen instructions on the right are not exotic: several are ordinary, unprivileged instructions a compiler emits without comment.	22
3.3	Ring depriving. Moving the guest kernel down one ring makes the hardware trap most of what the monitor needs; the instructions that still slip through are removed from the guest's source instead.	24
P2.1	Reading path through Part II. Chapter 4 is the silicon; the three that follow spend it — on platforms, on the processor and memory, and on everything plugged into the bus.	29
4.1	VT-x in one picture. The guest runs in ring 0 of a second operating mode, so nothing it executes behaves unexpectedly; the hypervisor regains control through a hardware transition rather than a trap it had to arrange.	31
4.2	A nested page walk, and the trade it represents. Shadow paging is cheaper on a TLB miss and ruinous on every guest page-table edit; extended page tables invert both, and edits are the more frequent event.	34

5.1	The four platforms, drawn to the same question: where are the drivers (red)? Xen's answer gives it the smallest hypervisor in the room and the most moving parts; ESXi's gives it the most predictable behaviour and the shortest hardware compatibility list.	39
6.1	Why a bigger guest can be a slower guest. Right-sizing downward is the most common successful intervention in a contended cluster, and the one administrators are most reluctant to make.	46
6.2	The reclamation ladder. A hypervisor works down it as pressure rises; an administrator's job is to ensure it never reaches the bottom rung.	47
6.3	NUMA on a two-socket host. The green guest is a sizing decision, not a tuning option: once a guest is wider than a socket, no amount of hypervisor configuration makes its memory local again.	49
7.1	The same I/O request, three ways. Reading down each lane gives the cost: the left lane crosses the boundary dozens of times per request, the middle once per batch, the right not at all.	54
P3.1	Reading path through Part III. The arc runs from kernel primitives to a cluster scheduler — and then back to Part II, because Chapter 11's answer to container isolation is to put a virtual machine underneath each one.	61
8.1	The whole of Part III in one picture. Nothing else about containers is a new idea — only the height at which the stack is cut.	63
8.2	Overlayfs. The layers are why pulling a second image from the same base downloads almost nothing, and why a thousand containers of one image cost little more on disk than one.	65
9.1	The runtime stack. Each layer was split out of the one above it as the ecosystem matured; the bottom two rows are the only ones that are not replaceable.	71
9.2	The same four instructions in two orders. Nothing else differs, and the rebuild costs differ by a factor of 300.	71
10.1	A cluster. The arrows all point at the API server, which is the architectural claim: components do not coordinate with each other, they each reconcile against one shared record.	78
11.1	The trade, with measured figures on the horizontal axis. The dashed line is the move the designs in this chapter make: a hardware-enforced boundary at something close to a container's cost.	86
P4.1	Reading path through Part IV. Chapters 12 and 13 complete the virtual machine; Chapter 14 is where the industry sells it, and Chapter 15 is the case where the thing virtualized is somebody's desk.	93
12.1	The path of one write. Each layer below the virtual disk file can be replaced without the guest noticing — which is what makes migration, snapshots and replication possible at all.	95
12.2	Thin provisioning, drawn to scale for the worked example. The gap between the second and third bars is the saving; the gap between the first and second is the exposure.	96

13.1	VXLAN encapsulation, to scale. The 24-bit VNI in those eight bytes is what replaces the 12-bit VLAN tag; the 50 bytes are what causes the fault in the worked example.	102
14.1	The service models as one picture. FaaS sits beside SaaS rather than beyond it, because what you still own there is code rather than configuration.	109
15.1	A remote desktop session. The asymmetry is extreme — megabits down, kilobits up — and the quality the user perceives is governed by the small return arrow, not the large one.	117
P5.1	Reading path through Part V. Live migration is the one in the middle because it is what the rest is for: a fleet you can manage is a fleet whose machines can move without stopping.	124
16.1	The shape every platform in this chapter has. The interesting differences are in scope — how much of the stack the platform claims — not in this diagram.	126
16.2	The pipeline. The dashed arrow is the discipline: the only supported way to change what a guest contains is to change the image and re-instantiate.	128
17.1	Pre-copy migration, drawn to scale for the worked example. The upper case finishes in 20 seconds with a 50 ms pause; the lower one never finishes at all.	134
18.1	A real latency distribution. The mean sits below almost all of the mass, and every pathology in this book lives to the right of it.	143
19.1	The attack surface. The hypervisor core is small and heavily reviewed; the device models around it are neither, and that is where the escapes are.	150
P6.1	Reading path through Part VI. Chapters 20 and 21 are where the hardware is still moving; Chapters 22 and 23 put the whole book to work, first on other people's decisions and then on your own.	158
20.1	Three ways to divide one accelerator. Only the right-hand one gives a guarantee, and only the left-hand one gives the whole card.	160
21.1	The inversion. The hypervisor still schedules the guest and allocates its memory; it simply loses the ability to read it. Everything red is in the attack path and outside the trust boundary.	166

PART I

Foundations of Virtualization

Every idea in this book is a special case of one move: put a layer underneath something, and make what sits above believe nothing has changed. Part I says what that layer is, names the ways it can be built — and then proves the uncomfortable fact that for twenty years the most widely sold processor in the world could not host one correctly.

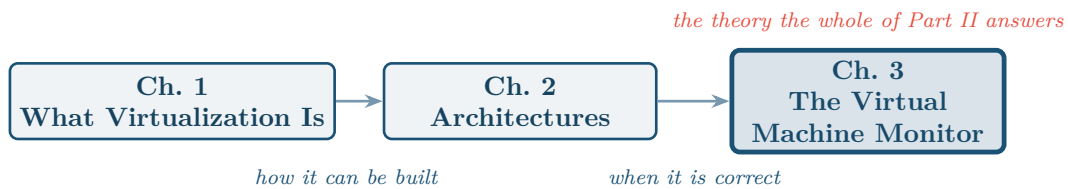


Figure P1.1: Reading path through Part I. Chapter 3 is the hinge: it states the conditions a correct hypervisor must meet, and every hardware feature in Part II exists because x86 did not meet them.

What Virtualization Is

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Virtualization

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** virtualization as a layer that presents a resource different from the one it sits on, and **identify** that layer in a system you are shown.
- **Distinguish** a virtual machine from a process, an emulator and a simulator, by what each one is allowed to assume about the hardware.
- **Explain** the four reasons an organisation inserts such a layer — consolidation, isolation, encapsulation and elasticity — and **state** which of the four motivated each milestone in the field's history.
- **Compute** a consolidation ratio and the power it saves, from measured utilisation figures.
- **Describe** what is being virtualized in a given product, and locate the chapter of this book that covers it.

🔗 Before You Start

Nothing in this book. You need the idea that an operating system stands between a program and the hardware, and that a program cannot tell the difference between a real device and a convincing imitation of one. If the second half of that sentence sounds improbable, this chapter is the right place to start.

📖 Key Terms in This Chapter

virtualization • virtual machine • hypervisor • virtual machine monitor • host • guest • abstraction • multiplexing • aggregation • emulation • consolidation • consolidation ratio • utilisation • server sprawl • isolation • encapsulation • elasticity • time-sharing

1.1 A Room Full of Idle Servers

Walk into a university machine room that has not been consolidated and you will find something strange: forty servers, each costing as much as a car, each running at eight per cent of its capacity. The library catalogue has a server. The examination system has a server. The staff portal has a server. None of them is busy. Together they draw fourteen kilowatts and need as much again in cooling, and the department pays for all of it.

Nobody planned this. It arrived one purchase order at a time, and each purchase was defensible. The examination system must not be slowed down by the library catalogue, so it gets its own machine. The portal needs a newer version of the database than the catalogue can tolerate, so it gets its own machine. The vendor's support contract is void if anything else runs on the box, so it gets its own machine. Every reason is a reason about *isolation*, and the only isolation mechanism available was a second computer.

This is **server sprawl**, and it is the problem virtualization was brought to industry to solve. The insight is that the forty applications genuinely do need to be isolated from one another — that part was never wrong — but that isolation does not have to be purchased one chassis at a time.

Virtualization

Virtualization is the insertion of a software layer that presents a resource which differs from the one physically present, while preserving the interface that software above the layer already expects.

The layer does one of three things, or several at once:

- **multiplexing** — one physical resource is presented as many (one processor as twelve vCPUs);
- **aggregation** — many physical resources are presented as one (twelve disks as one volume);
- **abstraction** — a resource is presented as a different kind of thing entirely (a file on a disk presented as a disk).

What makes it virtualization rather than merely software is the second clause: the interface is *preserved*. An unmodified operating system, written with no knowledge that any of this is happening, runs correctly above the layer.

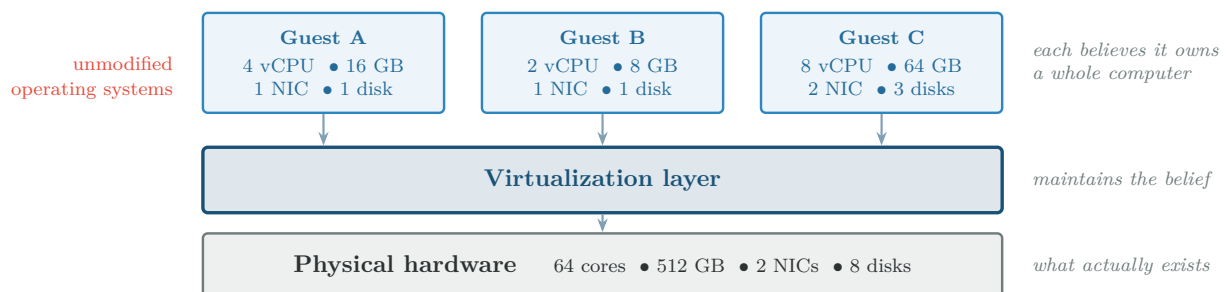


Figure 1.1: The layer, and the only claim it makes. Each guest is told a different machine exists, and none of the three was modified to be told. Every later chapter replaces one row of this diagram with something more specific.

Virtual Machine, Hypervisor, Host, Guest

A **virtual machine** is the illusion the layer sells: a complete computer — processor, memory, firmware, disks, network interfaces — implemented in software, on which an operating system can be installed as if on metal.

The **hypervisor**, equivalently the **virtual machine monitor** or VMM, is the software that creates and maintains that illusion. The **host** is the physical machine and the software running directly on it; a **guest** is an operating system running inside a virtual machine.

This book uses *hypervisor* and *VMM* interchangeably, as the literature does. Chapter 3 is the one place the distinction is sharpened, because that is where it begins to matter.

1.2 What It Is Not

Three neighbouring ideas are routinely confused with virtualization, and the confusion is worth clearing before it compounds.

Virtualization, Emulation, Simulation

Emulation interprets every guest instruction in software, translating from one instruction set to another. An ARM emulator on an x86 laptop reads an ARM instruction, works out what it means, and performs the equivalent x86 work. It is slow — commonly ten to a hundred times slower than native — and completely general: the guest architecture need not resemble the host at all.

Virtualization runs guest instructions *on the real processor* wherever it safely can, and intervenes only for the instructions it cannot allow to execute directly. It is therefore fast — commonly within a few per cent of native — and restricted: the guest must be built for the same architecture as the host.

Simulation models a system’s behaviour without claiming to run its software: a network simulator predicts throughput; it does not boot a router.

QEMU does the first two in different modes, which is why it appears on both sides of the comparison in Chapter 2. Running `qemu-system-aarch64` on an x86 host is emulation. Running `qemu-system-x86_64` with KVM enabled is virtualization, and the same binary does both.

⚠ A container is not a virtual machine, and both are virtualization

The two most common statements about containers in a first course contradict each other, and both are half right.

A container does not contain a virtual machine: there is no virtual processor, no virtual firmware, no guest kernel. Processes in a container run on the host’s kernel as ordinary processes, and `ps` on the host can see them.

But a container *is* virtualization by the definition above. The layer is inserted inside the kernel rather than beneath it, and what it multiplexes is the operating system’s own namespace of process identifiers, mount points, network interfaces and users. The interface preserved is the system-call interface, not the instruction set. Chapter 8 makes this precise; the reason it matters this early is that “virtual machine or container?” is a question about *which layer*, not about whether a layer exists.

1.3 Why Anyone Would Do This

Four motives, and they arrived in this order historically. Every product in this book was built for at least one of them, and the ones that lasted serve several.

Motive	What it buys	Where this book shows it
Consolidation	Many lightly-loaded workloads on one well-loaded machine. Fewer machines, less power, less cooling, less floor space, less to patch	The worked example below; Chapter 6 on how far it can be pushed; Chapter 18 on when it has gone too far
Isolation	A fault, a crash or a compromise in one workload does not reach the others. This is the reason the forty servers existed, and virtualization preserves it while removing the forty	Chapter 3 on what makes isolation sound; Chapter 19 on where it fails
Encapsulation	A running machine becomes a file. It can be copied, versioned, moved, snapshotted and restored — operations no physical server supports	Chapter 12 on the file; Chapter 17 on moving it while it runs
Elasticity	Capacity becomes a quantity you request rather than a chassis you procure. Minutes instead of months, and returnable	Chapter 14 entire; Chapter 10 for the same idea one layer up

💡 Encapsulation is the one with no physical analogue

Consolidation is an efficiency argument and isolation is a safety argument, and a sufficiently rich organisation could buy its way out of both. Encapsulation is different in kind.

Once a running computer is a file, every operation you know how to perform on a file becomes available to it. You can take a snapshot before a risky upgrade and return to it in four seconds when the upgrade fails. You can copy a production machine into a test environment, bit for bit, including the fault you are trying to reproduce. You can keep last Tuesday. None of this has a physical equivalent — there is no way to take a snapshot of a server in a rack — and it is the capability administrators miss most when they go back.

📊 Worked Example: The Forty Servers, Costed

The machine room of the opening section, measured over one semester. Decide how many hosts replace it, and what that saves.

What is there. 40 servers, each 2 sockets $\times$ 8 cores = 16 cores and 64 GB of memory. Measured per server: mean CPU utilisation 8%, peak 22%; mean memory in use 11 GB. Each draws 350 W.

Step 1 — what is actually consumed.

- CPU: $40 \times 16 \times 0.08 = 51.2$ cores' worth of work, spread over 640 physical cores.
- Memory: $40 \times 11 = 440$ GB in use, of 2,560 GB installed.

Step 2 — size the replacement on memory, not CPU. Memory is the binding constraint in almost every consolidation, because it cannot be oversubscribed as freely as processor time (Chapter 6 says why). Allow 30% headroom for growth and for the hypervisor itself:

$$440 \times 1.3 = 572 \text{ GB}$$

On hosts of 512 GB that is $\lceil 572/512 \rceil = 2$ hosts. Add one more so that a single host failure does not take the estate down — an $N+1$ design — giving **3 hosts**.

Step 3 — check that CPU is not the real constraint. Give each workload 4 vCPUs, which is generous against a measured peak of $16 \times 0.22 = 3.5$ cores. Three hosts of 2×32 cores give 192 physical cores:

$$\sigma = \frac{40 \times 4}{192} = \frac{160}{192} = 0.83$$

An overcommit ratio below 1 means that even if every guest demanded its full allocation simultaneously, the cores exist. CPU is not the constraint; memory was right.

Step 4 — the consolidation ratio.

$$R = \frac{40 \text{ workloads}}{3 \text{ hosts}} = \mathbf{13.3 : 1}$$

Step 5 — the power. The new hosts are larger and draw 700 W each.

$$40 \times 350 = 14,000 \text{ W} \quad 3 \times 700 = 2,100 \text{ W} \quad \text{saved: } 11.9 \text{ kW}$$

Cooling roughly tracks the load; at a power usage effectiveness of 1.6 the facility saves $11.9 \times 1.6 = 19.0$ kW. Over a year:

$$19.0 \times 8,760 = 166,400 \text{ kWh}$$

At PKR 45 per kWh that is **about PKR 7.5 million a year**, before counting the thirty-seven servers not bought, the rack space not rented, or the thirty-seven operating systems not patched.

What the number does not say. Those three hosts are now a shared fate. The arithmetic above bought a 13:1 reduction in machines and a 13:1 concentration of risk, and Chapter 17 is where that bill comes due.

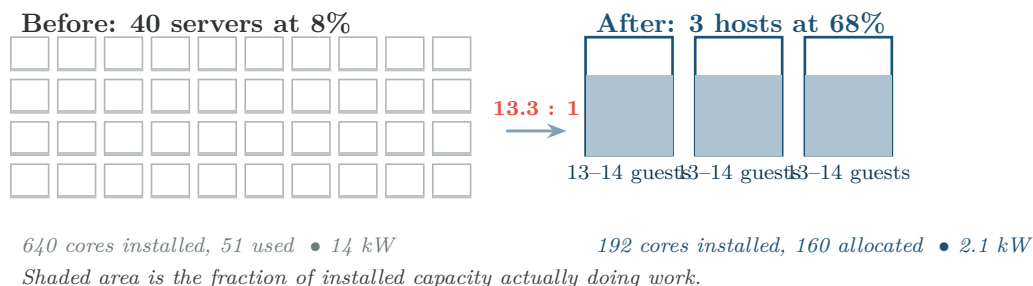


Figure 1.2: The same work, before and after. Nothing was made faster; the idle capacity was simply stopped from being bought. The right-hand picture is also the risk picture: three failure domains where there were forty.

1.4 Where It Came From

Virtualization is not a 2000s invention. It is a 1960s invention that the personal computer made irrelevant for thirty years and the data centre made urgent again.

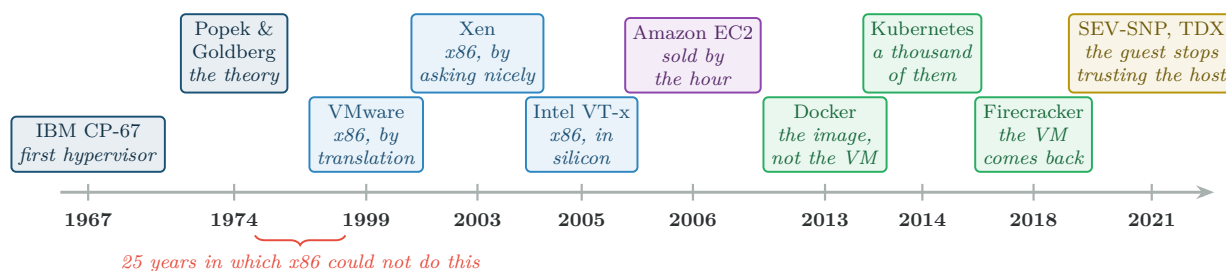


Figure 1.3: The field in ten dates, coloured by the part of this book that covers each. The gap in the middle is the subject of Chapter 3: the theory was settled in 1974, and the dominant architecture then spent a quarter of a century unable to satisfy it.

IBM built CP-67 for the System/360 Model 67 because its customers had a problem of scarcity rather than sprawl. A mainframe cost millions and there was exactly one; a dozen departments each needed their own operating environment on it, and several of those operating systems could not be persuaded to share. CP-67 gave each department a complete virtual System/360 of its own. Its successor VM/370 ran in production for decades, and its descendant z/VM still does.

Then the minicomputer and the personal computer made computers cheap, and the problem CP-67 solved stopped being a problem: if you needed a second computing environment you bought a second computer. The idea went quiet for twenty-five years, kept alive on mainframes and in research, while the architecture that won the market — x86 — turned out, for reasons Chapter 3 sets out exactly, to be incapable of supporting it correctly.

💡 Why the second wave looked nothing like the first

CP-67 was built for scarcity: one machine, many users. VMware was built for the opposite — forty machines, each almost idle. The motive inverted completely, and the mechanism was reused unchanged.

That is worth noticing because the same inversion happened again. Containers were adopted in 2013 largely for *packaging*: a reproducible image that runs the same on a laptop and in production. Only afterwards did the industry discover that the same mechanism gave it density — a thousand containers on a host that would hold fifty virtual machines — and density became the reason people cite today. A mechanism outlives the motive that produced it, which is why this book teaches mechanisms.

1.5 What Is Virtualized, and Where to Find It

“Virtualization” names a technique, not a product category. The useful question about any system is *which resource* it virtualizes, because that determines what it can promise.

What is virtualized	So that	Examples	Chapter
The whole machine	An unmodified operating system boots on it	ESXi, Hyper-V, KVM, Xen	Chapter 5
The operating system	Processes see a private system, sharing one kernel	Docker, containerd, LXC	Chapter 8, Chapter 9
The processor and memory	Several guests divide cores and RAM	vCPU scheduling, ballooning	Chapter 6
A device	A guest drives hardware it does not exclusively own	virtio, SR-IOV, VFIO	Chapter 7
Storage	A file behaves as a disk; many disks behave as one	qcow2, VMDK, Ceph, vSAN	Chapter 12
The network	A switch exists in software; a LAN spans data centres	Open vSwitch, VXLAN, SDN	Chapter 13
The desktop	The screen is here and the computer is elsewhere	VDI, Citrix, AVD	Chapter 15
An accelerator	One GPU serves several tenants	vGPU, MIG, SR-IOV	Chapter 20

! Common Pitfalls

- **Treating “virtual” as meaning “not real”.** A virtual processor executes real instructions on real silicon and consumes real electricity. What is virtual is the *mapping*, not the work. The fastest way to mislead yourself about capacity is to believe the vCPU count is free.
- **Sizing a consolidation from installed capacity rather than measured use.** The forty servers above had 2,560 GB installed and used 440 GB. Planning against the first number buys five times the hardware you need; it is the single most common and most expensive mistake in this subject.
- **Assuming consolidation is free.** It converts capital cost into shared risk and contention. Three hosts instead of forty is also three failure domains instead of forty, and every guest on a host is now a potential noisy neighbour to every other (Chapter 18).
- **Calling a hypervisor an operating system, or vice versa.** They schedule different things for different consumers. The confusion makes Chapter 3 incomprehensible, because the whole chapter turns on what the hypervisor is allowed to assume that an operating system is not.

- **Believing the marketing distinction between “virtualization” and “cloud”.** A cloud is virtualization with a billing system, an API and somebody else’s staff. Chapter 14 is explicit about which of the three is doing the work in any given claim.

Try It Yourself: Your First Virtual Machine

Runs on any Linux host, or on Windows under WSL2 with VirtualBox substituted at step 2. Appendix A installs the prerequisites. Expect fifteen minutes.

```
#!/usr/bin/env bash
# ch01_first_vm.sh -- create, snapshot, break and restore a virtual machine.
set -euo pipefail

# --- 1. Does this processor support virtualization at all? -----
# vmx = Intel VT-x, svm = AMD-V. Chapter 4 explains what these flags buy.
# A count of 0 means the feature is absent or disabled in firmware.
grep -c -E '\<(vmx|svm)\>' /proc/cpuinfo
lscpu | grep -i 'virtuali[sz]ation' || echo "no such line reported"

# --- 2. Install the hypervisor and confirm it is usable -----
sudo apt-get install -y qemu-kvm libvirt-daemon-system virtinst \
    libguestfs-tools cloud-image-utils
# virt-host-validate prints PASS/WARN/FAIL for every prerequisite.
sudo virt-host-validate qemu | head -20

# --- 3. Fetch a cloud image and give it a login -----
# A cloud image is a disk file with an operating system already installed:
# encapsulation (section 1.3) before you have even started.
BASE=https://cloud-images.ubuntu.com/jammy/current
wget -nc -O ubuntu.img "$BASE/jammy-server-cloudimg-amd64.img"
cat > user-data <<'YAML'
#cloud-config
password: vss2026
chpasswd: { expire: False }
ssh_pwauth: True
YAML
cloud-localds seed.iso user-data

# --- 4. Create the virtual machine -----
# Read the arguments as a parts list for an imaginary computer.
sudo cp ubuntu.img /var/lib/libvirt/images/vss1.qcow2
sudo qemu-img resize /var/lib/libvirt/images/vss1.qcow2 10G
sudo virt-install --name vss1 --memory 2048 --vcpus 2 \
    --disk /var/lib/libvirt/images/vss1.qcow2,format=qcow2 \
    --disk seed.iso,device=cdrom --os-variant ubuntu22.04 \
    --import --graphics none --noautoconsole

# --- 5. Look at it from the host -----
# dominfo is the guest as the hypervisor sees it: an allocation.
sudo virsh list --all
sudo virsh dominfo vss1

# --- 6. Snapshot it, then break it on purpose -----
sudo virsh snapshot-create-as vss1 clean "before I broke it"
sudo virsh console vss1      # log in as ubuntu / vss2026, then:
                             #   sudo rm -rf /etc/systemd ; sudo reboot
                             # Ctrl-] returns you to the host.

# --- 7. Undo four minutes of damage in four seconds -----
# No physical server supports this operation. That is section 1.3's point.
sudo virsh snapshot-revert vss1 clean --running
sudo virsh snapshot-list vss1
```

```
# --- 8. Tear down -----
sudo virsh destroy vss1 || true
sudo virsh undefine vss1 --remove-all-storage --snapshots-metadata
```

What to notice. Step 6 took longer to type than step 7 took to undo, and step 7 has no physical equivalent. Everything else in this book is detail; that asymmetry is the subject.

🔍 Checkpoint — Test Yourself

1. A department runs 24 servers, each with 128 GB of memory, of which a measured 19 GB is in use. You are sizing hosts with 384 GB each, with 30% headroom and an N+1 spare. How many hosts, and what consolidation ratio does that give?
2. A colleague says “containers are faster than virtual machines because there is no virtualization layer.” Which half of that sentence is wrong, and what is the correct statement?
3. Of the four motives in Section 1.3, which one has no equivalent for physical servers at all? Give an operation it makes possible and say why a physical machine cannot offer it.

📋 Chapter Summary

- Virtualization inserts a layer that presents a resource differing from the physical one, while preserving the interface above it. It multiplexes, aggregates or abstracts — often all three.
- The preserved interface is what distinguishes it: an unmodified operating system runs above the layer.
- Emulation interprets every instruction and is general and slow; virtualization executes most instructions natively and is restricted and fast. QEMU does both, in different modes.
- Four motives: consolidation, isolation, encapsulation, elasticity. Encapsulation — a running machine as a file — is the one with no physical analogue.
- Size a consolidation from measured use, not installed capacity, and expect memory rather than CPU to be the binding constraint.
- The technique is from 1967. The twenty-five-year gap in the middle of its history is an x86 problem, and Chapter 3 is what went wrong.
- Ask of any product: *which resource does it virtualize?* That determines what it can promise, and which chapter of this book covers it.

🔧 Review Questions

1. (MCQ) The defining property that separates virtualization from ordinary resource-sharing software is that: (a) it runs in kernel mode; (b) it preserves the interface, so unmodified software runs above it; (c) it improves performance; (d) it requires hardware support.
2. (MCQ) A tool runs ARM binaries on an x86 laptop at roughly one fiftieth of native speed. This is: (a) virtualization; (b) paravirtualization; (c) emulation; (d) containerisation.
3. (Short) Give the three things a virtualization layer can do to a resource, with one concrete example of each from a system you have used.
4. (Short) Explain why memory, rather than processor time, is usually the constraint that decides how many guests fit on a host.
5. (Short) The opening section argued that forty separate servers was a rational response to a real requirement. Name the requirement, and explain precisely what virtualization changed about it.

6. **(Applied)** A faculty runs 60 servers at a measured mean of 6% CPU and 14 GB of 96 GB memory each, drawing 300 W apiece. Size a replacement estate on 768 GB hosts drawing 850 W, with 30% headroom and N+1. Give the host count, the consolidation ratio and the annual energy saved at PUE 1.6 and PKR 45 per kWh. Then state the one risk your proposal has introduced and what you would do about it.
7. **(Applied)** For each of these, say which resource is virtualized and which chapter of this book covers it: a laptop running Windows inside a window on macOS; a Kubernetes cluster; an NVMe disk presented to eight tenants; an office where staff keep no local files and log in to the same desktop from any machine; a GPU shared between three training jobs.

Virtualization Architectures

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** Type-1 from Type-2 hypervisors by where the monitor runs, and **explain** why KVM does not fit the distinction cleanly.
- **Compare** full virtualization, paravirtualization and hardware-assisted virtualization by what each one does with an instruction the guest may not execute.
- **Place** any given product on a two-axis taxonomy: which layer it virtualizes, and which technique it uses.
- **Compute** virtualization overhead from measured timings, and **explain** why the figure differs by an order of magnitude between a CPU-bound and an I/O-bound workload.
- **Choose** an architecture for a stated requirement and defend the choice against the two strongest alternatives.

🔗 Before You Start

Chapter 1 — the definition of the layer, the distinction between virtualization and emulation, and the four motives. This chapter names the ways the layer can be built; Chapter 3 then proves which of them are correct.

📖 Key Terms in This Chapter

Type-1 hypervisor • Type-2 hypervisor • bare-metal • hosted • full virtualization • binary translation • paravirtualization • hypercall • hardware-assisted virtualization • guest additions • PV driver • OS-level virtualization • virtualization overhead • attack surface • trusted computing base

2.1 Two Places to Stand

A hypervisor has to be *underneath* the guest. There are only two places to put it: directly on the hardware, or on top of an operating system that is already there. Goldberg named these Type-1 and Type-2 in 1973 and the names have stuck, although — as the end of this section argues — they have worn less well than the people who use them tend to admit.

📑 Type-1 and Type-2 Hypervisors

A **Type-1** or **bare-metal** hypervisor runs directly on the hardware. It *is* the operating system of the machine, in the sense that nothing else mediates its access to the processor, memory and devices. VMware ESXi, Microsoft Hyper-V and Xen are Type-1.

A **Type-2** or **hosted** hypervisor runs as a program on an ordinary operating system, alongside the browser and the editor. It asks that operating system for memory and for device access, like any other application. VMware Workstation, Oracle VirtualBox and Parallels Desktop are Type-2.

The test is not “does a general-purpose operating system exist on this machine?” — ESXi has a management console and Hyper-V has a full Windows parent partition. The test is *whether the hypervisor needs that operating system’s permission to touch the hardware*.

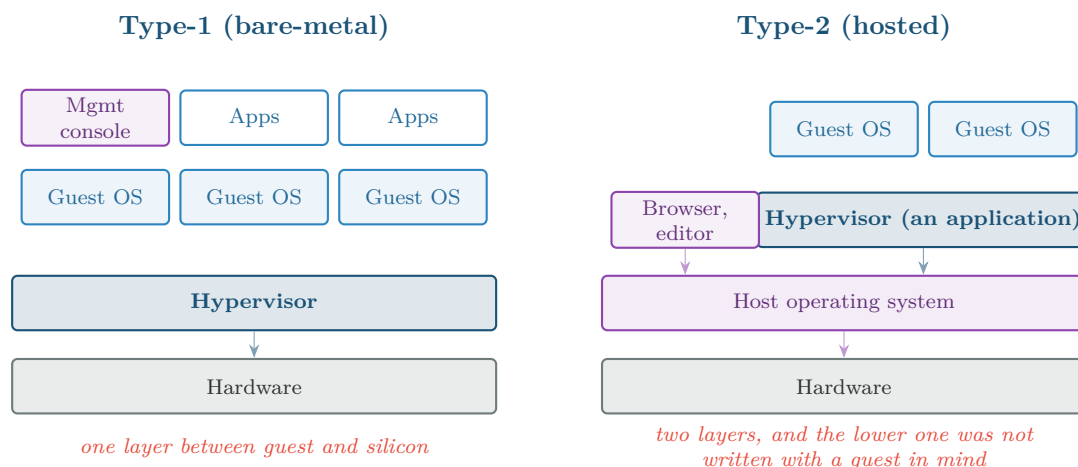


Figure 2.1: The only two places a hypervisor can stand. The right-hand stack is easier to install and slower at everything, because every device access the guest makes is translated twice.

💡 KVM Is the Case That Breaks the Taxonomy

Ask five administrators whether KVM is Type-1 or Type-2 and you will get both answers with equal confidence. Both are defensible, which tells you the question is poorly posed rather than hard.

KVM is a module loaded into the Linux kernel. Once loaded, Linux *is* the hypervisor: it schedules virtual CPUs with the same scheduler it uses for processes, and reaches the hardware with no layer beneath it. By the test above — does it need another operating system’s permission? — it is Type-1.

But Linux was a general-purpose operating system first and acquired this role second; it runs your shell and your web server on the same kernel. By the informal test most people actually apply — is this a dedicated hypervisor product? — it looks Type-2.

The resolution is to stop asking. The questions that have operational answers are: *how large is the trusted computing base?* (ESXi: roughly 150 MB; Linux with KVM: a kernel of several hundred megabytes and every driver in it), and *what else runs on this host?* Those two determine the security and the performance arguments, and the Roman numeral determines neither.

2.2 Three Ways to Keep the Promise

The promise of Chapter 1 was that an unmodified operating system runs above the layer. A guest kernel is written to execute privileged instructions — change the page tables, disable interrupts, talk to a disk controller — and the hypervisor cannot simply let it. There are three ways to deal with that, and the history of this field is largely the history of moving from the first to the third.

Technique	What happens to a privileged instruction	Guest changes	Cost
Full virtualization (binary translation)	The hypervisor scans the guest's kernel code before it runs and rewrites the dangerous instructions into safe calls into itself	None. The guest is unaware	Translation work, a cache of translated blocks, and great complexity in the hypervisor
Paravirtualization	The guest kernel is modified so that it never issues the instruction: it calls the hypervisor deliberately, through a hypercall	Source changes to the guest kernel	Fast and simple, but the guest must cooperate — which rules out an operating system you cannot recompile
Hardware-assisted	The processor itself traps the instruction and hands control to the hypervisor, with the guest's state saved automatically	None	A trap costs hundreds to thousands of cycles. Chapter 4 is how that cost was brought down

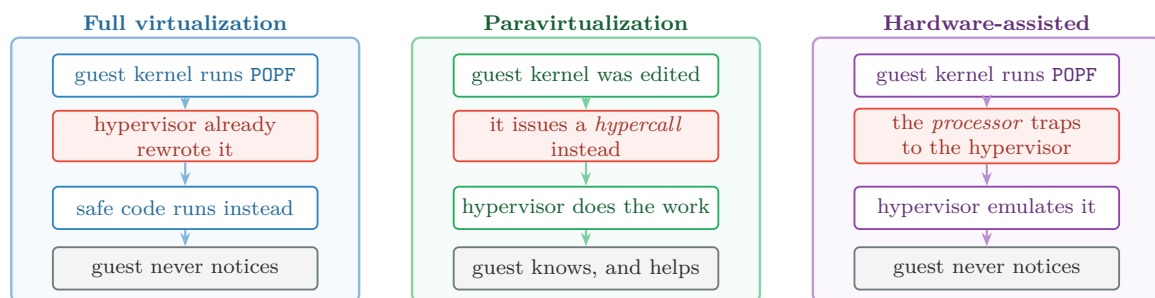


Figure 2.2: The same instruction, three architectures. Only the middle one requires the guest's cooperation; only the right-hand one requires the processor's. The red step is where the work is done, and it is a different party each time.

⚠ Nobody ships one of these alone any more

The table above is a taxonomy of techniques, not of products. Every current hypervisor is a hybrid.

A modern KVM guest uses *hardware assistance* for privileged instructions, *paravirtualized* drivers (virtio, Chapter 7) for disk and network because emulating real hardware is far slower, and occasionally *emulation* for a device nobody has optimised. Xen began as the paravirtualization project and has supported hardware-assisted guests since 2006. VMware built its reputation on binary translation and now uses it only where hardware assistance is unavailable or slower.

So the right question about a product is never “which one is it?” but “which one does it use *for this operation?*” — and the answer usually differs between the processor, the disk and the network card.

Worked Example: What the Layer Actually Costs

Four configurations, two workloads, measured on the same host. Compute the overhead of each configuration and explain the pattern.

Workload A — CPU-bound (compiling a kernel; lower is better):

- native 240 s • Type-1 245 s • Type-2 259 s • container 241 s

Workload B — I/O-bound (sequential read, MB/s; higher is better):

- native 980 • Type-1 with an *emulated* disk controller 185 • Type-1 with a *virtio* disk 940 • container 975

Step 1 — CPU overhead. Overhead is the extra time as a fraction of native:

$$\begin{aligned} \text{Type-1} &= \frac{245 - 240}{240} = \mathbf{2.1\%} & \text{Type-2} &= \frac{259 - 240}{240} = \mathbf{7.9\%} \\ \text{container} &= \frac{241 - 240}{240} = \mathbf{0.4\%} \end{aligned}$$

Step 2 — I/O overhead. Here express the loss as a fraction of native throughput:

$$\text{emulated} = 1 - \frac{185}{980} = \mathbf{81\% \text{ lost}} \quad \text{virtio} = 1 - \frac{940}{980} = \mathbf{4.1\% \text{ lost}}$$

Step 3 — read the pattern. The CPU numbers are small because a compile is almost entirely ordinary arithmetic, and ordinary arithmetic runs on the real processor at full speed in every configuration. The layer only costs you where it has to *intervene*.

The I/O numbers are enormous for exactly that reason. An emulated disk controller means the hypervisor pretends, in software, to be a specific piece of 1990s hardware: every register the guest driver writes is a trap, and a single disk read becomes dozens of them. Replacing it with virtio — paravirtualization, Chapter 7 — replaces those dozens of traps with one hypercall carrying a batch of requests, and 81% lost becomes 4%.

The lesson for a design. Virtualization overhead is not a property of the hypervisor; it is a property of how often the guest crosses the boundary. Measure the crossing rate, not the product.

2.3 Where Containers Sit

Everything so far virtualizes the machine. The alternative is to virtualize the operating system — to leave one kernel in place and give each group of processes a private view of it. Chapter 8 develops this properly; the purpose here is to put it on the same map, because the two are routinely presented as rivals when they are neighbours.

The Trade Is Isolation Against Density

A virtual machine carries its own kernel. That costs a few hundred megabytes of memory and tens of seconds of boot time, and buys an isolation boundary the hardware enforces: a guest that compromises its own kernel has still not escaped.

A container shares the host kernel. That costs almost nothing — megabytes, milliseconds — and buys an isolation boundary the *kernel* enforces: a process that finds a kernel bug is on the host, beside every other container. The Linux system-call interface is roughly 350 calls wide, and every one of them is attack surface.

Neither is better. A hosting provider running untrusted code from strangers wants the hardware boundary. A company running fifty of its own microservices wants the density.

	Full virtualization (binary translation)	Paravirtualization	Hardware-assisted	Namespaces and cgroups
Virtualize the <i>machine</i>	VMware Workstation (early), QEMU with TCG	Xen PV, virtio drivers, Hyper-V enlightenments	ESXi, Hyper-V, KVM, Xen HVM, Firecracker	not applicable
Virtualize the <i>operating system</i>	not applicable	not applicable	gVisor, Kata, Firecracker (both rows)	Docker, containerd, LXC, systemd-nspawn

The bottom-right cell is Part III. The bottom-middle cell is Chapter 11: designs that put a machine back underneath a container.

Figure 2.3: Two questions, not one: *which layer* is virtualized, and *by what technique*. Products are placed by what they principally do; most occupy more than one cell in practice.

Chapter 11 is what happened when people wanted both, and the answer turned out to be a very small virtual machine.

2.4 Choosing

If the requirement is	Choose	Because
A production server estate	Type-1, hardware-assisted	Lowest overhead, smallest trusted computing base, and the management and migration tooling only exists here
Running three operating systems on your laptop to learn	Type-2	Installs in minutes, coexists with everything else, and the 8% you lose does not matter
Untrusted code from strangers	Type-1, one guest per tenant	The isolation boundary is enforced by the processor, not by a 350-call kernel interface
Fifty of your own services, deployed often	Containers	Density, and seconds rather than minutes to start
Untrusted code <i>and</i> container density	Chapter 11 — microVMs	A kernel per tenant, with a 125 ms boot and 5 MB of overhead
An operating system you cannot modify (an appliance, a legacy system)	Anything but pure paravirtualization	Paravirtualization needs the guest's source
A different processor architecture	Emulation, not virtualization	The instruction set has to be translated, and that is a different and much slower job

! Common Pitfalls

- **Treating Type-1 and Type-2 as a performance ranking.** The difference in the worked example was six percentage points on CPU and nothing at all on properly-driven I/O. The real differences are the size of the trusted computing base, the management tooling, and whether the machine does anything else.
- **Saying “paravirtualization” when you mean “PV drivers”.** Almost every guest running today uses paravirtualized *drivers* while being a fully virtualized *guest*. The

kernel was not modified; a driver was installed. These are different claims and the second is much weaker.

- **Quoting a virtualization overhead figure without the workload.** “About 3%” is true of arithmetic and absurd of an emulated network card. Always state what was measured.
- **Forgetting that guest additions are not optional in practice.** A Type-2 guest without them has no shared clipboard, no resized display and a tenth of the I/O throughput. Students who conclude “virtual machines are slow” have usually just not installed them.
- **Assuming containers replaced virtual machines.** The dominant production pattern is containers *inside* virtual machines: every managed Kubernetes service runs its nodes as cloud instances, which are guests. The layers compose; they do not compete.

Try It Yourself: Three Architectures, One Workload

Measures the overhead of the same job under a Type-2 hypervisor, a Type-1 hypervisor and a container, on one laptop. Needs the stack from Appendix A. Expect thirty minutes, most of it waiting.

```
#!/usr/bin/env bash
# ch02_overhead.sh -- measure the cost of the layer, three ways.
set -euo pipefail
WORK='echo "scale=4000; 4*a(1)" | bc -l' # seconds of pure arithmetic

# The guest's address, however libvirt chose to hand it out today.
ip_of () {
    sudo virsh domifaddr "$1" | awk '/ipv4/{split($4,a,"/"); print a[1]}
}
in_guest () { ssh -o StrictHostKeyChecking=no "ubuntu@$(ip_of vss1)" "$1"; }

# --- 1. The baseline: no layer at all -----
echo "=== native ==="
time bash -c "$WORK" > /dev/null

# --- 2. Type-1: KVM, hardware-assisted -----
# vss1 is the guest built in Chapter 1's lab.
sudo virsh start vss1 || true
sleep 25
echo "=== type-1 (kvm) ==="
in_guest "time bash -c '$WORK' > /dev/null"

# --- 3. A container: no second kernel -----
echo "=== container ==="
time docker run --rm ubuntu:22.04 bash -c \
    "apt-get -qq update && apt-get -qq install -y bc && $WORK" > /dev/null

# --- 4. Now the part that actually matters: I/O -----
# Compare an emulated disk controller against a paravirtualized one.
# The guest must be stopped to change its disk bus.
sudo virsh destroy vss1
echo "=== emulated SATA controller ==="
sudo virt-xml vss1 --edit target=vda --disk bus=sata
sudo virsh start vss1 ; sleep 25
in_guest "sudo dd if=/dev/sda of=/dev/null bs=1M count=512 iflag=direct"

sudo virsh destroy vss1
echo "=== virtio (paravirtualized) ==="
sudo virt-xml vss1 --edit target=sda --disk bus=virtio
sudo virsh start vss1 ; sleep 25
in_guest "sudo dd if=/dev/vda of=/dev/null bs=1M count=512 iflag=direct"
```

```
# --- 5. Tear down -----
sudo virsh destroy vss1 || true
```

What to notice. The three CPU numbers will be within a few per cent of each other and the two disk numbers will not be within a factor of three. Write both pairs down: the worked example above claims a pattern, and this is you checking it on your own hardware rather than believing it.

? Checkpoint — Test Yourself

1. A hypervisor runs on a stripped Linux installation that exists only to boot it and offers no shell to users. Is it Type-1 or Type-2, and what is the more useful question to ask about it instead?
2. A guest completes a numerical job in 412 s where the host takes 400 s, and reads a disk at 210 MB/s where the host reads at 1,050 MB/s. Give both overheads, and say which single configuration change you would make first.
3. Why can paravirtualization not be used for a guest operating system whose vendor has gone out of business, and what is used instead?

✓ Chapter Summary

- Type-1 runs on the hardware; Type-2 runs on a host operating system. The useful questions behind the labels are the size of the trusted computing base and what else the machine does.
- KVM fits neither label cleanly, which is a fact about the labels.
- Three techniques handle a privileged instruction: binary translation rewrites it, paravirtualization replaces it with a hypercall the modified guest issues deliberately, and hardware assistance makes the processor trap it.
- No product uses one technique alone. Ask which one is used *for this operation*.
- Virtualization overhead is a property of how often the guest crosses the boundary, not of the hypervisor. A few per cent on arithmetic; 80% on an emulated device; 4% on the same device paravirtualized.
- Containers virtualize the operating system rather than the machine, trading a hardware-enforced isolation boundary for density. Neither is better, and in production they are usually stacked.

✍ Review Questions

1. (MCQ) The defining test for a Type-1 hypervisor is that: (a) no operating system is present on the host; (b) it does not need another operating system's permission to reach the hardware; (c) it has no graphical interface; (d) it is sold commercially.
2. (MCQ) A guest kernel is recompiled so that it calls the hypervisor instead of executing a privileged instruction. This is: (a) binary translation; (b) hardware-assisted virtualization; (c) paravirtualization; (d) emulation.
3. (Short) State what each of the three techniques requires of a party other than the hypervisor: the guest, the processor, or neither.
4. (Short) Explain why virtualization overhead on a compile is around 2% while overhead on an emulated network card can exceed 80%.
5. (Short) Give two concrete operational consequences of a hypervisor having a 150 MB trusted computing base rather than a 600 MB one.

6. **(Applied)** A hosting company will run customer-supplied code. It is considering (i) one container per customer on shared hosts, (ii) one virtual machine per customer, (iii) microVMs. Density, cost and isolation all matter. Argue for one, and state precisely what each of the other two would have to change for you to pick it instead.
7. **(Applied)** You inherit a Type-2 estate on which guests are reported as “unusably slow”. List, in the order you would check them, five things that could cause this, and say what measurement would confirm or eliminate each.

The Virtual Machine Monitor

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the three properties Popek and Goldberg require of a virtual machine monitor, and **explain** what each one rules out.
- **Distinguish** privileged from sensitive instructions, and **state** the theorem that connects them to virtualizability.
- **Explain** why x86 failed the theorem’s condition, using POPF as the worked case, and **say** precisely what goes wrong — which is not a crash.
- **Describe** binary translation and ring deprivileging as the two software answers, and **compare** what each costs.
- **Apply** the theorem to an architecture you are given a specification for, and decide whether trap-and-emulate alone would suffice.

🔗 Before You Start

Chapter 1 (the layer and what it promises) and Chapter 2 (the three techniques, named but not yet justified). You also need, from Operating Systems, what a privilege level is and what happens on a trap. If that is shaky, read the first page of Section 3.1 slowly; everything after it depends on the idea that the processor behaves differently according to who is asking.

🔑 Key Terms in This Chapter

virtual machine monitor • equivalence • resource control • efficiency • privileged instruction • sensitive instruction • control-sensitive • behaviour-sensitive • trap • trap-and-emulate • Popek–Goldberg theorem • silent failure • ring deprivileging • ring aliasing • binary translation • translation cache • hybrid virtualization

3.1 What a Monitor Must Guarantee

In 1974 Gerald Popek and Robert Goldberg asked a question that had not been asked precisely before: *what has to be true of a processor for it to be possible to write a correct virtual machine monitor for it?* Their answer is the only theorem in this book, and it explains the twenty-five-year gap in Figure 1.3.

They begin by saying what a monitor is *for*. Three properties, all three required.

📋 The Three Properties of a Virtual Machine Monitor

- **Equivalence.** A program running under the monitor behaves identically to the same program running directly on the hardware, except for timing and for resources that are genuinely absent. This is the “preserved interface” of Chapter 1, stated formally.
- **Resource control.** The monitor is in complete control of the resources it virtualizes. A guest cannot reach memory it was not given, cannot affect another guest, and cannot take control away from the monitor.

- **Efficiency.** A *statistically dominant* subset of the guest's instructions executes directly on the real processor, with no intervention.

💡 Efficiency Is the Clause That Does the Work

Drop the third property and the problem becomes trivial: write an interpreter. An emulator satisfies equivalence and resource control perfectly, because it never lets the guest touch the processor at all. It is also fifty times slower, which is why nobody consolidates a data centre with one.

So the entire difficulty of this chapter lives in the word *dominant*. The monitor must let the guest run on the real silicon almost all of the time, and must nevertheless remain in control — and those two demands pull in opposite directions. The rest of the chapter is about where exactly they tear.

Notice also what equivalence *excludes*: timing. A guest may legitimately observe that it is running slower, and a guest that reads the cycle counter can often tell it is virtualized. Popek and Goldberg were explicit that this is allowed. Software that uses timing to *detect* a hypervisor — malware avoiding analysis, licence enforcement — is exploiting a gap the theorem deliberately left open.

3.2 Privileged, Sensitive, and the Difference

The classical way to build a monitor is **trap-and-emulate**: run the guest at a reduced privilege level, let the hardware trap anything it may not do, and have the monitor emulate the trapped instruction in a way consistent with the guest's virtual machine.

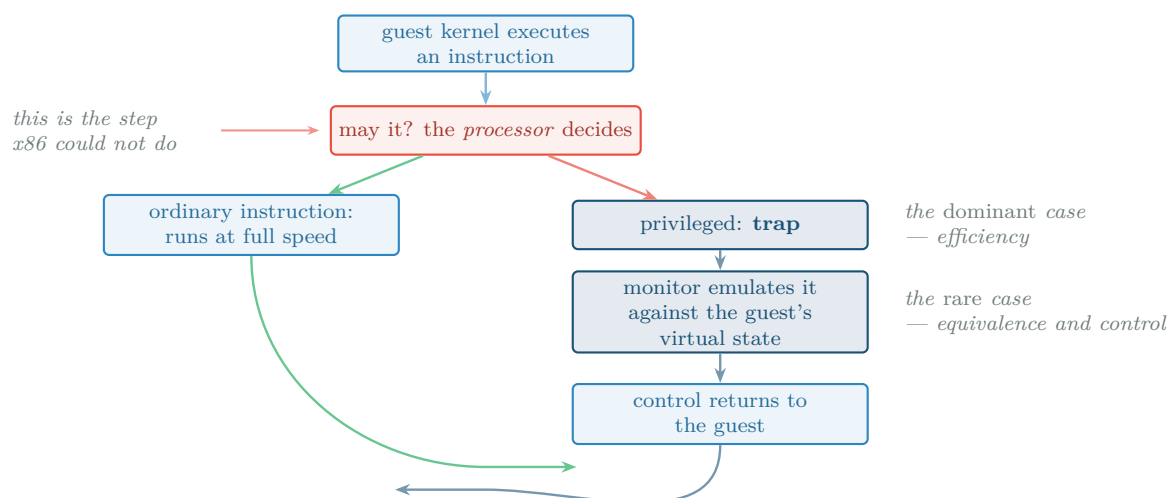


Figure 3.1: Trap-and-emulate. The design is sound and was old by 1974; the whole question is whether the processor traps everything the monitor needs it to trap.

For this to work, the processor must trap *every* instruction whose behaviour the monitor needs to control. Popek and Goldberg made that precise with two definitions.

📖 Privileged and Sensitive Instructions

An instruction is **privileged** if it traps when executed in user mode and does not trap in supervisor mode. This is a property of the *hardware*: the processor is built to refuse it.

An instruction is **sensitive** if it is one the monitor must know about. There are two kinds:

- **control-sensitive** — it changes the configuration of resources: the page tables, the interrupt mask, the privilege level.
- **behaviour-sensitive** — its *result* depends on the configuration: it reads a register whose value differs between privilege levels, or depends on where in memory it is running.

Sensitivity is a property of what the *monitor needs*; privilege is a property of what the *hardware does*. Nothing guarantees that they coincide.

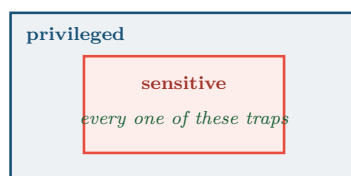
💡 The Theorem

Popek and Goldberg, 1974. For any conventional third-generation computer, a virtual machine monitor may be constructed if the set of sensitive instructions is a *subset* of the set of privileged instructions.

That is the whole condition. Every instruction the monitor needs to know about must be one the hardware will trap. If so, trap-and-emulate works: run the guest unprivileged, catch the sensitive instructions as traps, emulate them, and everything else runs at full speed — equivalence, control and efficiency together.

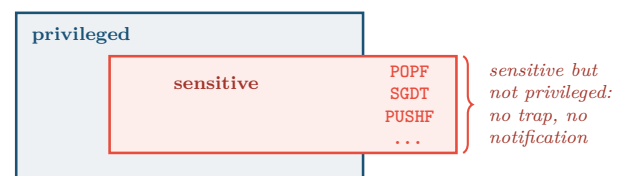
And the contrapositive is the interesting half. If even one instruction is sensitive but *not* privileged, the guest can execute it, the hardware will not trap it, and the monitor will never find out.

Virtualizable: sensitive $\subseteq$ privileged



trap-and-emulate alone is enough

x86 before 2005: 17 instructions outside



trap-and-emulate silently produces wrong answers

Figure 3.2: The condition, and the architecture that failed it. The seventeen instructions on the right are not exotic: several are ordinary, unprivileged instructions a compiler emits without comment.

3.3 Why x86 Failed

In 2000 John Robin and Cynthia Irvine went through the Pentium instruction set against Popek and Goldberg's condition and found **seventeen** instructions that are sensitive but not privileged. The IBM System/370 had satisfied the condition; x86, designed for a single operating system on a desktop, had never needed to.

The clearest of the seventeen is POPF.

📖 Worked Example: How POPF Breaks a Hypervisor Without Crashing It

POPF pops a word off the stack into the EFLAGS register. EFLAGS holds ordinary arithmetic flags — carry, zero, sign — and also the **interrupt flag** IF, which controls whether the processor accepts hardware interrupts.

What the guest kernel intends. It is entering a critical section and wants interrupts off:

- push a flags word with IF = 0;
- execute POPF;
- expect interrupts to be disabled.

Step 1 — is POPF sensitive? Yes, control-sensitive: it changes the interrupt configuration, which the monitor must control. If a guest could really disable interrupts, it could refuse to give the processor back, destroying resource control.

Step 2 — is POPF privileged? No. In user mode the x86 POPF does not fault. It updates the arithmetic flags and *silently ignores the write to IF*. The designers chose this deliberately in 1985, so that user programs manipulating arithmetic flags would not need to care about IF.

Step 3 — so what happens under a monitor? The guest kernel runs unprivileged. It executes POPF:

- no trap occurs — the monitor is never told;
- the arithmetic flags are updated, so the instruction appears to work;
- IF is unchanged, so interrupts remain *enabled*;
- the instruction completes and the guest proceeds, believing it is in a critical section.

Step 4 — what the failure looks like. Not a crash. An interrupt arrives during the critical section the guest believes it has protected. A data structure is modified halfway through an update. Perhaps nothing happens. Perhaps the guest’s scheduler corrupts its run queue, at a rate of once a fortnight, in a way that is not reproducible and cannot be traced to the instruction that caused it.

The point. A *crash* would be a gift — it is a signal. The seventeen instructions do something far worse: they make the equivalence property fail *quietly*. The guest cannot detect it, the monitor cannot detect it, and the resulting bug reports are unfalsifiable. This, and not performance, is why x86 was called non-virtualizable for twenty-five years.

⚠ “Non-virtualizable” never meant impossible

It meant that *trap-and-emulate alone* is not sufficient — that no correct monitor can be built the classical way. VMware shipped a correct x86 hypervisor in 1999 and Xen in 2003, so clearly monitors could be built. What the theorem denies is that they can be built cheaply and simply. Both products are, in effect, elaborate workarounds for seventeen instructions, and both workarounds cost something the next section quantifies.

3.4 Two Ways Round

3.4.1 Binary translation

VMware’s answer, and the one that founded the industry. The monitor never lets the guest kernel’s instructions reach the processor unexamined. Instead it reads the kernel’s code a basic block at a time, *translates* it — copying ordinary instructions unchanged and replacing each of the seventeen with a call into the monitor — and executes the translated block. Translations are cached, so a hot loop is translated once and then runs many times.

Guest *user* code is not translated at all. It runs directly, at full speed, because an application executing POPF and losing the write to IF is exactly what would happen on real hardware: equivalence is preserved for free.

💡 Why Translation Was Fast Enough to Sell

It sounds ruinous and it was not, for two reasons.

First, the ratio. A kernel is a small fraction of what runs on a busy machine, and the translation cost is paid once per block rather than once per execution.

Second — and this surprised people — a translator can be *faster* than the hardware trap it replaces. A trap on 2005-era x86 cost well over a thousand cycles of saving and restoring state. A translated call into the monitor cost tens. For several years after hardware assistance arrived, VMware’s translator beat the first generation of VT-x on system-call-heavy workloads, and VMware said so publicly. Chapter 4 is the story of Intel and AMD closing that gap.

3.4.2 Ring deprivileging and paravirtualization

Xen’s answer. x86 provides four privilege levels — rings 0 to 3 — of which operating systems historically used only two. Xen put the hypervisor in ring 0, moved the guest kernel to ring 1, and left applications in ring 3.

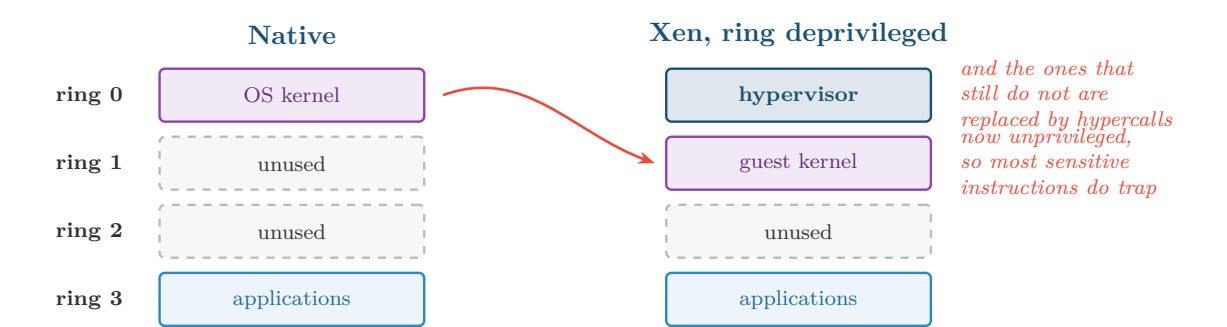


Figure 3.3: Ring deprivileging. Moving the guest kernel down one ring makes the hardware trap most of what the monitor needs; the instructions that still slip through are removed from the guest’s source instead.

Deprivileging alone does not solve the problem — POPF still fails silently in ring 1 — so Xen took the second step: it *modified the guest kernel*. Where Linux would have executed one of the seventeen, the Xen port issues a **hypercall** instead. That is paravirtualization, and the price is exactly the one named in Chapter 2: you need the guest’s source. Xen ran Linux and NetBSD from the first release and could not run Windows at all.

⚠ Ring aliasing: the guest can tell

Deprivileging breaks equivalence in a second, subtler way. The unprivileged instruction `PUSHF` pushes `EFLAGS` — including the current privilege level — onto the stack, and `SGDT` and `SIDT` read descriptor-table registers. None of them traps. A guest kernel that executes any of them sees ring 1 where it expects ring 0, and a descriptor table that is not its own. This is **ring aliasing**, and it is why `SGDT` is on the list of seventeen. It also makes hypervisor detection trivial: a few unprivileged instructions and the guest knows. Chapter 21 is where that question comes back inverted — there, the guest *wants* to prove what it is running on.

	Binary translation	Paravirtualization	Hardware assistance
Who changes	The monitor, at run time	The guest, at compile time	The processor, at design time
Unmodified guests	Yes	No	Yes
Where the cost falls	Translating, and a cache to maintain	Nothing at run time; everything at porting time	A trap per sensitive instruction

	Binary translation	Paravirtualization	Hardware assistance
Monitor complexity	Very high — a working x86 translator	Low	Low
Shipped	VMware, 1999	Xen, 2003	Intel VT-x 2005, AMD-V 2006
Still used	Where hardware assistance is absent or slower	As <i>drivers</i> , almost universally (Chapter 7)	Everywhere (Chapter 4)

! Common Pitfalls

- **Saying that non-virtualizable means “cannot be virtualized”.** It means trap-and-emulate alone is insufficient. Two companies built correct monitors anyway, at considerable cost.
- **Expecting the failure to be a crash.** The POPF failure is silent and intermittent. Students who look for an exception in the worked example have missed what makes those seventeen instructions dangerous.
- **Confusing privileged with sensitive.** Privileged is what the hardware traps; sensitive is what the monitor needs. The theorem is precisely the claim that the first set must contain the second.
- **Believing hardware assistance made the theorem obsolete.** It made x86 satisfy the condition. The theorem is what tells you *why* VT-x was needed and what it had to do — and it still decides the question for any new architecture, which is why RISC-V designers argued about exactly this.
- **Assuming efficiency means “fast”.** It is a formal property: a statistically dominant subset of instructions runs natively. An emulator fails it at any speed.

🔧 Try It Yourself: Catching a Sensitive Instruction in the Act

No hypervisor needed: this runs on the host and shows the hardware behaviour the whole chapter rests on. Needs gcc. Expect twenty minutes.

```
/* ch03_popf.c -- show that POPF fails silently in user mode.
   Build: gcc -O0 -o ch03_popf ch03_popf.c      Run: ./ch03_popf */
#include <stdio.h>

/* --- 1. Read EFLAGS, so we can see what the processor really did. */
static unsigned long read_flags(void) {
    unsigned long f;
    __asm__ volatile ("pushfq ; popq %0" : "=r"(f));
    return f;
}

/* --- 2. Try to write EFLAGS, clearing the interrupt flag (bit 9). */
static void write_flags(unsigned long f) {
    __asm__ volatile ("pushq %0 ; popfq" : : "r"(f) : "cc");
}

int main(void) {
    unsigned long before = read_flags();
    printf("before : EFLAGS=0x%08lx  IF=%lu\n", before, (before >> 9) & 1);

    /* --- 3. Ask for interrupts off, and for the carry flag on.
```

```

    A kernel entering a critical section does exactly this. */
    unsigned long want = (before & ~(1UL << 9)) | 1UL;    /* IF=0, CF=1 */
    write_flags(want);

    /* --- 4. Look at what we actually got. */
    unsigned long after = read_flags();
    printf("wanted : EFLAGS=0x%08lx  IF=%lu  CF=%lu\n",
        want, (want >> 9) & 1, want & 1);
    printf("after  : EFLAGS=0x%08lx  IF=%lu  CF=%lu\n",
        after, (after >> 9) & 1, after & 1);

    /* --- 5. The verdict. No signal was raised; nothing faulted. */
    printf("\ncarry flag taken?      %s\n",
        (after & 1) ? "yes -- the instruction 'worked'" : "no");
    printf("interrupt flag taken? %s\n",
        (((after >> 9) & 1) == 0) ? "yes" :
        "NO -- written, ignored, and no trap. This is the bug.");
    return 0;
}

```

What to notice. The program does not crash and reports no error. The carry flag changed, so by every signal available to the program the instruction succeeded — and the interrupt flag did not change. Now reread Step 4 of the worked example: a guest kernel in this position believes it holds a lock it does not hold.

🔍 Checkpoint — Test Yourself

1. An architecture has an instruction `RDCFG` that returns the current privilege level and does not fault in user mode. Classify it using the two definitions, and say whether this architecture satisfies the Popek–Goldberg condition.
2. Why would a *crash* on executing `POPF` in a guest have been a better outcome for hypervisor authors than what x86 actually does?
3. Binary translation needs no cooperation from the guest, and paravirtualization needs the guest's source code. Give one situation where the second is nevertheless the better engineering choice, and say why.

📋 Chapter Summary

- A virtual machine monitor must provide equivalence, resource control and efficiency. Efficiency — a statistically dominant subset running natively — is what rules out an emulator and makes the problem hard.
- Privileged means the hardware traps it; sensitive means the monitor needs to know about it. They are different properties of different things.
- Popek and Goldberg: a monitor can be built by trap-and-emulate if the sensitive instructions are a subset of the privileged ones.
- x86 had seventeen instructions that are sensitive and not privileged. `POPF` silently discards a write to the interrupt flag in user mode, so the guest believes it disabled interrupts and did not.
- The failure is silent, intermittent and untraceable — which is worse than a crash, and is why x86 was called non-virtualizable.
- Binary translation rewrites the guest's kernel code at run time, needs no guest changes, and was for some years faster than the first hardware assistance. Ring deprivileging plus paravirtualization is simpler and needs the guest's source.
- The theorem is not obsolete. It is what Chapter 4 is an answer to, and it still decides the question for any new architecture.

 Review Questions

1. **(MCQ)** Popek and Goldberg’s efficiency property requires that: (a) the monitor adds less than 5% overhead; (b) a statistically dominant subset of guest instructions executes directly on the processor; (c) the guest boots in under a minute; (d) no instruction is ever trapped.
2. **(MCQ)** An instruction that is sensitive but not privileged causes: (a) a general protection fault; (b) the guest to crash; (c) the monitor to lose control or correctness with no notification; (d) a performance penalty only.
3. **(Short)** State the Popek–Goldberg condition in one sentence, and explain what each of the two instruction sets is a property of.
4. **(Short)** Describe what POPF does in user mode on x86 and why that single design decision cost the industry twenty-five years.
5. **(Short)** Explain how binary translation could outperform hardware-assisted virtualization on a system-call-heavy workload in 2006, even though the hardware was doing the same job.
6. **(Applied)** You are reviewing the specification of a new 64-bit architecture. Write the procedure you would follow to decide whether it can be virtualized by trap-and-emulate alone, say what evidence you would need from the vendor, and state what you would recommend if you found three offending instructions.
7. **(Applied)** A colleague proposes to run an unmodified legacy operating system, whose vendor no longer exists, on current hardware. Using this chapter and Chapter 2, set out which architectures are available to them, which are ruled out and why, and what you would check first.

PART II

Hypervisors and Compute Virtualization

Part I ended with a problem: the architecture could not be virtualized, and the software workarounds were slow and fragile. Part II is the answer, in the order the industry found it — first in silicon, then in four platforms built on that silicon, then in the two resources every guest competes for, and finally in the devices the guest thinks it owns.

the answer to Chapter 3

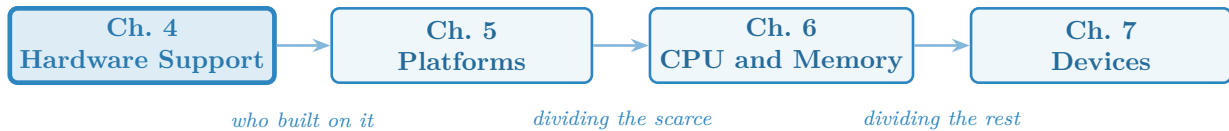


Figure P2.1: Reading path through Part II. Chapter 4 is the silicon; the three that follow spend it — on platforms, on the processor and memory, and on everything plugged into the bus.

Hardware Support for Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** how VMX root and non-root operation make x86 satisfy the Popek–Goldberg condition without changing a single instruction’s encoding.
- **Describe** what the VMCS holds and **account** for the cost of a VM exit in terms of the work the processor must do.
- **Count** the memory accesses in a nested page walk, and **explain** why two-dimensional paging replaced shadow page tables despite being slower on a TLB miss.
- **State** what an IOMMU does and why device passthrough is unsafe without one.
- **Diagnose** a performance problem as an exit-rate problem from measured counters, and **name** the feature that addresses it.

🔗 Before You Start

Chapter 3 above all — the Popek–Goldberg condition, the seventeen offending instructions, and why trap-and-emulate could not be used. This chapter is the hardware answer to that chapter’s problem, and almost nothing in it makes sense without it. You also need virtual memory and page tables from Operating Systems.

📖 Key Terms in This Chapter

VT-x • AMD-V • SVM • VMX root operation • VMX non-root operation • VMCS • VMCB • VM entry • VM exit • exit reason • world switch • shadow page table • extended page tables • nested page tables • two-dimensional paging • nested page walk • VPID • IOMMU • VT-d • DMA remapping • interrupt remapping

4.1 What Intel and AMD Had to Fix

Chapter 3 left the architecture in an awkward position. Seventeen instructions were sensitive but not privileged, so trap-and-emulate was unsound; the two workarounds — binary translation and paravirtualization — each cost something substantial. Intel shipped VT-x in 2005 and AMD shipped AMD-V in 2006, and both took the same approach.

The obvious fix would have been to make the seventeen instructions privileged. Both vendors rejected it, for a reason worth understanding: it would break existing software. A user program that executes `POPF` today and has its write to `IF` quietly ignored would begin faulting, and backward compatibility is the entire commercial basis of the x86 architecture.

So instead of changing what the instructions do, they changed *the mode the processor can be in*.

📖 VMX Root and Non-Root Operation

VT-x adds a new dimension to the privilege model. Orthogonally to rings 0–3, the processor is in one of two **operations**:

- **VMX root operation** — what the hypervisor runs in. Behaves like an ordinary processor, plus a set of new instructions (VMXON, VMLAUNCH, VMRESUME, VMREAD, VMWRITE) for managing guests.
- **VMX non-root operation** — what a guest runs in. All four rings are available, so a guest kernel runs in **ring 0** exactly as it expects. But certain instructions and events cause a **VM exit**: control transfers to the hypervisor in root operation.

The guest kernel is no longer deprived. It runs in ring 0, reads its own privilege level as 0, and sees the descriptor tables it installed. Ring aliasing is gone, the seventeen instructions trap, and equivalence holds — without a single instruction changing its meaning.

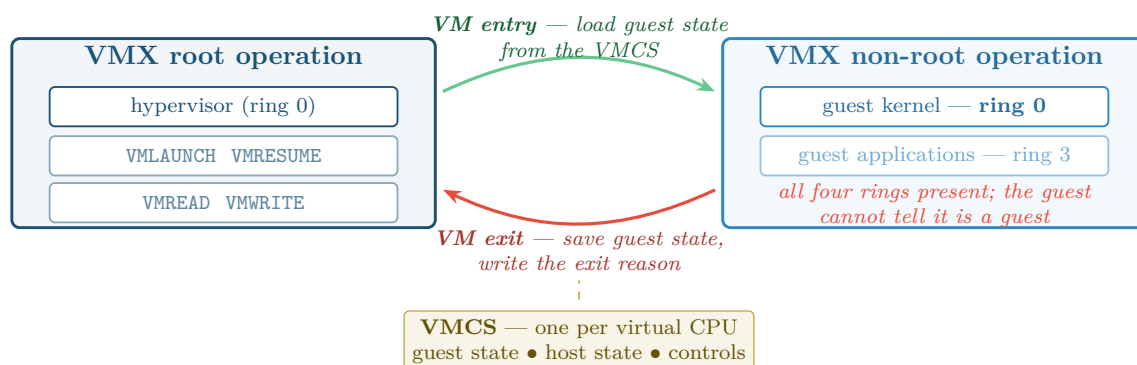


Figure 4.1: VT-x in one picture. The guest runs in ring 0 of a second operating mode, so nothing it executes behaves unexpectedly; the hypervisor regains control through a hardware transition rather than a trap it had to arrange.

💡 AMD Did the Same Thing with Different Names

AMD-V (originally *Pacifica*, architecturally **SVM**) solves the same problem with the same structure and an incompatible vocabulary. Guest mode corresponds to non-root operation; the **VMCB** (virtual machine control block) corresponds to the VMCS; **VMRUN** corresponds to VMLAUNCH; **nested page tables** correspond to *extended* page tables.

The differences that matter in practice are small but real: the VMCB is a plain 4 KB structure a hypervisor reads and writes with ordinary memory instructions, whereas the VMCS is opaque and must be accessed through VMREAD and VMWRITE, which cost cycles. AMD's design is simpler to program; Intel's leaves the processor free to cache VMCS fields in whatever internal format it likes.

This book uses the Intel names throughout and notes the AMD equivalent where it differs. Every hypervisor in Chapter 5 supports both, through an abstraction layer written once.

4.2 The VMCS, and What a Transition Costs

A **VMCS** — virtual machine control structure — exists per virtual CPU and holds everything needed to switch between the two worlds.

VMCS area	What it holds
Guest-state area	The guest's registers, control registers, segment registers, RIP and RSP. Saved automatically on exit, loaded automatically on entry

VMCS area	What it holds
Host-state area	The hypervisor's corresponding state, loaded on exit. This is what makes the return to the hypervisor a single hardware operation rather than a software convention
VM-execution controls	<i>Which events cause an exit.</i> A bitmap per instruction class, an exception bitmap, an I/O-port bitmap and an MSR bitmap. This is the dial the hypervisor turns to trade control against speed
VM-exit information	Written by the processor on every exit: the exit reason , the faulting address, the instruction length. The hypervisor reads this first and dispatches on it

💡 The Execution Controls Are the Real Design Surface

A novice reading of VT-x is that the processor decides what to trap. It does not: the *hypervisor* decides, by setting bits in the execution controls, and the engineering is in choosing well.

Exit on every CPUID? You must, because the guest must not see the host's true feature set. Exit on every RDTSC? Only if you care about hiding the passage of time, and most hypervisors do not, because the exit cost would be ruinous. Exit on every write to CR3? With shadow page tables you must; with EPT you need not, and that single bit is most of the next section.

Every bit set buys correctness or isolation and costs an exit. Hypervisor performance work is, to a first approximation, the work of clearing bits that turn out not to be needed.

⚠️ A VM exit is a world switch, not a function call

On exit the processor must save the guest's entire architectural state, load the hypervisor's, and flush or tag anything that was speculating on the guest's address space. On the first VT-x parts this cost on the order of 1,500 cycles in each direction; a decade of optimisation brought it to a few hundred, and it has not fallen much since because the work is irreducible. That number is why Chapter 2 could report that binary translation beat early hardware assistance: a translated call into the monitor cost tens of cycles and did not disturb the pipeline. Hardware assistance won in the end not by making exits cheap but by *removing the reasons to exit* — which is what extended page tables and the MSR bitmap are for.

4.3 Two-Dimensional Paging

Virtualizing the processor was half the problem. The other half is memory, and it is harder.

A guest kernel maintains page tables mapping *guest virtual* addresses to what it believes are physical addresses. They are not physical: they are **guest physical** addresses, which the hypervisor must in turn map to **host physical** addresses. Two translations, and the hardware originally performed only one.

4.3.1 The first answer: shadow page tables

The hypervisor builds its own page tables mapping guest virtual directly to host physical — **shadow page tables** — and points the real CR3 at those. The processor walks them normally, so a TLB miss costs exactly what it costs natively.

The price is keeping them consistent. The hypervisor must trap every guest write to its own page tables, which means write-protecting them and taking a VM exit on each modification. A process fork, which rewrites a great many page table entries, becomes thousands of exits.

4.3.2 The second answer: let the hardware do both

Extended page tables (Intel) and **nested page tables** (AMD) add a second, hardware-walked translation. The guest's own page tables stay in charge of guest-virtual to guest-physical, and the guest may edit them freely with no exits at all. A second set of tables, owned by the hypervisor, maps guest-physical to host-physical, and the processor walks both.

Worked Example: Counting a Nested Page Walk

How much does the hardware actually do on a TLB miss under EPT? Count memory accesses for 4-level paging on x86-64.

Native, for comparison. Four levels — PML4, PDPT, PD, PT — so a TLB miss costs **4 memory accesses**.

Under EPT. Every address the walker touches is a *guest physical* address, and each one must itself be translated by a 4-level EPT walk before it can be used.

- Guest CR3 holds the GPA of the guest PML4. Translate it: **4** EPT accesses. Then read the guest PML4 entry: **1**.
- That entry holds the GPA of the PDPT. Translate it: **4**. Read the PDPT entry: **1**.
- GPA of the PD. Translate: **4**. Read the PD entry: **1**.
- GPA of the PT. Translate: **4**. Read the PT entry: **1**.
- The guest PTE yields the final GPA of the data page. Translate it: **4**.

The total.

$$\underbrace{4 \times (4 + 1)}_{\text{four guest levels}} + \underbrace{4}_{\text{final translation}} = \mathbf{24} \text{ memory accesses}$$

So EPT is six times worse than native on a TLB miss, and worse than shadow paging, which costs 4. If that were the whole story, shadow page tables would have won.

Why EPT won anyway. Count the *other* operation. Under shadow paging every guest page-table write is a VM exit at several hundred cycles; under EPT it is an ordinary memory write at a few cycles. A workload that creates processes, maps files or frees memory does this constantly.

Measured on a fork-heavy build: shadow paging took tens of thousands of exits per second; EPT took almost none. The 24-access walk is also mitigated in hardware by caching the intermediate EPT translations — a page-walk cache — so the full 24 is the worst case rather than the common one.

The lesson. The right structure is not the one with the better best case; it is the one whose expensive case is rare. Shadow paging is cheaper per TLB miss and catastrophic per page-table edit. EPT is the reverse, and page-table edits are far more common than the arithmetic suggests.

VPID: Not Flushing the TLB Every Time

One more cost was hiding in the transitions. Early VT-x flushed the entire TLB on every VM entry and exit, because the hypervisor and the guest use different address spaces and the processor had no way to tell their entries apart. A busy guest therefore began every re-entry with a cold TLB.

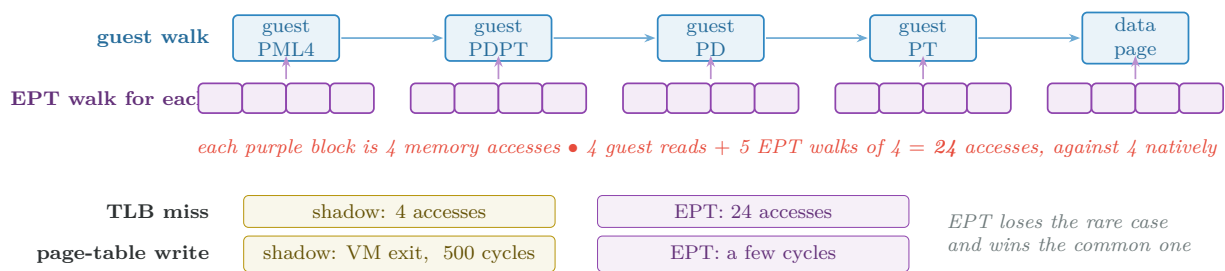


Figure 4.2: A nested page walk, and the trade it represents. Shadow paging is cheaper on a TLB miss and ruinous on every guest page-table edit; extended page tables invert both, and edits are the more frequent event.

VPID — virtual processor identifier, AMD’s equivalent is the ASID — tags each TLB entry with the virtual CPU it belongs to. Entries survive the transition and are simply ignored while another context runs. It is a small feature with a large effect, and it is the clearest illustration of the pattern in this chapter: the exits were not made cheaper, the work done around them was removed.

4.4 The IOMMU

The processor is virtualized and memory is virtualized. Devices are not, and devices are dangerous, because a device performing DMA writes to *physical* addresses and knows nothing about any of this.

Hand a network card directly to a guest — which Chapter 7 will want to do for speed — and the guest’s driver programs it with addresses from its own guest-physical space. The card would write wherever those happen to land in real memory: another guest, or the hypervisor. A single passthrough device without protection destroys the resource-control property for the entire host.

IOMMU, DMA Remapping, Interrupt Remapping

An **IOMMU** — Intel calls the feature **VT-d**, AMD calls it AMD-Vi — is a memory management unit that sits between devices and memory, as the MMU sits between the processor and memory.

DMA remapping: every address a device emits is translated through per-device page tables that the hypervisor controls. A device assigned to a guest can reach that guest’s memory and nothing else. An address outside the mapping faults rather than being written.

Interrupt remapping: a device cannot raise an arbitrary interrupt vector on an arbitrary core. Interrupts are translated too, and delivered only where the hypervisor allows.

Without an IOMMU, device passthrough is not merely risky; it is equivalent to giving the guest write access to all of physical memory.

⚠ Nested virtualization, and why your lab needs it

Running a hypervisor *inside* a guest — nested virtualization — means the processor must present VMX itself as a virtualizable feature. The outer hypervisor intercepts the inner one’s **VMLAUNCH**, merges the inner VMCS with its own, and runs the result. It works, and it is how the labs in this book run a Type-1 hypervisor on a machine that is already a guest.

It is also slow: every exit the inner guest takes may become several exits in the outer one. Expect a nested guest to be noticeably slower than a single-level one, and do not use nesting to measure anything.

On a laptop it must usually be enabled explicitly. Appendix A gives the commands; the lab below checks it first, because every later chapter’s lab assumes it.

! Common Pitfalls

- **Believing VT-x made virtualization fast.** It made it *correct* — the architecture now satisfies Popek–Goldberg. The first generation was often slower than binary translation. Speed came from EPT, VPID and the bitmaps, over roughly a decade.
- **Assuming EPT is faster at everything.** It is six times slower than native on a TLB miss and six times slower than shadow paging. It wins on the workload, not on the microbenchmark, and a workload with a huge working set and few page-table edits is exactly where shadow paging would still win.
- **Confusing the MMU with the IOMMU.** The MMU protects memory from the processor; the IOMMU protects memory from *devices*. A host with VT-x but no VT-d can virtualize fine and cannot safely pass a device through.
- **Leaving virtualization disabled in firmware and blaming the software.** A large share of “KVM will not start” is a cleared BIOS bit. Check `/proc/cpuinfo` first — the lab’s step 1 is there for this reason.
- **Benchmarking inside a nested guest.** Every measurement is inflated by the outer hypervisor’s exits, and by a variable amount. Nesting is for learning, not for numbers.

🔧 Try It Yourself: Watching the Hardware Work

Counts real VM exits on a real guest and shows what changes them. Needs KVM and a Linux host; `perf` comes from `linux-tools`. Expect thirty minutes.

```
#!/usr/bin/env bash
# ch04_exits.sh -- see VMX, EPT and the exit rate with your own eyes.
set -euo pipefail

# --- 1. Does this processor have the features this chapter describes? ---
grep -o -m1 -E '\<(vmx|svm)\>' /proc/cpuinfo # VT-x or AMD-V at all
grep -o -m1 -E '\<ept\>|\<npt\>' /proc/cpuinfo # two-dimensional paging
grep -o -m1 '\<vpid\>' /proc/cpuinfo # tagged TLB entries
# VT-d / AMD-Vi shows up as IOMMU groups once enabled in firmware:
ls /sys/kernel/iommu_groups 2>/dev/null | wc -l

# --- 2. Is nested virtualization on? Every later lab wants it. -----
cat /sys/module/kvm_intel/parameters/nested 2>/dev/null \
|| cat /sys/module/kvm_amd/parameters/nested 2>/dev/null
# If that prints N or 0, see Appendix A for how to turn it on.

# --- 3. Start the guest from Chapter 1 and find its thread. -----
sudo virsh start vss1 || true
sleep 25
PID=$(pgrep -f 'qemu.*vss1' | head -1)
echo "guest qemu pid: $PID"

# --- 4. Count exits by reason, for ten seconds of an idle guest. -----
# Every line is a different entry in the VM-execution controls.
sudo perf stat -e 'kvm:kvm_exit' -p "$PID" -- sleep 10
sudo perf record -e 'kvm:kvm_exit' -p "$PID" -- sleep 10 2>/dev/null
sudo perf script \
| awk '{for(i=1;i<=NF;i++) if($i ~ /reason/) print $(i+1)}' \
| sort | uniq -c | sort -rn | head -12

# --- 5. Now give the guest work that forces exits, and compare. -----
# Network I/O through an emulated device is an exit per register access.
```

```

in_guest () { ssh -o StrictHostKeyChecking=no "ubuntu@$1" "$2"; }
IP=$(sudo virsh domifaddr vss1 | awk '/ipv4/{split($4,a,"/"); print a[1]}')
in_guest "$IP" "ping -f -c 2000 10.0.2.2 >/dev/null 2>&1" &
sudo perf stat -e 'kvm:kvm_exit' -p "$PID" -- sleep 10
wait

# --- 6. Watch EPT doing its job: no exit for a guest page-table edit. --
# Fork a few thousand processes; under shadow paging this storms the host.
in_guest "$IP" "for i in $(seq 2000); do /bin/true; done"
sudo perf stat -e 'kvm:kvm_exit' -p "$PID" -- sleep 5

# --- 7. Tear down -----
sudo virsh destroy vss1 || true

```

What to notice. The idle guest still exits, a few thousand times a second, almost all of it timer and interrupt work. The flood ping multiplies that by a large factor, and the two thousand forks barely move it — which is the worked example's claim, measured. Write the three numbers down.

🔍 Checkpoint — Test Yourself

1. Why did Intel and AMD choose to add a new processor mode rather than simply making the seventeen offending instructions privileged?
2. Under 5-level paging, a guest page walk has five levels and so does the EPT walk. How many memory accesses does a nested walk cost, and what is the general formula for an n -level scheme?
3. A host has VT-x and EPT but the IOMMU is disabled in firmware. State exactly which capability is lost, and why the hypervisor must refuse it rather than merely warn.

📋 Chapter Summary

- VT-x and AMD-V add a second operating mode rather than changing any instruction, so the guest kernel runs in ring 0 and backward compatibility is preserved. x86 now satisfies the Popek–Goldberg condition.
- The VMCS holds guest state, host state, the controls that decide what causes an exit, and the exit information. The controls are the design surface: every bit set buys control and costs exits.
- A VM exit is a world switch — around 1,500 cycles originally, a few hundred now. Hardware assistance won by removing reasons to exit, not by making exits cheap.
- Shadow page tables cost 4 accesses per TLB miss and a VM exit per guest page-table write. Extended page tables cost 24 accesses per miss and nothing per write. Writes are commoner, so EPT won.
- VPID tags TLB entries so they survive a transition.
- An IOMMU translates and confines device DMA. Without one, passthrough gives a guest write access to all physical memory.
- Nested virtualization works and is slow. Use it to learn, never to measure.

🔧 Review Questions

1. (MCQ) Under VT-x a guest kernel runs in: (a) ring 1 of root operation; (b) ring 0 of non-root operation; (c) ring 3 of non-root operation; (d) ring 0 of root operation.
2. (MCQ) Extended page tables principally reduce: (a) the cost of a TLB miss; (b) the number of VM exits caused by guest memory management; (c) the size of the VMCS; (d) interrupt latency.

3. **(Short)** Explain what the VM-execution controls are for, and give one example of a bit you would set and one you would clear, with the reason for each.
4. **(Short)** Derive the 24-access figure for a nested page walk, showing where each group of accesses comes from.
5. **(Short)** State what an IOMMU protects against, and describe the concrete failure that occurs without one.
6. **(Applied)** A guest is slow and `perf` shows 180,000 VM exits per second, 80% of them with an I/O-instruction exit reason. Explain what the guest is most likely doing, name the change you would make, and predict what the exit count would become and why.
7. **(Applied)** You are asked whether a six-year-old server is suitable as a teaching host for this course. List the processor and firmware features you would check, in order, and say which lab in this book each one is needed for.

Major Hypervisor Platforms

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the architecture of ESXi, Hyper-V, KVM and Xen, and **say** for each one where the device drivers live.
- **Explain** why the location of the drivers is the single fact that most determines a platform's security and performance character.
- **Compare** the four on licensing, ecosystem, guest support and operational tooling.
- **Compute** the three-year cost of a small cluster on two of them, and **identify** which cost dominates.
- **Recommend** a platform for a stated requirement and defend it against the strongest alternative.

🔗 Before You Start

Chapter 2 (Type-1 and Type-2, and the three techniques) and Chapter 4 (VT-x, EPT, the IOMMU). Every platform here is a different set of engineering answers built on the same silicon, so the hardware chapter is what lets you see what they actually differ about.

📖 Key Terms in This Chapter

ESXi • VMkernel • vSphere • vCenter • Hyper-V • parent partition • child partition • VMBus • enlightenment • KVM • QEMU • libvirt • Xen • dom0 • domU • PV • HVM • PVH • disaggregation • driver domain • trusted computing base

5.1 The Question That Separates Them

All four platforms in this chapter run on the same processors, use VT-x or AMD-V the same way, and use extended page tables for memory. At the level of Chapter 4 they are nearly identical.

What separates them is a question Chapter 4 did not ask: *where do the device drivers live?* A hypervisor needs drivers for every disk controller and network card in the machine, and drivers are large, written by hardware vendors, and the commonest source of kernel bugs. Any code that can crash the host is part of the **trusted computing base**, and a driver can always crash the host. The four platforms give four different answers, and almost every other difference between them follows.

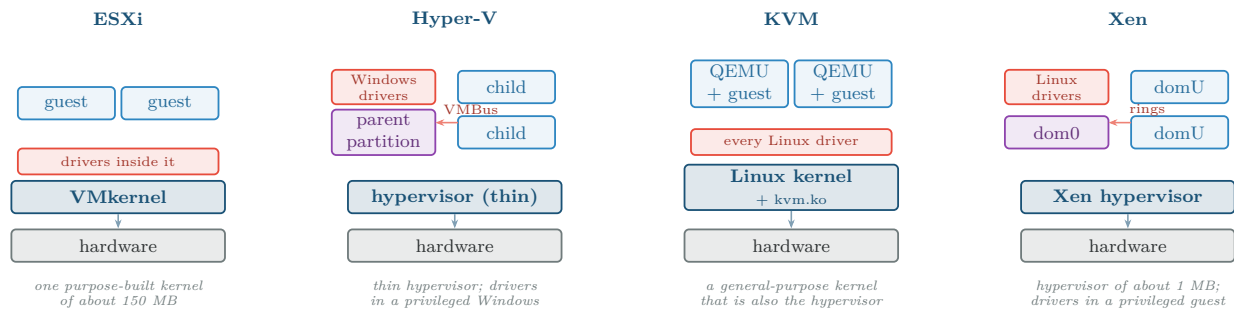


Figure 5.1: The four platforms, drawn to the same question: where are the drivers (red)? Xen’s answer gives it the smallest hypervisor in the room and the most moving parts; ESXi’s gives it the most predictable behaviour and the shortest hardware compatibility list.

5.2 VMware ESXi and vSphere

ESXi is a purpose-built Type-1 hypervisor. Its kernel, the **VMkernel**, was written for one job and contains the scheduler, the memory manager and the device drivers. Nothing general-purpose runs beside it; the shell is deliberately limited and disabled by default.

vSphere is the product that matters commercially: ESXi hosts plus **vCenter**, the management server that turns a set of hosts into a cluster with shared storage, automated placement, live migration and failover. Almost everything an enterprise buys VMware for lives in vCenter rather than in ESXi.

💡 The Compatibility List Is the Point, Not a Limitation

ESXi runs on a short list of certified hardware, and students usually read this as a weakness. It is the central design decision.

Because VMware controls which drivers exist, it can test them, and because the VMkernel does nothing else, its behaviour under load is predictable in a way a general-purpose kernel’s is not. An estate of certified hardware running a tested driver set is why vSphere acquired a reputation for not surprising people at three in the morning — and that reputation, rather than any technical feature, is what the licence buys.

The cost is real: a host that is not on the list may not install at all, and a new network card may wait months for a driver.

5.3 Microsoft Hyper-V

Hyper-V is also Type-1, and the picture most people hold of it is wrong. Installing the Hyper-V role on Windows Server does not make Windows the hypervisor; it slides a thin hypervisor *underneath* the running Windows, which is then demoted to the **parent partition** — a privileged guest.

The parent partition owns the physical devices and runs the Windows driver stack. **Child partitions** — the ordinary guests — have no direct device access and talk to the parent over **VMBus**, a shared-memory channel. A guest with Hyper-V’s integration services installed uses **enlightened** paravirtualized drivers over VMBus; a guest without them falls back to emulated devices and the performance of Chapter 2’s worked example.

⚠️ Hyper-V changes the machine underneath you

Once the Hyper-V role is enabled, the Windows you were using is a guest. This has consequences people meet by accident:

Other hypervisors stop working, or quietly become nested. VirtualBox and VMware Workstation on a Hyper-V-enabled Windows run on top of Hyper-V's platform rather than on the hardware, and historically simply refused. Benchmarks taken on that machine are measuring a nested guest.

It is also enabled by features that do not mention it: Windows Subsystem for Linux 2, Windows Sandbox, Docker Desktop and memory-integrity protection all turn Hyper-V on. A great deal of “my virtual machines got slow after a Windows update” is this.

5.4 KVM and QEMU

KVM is a kernel module. Loading `kvm.ko` and `kvm_intel.ko` turns Linux into a hypervisor by giving it the ability to enter VMX non-root operation; the module is about twenty thousand lines, because the scheduler, memory manager and drivers it needs are already there.

That is also the whole architectural claim: **a guest is a process**. It is scheduled by the ordinary Linux scheduler, its memory is ordinary process memory, `top` shows it, `kill` ends it, `cgroups` limit it. Every tool that works on a Linux process works on a virtual machine.

KVM provides only the processor and memory virtualization. The virtual hardware — disk controllers, network cards, graphics, BIOS — is QEMU's job, running in user space as that process. `libvirt` sits above both and provides the stable API that `virsh`, `virt-manager`, OpenStack and oVirt all drive.

💡 One Design Decision, Two Opposite Consequences

For: KVM inherits everything. Every driver Linux has, every filesystem, every scheduler improvement, every networking feature, with no porting work. When a new network card ships with a Linux driver, KVM supports it that day. This is why KVM runs most of the world's public cloud.

Against: KVM inherits everything. The trusted computing base is the entire Linux kernel — millions of lines, every driver loaded, every filesystem — and a bug in any of it is a host compromise. ESXi's 150 MB and Xen's 1 MB are the competing answers to exactly this.

Neither consequence can be had without the other, and which one dominates depends on whether you are optimising for hardware support or for attack surface.

5.5 Xen

Xen is the smallest hypervisor here — on the order of a megabyte — because it deliberately contains almost nothing. No drivers, no filesystems, no network stack. It schedules domains, manages memory, and passes messages.

Everything else lives in `dom0`, a privileged guest (normally Linux) that owns the physical devices and runs the driver stack. Unprivileged guests are `domU` and reach devices through split drivers terminating in `dom0`.

Xen was the paravirtualization project of Chapter 3, and its guest modes record that history: **PV** (fully paravirtualized, no hardware assistance needed, guest kernel modified), **HVM** (hardware-assisted, unmodified guests, from 2006), and **PVH** (hardware assistance for the processor, paravirtualized devices, no emulated chipset — the modern default).

💡 Disaggregation: the idea Xen got right early

Because `dom0` is a guest rather than part of the hypervisor, it can be split up. A **driver domain** is an unprivileged domain given one device via the IOMMU, running that device's

driver and nothing else. A buggy or hostile network driver then crashes a domain that can be restarted, instead of the host.

This is **disaggregation**, and it is the strongest security argument any platform in this chapter can make: it is the only one that can reduce the blast radius of a driver bug to something smaller than the whole machine. It is also operationally fiddly, which is why it is common in security-focused deployments such as Qubes OS and rare elsewhere.

Note where the idea goes next. Chapter 11 removes device emulation from the trusted path almost entirely, and Chapter 20 uses the same IOMMU machinery to hand accelerators to guests. Xen’s answer was early rather than wrong.

5.6 Choosing

	ESXi	Hyper-V	KVM	Xen
Hypervisor size	≈150 MB	thin, plus a full Windows parent	the Linux kernel	≈1 MB, plus dom0
Drivers live in	the VMkernel	the parent partition	the Linux kernel	dom0 or a driver domain
Licence	Commercial; free tier withdrawn in 2024	Included with Windows Server; free Hyper-V Server discontinued	GPL, free	GPL, free
Guest support	Very broad, certified	Windows strongest; Linux good	Anything x86 or ARM	Anything, three guest modes
Management	vCenter — the strongest in the industry	System Center, Windows Admin Center	libvirt, oVirt, Proxmox, OpenStack	XenCenter, XCP-ng
Used by	Enterprise data centres	Windows-centred estates	AWS Nitro, Google, DigitalOcean, most OpenStack	AWS before Nitro, Citrix, Qubes
Strongest when	Mixed hardware, mixed guests, operations staff who must not be surprised	The estate is already Windows and licensed	You want control, no licence cost, and newest hardware	Minimal trusted base, or driver isolation

📊 Worked Example: Three Years of a Small Cluster

Size and cost the three-host cluster of Chapter 1’s worked example on vSphere and on KVM. Each host has 2 sockets × 32 cores.

vSphere. VMware licenses per core with a 16-core minimum per socket. Here each socket has 32 cores, so the minimum does not bite:

$$3 \text{ hosts} \times 2 \text{ sockets} \times 32 = 192 \text{ cores}$$

At a list price of about \$350 per core per year for a standard edition, plus vCenter at about \$12,000 for three years:

$$192 \times 350 \times 3 + 12,000 = 201,600 + 12,000 = \textbf{\$213,600}$$

KVM. The software is free. A supported distribution with a virtualisation subscription runs about \$1,500 per host per year:

$$3 \times 1,500 \times 3 = \textbf{\$13,500}$$

Unsupported, it is \$0.

Step 1 — the gap. \$200,100 over three years, or about \$66,700 a year.

Step 2 — what the gap must buy. At a loaded cost of roughly \$45,000 a year for an experienced Linux engineer, the licence is worth paying if vSphere saves about **1.5 full-time engineers** of effort against a self-managed KVM estate — or if it prevents outages worth that much.

Step 3 — read it honestly in both directions. For an estate of three hosts the answer is almost always KVM: three hosts do not need 1.5 engineers of management either way, so the licence buys little. For an estate of three hundred hosts with a mixed hardware fleet, staff who know vCenter and an availability target with contractual penalties, the same arithmetic inverts.

What the arithmetic leaves out. Migration cost. An estate already on vSphere carries years of runbooks, automation, backup integration and staff knowledge, and moving is a project, not a decision. Prices also change: VMware’s 2024 move to subscription-only bundles raised many renewals severalfold and sent a visible part of the market to Proxmox and OpenStack. Check current pricing before quoting any of these figures.

! Common Pitfalls

- **Thinking Hyper-V runs on top of Windows.** It runs underneath it. Once enabled, your Windows is a privileged guest — which is why other hypervisors on the same machine become nested or refuse.
- **Describing KVM as “just a kernel module” and stopping there.** The module is the small part. QEMU supplies every virtual device and libvirt supplies the management API; a KVM deployment is all three.
- **Comparing hypervisor sizes without counting dom0 or the parent partition.** Xen’s hypervisor is a megabyte and its dom0 is a full Linux kernel. The honest comparison is of the whole privileged path.
- **Assuming the free tier still exists.** VMware withdrew free ESXi in 2024 and Microsoft discontinued the free Hyper-V Server. Student and evaluation routes change; check before building a course or a budget on one.
- **Choosing a platform on hypervisor features.** All four virtualize a processor competently. The decision is made on management tooling, licence cost, hardware support and the skills of the people who will be on call.

🔧 Try It Yourself: The Same Guest on Two Platforms

Builds one guest under KVM and the same guest under VirtualBox, then compares what each platform exposes to it. Appendix A has both. Expect forty minutes.

```
#!/usr/bin/env bash
# ch05_platforms.sh -- one workload, two platforms, and what differs.
```

```

set -euo pipefail

# --- 1. What is this host already running, if anything? -----
systemd-detect-virt || echo "bare metal"
lsmod | grep -E '^(kvm|kvm_intel|kvm_amd|vboxdrv)' || true

# --- 2. Define a guest on KVM, from the Chapter 1 image. -----
sudo virsh dumpxml vss1 > vss1.xml
grep -E '<(domain|emulator|controller|interface|model)' vss1.xml | head -20
# Note the machine type: QEMU presents a specific, named virtual chipset.

# --- 3. Ask the guest what hardware it thinks it has. -----
sudo virsh start vss1 ; sleep 25
IP=$(sudo virsh domifaddr vss1 | awk '/ipv4/{split($4,a,"/"); print a[1]}')
G () { ssh -o StrictHostKeyChecking=no "ubuntu@$IP" "$1"; }
G "sudo dmidecode -s system-manufacturer"
G "sudo dmidecode -s system-product-name"
G "lspci | head; systemd-detect-virt"

# --- 4. Build the same guest under VirtualBox. -----
VBoxManage createvm --name vss2 --ostype Ubuntu_64 --register
VBoxManage modifyvm vss2 --memory 2048 --cpus 2 --nic1 nat
VBoxManage clonehd ubuntu.img vss2.vdi --format VDI
VBoxManage storagectl vss2 --name SATA --add sata
VBoxManage storageattach vss2 --storagectl SATA --port 0 \
    --type hdd --medium vss2.vdi
VBoxManage startvm vss2 --type headless

# --- 5. Compare what the two platforms claim to be. -----
# Each hypervisor identifies itself: the manufacturer string, the PCI
# device list and the CPUID hypervisor leaf all differ.
echo "KVM guest reports:" ; G "sudo dmidecode -s system-manufacturer"
echo "VirtualBox guest reports: innotek GmbH (check in the guest console)"

# --- 6. The feature that is not in the free product. -----
# Live migration needs two hosts and shared storage. virsh offers it;
# VirtualBox does not. Chapter 17 is where this becomes the whole point.
sudo virsh migrate --help | head -5

# --- 7. Tear down. -----
sudo virsh destroy vss1 || true
VBoxManage controlvm vss2 poweroff || true
VBoxManage unregistervm vss2 --delete || true

```

What to notice. Step 5 is the honest answer to “can a guest tell?” — every platform identifies itself in several places, and none of them tries very hard to hide. Step 6 is the difference that actually decides procurement: the hypervisors are comparable, and the operational features are not.

? Checkpoint — Test Yourself

1. For each of ESXi, Hyper-V, KVM and Xen, say in one phrase where the device drivers run, and give the consequence for the trusted computing base.
2. A colleague reports that after installing Docker Desktop on Windows, their VirtualBox guests became much slower. Explain what happened.
3. Using the worked example’s method, at what cluster size does a vSphere licence begin to look defensible, and what assumption in your answer is doing the most work?

 Chapter Summary

- All four platforms use the same hardware features. What separates them is where the device drivers live, and almost every other difference follows from that.
- ESXi: a purpose-built kernel of about 150 MB with drivers inside it, a certified hardware list, and vCenter — which is what the licence actually buys.
- Hyper-V: a thin hypervisor that slides underneath a running Windows, demoting it to a privileged parent partition that owns the drivers. Enabling it changes the machine, often by accident.
- KVM: a kernel module that makes Linux a hypervisor, so a guest is a process. It inherits every Linux driver and the entire Linux kernel as trusted base — the same fact read twice.
- Xen: about a megabyte of hypervisor with nothing in it; dom0 owns the drivers, and driver domains can shrink the blast radius of a driver bug to less than the host.
- Licence cost differences are large — roughly \$200,000 over three years for a three-host cluster — and are decided by staff effort and risk, not by hypervisor features.

 Review Questions

1. **(MCQ)** In Hyper-V, the Windows installation on which the role was enabled becomes:
(a) the hypervisor; (b) the parent partition, a privileged guest; (c) an ordinary child partition; (d) unchanged.
2. **(MCQ)** Xen's hypervisor is roughly a megabyte because: (a) it is written in assembly; (b) it contains no device drivers, filesystems or network stack; (c) it supports fewer guests; (d) it is compiled with optimisation.
3. **(Short)** State the single architectural claim KVM makes about what a guest is, and give three operational consequences of it.
4. **(Short)** Explain why ESXi's short hardware compatibility list is a design decision rather than a shortcoming, and what it costs.
5. **(Short)** Describe a driver domain and the specific failure it is meant to contain.
6. **(Applied)** A department has six hosts, two Linux engineers, no existing virtualization and a small budget. A vendor proposes vSphere. Write the recommendation you would give, including the two questions you would ask before committing and what answer to each would change your mind.
7. **(Applied)** A hosting provider must run untrusted guests and is choosing between KVM and Xen with driver domains. Set out the security argument for each, name what the Xen design costs operationally, and say what you would need to measure before deciding.

CPU and Memory Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** how a hypervisor schedules virtual processors onto physical ones, and **interpret** ready time as the measure of whether it is succeeding.
- **Explain** why giving a guest more virtual processors can make it slower, using co-scheduling.
- **Compute** CPU and memory overcommit ratios, and **decide** from measured figures whether a host can take another guest.
- **Order** the four memory reclamation techniques by cost and **state** what each one does to guest performance.
- **Describe** NUMA and **explain** why a guest sized larger than a socket may lose a third of its memory bandwidth.

🔗 Before You Start

Chapter 4 (extended page tables, which are what make guest memory management cheap) and Chapter 5 (the four schedulers). From Operating Systems you need process scheduling and the idea of a working set.

📖 Key Terms in This Chapter

vCPU • pCPU • time slice • ready time • %RDY • co-scheduling • co-stop • relaxed co-scheduling • overcommit ratio • working set • active memory • transparent page sharing • ballooning • balloon driver • memory compression • hypervisor swapping • memory tax • NUMA • NUMA node • vNUMA • node interleaving

6.1 Scheduling a Processor That Is Not There

A guest configured with four virtual processors believes it has four processors. It runs four run queues, balances threads across them, and spins on locks expecting the other holder to be making progress. None of that is true: a **vCPU** is a thread the hypervisor schedules onto a **pCPU**, and between one guest instruction and the next it may have been off the processor for milliseconds.

For CPU-bound work this is nearly free — the guest's instructions run on the real processor at full speed, as Chapter 2 measured. The costs appear when guests contend, and they are measured by one number.

📄 Ready Time

Ready time is the proportion of time a vCPU was *runnable but not scheduled* — the guest had work, and no physical core was free for it. VMware reports it as **%RDY** per vCPU; KVM exposes the same quantity as steal time, which the guest itself can see in **top** as **%st**.

It is the single most useful number in capacity planning, because it distinguishes the two cases that look identical from inside the guest: a guest that is slow because its own work is slow, and a guest that is slow because it is not being given a processor.

Rules of thumb, widely used and worth knowing: under 5% is healthy, 5–10% deserves investigation, and sustained figures above 10% mean the host is oversubscribed. Chapter 18 treats the measurement properly.

6.2 Why More vCPUs Can Be Slower

The first instinct on finding a slow guest is to give it more processors. It is often exactly wrong, and the reason is **co-scheduling**.

A guest kernel assumes its processors run simultaneously. A thread holding a spinlock on vCPU 0 will be released shortly, so a thread on vCPU 1 spins rather than sleeping. If vCPU 0 is *descheduled* while holding that lock, vCPU 1 spins for a whole time slice — burning a physical core to wait for a virtual one that is not running.

Early hypervisors avoided this by strict co-scheduling: never run a guest’s vCPUs unless *all* of them can run together. That removes the problem and creates a worse one — an eight-vCPU guest waits for eight free cores, and on a busy host it waits a long time. Modern schedulers use **relaxed co-scheduling**: a vCPU is allowed to fall behind its siblings by a small margin, and is forced to catch up only when the skew grows too large. The waiting that results is reported as **co-stop**.

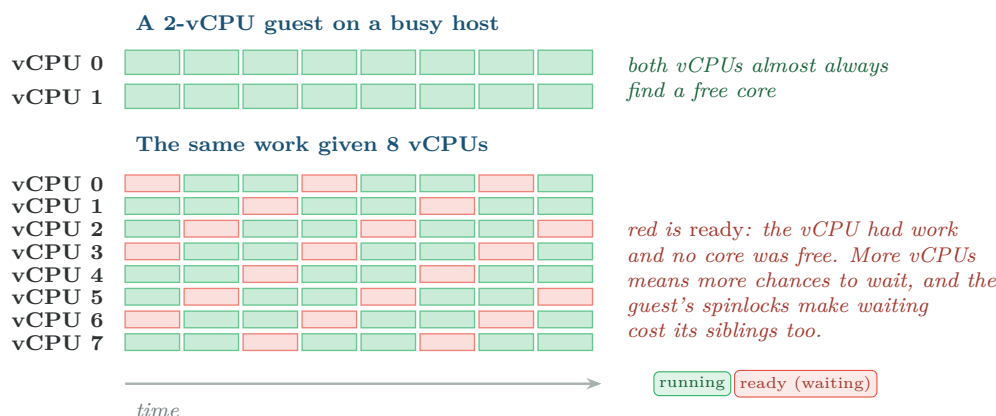


Figure 6.1: Why a bigger guest can be a slower guest. Right-sizing downward is the most common successful intervention in a contended cluster, and the one administrators are most reluctant to make.

⚠ The rule that follows: size for what is used, not what is asked for

Users ask for vCPUs the way they ask for disk — more is safer. It is not. A guest given eight vCPUs and using two has not merely wasted six; it has made itself harder to schedule, raised its own ready time and raised everyone else’s.

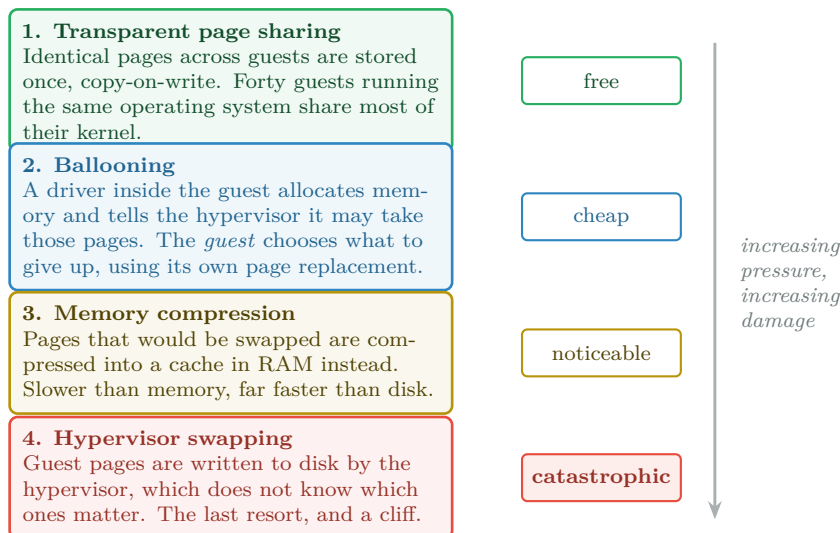
The operational rule is to allocate what measurement justifies and add more when ready time and utilisation both say so. Chapter 18 gives the procedure; the point here is that the arithmetic of overcommit does not capture it, because two guests with the same total vCPU count behave differently according to how those vCPUs are grouped.

6.3 Memory, and Why It Is the Constraint

Chapter 1 asserted that memory, not processor time, usually decides how many guests fit. The reason is that the two resources are elastic in different ways.

A processor can be oversubscribed because *time* is divisible: two guests each wanting a core can have it alternately, and the result is slower, not wrong. Memory cannot be divided in time. A page either holds a guest's data or it does not, and a guest denied a page it is using does not slow down slightly — it swaps, and swapping is a hundredfold penalty, not a percentage.

So hypervisors oversubscribe memory cautiously, using four techniques of increasing desperation.



The gap between steps 3 and 4 is the one that matters: the first three degrade gracefully and the fourth does not. A host reaching step 4 has been oversubscribed, not tuned.

Figure 6.2: The reclamation ladder. A hypervisor works down it as pressure rises; an administrator's job is to ensure it never reaches the bottom rung.

💡 Why Ballooning Works, and Why It Feels Wrong

Ballooning looks like a trick, and students reasonably ask why the hypervisor does not simply take the pages.

Because it does not know which ones to take. From outside, every guest page looks alike; the hypervisor cannot tell a hot database buffer from a page of cache the guest would discard without noticing. Taking the wrong page costs a disk read in the guest, and the hypervisor would be guessing.

The guest kernel, by contrast, has spent decades getting good at exactly this question. So the balloon driver asks the guest for memory in the one language the guest understands — it allocates it, as any program would — and the guest runs its own page replacement to decide what to give up. The hypervisor then takes the pages backing the balloon, which it knows are not in use.

The consequence worth remembering: **ballooning requires the guest to cooperate**. A guest without the balloon driver installed cannot be reclaimed from gracefully, and pressure on that host goes straight from step 1 to step 3. Installing guest tools is not cosmetic.

Worked Example: Can This Host Take Four More Guests?

A host of 2×32 cores and 512 GB currently runs 28 guests, each configured with 4 vCPUs and 16 GB. Measured over a month: mean %RDY 4%, 95th percentile 9%; mean active memory 6.2 GB per guest. A team asks for four more identical guests. Decide.

Step 1 — CPU overcommit now and after.

$$\sigma_{\text{cpu}} = \frac{28 \times 4}{64} = \frac{112}{64} = \mathbf{1.75} \quad \text{after: } \frac{32 \times 4}{64} = \frac{128}{64} = \mathbf{2.00}$$

Ready time is already at 9% at the 95th percentile, which is the edge of the acceptable band. Raising oversubscription by 14% will not improve it.

Step 2 — memory allocated.

$$28 \times 16 = 448 \text{ GB} \quad \text{after: } 32 \times 16 = \mathbf{512 \text{ GB}}$$

The host has 512 GB *in total*, and the hypervisor itself needs some — call it 6%, about 32 GB, for its own code, the per-guest overhead and the page tables of Chapter 4. So the request allocates 512 GB of 480 GB usable: **32 GB short** before a single guest does anything.

Step 3 — memory active. Today the guests actually touch

$$28 \times 6.2 = 173.6 \text{ GB}$$

of 448 GB allocated — 39%. After the change, $32 \times 6.2 = 198.4$ GB, still far below 480 GB. So on *today's behaviour* the memory fits comfortably.

Step 4 — which of Steps 2 and 3 should decide? This is the real question, and the answer is: Step 3 for the normal case, Step 2 for the bad one. Overcommitting memory is safe exactly as long as the guests do not all become active together — and the events that make them active together (a patch window, a backup run, a traffic spike, a failover from another host) are correlated by construction.

If all 32 guests touched their full allocation, demand would be 512 GB against 480 GB usable: 32 GB short, about 1 GB per guest to be reclaimed. Page sharing and ballooning would absorb that — rungs 1 and 2 — so the host would not swap. That is survivable.

Step 5 — the recommendation. Refuse the request as stated, and make a counter-offer. The binding problem is CPU, not memory: 9% ready at the 95th percentile is the symptom of a host already at its limit, and $\sigma_{\text{cpu}} = 2.0$ will make it worse for all 32 guests, not just the new four.

The counter-offer is to right-size. If the existing 28 guests are, like most, using two vCPUs of their four, reducing them to 2 vCPUs gives $\sigma_{\text{cpu}} = (28 \times 2 + 4 \times 4)/64 = 72/64 = 1.13$ — and the four new guests fit with ready time *lower* than it is today. Chapter 18 is how you establish whether that premise holds.

Do not read a memory overcommit ratio without an active figure

$\sigma_{\text{mem}} = 1.25$ tells you almost nothing on its own. If the guests are using a third of what they were given, it is comfortable. If they are using all of it, the host is already swapping.

Allocation is a promise; active memory is a fact. Plan against the fact, size the safety margin against the promise, and know which of the two you are quoting whenever you quote a number.

6.4 NUMA

A two-socket server does not have one pool of memory. Each socket has its own memory controller and its own attached DIMMs; a core reaches its own socket's memory quickly and the other socket's memory across an interconnect, more slowly. This is **NUMA** — non-uniform memory access — and a remote access typically costs something in the region of 1.5 to 2 times a local one in latency, with lower bandwidth.

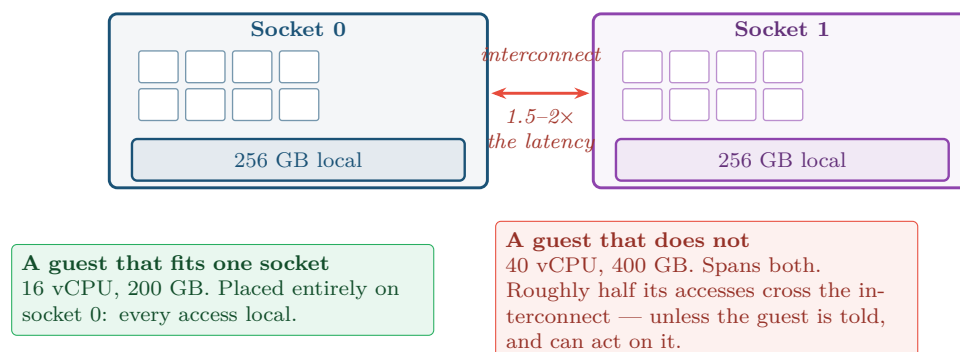


Figure 6.3: NUMA on a two-socket host. The green guest is a sizing decision, not a tuning option: once a guest is wider than a socket, no amount of hypervisor configuration makes its memory local again.

💡 vNUMA: Telling the Guest the Truth

A guest that must span sockets can still be scheduled well, but only if it knows. **vNUMA** exposes the host's NUMA topology to the guest, so the guest kernel's own NUMA-aware scheduler and allocator can keep each thread's memory on the node where the thread runs. Databases and JVMs do this competently.

Two consequences follow, and both are commonly missed. First, vNUMA is usually enabled automatically only above a threshold — historically eight or nine vCPUs — so a guest just under it that still spans sockets gets no topology at all. Second, the topology is fixed when the guest boots: migrating it (Chapter 17) to a host with a different socket layout leaves the guest optimising for a machine it is no longer on.

The rule that survives all of this: **size guests to fit inside a NUMA node wherever you can**. It is the cheapest performance decision in this chapter and it is made at provisioning time, where nobody is looking.

❗ Common Pitfalls

- **Adding vCPUs to a slow guest.** If the guest is slow because the host is contended, more vCPUs raise its ready time and everyone else's. Measure %RDY before changing anything.
- **Quoting an overcommit ratio with no active-memory figure.** Allocation is a promise and activity is a fact; a ratio without the second is not a capacity statement.
- **Treating the four reclamation techniques as interchangeable.** Page sharing is free and ballooning is cheap; swapping is a cliff, not a slope. A host that swaps is misconfigured, not merely busy.
- **Running guests without the balloon driver and then overcommitting.** Without guest cooperation the hypervisor skips the graceful rung entirely.
- **Ignoring NUMA until there is a problem.** It is decided when the guest is sized. A 40-vCPU guest on a 32-core socket cannot be tuned back into locality.

- **Disabling page sharing because of side-channel advice, without noticing the memory cost.** Sharing between guests was restricted by default after 2014 for good reasons, but on a host of forty identical guests it was recovering a substantial fraction of memory. Know which trade you have made.

Try It Yourself: Making a Host Hurt, on Purpose

Drives one host into CPU contention and then into memory pressure, and watches each symptom appear. Use a laptop you are not relying on; it will become unresponsive at step 5. Expect forty minutes.

```
#!/usr/bin/env bash
# ch06_pressure.sh -- ready time and the reclamation ladder, observed.
set -euo pipefail
G () { ssh -o StrictHostKeyChecking=no "ubuntu@$1" "$2"; }

# --- 1. How many physical cores are we actually working with? -----
nproc ; lscpu | grep -E 'Socket|Core|NUMA'

# --- 2. Build four guests, each with as many vCPUs as the host has. ----
# Deliberate oversubscription: 4 guests x nproc vCPUs on nproc cores.
for n in 1 2 3 4; do
    sudo qemu-img create -f qcow2 -b /var/lib/libvirt/images/vss1.qcow2 \
        -F qcow2 "/var/lib/libvirt/images/p$n.qcow2" 10G
    sudo virt-install --name "p$n" --memory 2048 --vcpus "$(nproc)" \
        --disk "/var/lib/libvirt/images/p$n.qcow2",format=qcow2 \
        --disk seed.iso,device=cdrom --os-variant ubuntu22.04 \
        --import --graphics none --noautoconsole
done
sleep 40

# --- 3. Baseline: one guest busy, three idle. -----
ip_of () {
    sudo virsh domifaddr "$1" | awk '/ipv4/{split($4,a,"/"); print a[1]}'
}
IP1=$(ip_of p1)
G "$IP1" "sudo apt-get -qq install -y stress-ng sysstat"
G "$IP1" "stress-ng --cpu $(nproc) --timeout 30s --metrics-brief"

# --- 4. Now make all four busy and read steal time in the guest. -----
# %st in the guest IS the ready time of section 6.1, seen from inside.
for n in 2 3 4; do
    IP=$(ip_of "p$n")
    G "$IP" "sudo apt-get -qq install -y stress-ng" || true
    G "$IP" "nohup stress-ng --cpu $(nproc) --timeout 60s >/dev/null 2>&1 &"
done
sleep 5
G "$IP1" "mpstat 2 5 | tail -4"          # watch the %steal column climb

# --- 5. Memory pressure: ask for far more than the host has. -----
# Watch the balloon move before anything swaps.
sudo virsh dommemstat p1
for n in 1 2 3 4; do
    G "$(ip_of "p$n")" "nohup stress-ng --vm 1 --vm-bytes 85% \
        --timeout 60s >/dev/null 2>&1 &" || true
done
sleep 20
sudo virsh dommemstat p1                # balloon / swap_in / swap_out
free -m ; vmstat 2 4                    # the host's own view

# --- 6. Right-size, and measure the same thing again. -----
```

```
# Two vCPUs each instead of nproc: the overcommit ratio drops fourfold.
for n in 1 2 3 4; do sudo virsh destroy "p$n"; \
    sudo virsh setvcpus "p$n" 2 --config --maximum --config; \
    sudo virsh setvcpus "p$n" 2 --config; sudo virsh start "p$n"; done
sleep 40
G "$IP1" "mpstat 2 5 | tail -4"          # %steal should be far lower

# --- 7. Tear down. -----
for n in 1 2 3 4; do
    sudo virsh destroy "p$n" || true
    sudo virsh undefine "p$n" --remove-all-storage || true
done
```

What to notice. Step 4's %steal is the worked example's %RDY, measured from inside the guest, and step 6 shows that halving the vCPU count made every guest *faster*. That is the chapter's least intuitive claim and you have now produced it yourself.

? Checkpoint — Test Yourself

1. A host of 48 cores runs 20 guests of 6 vCPUs each. Give the CPU overcommit ratio. Mean %RDY is 14%. State what you would do and what you would measure first.
2. Explain why the hypervisor asks the guest to give up memory through a balloon driver rather than simply reclaiming pages itself.
3. A guest is configured with 40 vCPUs and 400 GB on a host with two 32-core sockets and 256 GB per socket. Name the problem, say why no hypervisor setting fixes it, and give the one change that would.

✓ Chapter Summary

- A vCPU is a thread scheduled onto a physical core. Ready time — %RDY, or steal time inside the guest — measures how often it had work and no core. Under 5% healthy, over 10% oversubscribed.
- More vCPUs can make a guest slower: a guest's spinlocks assume its processors run together, and relaxed co-scheduling makes a wide guest wait. Right-sizing downward is the commonest effective fix.
- Processor time is divisible and memory is not, which is why memory usually decides guest density.
- The reclamation ladder: page sharing (free), ballooning (cheap, needs the guest driver), compression (noticeable), hypervisor swapping (a cliff). The job is to never reach the fourth.
- Overcommit ratios mean nothing without an active-memory figure. Allocation is a promise; activity is a fact.
- NUMA is decided at provisioning time. Size guests inside a node where you can; where you cannot, make sure vNUMA tells the guest the truth.

✍ Review Questions

1. (MCQ) Ready time measures: (a) how long the guest took to boot; (b) the fraction of time a vCPU was runnable but not scheduled; (c) the hypervisor's own CPU use; (d) time spent in VM exits.
2. (MCQ) Ballooning requires: (a) extended page tables; (b) an IOMMU; (c) a cooperating driver inside the guest; (d) shared storage.
3. (Short) Explain co-scheduling and why relaxed co-scheduling replaced the strict form.

4. **(Short)** Give the four reclamation techniques in order and say what each costs the guest.
5. **(Short)** Why is a memory overcommit ratio of 1.3 potentially safe while a CPU overcommit ratio of 1.3 is almost always safe?
6. **(Applied)** A host of 2×24 cores and 384 GB runs 30 guests of 4 vCPUs and 12 GB. Measured: %RDY mean 7%, p95 15%; active memory 4.1 GB per guest. A team wants six more identical guests. Work through the same five steps as the chapter's worked example and give a recommendation with a counter-offer.
7. **(Applied)** A database guest is reported as slow. It has 32 vCPUs and 512 GB on a two-socket host with 24 cores and 384 GB per socket. List what you would measure, in order, and for each possible finding say what you would change.

Paravirtualization and Device Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** the three ways a guest can be given a device — emulation, paravirtualization and passthrough — by the path an I/O request takes in each.
- **Explain** the virtio split-driver model and the virtqueue, and **say** why batching is what makes it fast.
- **Describe** SR-IOV in terms of physical and virtual functions, and **state** what the IOMMU contributes.
- **Quantify** the throughput and CPU cost of each option from measured figures.
- **Choose** a device model for a stated requirement, and **state** the capability each choice gives up.

🔗 Before You Start

Chapter 2 (paravirtualization as a technique, and the worked example where an emulated disk lost 81% of its throughput) and Chapter 4 (VM exits, and the IOMMU without which passthrough is unsafe). This chapter is where that 81% is explained and removed.

📖 Key Terms in This Chapter

device emulation • full device model • virtio • split driver • front-end driver • back-end driver • virtqueue • descriptor ring • available ring • used ring • doorbell • vhost • vhost-user • passthrough • device assignment • VFIO • SR-IOV • physical function • virtual function • DPDK • guest additions

7.1 Three Ways to Give a Guest a Device

A guest needs a disk and a network card. The hypervisor has one of each, or several, and must serve many guests. There are exactly three strategies, and they differ in how much of the real device the guest is allowed to see.

7.1.1 Emulation

The hypervisor implements, in software, a specific piece of real hardware — classically an Intel e1000 network card or an IDE controller — chosen because every operating system since the 1990s already has a driver for it. Nothing is installed in the guest and nothing is configured. It simply works.

It is also catastrophic for throughput, and Chapter 2 measured why. The guest driver was written to talk to hardware, so it reads and writes device registers, and each of those is a VM exit at several hundred cycles. One packet costs dozens.

Emulation survives because it is the only option that needs no cooperation: an installer booting from an unknown ISO, an appliance you cannot modify, a guest before its drivers are installed. Every platform offers it, and no platform expects you to keep using it.

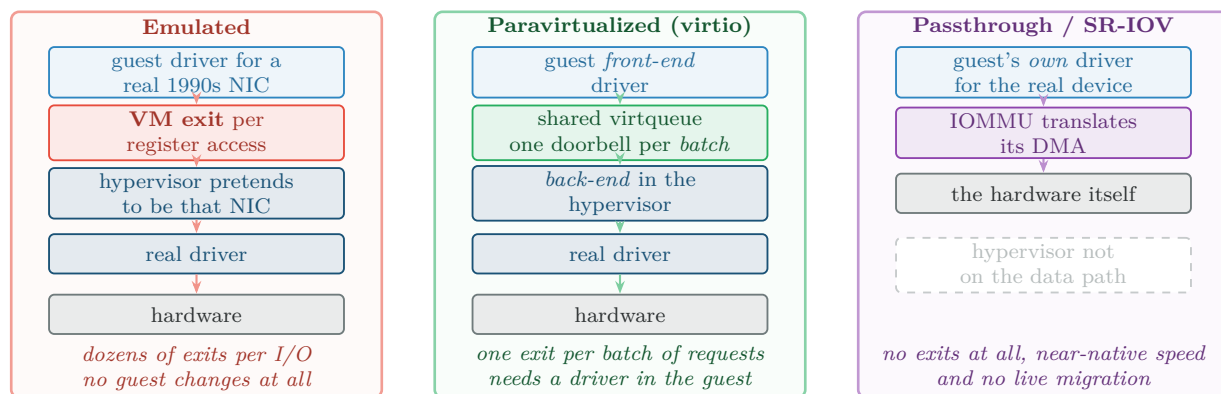


Figure 7.1: The same I/O request, three ways. Reading down each lane gives the cost: the left lane crosses the boundary dozens of times per request, the middle once per batch, the right not at all.

7.1.2 Paravirtualization: virtio

If the guest can be given a driver, a far better interface is available — one designed for the situation rather than inherited from it.

virtio is that interface. It is a standard — an OASIS specification, not a vendor's product — which is why one virtio driver in Linux works under KVM, and under cloud hypervisors that are not KVM at all.

The Split Driver Model and the Virtqueue

A virtio device is split in two. The **front-end driver** runs in the guest and presents a normal disk or network interface to the guest kernel. The **back-end driver** runs in the hypervisor and does the real work. Between them sits one or more **virtqueues**, in memory both can see.

A virtqueue is three rings in shared memory:

- the **descriptor table** — buffers, each an address and a length;
- the **available ring** — which descriptors the guest has offered;
- the **used ring** — which the host has finished with.

The guest writes buffers and advances the available ring with *ordinary memory writes*. No exit. It then rings a **doorbell** — one write to one register, one exit — to say that work is waiting. The host takes everything in the ring, does it, and advances the used ring.

Batching Is the Whole Trick

The emulated card costs an exit per register access. virtio costs an exit per *doorbell*, and one doorbell can cover a hundred queued requests.

Under load it gets better rather than worse, which is the opposite of the usual pattern. A busy back-end polls the ring and finds new work already waiting, so the guest can skip the doorbell entirely — the specification has an explicit flag for suppressing notifications. At high rates the exit count per request tends toward zero.

That is why Chapter 2's disk measurement went from 81% lost to 4% lost by changing one attribute. Nothing about the hardware changed; the number of boundary crossings did.

💡 **vhost: Moving the Back-End Out of QEMU**

In plain KVM the virtio back-end lives in QEMU, in user space. A packet therefore crosses from the guest into the kernel, up to QEMU, back into the kernel to reach the real driver, and out.

vhost moves the back-end into the host *kernel*, so the data path never enters user space; QEMU only sets the queues up. **vhost-user** goes the other way for a different reason: it hands the back-end to a dedicated user-space process — typically a DPDK-based virtual switch — which polls the queues on a core of its own and never sleeps.

Both are the same instinct as the ladder in Chapter 6: the mechanism is settled, and all remaining performance work is removing crossings.

7.1.3 Passthrough and SR-IOV

The third option is to stop virtualizing the device at all and give it to one guest. The guest loads the vendor's own driver and programs the real hardware; the hypervisor is not on the data path. On Linux this is done with **VFIO**, which unbinds the device from the host driver and exposes it to QEMU, with the IOMMU confining its DMA to that guest's memory. Performance is native, because nothing is in the way.

The limitation is arithmetic: one device, one guest. A host with two network cards can serve two guests this way.

📖 **SR-IOV, Physical and Virtual Functions**

SR-IOV — single-root I/O virtualization — is a PCIe capability that lets one physical device present itself as many.

The **physical function** (PF) is the real device, managed by the host: it configures the card, sets rate limits, and creates the virtual functions. Each **virtual function** (VF) is a lightweight PCIe function with its own queues, its own MAC address and its own configuration space, which can be assigned to a guest exactly as a whole device would be.

The card's own hardware switch sorts traffic between the VFs. A modern server NIC supports 64 to 128 of them, so one card serves many guests at close to line rate, with the IOMMU keeping each VF's DMA inside its guest.

🧮 **Worked Example: What Each Device Model Costs**

One 25 Gb/s network card, one guest, a small-packet forwarding test. Compute the throughput and the cost per unit of work.

Measured.

Model	Mpps	Host CPU, cores	Guest CPU, cores
Emulated e1000	0.12	1.8	0.9
virtio (in QEMU)	1.40	1.1	0.8
virtio + vhost	2.90	0.9	0.8
SR-IOV VF	8.60	0.0	0.9
Native (no guest)	8.90	—	—

Step 1 — throughput against native.

$$\text{emulated} = \frac{0.12}{8.90} = \mathbf{1.3\%} \quad \text{virtio} = \frac{1.40}{8.90} = \mathbf{15.7\%}$$

$$\text{vhost} = \frac{2.90}{8.90} = \mathbf{32.6\%} \quad \text{SR-IOV} = \frac{8.60}{8.90} = \mathbf{96.6\%}$$

Step 2 — the cost of a packet, in host cores per Mpps.

$$\text{emulated} = \frac{1.8}{0.12} = 15.0 \quad \text{virtio} = \frac{1.1}{1.40} = 0.79 \quad \text{vhost} = \frac{0.9}{2.90} = 0.31$$

$$\text{SR-IOV} = \frac{0.0}{8.60} = \mathbf{0}$$

Emulation is roughly **48 times** more expensive per packet than vhost, and SR-IOV spends no host processor at all because the host is not involved.

Step 3 — the figure that decides most designs. Look at the host-CPU column, not the throughput column. The emulated card burns 1.8 cores to move almost nothing; on a host of 64 cores running 30 guests, that is nearly 3% of the machine, per guest, spent on pretending to be a 1999 network card. The consolidation ratio of Chapter 1 is quietly destroyed by a default nobody changed.

Step 4 — so why is SR-IOV not the answer to everything? Because of what it gives up, which Section 7.4 sets out: that guest can no longer be live migrated, snapshotted with its device state, or overcommitted. Most estates value those more than they value the last 64% of line rate, and run virtio with vhost. SR-IOV is for the specific guests that cannot — a virtual firewall, a packet broker, a trading system.

7.2 What Passthrough Costs You

Passthrough is the fastest option and it is rare, because speed is not the thing most estates are optimising.

Capability	What happens with passthrough	Why
Live migration (Chapter 17)	Lost , or needs an elaborate bonded-failover arrangement	The device's internal state is in silicon the hypervisor cannot read, so there is nothing to copy to the other host
Memory overcommit (Chapter 6)	Lost for that guest	The device performs DMA to guest memory at any moment, so every page must be pinned in real memory and cannot be ballooned or swapped
Snapshots	Lost or incomplete	The device's state is not part of the guest's memory image
Density	One VF per guest, 64–128 per card	A hard hardware limit, where virtio has none
Hardware independence	Lost	The guest now has a driver for a specific card. Move it to a host with a different NIC and it will not work

Capability	What happens with passthrough	Why
Host visibility	Reduced	The host cannot see or shape that traffic; firewall and monitoring must move into the guest or onto the card

⚠ Memory pinning is the consequence people forget

Live migration is the loss everyone cites. Pinning is the one that bites quietly.

A device doing DMA writes to host physical addresses it was given earlier. If the hypervisor moved, compressed, ballooned or swapped that page in the meantime, the device writes over whatever is there now. So every page of a passthrough guest's memory must be locked in RAM for the guest's whole life.

A 64 GB guest with an assigned device consumes 64 GB of the host whether it uses it or not — and the entire reclamation ladder of Chapter 6 is unavailable for it. On a host sized with overcommit in mind, two or three such guests change the arithmetic completely.

! Common Pitfalls

- **Leaving guests on emulated devices and concluding virtualization is slow.** It is the commonest cause of a bad first impression, and it is one attribute in the guest's configuration.
- **Installing guest tools “for the clipboard”.** The drivers are the point. The clipboard is a side effect.
- **Assigning a device without checking IOMMU groups.** Devices share groups, and the whole group must be assigned together. Passing one function of a multi-function card may drag its siblings — which might include a device the host needs.
- **Expecting to migrate a guest with an assigned device.** Plan the exception before you build it, not when you first need to patch the host.
- **Forgetting that SR-IOV traffic bypasses the host switch.** Packets between two VFs on the same card never reach the host, so host firewall rules, flow logging and traffic shaping do not see them. Chapter 13 and Chapter 19 both depend on this.
- **Treating virtio as a Linux feature.** It is an open specification with signed Windows drivers (virtio-win) that are free and are what every cloud Windows image already uses.

🔧 Try It Yourself: The Same Card, Three Ways

Measures the throughput and the host CPU cost of an emulated NIC, a virtio NIC and virtio with vhost. SR-IOV is included as a read-only step because it needs a server NIC. Expect forty minutes.

```
#!/usr/bin/env bash
# ch07_devices.sh -- emulation, virtio and vhost, measured.
set -euo pipefail
ip_of () {
    sudo virsh domifaddr "$1" | awk '/ipv4/{split($4,a,"/"); print a[1]}'
}
G () { ssh -o StrictHostKeyChecking=no "ubuntu@$(ip_of vss1)" "$1"; }

# --- 1. What device model is the guest using right now? -----
sudo virsh dumpxml vss1 | sed -n '/<interface/,/<\</interface>/p'
# <model type='virtio'> or type='e1000'. This single line is the chapter.
```

```
# --- 2. Set up the measurement: iperf3 on the host, client in guest. ---
sudo apt-get install -y iperf3 sysstat
iperf3 -s -D # server on the host, port 5201
sudo virsh start vss1 || true ; sleep 25
G "sudo apt-get -qq install -y iperf3"

# --- 3. Emulated e1000: a software imitation of a 1999 card. -----
sudo virsh destroy vss1
sudo virt-xml vss1 --edit --network model=e1000
sudo virsh start vss1 ; sleep 25
mpstat 1 12 > host-e1000.txt & # host CPU while the test runs
G "iperf3 -c 192.168.122.1 -t 10 -P 4" | tail -4
wait ; tail -2 host-e1000.txt

# --- 4. virtio: the split-driver model of section 7.2. -----
sudo virsh destroy vss1
sudo virt-xml vss1 --edit --network model=virtio
sudo virsh start vss1 ; sleep 25
mpstat 1 12 > host-virtio.txt &
G "iperf3 -c 192.168.122.1 -t 10 -P 4" | tail -4
wait ; tail -2 host-virtio.txt

# --- 5. virtio + vhost: back-end moved into the host kernel. -----
# vhost_net is usually already on; confirm, and confirm the guest uses it.
lsmod | grep vhost_net || sudo modprobe vhost_net
sudo virsh destroy vss1
sudo virt-xml vss1 --edit --network driver.name=vhost
sudo virsh start vss1 ; sleep 25
mpstat 1 12 > host-vhost.txt &
G "iperf3 -c 192.168.122.1 -t 10 -P 4" | tail -4
wait ; tail -2 host-vhost.txt

# --- 6. Look at the virtqueues themselves. -----
# One queue pair per vCPU is the usual arrangement; each is a ring in
# memory that both sides can see, exactly as the definition box says.
G "ls /sys/class/net/*/queues/ ; ethtool -l ens3 2>/dev/null || true"

# --- 7. SR-IOV, if the host has a server NIC that supports it. -----
# Read-only: creating VFs needs a capable card and IOMMU enabled.
for d in /sys/class/net/*/device/sriov_totalvfs; do
    [ -e "$d" ] && echo "$d -> $(cat "$d") VFs available"
done
ls /sys/kernel/iommu_groups 2>/dev/null | wc -l

# --- 8. Restore and tear down. -----
sudo virsh destroy vss1 || true
pkill iperf3 || true
```

What to notice. The three throughput figures should differ by roughly the ratios in the worked example, but look at the `mpstat` output rather than the bandwidth: the emulated run burns host processor to move very little, and that — not the bandwidth — is what decides how many guests the host can hold.

? Checkpoint — Test Yourself

1. A guest moves 0.2 Mpps while the host spends 1.6 cores serving it. Give the cost in host cores per Mpps, say which device model this almost certainly is, and give the one change you would make.
2. Explain why a virtio guest's exit rate per request *falls* as load rises, and name the mechanism.

3. A team wants SR-IOV for twelve guests on a host that must stay patchable without downtime. Name the conflict, and give two ways to resolve it.

✓ Chapter Summary

- Three device models: emulation (no guest changes, dozens of exits per I/O), paravirtualization (a driver in the guest, one exit per batch), and passthrough (the guest drives real hardware, no exits).
- virtio splits the driver in two around a virtqueue in shared memory. The guest fills the ring with ordinary writes and rings a doorbell once per batch, so the crossing rate per request falls as load rises.
- vhost moves the back-end into the host kernel; vhost-user moves it to a polling user-space switch. Both remove crossings rather than making them cheaper.
- SR-IOV makes one card present many virtual functions, each assignable to a guest, with the IOMMU confining each one's DMA.
- Passthrough costs live migration, memory overcommit, snapshots, hardware independence and host visibility. Memory must be pinned for the guest's whole life.
- Most estates run virtio with vhost, and reserve passthrough for the few guests that genuinely need line rate.

✍ Review Questions

1. **(MCQ)** A virtqueue consists of: (a) a single FIFO buffer; (b) a descriptor table, an available ring and a used ring; (c) one ring per device register; (d) a copy of the guest's page tables.
2. **(MCQ)** Assigning a physical device to a guest requires every page of that guest's memory to be pinned because: (a) the guest runs faster; (b) the device performs DMA to addresses that must stay valid; (c) the IOMMU has no page tables; (d) live migration needs it.
3. **(Short)** Explain the difference between a physical function and a virtual function, and say what each is used for.
4. **(Short)** Why does emulation still exist, given that it is roughly fifty times more expensive per packet than the alternatives?
5. **(Short)** State three capabilities a guest loses when a device is assigned to it, with the reason for each.
6. **(Applied)** A virtual firewall must inspect 10 Gb/s of small-packet traffic. The estate's standard is virtio with vhost and all guests are live migrated for monthly patching. Recommend a device model, state what it costs, and describe how you would keep the host patchable anyway.
7. **(Applied)** A host of 64 cores runs 30 guests, all on emulated e1000 adapters because that was the template default. Using the worked example's figures, estimate the host processor currently spent on device emulation, say what you would change, and describe how you would roll the change out without an outage.

PART III

Containers and Lightweight Virtualization

A virtual machine virtualizes the hardware, and pays for a whole operating system to do it. Part III asks a cheaper question: if the processes already share one kernel, what would it take to stop them seeing each other? The answer turns out to be two features Linux already had — and an industry grew out of combining them.

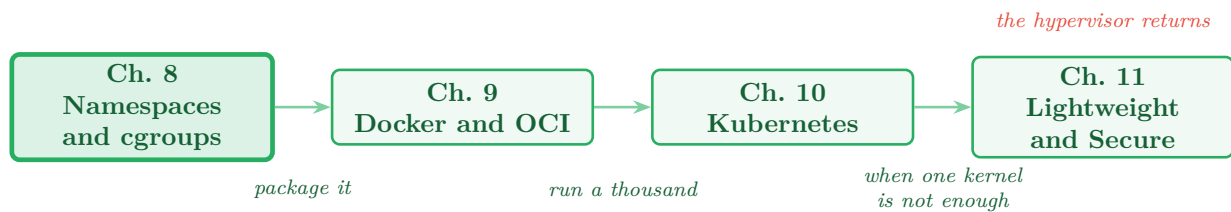


Figure P3.1: Reading path through Part III. The arc runs from kernel primitives to a cluster scheduler — and then back to Part II, because Chapter 11’s answer to container isolation is to put a virtual machine underneath each one.

From Virtual Machines to Containers

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** where a container cuts the stack, and **contrast** it with where a hypervisor cuts it.
- **Name** the Linux namespaces and **state** what each one isolates.
- **Use** control groups to limit a process tree's CPU and memory, and **explain** what happens when a limit is reached.
- **Describe** how an overlay filesystem gives every container a private root from shared read-only layers.
- **Compute** the density and start-up advantage of containers over virtual machines, and **state** precisely what is given up for it.

🔗 Before You Start

Chapter 2 (the taxonomy figure, which placed containers on the second row) and Chapter 6 (overcommit and the working set). From Operating Systems you need processes, the process tree, file descriptors and mount points. Nothing in Part II's hardware material is required — which is itself the point of this chapter.

📖 Key Terms in This Chapter

OS-level virtualization • container • namespace • mount namespace • PID namespace • network namespace • user namespace • control group • cgroup v2 • cgroup controller • union filesystem • overlayfs • lower dir • upper dir • copy-up • capability • seccomp • chroot • rootless container • shared kernel

8.1 A Different Place to Cut

Every chapter so far has virtualized *hardware*: a processor that is not a processor, memory that is not where the guest thinks, a disk that is a file. The guest runs its own kernel on that imaginary machine.

A container asks a cheaper question. If all the workloads are going to run on Linux anyway, why buy forty copies of Linux? Leave one kernel in place, and give each group of processes a private *view* of it: its own filesystem root, its own process numbers, its own network interfaces, its own users. The processes are ordinary processes on the host kernel. They simply cannot see anything they were not shown.

📄 Container

A **container** is a group of processes on a shared kernel, given a private view of the system by **namespaces** and a bounded share of its resources by **control groups**, usually running from a filesystem image assembled by a **union filesystem**.

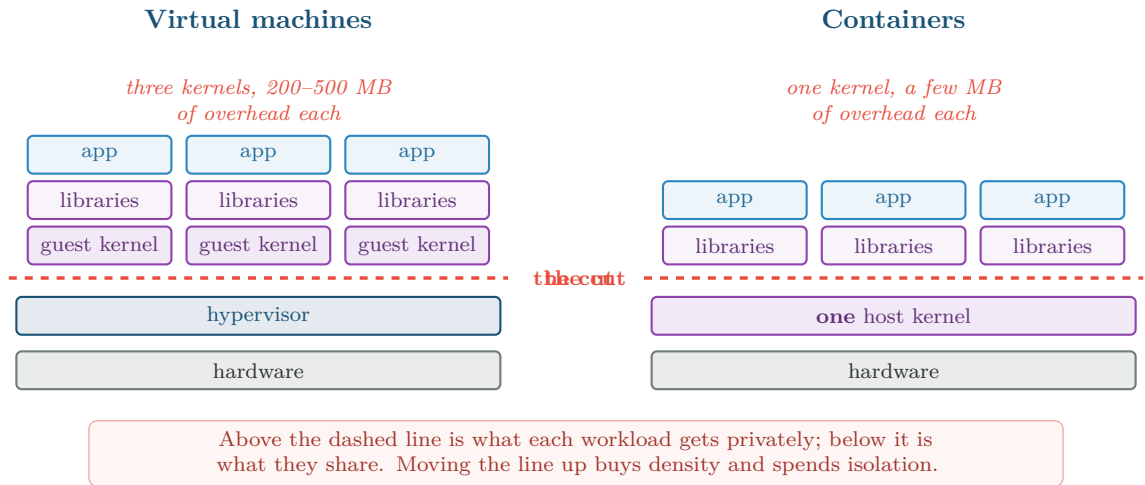


Figure 8.1: The whole of Part III in one picture. Nothing else about containers is a new idea — only the height at which the stack is cut.

There is no “container” object in the Linux kernel. The kernel has namespaces, cgroups, capabilities and mounts; a container is a convention about using them together, and Docker’s contribution was the convention and the packaging, not the mechanism.

8.2 Namespaces

A namespace wraps one global system resource so that processes inside it see their own instance of it. Eight exist, added over eighteen years.

Namespace	Year	What it isolates	Consequence inside
mount	2002	The set of mount points	A private filesystem tree — this is the one that makes an image work
UTS	2006	Hostname and domain name	<code>hostname</code> returns something of its own
IPC	2006	System V IPC, POSIX message queues	Shared memory segments cannot be reached from outside
PID	2008	Process identifiers	The first process is PID 1, and the host’s processes are invisible
network	2009	Interfaces, routes, firewall rules, ports	Its own <code>lo</code> and <code>eth0</code> ; two containers may both bind port 80
user	2013	User and group identifiers	<code>root</code> inside can map to an unprivileged user outside — the basis of rootless containers
cgroup	2016	The cgroup hierarchy root	A container cannot see where it sits in the host’s resource tree
time	2020	The boot and monotonic clocks	Used mainly for checkpoint and restore

💡 PID 1 Is Not a Detail

Inside a PID namespace the first process is numbered 1, and the kernel treats PID 1 specially everywhere: it must reap orphaned children, and signals with default actions are *not* delivered to it unless it has installed a handler.

Two consequences bite in production. A container whose PID 1 is an ordinary application does not reap orphans, so a process that forks accumulates zombies until the PID limit is reached. And `docker stop` sends `SIGTERM`, which an application that installed no handler simply ignores — the runtime then waits ten seconds and sends `SIGKILL`, which is why so many containers take exactly ten seconds to stop and lose in-flight work.

Both are fixed by running a tiny init as PID 1, which is what `docker run -init` does. Knowing *why* is the difference between applying the flag and diagnosing the symptom.

⚠️ The user namespace is the one that changed the security story

Before user namespaces, `root` inside a container was `root` on the host: the same UID 0, with the same power over the shared kernel, and one escape was a total compromise.

A user namespace maps identifiers. UID 0 inside can be UID 100000 outside, so a process that believes it is root holds no privilege the host recognises. This is what makes **rootless containers** possible — an unprivileged user running containers with no daemon and no `setuid` helper, which is Podman's default and now an option in Docker.

It is not enabled by default in every runtime, and a container run as root without it is still a process running as real root beside your other services. Chapter 19 returns to this.

8.3 Control Groups

Namespaces decide what a container can *see*. They say nothing about how much it can *use*: a process in a private namespace can still consume every core and all the memory on the host.

Control groups supply the limits. Processes are arranged in a tree, and each node carries controllers — `cpu`, `memory`, `io`, `pids` — that account for and cap consumption. Version 2, now the default everywhere, replaced v1's separate per-controller hierarchies with a single unified tree.

💡 What a Limit Does When It Is Reached

The two important limits fail in completely different ways, and conflating them is a common source of mystification.

CPU is throttled. A container limited to 0.5 of a core that asks for more is simply scheduled less: it runs at half speed. Nothing fails. You see it as latency, and in `cpu.stat` as a rising `throttled_usec`.

Memory is killed. A container that exceeds its memory limit has a process killed by the OOM killer — usually the largest, usually the one you cared about. There is no graceful degradation, because there is nothing to degrade: the page is needed or it is not.

This is the same asymmetry as Chapter 6: time is divisible and memory is not. It is why the commonest container incident is an application restarting every few hours with exit code 137 — which is $128 + 9$, signal 9, the OOM kill — and why memory limits deserve far more thought than CPU limits.

8.4 The Image, and the Union Filesystem

A container needs a root filesystem. Giving each one a private copy would cost hundreds of megabytes apiece and lose most of the density the chapter has been claiming.

Overlayfs solves it by stacking directories. Several read-only **lower** layers are presented as one tree, with a single writable **upper** layer on top. A read finds the file in the topmost layer that has it. A write triggers a **copy-up**: the file is copied into the upper layer and modified there, leaving the shared layer untouched.

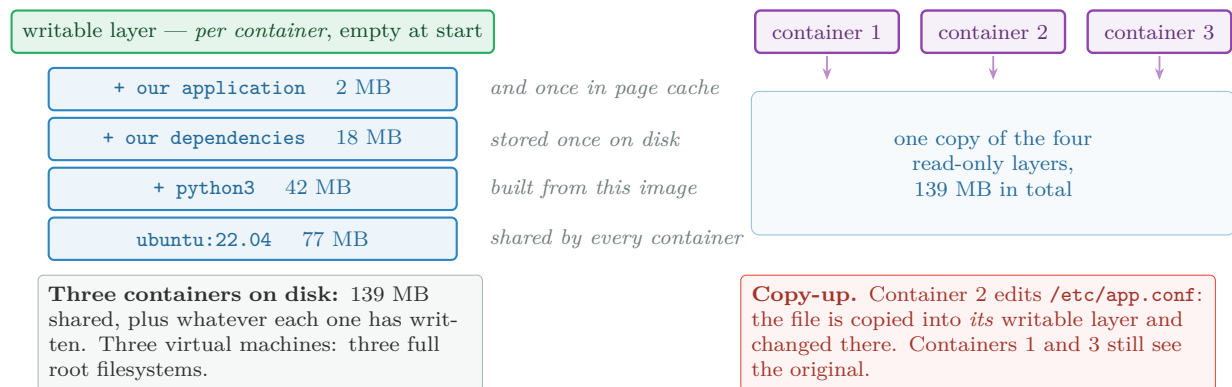


Figure 8.2: Overlayfs. The layers are why pulling a second image from the same base downloads almost nothing, and why a thousand containers of one image cost little more on disk than one.

Worked Example: Density and Start-Up, Costed

A host of 64 cores and 512 GB must run as many copies as possible of one service that needs 256 MB of memory. Compare virtual machines with containers.

Step 1 — memory per instance, as a virtual machine.

- the application: 256 MB;
- a guest Linux that can run it: 512 MB is a realistic floor;
- per-guest hypervisor overhead — the VMCS, the EPT tables, the device models of Chapter 7: call it 100 MB.

$$256 + 512 + 100 = 868 \text{ MB per guest}$$

Step 2 — memory per instance, as a container.

- the application: 256 MB;
- no guest kernel — the host's is shared;
- runtime overhead: a few MB. Call it 4 MB.

$$256 + 4 = 260 \text{ MB per container}$$

Step 3 — how many fit. Reserve 32 GB for the host, leaving 480 GB:

$$\text{VMs} = \frac{480 \times 1024}{868} = 566 \quad \text{containers} = \frac{480 \times 1024}{260} = \mathbf{1,890}$$

A ratio of **3.3 : 1** — and note it is not the hundredfold figure containers are often credited with, because the *application's* 256 MB dominates both sides. The container advantage is large exactly when the workload is small.

Step 4 — the disk, where the ratio is larger. 566 guests each with a 2 GB root disk is 1.1 TB, even thin-provisioned. 1,890 containers from one 139 MB image share that image: 139 MB plus each container's writes. Call it 10 MB each:

$$139 + 1,890 \times 10 = 19 \text{ GB, against } 1,100 \text{ GB} \approx \mathbf{58 : 1}$$

Step 5 — start-up, where it is larger still. A guest boots a kernel, initialises virtual hardware and starts an init system: 20–40 s. A container is `clone()` with some flags, a mount and an `execve`: 50–200 ms.

$$\frac{30 \text{ s}}{0.1 \text{ s}} = 300 : 1$$

Step 6 — what the three ratios are actually telling you. Memory density, 3:1, is the least impressive number and the one usually quoted. The ones that changed how software is deployed are disk and start-up: when an instance costs 10 MB and starts in a tenth of a second, you can afford to run one per request, scale on demand, and redeploy continuously. Chapter 10 is what people built once that became true.

⚠ What the ratios cost: one kernel, one failure

Every container on that host shares one kernel. A kernel panic takes all 1,890 down; a kernel privilege-escalation bug gives any one of them the host. The Linux system-call interface is around 350 calls wide and every one is reachable from inside a container, where a virtual machine’s guest would have to escape its own kernel first and then the hypervisor.

That is the trade, stated numerically: 3× the density, 300× the start-up speed, and an isolation boundary enforced by software rather than by the processor. Chapter 11 is the chapter where people decline to choose.

! Common Pitfalls

- **Calling a container a lightweight virtual machine.** There is no virtual machine and no guest kernel. The phrase predicts the wrong things about security, about kernel modules, and about what `uname` reports inside.
- **Expecting a container to run a different kernel.** It cannot. A container “running CentOS” on an Ubuntu host is CentOS *user space* on Ubuntu’s kernel. Nothing needing a specific kernel version or a kernel module will work.
- **Running an application directly as PID 1 and wondering about zombies or ten-second stops.** Use an init, and know why.
- **Setting CPU limits carefully and memory limits by guesswork.** CPU throttles; memory kills. Exit code 137 is the OOM killer and it is the commonest container incident there is.
- **Writing large amounts of data to the container’s writable layer.** Copy-up through overlaysfs is slow, and the layer disappears with the container. Mount a volume.
- **Assuming a container is isolated because it has a private namespace.** Namespaces hide things; they do not reduce the kernel attack surface. Chapter 19 separates the two properly.

🔧 Try It Yourself: Building a Container by Hand

No Docker. Builds an isolated process tree from the kernel primitives, so that the next chapter’s tooling is recognisable rather than magical. Needs a Linux host and root. Expect thirty minutes.

```
#!/usr/bin/env bash
# ch08_byhand.sh -- namespaces, cgroups and overlaysfs, without a runtime.
set -euo pipefail
ROOT=/tmp/vsroot

# --- 1. What namespaces does an ordinary process already have? -----
```

```

# Every process is in a namespace; a container just gets different ones.
ls -l /proc/self/ns/

# --- 2. A root filesystem to be the container's world. -----
sudo apt-get install -y debootstrap util-linux
sudo mkdir -p "$ROOT"
sudo debootstrap --variant=minbase jammy "$ROOT" \
    http://archive.ubuntu.com/ubuntu/

# --- 3. Enter new namespaces. This is the whole mechanism. -----
# --pid --mount --uts --ipc --net: one flag per namespace from the table.
sudo unshare --pid --mount --uts --ipc --net --fork --mount-proc \
    chroot "$ROOT" /bin/bash -c '
    hostname container-1
    mount -t proc proc /proc
    echo "--- inside ---"
    hostname           # the UTS namespace
    ps aux             # the PID namespace: we are PID 1, host invisible
    ip addr            # the network namespace: only lo, and it is down
    exit'

# --- 4. Prove the host is unaffected. -----
hostname ; ps aux | wc -l

# --- 5. Limit it with a control group. -----
# cgroup v2: one unified tree under /sys/fs/cgroup.
sudo mkdir -p /sys/fs/cgroup/vsdemo
echo "+cpu +memory" | sudo tee /sys/fs/cgroup/cgroup.subtree_control
echo "50000 100000" | sudo tee /sys/fs/cgroup/vsdemo/cpu.max    # half a core
echo "100M"         | sudo tee /sys/fs/cgroup/vsdemo/memory.max

# Put a busy process in it and watch it be throttled, not killed.
sudo bash -c 'echo $$ > /sys/fs/cgroup/vsdemo/cgroup.procs; \
    timeout 10 yes > /dev/null' &

sleep 11
cat /sys/fs/cgroup/vsdemo/cpu.stat | grep throttled

# --- 6. Now cross the memory limit, and watch it be killed. -----
# CPU throttles; memory kills. This is the asymmetry of section 8.3.
sudo bash -c 'echo $$ > /sys/fs/cgroup/vsdemo/cgroup.procs; \
    head -c 200M /dev/zero | tail -c 1' || \
    echo "killed, exit $? -- 137 is 128+9, the OOM killer"
dmesg | tail -3 | grep -i 'out of memory' || true

# --- 7. Share one root between two containers with overlayfs. -----
sudo mkdir -p /tmp/ovl/{upper1,upper2,work1,work2,m1,m2}
for i in 1 2; do
    sudo mount -t overlay overlay \
        -o lowerdir="$ROOT",upperdir=/tmp/ovl/upper$i,workdir=/tmp/ovl/work$i \
        /tmp/ovl/m$i
done
# Write in one; the other and the shared lower layer are untouched.
sudo touch /tmp/ovl/m1/etc/only-in-one
ls /tmp/ovl/m1/etc/only-in-one
ls /tmp/ovl/m2/etc/only-in-one 2>&1 | tail -1
sudo du -sh /tmp/ovl/upper1 /tmp/ovl/upper2 "$ROOT"

# --- 8. Tear down. -----
for i in 1 2; do sudo umount /tmp/ovl/m$i || true; done
sudo rmdir /sys/fs/cgroup/vsdemo || true
sudo rm -rf /tmp/ovl "$ROOT"

```

What to notice. Step 3 is a container. There is no daemon, no image format and no runtime — one command, five flags and a directory. Everything Chapter 9 adds is packaging and distribution on top of exactly this.

🔍 Checkpoint — Test Yourself

1. A container restarts every few hours with exit code 137. Say what that code means, which limit is responsible, and why the symptom is a kill rather than a slowdown.
2. A team wants to run a container that loads a kernel module for a custom device. Explain why this cannot work as stated, and give the two options available to them.
3. Three containers run from the same 400 MB image, and each writes 50 MB of logs into its own filesystem. How much disk is used, and what single change would stop the logs counting at all?

📋 Chapter Summary

- A container cuts the stack above the kernel instead of below it: one kernel, many private views of it. Everything else follows from that.
- Eight namespaces isolate what a process can see — mount, UTS, IPC, PID, network, user, cgroup, time. The user namespace is what makes rootless containers possible.
- PID 1 is special: it must reap orphans and ignores default-action signals. Run an init unless you have handled both.
- Control groups bound what a process tree may use. CPU limits throttle; memory limits kill. The asymmetry is the same one as in Chapter 6.
- Overlayfs stacks read-only layers under one writable layer, with copy-up on write, so many containers share one image.
- Measured against virtual machines: roughly 3:1 on memory, 58:1 on disk, 300:1 on start-up. The last two are what changed how software is deployed.
- The cost is a single shared kernel and a 350-call attack surface.

🔧 Review Questions

1. (MCQ) The namespace that gives a container its own filesystem tree is: (a) PID; (b) mount; (c) UTS; (d) user.
2. (MCQ) A container exceeding its memory limit is: (a) throttled; (b) swapped by the host; (c) has a process killed by the OOM killer; (d) given memory from another container.
3. (Short) Explain copy-up, and say what it implies about writing large files inside a container.
4. (Short) Why can a container not run a different kernel from its host, and what is actually different about a “CentOS container” on an Ubuntu host?
5. (Short) State the two behaviours that make PID 1 special, and the symptom each produces when ignored.
6. (Applied) Repeat the worked example for a service needing 2 GB rather than 256 MB, on the same host. Give the three ratios and explain why the memory ratio changed so much while the start-up ratio did not.
7. (Applied) A team proposes moving forty internal services from virtual machines to containers on shared hosts. Write the three questions you would ask before agreeing, and say what answer to each would make you recommend Chapter 11’s approach instead.

Docker and the Container Ecosystem

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what Docker added to the kernel features of Chapter 8, and why that — rather than isolation — is what changed the industry.
- **Describe** an image as a stack of content-addressed layers plus a manifest, and **explain** what a digest guarantees.
- **Name** the components of the OCI runtime stack and **say** what each one does.
- **Order** the instructions in a Dockerfile so that the build cache is not needlessly invalidated, and **quantify** what the wrong order costs.
- **Write** a Dockerfile that is multi-stage, pinned and non-root, and **justify** each of those three properties.

🔗 Before You Start

Chapter 8 entire — namespaces, cgroups and especially overlayfs, since an image is layers and a layer is an overlayfs directory. The lab in that chapter built a container by hand; this one is about everything that was missing from it.

📖 Key Terms in This Chapter

image • layer • manifest • image index • digest • content addressing • tag • registry • Dockerfile • build cache • cache invalidation • multi-stage build • distroless • OCI image specification • OCI runtime specification • containerd • runc • shim • BuildKit • SBOM • volume

9.1 What Docker Actually Added

Everything in Chapter 8 existed before Docker. Namespaces had been in Linux for a decade, cgroups for six years, and LXC had been packaging both since 2008. Google was running billions of containers a week.

Docker's 2013 release added almost nothing to the kernel. What it added was a *format* and a *workflow*:

- a declarative build file producing a reproducible artefact;
- an image format that is layered, so sharing and incremental transfer are free;
- a registry protocol, so an image could be published and pulled by name;
- one command to run any of it.

💡 The Problem It Solved Was Not Isolation

LXC already isolated things perfectly well. What it did not do was let a developer hand an operator something that would behave identically on both machines.

The before-and-after is the whole story. Before: a wiki page saying “install Python 3.9, these fourteen packages, this version of libxml2, set these three environment variables” — a procedure that worked on the author’s laptop and drifted everywhere else. After: a file that *is* the procedure, executed in a defined base, producing an artefact identified by a hash that either matches or does not.

This is **encapsulation** — the third motive of Chapter 1 — applied to an application rather than a machine, and it is why the format won against better-engineered rivals. The density of Chapter 8 was discovered to be a bonus afterwards.

9.2 Images, Layers and Digests

An image is not a file. It is a **manifest** — a small JSON document — listing a configuration object and an ordered set of layers, each referenced by the SHA-256 hash of its contents.

Content Addressing, Digest and Tag

Every layer, configuration and manifest is identified by a **digest**: the SHA-256 of the bytes themselves. Content addressing has three consequences the whole ecosystem rests on:

- **Deduplication is automatic.** Two images sharing a base share its layer digest, so it is stored and transferred once.
- **Integrity is free.** If the bytes are intact the digest matches; if anything changed, it does not.
- **Identity is exact.** `ubuntu@sha256:9b8c...` is one specific image, for ever.

A **tag** — `ubuntu:22.04` — is not identity. It is a mutable pointer that the publisher may move whenever they like, and routinely does. Two builds a week apart from the same tag may use different bytes, which is precisely the reproducibility problem Docker was invented to solve, reappearing one level up. Pin digests where reproducibility matters.

9.3 The OCI Stack

By 2015 the format mattered enough that it needed an owner who was not a company. Docker donated it, and the **Open Container Initiative** now maintains three specifications — image, runtime and distribution — which is why containers built with Docker run under containerd, CRI-O, Podman and Kubernetes without anyone negotiating.

The implementation split up at the same time.

⚠ “Kubernetes deprecated Docker” was widely misread

In 2022 Kubernetes removed `dockershim`, the adapter that let the kubelet drive `dockerd`. Headlines announced that Docker was dead, and a great many teams rebuilt images they did not need to rebuild.

What was removed was one *adapter*. Kubernetes now talks to containerd directly — which is what `dockerd` was talking to anyway, so a layer was deleted rather than a capability. Images built with Docker are OCI images and run unchanged, because that is what standardising the format was for.

Worth remembering as a general lesson: in this ecosystem the *format* is the durable thing and the tooling is not.

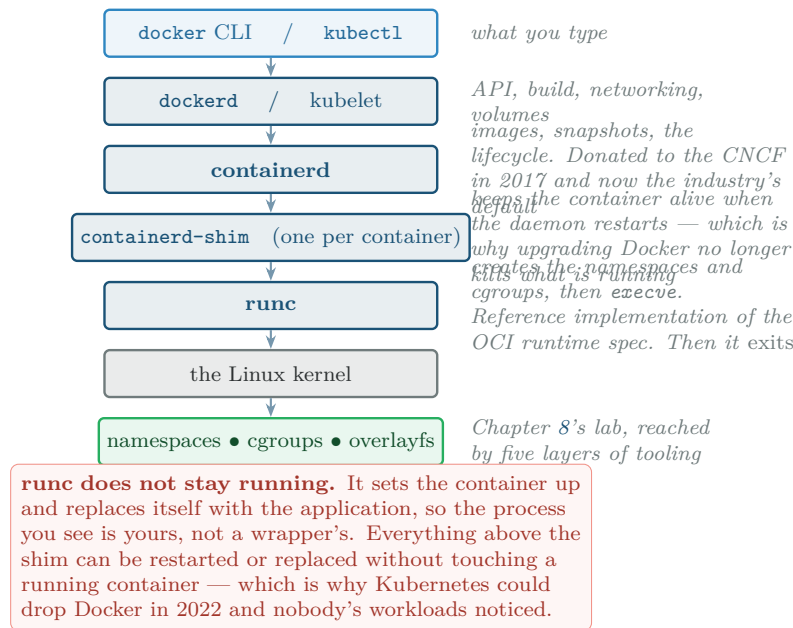


Figure 9.1: The runtime stack. Each layer was split out of the one above it as the ecosystem matured; the bottom two rows are the only ones that are not replaceable.

9.4 The Build Cache, and the Mistake Everyone Makes

Each instruction in a Dockerfile produces a layer. On rebuild, Docker reuses a cached layer if the instruction and all the layers beneath it are unchanged — and crucially, once one layer is invalidated, *every layer above it is rebuilt too*.

That single rule decides whether a build takes four seconds or ninety-five, and almost every slow build in existence gets it wrong the same way.

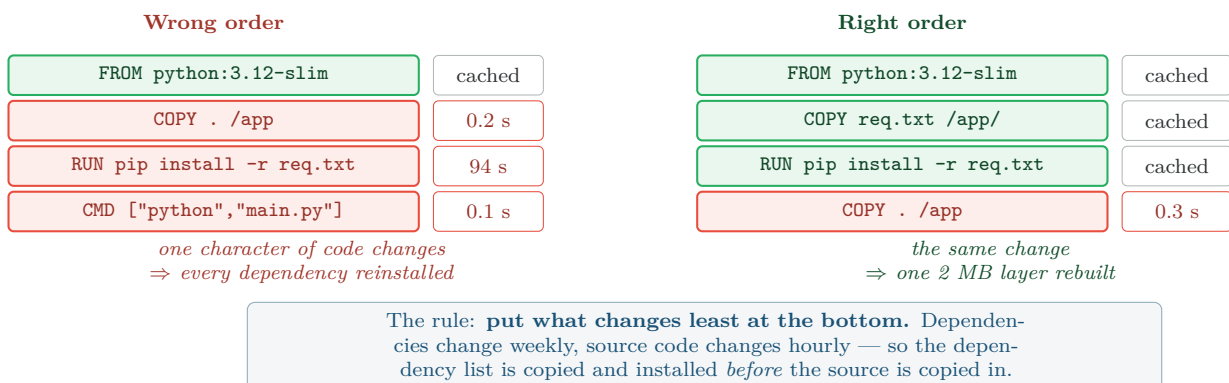


Figure 9.2: The same four instructions in two orders. Nothing else differs, and the rebuild costs differ by a factor of 300.

Worked Example: What the Wrong Order Costs a Team

A team of six rebuilds this image about 30 times a day between them. Measure the cost of the left-hand order against the right-hand one.

Step 1 — per-build time. From the figure:

$$\text{wrong} = 0.2 + 94 + 0.1 = 94.3 \text{ s} \quad \text{right} = 0.3 \text{ s}$$

Step 2 — per day.

$$30 \times 94.3 = 2,829 \text{ s} = \mathbf{47} \text{ minutes} \quad 30 \times 0.3 = 9 \text{ s}$$

Step 3 — the bytes pushed. A rebuilt layer must be uploaded; a cached one need not. The dependency layer is 180 MB and the application layer is 2 MB:

$$\text{wrong} = 30 \times 182 = 5,460 \text{ MB} = \mathbf{5.3} \text{ GB a day}$$

$$\text{right} = 30 \times 2 = 60 \text{ MB a day}$$

Step 4 — and the pulls. Every deployment target pulls what changed. With 12 nodes and 30 deployments:

$$\text{wrong} = 12 \times 30 \times 182 \text{ MB} = \mathbf{65} \text{ GB a day} \quad \text{right} = 12 \times 30 \times 2 = 0.7 \text{ GB}$$

Step 5 — read the result. Reordering two lines saves 47 minutes of engineer waiting, 5.3 GB of upload and 64 GB of download, every day. Over a working year that is roughly 190 hours and 17 TB.

What the arithmetic does not capture. The 47 minutes are not spread evenly: they are 94 seconds at a time, which is exactly long enough to lose concentration and not long enough to do anything else. That cost is larger than the number, and it is the real argument for getting the order right.

9.5 A Dockerfile That Holds Up

Three properties, each with a reason.

💡 Multi-Stage: Build Tools Are Not Runtime Dependencies

A compiler, a package manager and a full toolchain are needed to *build* and are liabilities at *run time*: they are weight, and they are attack surface for anyone who gets a shell.

A **multi-stage build** uses one stage with the toolchain and a second containing only the artefact:

- A Go service: builder stage with the Go toolchain, about 900 MB; final stage **FROM scratch** with one static binary, about 15 MB. A **60:1** reduction.
- A Python service: compilers for the C extensions in the builder stage; only the slim base and the installed packages in the final one. Typically 1.1 GB down to 210 MB.

Distroless images go further — a language runtime and no shell, no package manager, no **ls**. An attacker with code execution has nothing to run. The cost is that you cannot **exec** into it to debug, which is a real operational trade and not everyone's.

⚠️ Pin the base, and know what `:latest` promises

FROM ubuntu:latest promises nothing. It resolves to whatever the publisher last pushed, so a build that worked this morning may not this afternoon, and the failure appears in CI with no change in your repository.

FROM ubuntu:22.04 is better and is still a moving pointer — the tag is republished with security updates, which you usually want. **FROM ubuntu@sha256:9b8c...** is exact and never changes, which means you must update it deliberately to get security fixes.

The usual resolution: pin digests in production manifests, and run an automated job that proposes digest updates as reviewable changes. Choosing between reproducibility and patching by accident is the only wrong answer.

💡 Do Not Run as Root

A container process running as UID 0 is, without a user namespace (Chapter 8), really root — and a kernel bug then yields the host rather than a container.

USER in the Dockerfile costs nothing and removes the whole class. Combine it with a read-only root filesystem and dropped capabilities at run time, and an attacker who achieves code execution has an unprivileged process in a filesystem it cannot write. Chapter 19 covers the rest of the hardening set.

❗ Common Pitfalls

- **Copying source before installing dependencies.** The worked example above. Re-ordering two lines is the single highest-return change available in most repositories.
- **Running apt-get update in a separate layer from install.** The update layer caches, so a later install can fetch a package list months out of date. Always one RUN.
- **Leaving secrets in a build.** Deleting a file in a later layer does not remove it — the earlier layer still contains it, and anyone who pulls the image can read it. Use build secrets or a mount.
- **Treating a tag as an identity.** It is a mutable pointer. Only a digest identifies an image.
- **Storing data in the container.** The writable layer dies with the container and copy-up is slow. Use a volume.
- **Equating image size with security.** A 50 MB image with an unpatched library is worse than a 400 MB patched one. Size reduces attack surface and transfer cost; it does not replace patching.

🔧 Try It Yourself: Measuring the Build Cache, and Fixing It

Builds the same application both ways and measures the difference; then makes it multi-stage and non-root. Needs Docker. Expect thirty minutes.

```
#!/usr/bin/env bash
# ch09_build.sh -- the build cache, measured, then a sound image.
set -euo pipefail
mkdir -p ~/vsapp && cd ~/vsapp

# --- 1. A small application with real dependencies. -----
cat > requirements.txt <<'REQ'
flask==3.0.3
requests==2.32.3
REQ
cat > main.py <<'PY'
from flask import Flask
app = Flask(__name__)

@app.route("/")
def index():
    return "virtual systems and services\n"
PY

# --- 2. The wrong order: source copied before dependencies. -----
cat > Dockerfile.wrong <<'DF'
```

```

FROM python:3.12-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "-m", "flask", "run", "--host=0.0.0.0"]
DF
docker build -q -f Dockerfile.wrong -t vsapp:wrong . > /dev/null

# --- 3. The right order: dependency list first, then source. -----
cat > Dockerfile.right <<'DF'
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt
COPY . /app
CMD ["python", "-m", "flask", "run", "--host=0.0.0.0"]
DF
docker build -q -f Dockerfile.right -t vsapp:right . > /dev/null

# --- 4. Change one character of source and time both rebuilds. -----
sed -i 's/services/services!/' main.py
echo "=== wrong order ===" ; time docker build -q -f Dockerfile.wrong \
    -t vsapp:wrong . > /dev/null
echo "=== right order ===" ; time docker build -q -f Dockerfile.right \
    -t vsapp:right . > /dev/null

# --- 5. Look at the layers, and at what is shared. -----
docker history vsapp:right --format '{{.Size}}\t{{.CreatedBy}}' | head -8
docker image inspect vsapp:right \
    --format '{{json .RootFS.Layers}}' | tr ', ' '\n'
# Both images share the python:3.12-slim layers: content addressing at work.

# --- 6. Multi-stage and non-root. -----
cat > Dockerfile.good <<'DF'
FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt /app/
RUN pip install --no-cache-dir --target=/deps -r requirements.txt

FROM python:3.12-slim
RUN useradd --create-home --uid 10001 app
WORKDIR /app
COPY --from=build /deps /deps
COPY --chown=app:app . /app
ENV PYTHONPATH=/deps
USER app
CMD ["python", "-m", "flask", "run", "--host=0.0.0.0"]
DF
docker build -q -f Dockerfile.good -t vsapp:good . > /dev/null
docker images vsapp --format '{{.Tag}}\t{{.Size}}'

# --- 7. Prove it is not running as root, and cannot write its root. ---
docker run --rm vsapp:good id
docker run --rm --read-only vsapp:good sh -c 'touch /nope' 2>&1 | tail -1

# --- 8. A secret deleted in a later layer is still in the image. -----
printf 'FROM alpine\nRUN echo hunter2 > /k && rm /k\n' > Dockerfile.leak
docker build -q -f Dockerfile.leak -t vsapp:leak . > /dev/null
docker save vsapp:leak | tar -xO 2>/dev/null | grep -ac hunter2 || \
    echo "search the layer tarballs: the secret is in one of them"

# --- 9. Tear down. -----

```

```
docker rmi -f vsapp:wrong vsapp:right vsapp:good vsapp:leak || true
```

What to notice. Step 4 gives you the worked example's two numbers on your own machine. Step 8 is the one worth remembering: `rm` in a Dockerfile removes a file from the *view*, not from the image, and everything you ever put in a layer is still there for whoever pulls it.

? Checkpoint — Test Yourself

1. An image is built from `FROM node:20` on Monday and again on Friday with no repository changes, and the two behave differently. Explain how, and give the change that prevents it.
2. A Dockerfile copies a private key, uses it to clone a repository, then deletes it in the same `RUN`. Is the key in the image? What if the delete is a separate `RUN`?
3. A team's CI rebuilds a 1.2 GB image on every commit and deploys it to 40 nodes. Using the worked example's method, estimate what reordering the Dockerfile would save per day in transfer alone, stating your assumptions.

✓ Chapter Summary

- Docker added a build file, a layered image format, a registry protocol and one command — not isolation, which already existed. The problem it solved was reproducible delivery.
- An image is a manifest plus content-addressed layers. A digest is identity and integrity; a tag is a mutable pointer and neither.
- The OCI specifications made the format portable, which is why the tooling could be replaced underneath it without breaking anything.
- The runtime stack is CLI, daemon, containerd, shim, runc, kernel. `runc` exits after setting the container up; the shim is why restarting the daemon does not kill running containers.
- A cache miss invalidates every layer above it. Put what changes least at the bottom: dependency lists before source. The wrong order cost the worked example's team 47 minutes and 70 GB a day.
- Multi-stage, pinned and non-root are the three properties a Dockerfile should have. A file deleted in a later layer is still in the image.

✍ Review Questions

1. (MCQ) A digest identifies an image by: (a) its tag; (b) the SHA-256 of its contents; (c) its creation date; (d) its registry URL.
2. (MCQ) `runc`, after setting up a container: (a) supervises it until exit; (b) exits, having replaced itself with the application; (c) becomes the container's PID 1 wrapper; (d) restarts the daemon.
3. (Short) State the build-cache rule in one sentence, and give the ordering principle that follows from it.
4. (Short) Explain why a secret removed in a later layer is still recoverable from the image.
5. (Short) Give the two reasons a multi-stage build is preferred, and the operational cost of going all the way to distroless.
6. (Applied) You inherit a repository whose image is 1.4 GB, builds in four minutes, runs as root and uses `FROM python:latest`. Write the four changes you would make in priority order, with the measurable improvement you expect from each.

7. **(Applied)** A regulated client requires that any deployed image can be reproduced exactly two years later. Describe what you would pin, what you would store, and what you would automate so that security patching still happens.

Container Orchestration with Kubernetes

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** the control loop and **distinguish** declarative from imperative management.
- **Name** the control-plane and node components and **trace** what each does when a Deployment is created.
- **Describe** a Pod, a Deployment and a Service, and **say** which problem each one solves.
- **Write** requests and limits for a workload, and **predict** the scheduling and eviction behaviour that follows.
- **Diagnose** a workload that will not schedule or will not stay running, from the resource arithmetic and the probe configuration.

🔗 Before You Start

Chapter 8 (namespaces, cgroups, and above all that CPU limits throttle while memory limits kill — Kubernetes inherits both) and Chapter 9 (images, digests, and the OCI runtime stack the kubelet drives).

🔑 Key Terms in This Chapter

orchestration • declarative • reconciliation • control loop • desired state • API server • etcd • scheduler • controller manager • kubelet • kube-proxy • Pod • sidecar • ReplicaSet • Deployment • Service • ClusterIP • Ingress • request • limit • QoS class • eviction • liveness probe • readiness probe • taint • toleration

10.1 The Problem Containers Created

Chapter 9 gave us an artefact that starts in a tenth of a second and costs ten megabytes. Chapter 8 showed a host could hold two thousand of them. Neither says anything about the questions that follow immediately:

Which host should this container run on? What happens when it dies at three in the morning? When the host dies? How does traffic find it, given that its address changes every restart? How do you replace all sixty copies with a new version without dropping a request — and undo that in forty seconds when it is wrong?

Answering those by hand is possible for ten containers and impossible for a thousand. **Orchestration** is the automation of those answers, and Kubernetes is the one that won.

📖 Declarative Management and the Control Loop

You do not tell Kubernetes what to *do*. You tell it what you *want*, and it works continuously to make reality match.

A **controller** is a loop that:

- reads the **desired state** — what you declared;
- observes the **actual state** — what exists;

- takes one step to reduce the difference;
- repeats, for ever.

This is **reconciliation**, and it is the single idea the whole system is built from. “Run five copies” is not a command executed once; it is a condition continuously enforced. Kill one and a controller notices the difference and creates another — not because anything handled a failure, but because four is not five.

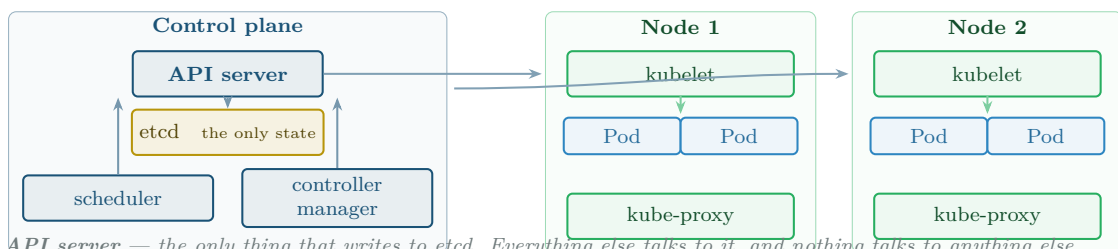
💡 Why Declarative Beats Imperative Here

An imperative system executes instructions: *start a container, stop that one*. It must therefore anticipate every way reality can diverge — a node rebooting, a network partition healing, a half-finished rollout — and have an instruction for each. The failure modes multiply faster than anyone can enumerate them.

A declarative system has one failure mode: the difference between desired and actual is not shrinking. Every problem reduces to that, and every recovery is the same loop running again. It is why a Kubernetes cluster can be left alone over a weekend and why `kubectl apply` is safe to run twice.

The cost is that you give up knowing exactly *when* things happen. The system converges; it does not promise a schedule. Operators used to imperative tooling find this unnerving, and the answer is to watch the difference, not the steps.

10.2 What the Cluster Is Made Of



API server — the only thing that writes to *etcd*. Everything else talks to it, and nothing talks to anything else.
etcd — the cluster's entire state, in one consistent key-value store. Lose it and you have lost the cluster.
scheduler — decides which node an unassigned Pod belongs on. It only writes that decision down; it does not start anything.
controller manager — the reconciliation loops: Deployments, ReplicaSets, nodes, endpoints.
kubelet — the only component that touches containers. Reads what should be on its node, drives containerd, reports back.
kube-proxy — programmes the node's routing so a Service address reaches a healthy Pod.

Figure 10.1: A cluster. The arrows all point at the API server, which is the architectural claim: components do not coordinate with each other, they each reconcile against one shared record.

💡 Trace a Deployment Through the Loops

`kubectl apply` a Deployment asking for three replicas. Nothing is started by the thing that handled your request.

- The **API server** validates it and writes it to **etcd**. Your command returns. Nothing is running.
- The **Deployment controller** notices a Deployment with no ReplicaSet and creates one.
- The **ReplicaSet controller** notices a ReplicaSet wanting three Pods and zero existing, and creates three Pod objects — with no node assigned.
- The **scheduler** notices three unassigned Pods, scores the nodes on free resources and constraints, and writes a node name into each. It still has not started anything.

- Each **kubelet** notices a Pod assigned to its node, pulls the image and asks containerd to run it.
- The **endpoints controller** notices Pods that match a Service’s selector and adds their addresses; **kube-proxy** programmes the routing.

Six independent loops, none of which called another. Each noticed a difference and reduced it. Now kill a Pod: step 3’s controller sees two where three are wanted, and the same sequence runs from there. *Failure recovery is not a feature; it is the ordinary behaviour of the system.*

10.3 Pods, Deployments and Services

Object	What it is	The problem it solves
Pod	One or more containers sharing a network namespace, IP address and storage volumes. The smallest schedulable unit	Some containers must be co-located and share <code>localhost</code> — an application and its log shipper, or a sidecar proxy. You almost never create one directly
ReplicaSet	Keeps n identical Pods running	The reconciliation loop for “how many”. You almost never create one directly either
Deployment	Manages ReplicaSets to roll out and roll back versions	Changing the image without an outage. It creates a second ReplicaSet, shifts Pods across gradually, and keeps the old one so a rollback is one command
Service	A stable virtual IP and DNS name in front of a changing set of Pods	Pod addresses change on every restart. A ClusterIP does not, and kube-proxy keeps its backend list current
Ingress	HTTP routing from outside the cluster to Services	A Service is reachable inside the cluster; this is how the world gets in, with host and path rules and TLS

⚠ A Pod is not a container, and the difference bites

A Pod is the unit of *scheduling* and of *IP address*. Every container in it runs on the same node, shares one network namespace — so they reach each other on `localhost` and cannot both bind port 8080 — and can share volumes.

The error this produces is putting an application and its database in one Pod because they belong together. They do not: they scale differently, fail differently and are updated differently, and now they cannot be scheduled separately or scaled separately. The test is whether the two must be on the same machine and share a lifetime. A log shipper must. A database must not.

10.4 Requests, Limits and What the Scheduler Does With Them

Every container may declare a **request** and a **limit** for CPU and memory. They do completely different jobs and conflating them is the commonest source of Kubernetes capacity trouble.

Request and Limit

The **request** is what the **scheduler** uses. It reserves that much on the node, and a node whose remaining allocatable capacity is less than the request will not be chosen. The request is a claim on *placement* and is reserved whether or not the container uses it.

The **limit** is what the **kernel** enforces, through the cgroups of Chapter 8. CPU above the limit is throttled; memory above it is an OOM kill.

The gap between them is where a workload may burst, and where the cluster is oversubscribed. A Pod requesting 100m of CPU with a 2-core limit is scheduled as if it were tiny and may consume twenty times that — from capacity the scheduler has promised to somebody else.

Worked Example: Why This Pod Will Not Schedule

A cluster has four nodes of 16 cores and 64 GB. A team reports that their new Deployment sits in **Pending** and asks for more nodes. Decide whether they need any.

What is declared. 20 replicas, each requesting 2 cores and 8 GB.

Step 1 — what the nodes actually offer. Allocatable capacity is not total capacity: the kubelet, the container runtime and the operating system are reserved first. Typically about 10% of CPU and 15% of memory:

$$\text{per node} \approx 16 \times 0.90 = 14.4 \text{ cores}, \quad 64 \times 0.85 = 54.4 \text{ GB}$$

$$\text{cluster} = 4 \times 14.4 = 57.6 \text{ cores}, \quad 4 \times 54.4 = 217.6 \text{ GB}$$

Step 2 — what is already running. Existing workloads request 30 cores and 120 GB. Remaining:

$$57.6 - 30 = 27.6 \text{ cores} \quad 217.6 - 120 = 97.6 \text{ GB}$$

Step 3 — what the new Deployment asks for.

$$20 \times 2 = 40 \text{ cores} \quad 20 \times 8 = 160 \text{ GB}$$

Both exceed what is left. It cannot schedule, and the team is right that something must change.

Step 4 — how many will fit? The scheduler places Pods one at a time, so some will run. The binding constraint is memory:

$$\left\lfloor \frac{97.6}{8} \right\rfloor = 12 \text{ Pods}$$

but Pods cannot be split across nodes, so compute it per node. If the existing load is even, each node has $54.4 - 30 = 24.4$ GB free, giving $\lfloor 24.4/8 \rfloor = 3$ Pods per node = **12**. Eight stay **Pending** — which matches what they are seeing.

Step 5 — are the requests honest? This is the question nobody asks, and it is usually the answer. Measured over the previous week the same application used 0.3 cores and 1.2 GB at the 95th percentile. The request of 2 cores and 8 GB is roughly **seven times** its actual demand.

Set requests to p95 plus a margin — say 0.5 cores and 2 GB — and limits somewhat above:

$$20 \times 0.5 = 10 \text{ cores} \quad 20 \times 2 = 40 \text{ GB}$$

Against 27.6 cores and 97.6 GB free, all twenty schedule with room to spare.

Step 6 — the recommendation. No new nodes. The cluster was not full; it was *reserved*. A request is a promise the scheduler keeps whether the workload uses it or not, so inflated requests buy hardware that then sits idle — the server sprawl of Chapter 1, re-created inside a cluster that was bought to prevent it.

💡 QoS Classes: Who Gets Evicted First

From the requests and limits, Kubernetes derives a **QoS class** that decides what happens when a node runs out of memory.

- **Guaranteed** — every container has limits equal to requests. Evicted last.
- **Burstable** — requests set, limits higher or absent. Evicted in the middle, worst offender against its request first.
- **BestEffort** — nothing set at all. Evicted first, always.

So omitting requests and limits is not neutral — it is choosing to be killed first. This surprises teams who leave them out to “keep it simple”, and the symptom arrives weeks later as a workload that dies only when the cluster is busy.

💡 Liveness and Readiness Are Not the Same Question

Two probes, routinely confused, with opposite consequences.

A **readiness** probe asks *should traffic go here?* Failing it removes the Pod from its Service’s endpoints. Nothing is killed; the Pod is simply left alone until it recovers. Use it for warm-up, for a lost database connection, for shedding load.

A **liveness** probe asks *is this process broken beyond recovery?* Failing it **kills the container**. Use it only for genuine deadlock.

The classic outage: a liveness probe that hits an endpoint which also checks the database. The database slows, every replica fails liveness, Kubernetes kills all of them, they restart, reconnect in a thundering herd and fail again. A readiness probe there would have removed them from rotation and let them recover. *If in doubt, use readiness.*

❗ Common Pitfalls

- **Setting requests from the limit, or from a guess.** The scheduler reserves the request. Inflated requests waste the cluster exactly as oversized virtual machines waste a host.
- **Omitting requests entirely.** That is BestEffort, which is evicted first. Not setting them is a decision.
- **Using a liveness probe where readiness is meant.** Liveness kills. The cascading restart above is the single most common self-inflicted Kubernetes outage.
- **Setting CPU limits aggressively.** CPU throttling is invisible in most dashboards and shows up as latency. Many teams set CPU requests and no CPU limit deliberately; check `throttled_usec` before deciding.
- **Putting unrelated containers in one Pod.** They now share a lifetime, a node and a scaling decision.
- **Using `:latest` in a manifest.** The rollout is not reproducible and a rollback may not roll anything back (Chapter 9).
- **Treating Kubernetes as a hypervisor.** It schedules containers on a shared kernel. Every isolation caveat in Chapter 8 still applies, and Chapter 11 is what to do about it.

🔧 Try It Yourself: A Cluster on One Laptop

Builds a real cluster with **kind** — a real API server, scheduler and kubelet; only the node count is a pretence — and produces each failure mode above on purpose. Expect forty-five minutes.

```
#!/usr/bin/env bash
# ch10_kind.sh -- reconciliation, scheduling and probes, observed.
```

```

set -euo pipefail

# --- 1. A three-node cluster in containers on this machine. -----
kind create cluster --name vss --config - <<'YAML'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
YAML
kubectl get nodes -o wide
# Each "node" is a container (Chapter 9) running a kubelet. Containers
# all the way down.

# --- 2. Declare a desired state and watch six loops reach it. -----
kubectl create deployment web --image=nginx:1.27 --replicas=3
kubectl get deploy,rs,pods -o wide
kubectl get events --sort-by=.lastTimestamp | tail -12
# Read the events bottom-up: scheduled, pulling, created, started.

# --- 3. Reconciliation: delete a Pod and time the replacement. -----
kubectl get pods -l app=web
kubectl delete pod -l app=web --field-selector status.phase=Running \
  --wait=false | head -1
sleep 3 ; kubectl get pods -l app=web
# Nothing "handled" the deletion. Two is not three, so a loop acted.

# --- 4. A Service: a stable address in front of changing Pods. -----
kubectl expose deployment web --port=80 --target-port=80
kubectl get svc web
kubectl get endpoints web          # the Pod IPs, maintained by a controller
kubectl delete pod -l app=web --wait=false | head -1
sleep 5 ; kubectl get endpoints web # different Pod IPs, same Service IP

# --- 5. Make a Pod unschedulable, and read why. -----
kubectl create deployment big --image=nginx:1.27
kubectl set resources deployment big --requests=cpu=8,memory=32Gi
kubectl get pods -l app=big
kubectl describe pod -l app=big | sed -n '/Events/, $p' | tail -6
# "0/3 nodes are available: Insufficient cpu" -- the worked example,
# reported by the scheduler itself.

# --- 6. QoS classes, derived from what you declared. -----
kubectl run guaranteed --image=nginx:1.27 \
  --overrides='{ "spec": { "containers": [ { "name": "c", "image": "nginx:1.27",
    "resources": { "requests": { "cpu": "100m", "memory": "64Mi" },
    "limits": { "cpu": "100m", "memory": "64Mi" } } } ] } }'
kubectl run besteffort --image=nginx:1.27
sleep 5
kubectl get pod guaranteed besteffort \
  -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass

# --- 7. The cascading-restart outage, on purpose. -----
# A liveness probe pointing at something that will fail.
kubectl create deployment fragile --image=nginx:1.27 --replicas=3
kubectl patch deployment fragile --type=json -p='[{"op": "add",
  "path": "/spec/template/spec/containers/0/livenessProbe",
  "value": {"httpGet": {"path": "/nope", "port": 80}, "periodSeconds": 5}]'
sleep 45
kubectl get pods -l app=fragile          # RESTARTS climbing on every replica
kubectl describe pod -l app=fragile | grep -A2 'Liveness' | head -6

```

```
# Now imagine /nope were a real endpoint that checks a slow database.
```

```
# --- 8. Tear down. -----  
kind delete cluster --name vss
```

What to notice. Step 3 is the whole chapter: nothing detected a failure or ran a recovery procedure. A controller compared two numbers and acted. Step 7 is the chapter’s warning, reproduced in forty-five seconds.

🔍 Checkpoint — Test Yourself

1. A cluster of six nodes has 12 allocatable cores and 48 GB each. Existing workloads request 40 cores and 190 GB. A Deployment of 15 replicas requesting 1 core and 6 GB each is created. How many schedule, and what is the binding resource?
2. A team sets no requests or limits anywhere “to keep things simple”. Name the QoS class this produces and describe the symptom they will eventually see, including when.
3. Explain why pointing a liveness probe at an endpoint that checks a shared database can take down every replica at once, and what to use instead.

📋 Chapter Summary

- Kubernetes is declarative: you state desired state and controllers reconcile towards it for ever. Failure recovery is not a feature but the ordinary behaviour of a loop comparing two numbers.
- Everything talks to the API server and nothing talks to anything else. etcd holds the entire cluster state.
- Creating a Deployment runs six independent loops; none calls another.
- A Pod is the unit of scheduling and of IP address. Put containers together only if they must share a node and a lifetime.
- Requests are for the scheduler and are reserved whether used or not; limits are enforced by cgroups, throttling CPU and killing on memory.
- QoS class follows from requests and limits, and decides eviction order. Declaring nothing means being evicted first.
- Readiness removes traffic; liveness kills the container. Using liveness where readiness was meant is the commonest self-inflicted outage.

🔧 Review Questions

1. (MCQ) The component that decides which node a Pod runs on, and then does nothing further, is: (a) the kubelet; (b) the scheduler; (c) kube-proxy; (d) the API server.
2. (MCQ) A container with no requests or limits is in QoS class: (a) Guaranteed; (b) Burstable; (c) BestEffort; (d) none.
3. (Short) State the control loop in four steps and explain why it makes `kubectl apply` safe to run twice.
4. (Short) Explain the difference between a request and a limit, naming which component enforces each.
5. (Short) Give a workload for which two containers genuinely belong in one Pod, and one where they do not, with the reason.

6. **(Applied)** A 10-node cluster of 32 cores and 128 GB each is reported “full” at 40% measured utilisation. Explain how both facts can be true, describe what you would measure, and set out the change you would propose and its risk.
7. **(Applied)** During a database slowdown, every replica of three different services restarted repeatedly and the outage lasted far longer than the database problem. Explain the mechanism, and write the probe configuration you would have had in place.

Lightweight and Secure Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the isolation–density–start-up trade and **place** any runtime on it.
- **Explain** how a microVM reaches a 125 ms boot, and **name** what was removed to get there.
- **Contrast** gVisor’s approach — intercepting system calls in user space — with Kata’s, which puts a kernel underneath each container.
- **Describe** a unikernel and **say** why the idea has not displaced the alternatives.
- **Choose** a runtime for a stated workload and **quantify** what the choice costs.

🔗 Before You Start

Chapter 8 (the shared kernel and the 350-call attack surface) and Chapter 5 (what a hypervisor contains, and Xen’s disaggregation argument). Chapter 7 is also assumed: every design here is fast because it has *virtio* and nothing else.

📖 Key Terms in This Chapter

microVM • Firecracker • Cloud Hypervisor • device model • boot time • jailer • gVisor • Sentry • Gofer • system-call interception • Kata Containers • RuntimeClass • unikernel • library operating system • WebAssembly • WASI • attack surface • cold start • oversubscription

11.1 The Choice Nobody Wanted to Make

Part II gave an isolation boundary enforced by the processor, at a few hundred megabytes and half a minute per instance. Part III gave a boundary enforced by the kernel, at a few megabytes and a tenth of a second.

For most workloads the second is right. For one case it is unacceptable: running *other people’s* code. A public cloud function service executes whatever a stranger uploads, beside another stranger’s, on hardware shared with a third. The 350-call Linux system-call interface is then not an abstraction — it is the perimeter, and it is enormous.

The honest statement of the problem is that this is a three-way trade and you may normally have two.

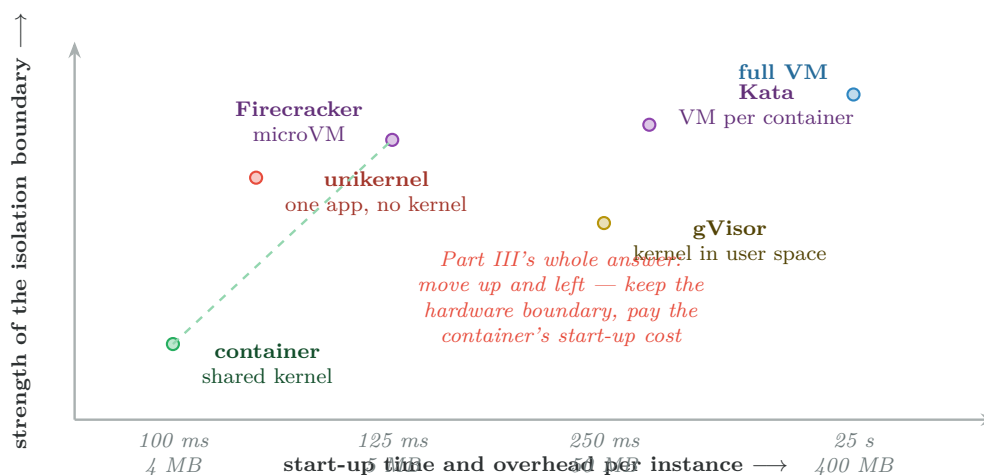


Figure 11.1: The trade, with measured figures on the horizontal axis. The dashed line is the move the designs in this chapter make: a hardware-enforced boundary at something close to a container’s cost.

11.2 MicroVMs

A virtual machine is slow to start and heavy because of what the hypervisor emulates: a BIOS, a PCI bus, an IDE controller, a VGA adapter, a PS/2 keyboard — a 1990s PC, assembled in software so that an unmodified operating system recognises it. Chapter 7 showed this is also where the throughput goes.

A **microVM** deletes all of it. If the guest is going to use virtio for everything anyway, the emulated chipset is dead weight.

MicroVM

A **microVM** is a virtual machine with a minimal device model: typically virtio-net, virtio-block, a serial console and a one-shot timer, and nothing else. No BIOS, no PCI enumeration, no legacy devices. The kernel is loaded directly rather than booted through firmware.

Firecracker, written at Amazon in Rust and released in 2018, is the reference example. It is about 50,000 lines against QEMU’s 1.4 million, boots a Linux guest in roughly **125 ms**, and adds under **5 MB** of memory per instance. It runs AWS Lambda and AWS Fargate: every function invocation you have ever made on Lambda ran in one.

Where the Boot Time Went

A conventional guest spends its 25 seconds roughly like this: firmware initialisation and PCI enumeration, bootloader, kernel device probing for hardware that is not there, then an init system bringing up services. Almost none of it is the application.

Firecracker removes each in turn. There is no firmware, so nothing to initialise. There is no PCI bus, so nothing to enumerate. The kernel is loaded into memory by the VMM and entered directly, with a command line that names the devices, so probing finds exactly what exists and stops. What remains is roughly 125 ms, most of it the Linux kernel’s own startup.

The lesson generalises past this chapter: *the cost was compatibility, not virtualization*. A guest willing to give up looking like a PC can have a hardware isolation boundary for about the price of a container — and that one observation is what made serverless computing economic.

⚠ The jailer, and defence in depth

Firecracker’s small size is a security argument only if you also assume the VMM process itself can be attacked. Amazon assumes it can.

Each Firecracker process is therefore started by a **jailer** that places it in its own namespaces and cgroup, chroots it, drops every capability it does not need and applies a seccomp filter restricting it to a few dozen system calls. An attacker who escapes the guest lands in a process that can barely talk to the kernel — the container mechanisms of Chapter 8 used to contain the hypervisor rather than the workload.

This is the strongest practical illustration in this book that the layers compose. It is not virtual machines *or* containers; it is a microVM inside a container, and the combination is stronger than either.

11.3 gVisor: a Kernel in User Space

Google’s answer inverts the problem. Instead of putting a real kernel underneath the container, **gVisor** puts a *fake* one in front of it.

A gVisor sandbox runs the application’s system calls into **Sentry**, a user-space process written in Go that implements most of the Linux system-call interface itself. Sentry handles the call in user space wherever it can; where it genuinely needs the host, it makes a far narrower call of its own. File access goes through a second process, the **Gofer**, so Sentry itself need not hold file descriptors.

💡 What gVisor Buys and What It Costs

Buys: the host kernel sees perhaps 60 distinct system calls from Sentry rather than 350 arbitrary ones from untrusted code. The attack surface shrinks by roughly a factor of six, without a hypervisor and without a second kernel’s memory.

Costs: every system call is now handled by a Go program instead of by the kernel. System-call-heavy workloads — anything doing many small reads and writes — lose substantially, commonly 10–30% and occasionally much more. CPU-bound work is barely affected, because it makes few calls.

And a third thing: Sentry implements *most* of Linux, not all of it. Unusual system calls, exotic `ioctl`s and some `/proc` behaviour are missing or approximated, so an application can fail in ways it never would natively. It is used in production at Google and in Google Cloud Run; it needs testing with your workload rather than faith.

11.4 Kata Containers

Kata takes the most direct route: make every container a virtual machine, and hide that fact completely.

Kata implements the OCI runtime interface, so Kubernetes drives it exactly as it drives `runc`. Underneath, each Pod gets its own lightweight VM — using Firecracker, Cloud Hypervisor or QEMU — with a minimal guest kernel and an agent that starts the container inside. The user writes ordinary manifests; a **RuntimeClass** selects Kata for the workloads that need it.

💡 Per-Workload Isolation Is the Deployable Idea

The important property is not Kata’s architecture but its granularity. A cluster does not have to choose one isolation level for everything.

Internal services, whose code you wrote and review, run on `runc` at full density. A tenant-supplied plugin, a customer’s build job, an untrusted PDF converter — these get

`runtimeClassName:` `kata` and a hardware boundary, at perhaps 50 MB and 250 ms each. The manifests are otherwise identical, and the decision is made per workload by the people who know the trust level.

That is the shape of the answer this chapter has been building towards: isolation stops being an architectural commitment and becomes a per-workload setting.

11.5 Unikernels and WebAssembly

Two more positions on the same axis, both further out than anything above.

A **unikernel** compiles the application together with just the operating system functions it actually calls, producing a single-address-space image that boots directly on a hypervisor. There is no shell, no package manager, no second process and often no notion of a user. Images are measured in megabytes and boot in milliseconds, and the attack surface is tiny because almost nothing is there.

They have not displaced anything, and the reasons are instructive: you cannot debug one with the usual tools, you cannot run arbitrary existing software in one without porting it, and the build toolchains have stayed difficult. The idea is sound and the operational cost has never been paid down.

WebAssembly outside the browser — Wasm with **WASI** — is the current attempt at the same goal from the other direction: a sandboxed bytecode with a deny-by-default capability model, millisecond start-up and images in the low megabytes. It runs under Kubernetes today through `runwasi` and similar shims. Its limitations are the obvious ones: your application must compile to it, and the ecosystem is much younger than the alternatives in this chapter.

Worked Example: Sizing a Function Service

A provider will run untrusted functions. Expected load: 2,000 concurrent instances, average lifetime 400 ms, memory limit 128 MB each, on hosts of 64 cores and 256 GB. Compare plain containers, Kata and Firecracker.

Step 1 — memory per instance.

- container: $128 + 4 = 132$ MB
- Kata on QEMU: $128 + 50 = 178$ MB
- Firecracker: $128 + 5 = 133$ MB

Step 2 — instances per host, reserving 32 GB, so 224 GB usable:

$$\begin{aligned} \text{container} &= \frac{224 \times 1024}{132} = 1,737 & \text{Kata} &= \frac{224 \times 1024}{178} = 1,288 \\ \text{Firecracker} &= \frac{224 \times 1024}{133} = \mathbf{1,724} \end{aligned}$$

Step 3 — hosts needed for 2,000 concurrent.

$$\text{container} = \left\lceil \frac{2000}{1737} \right\rceil = 2 \quad \text{Kata} = \left\lceil \frac{2000}{1288} \right\rceil = 2 \quad \text{Firecracker} = 2$$

All three need two hosts. On this workload the density difference does not change the bill at all.

Step 4 — the cost that does differ: cold start. With a 400 ms lifetime, start-up is not overhead, it is a large fraction of the work:

$$\text{container} = \frac{0.10}{0.40 + 0.10} = 20\% \quad \text{Kata} = \frac{0.25}{0.40 + 0.25} = 38\%$$

$$\text{Firecracker} = \frac{0.125}{0.40 + 0.125} = \mathbf{24\%}$$

Kata spends 38% of every host-second starting machines. Firecracker spends 24%, nearly matching the container, and gives a hardware boundary for it.

Step 5 — the decision. Firecracker. It is within 1% of the container's density and within 4 percentage points of its start-up overhead, and it is the only one of the three that puts the processor between one stranger's code and the next.

Step 6 — and the honest caveat. These figures assume a cold start every time. Real function services keep warm instances pooled, which cuts the effective cold-start rate by an order of magnitude and moves the whole comparison. Reproduce Step 4 with your measured cold-start *rate*, not the worst case, before committing hardware.

! Common Pitfalls

- **Treating this as a ranking.** There is no best runtime. A microVM for an internal service is wasted money; `runc` for a stranger's code is negligence.
- **Assuming gVisor is free.** The system-call path is slower, and how much depends entirely on how many calls your workload makes. Measure it, do not assume 10%.
- **Expecting every program to run under gVisor.** It implements most of Linux, not all. Test the actual application.
- **Thinking a microVM removes the need to harden the container around it.** Firecracker ships a jailer precisely because its designers do not make that assumption.
- **Choosing a unikernel without pricing the debugging.** The runtime properties are excellent and the day the thing misbehaves in production you have no shell, no `strace` and no package to install one from.
- **Applying one isolation level to a whole cluster.** `RuntimeClass` exists so that the decision can be made per workload. Most clusters should mix.

🔧 Try It Yourself: Booting a MicroVM, and Timing It

Starts a Firecracker microVM and compares its boot time and memory footprint with a full QEMU guest and a container. Needs KVM. Expect forty minutes.

```
#!/usr/bin/env bash
# ch11_microvm.sh -- 25 s, 125 ms and 100 ms, measured on one machine.
set -euo pipefail
cd ~ && mkdir -p fc && cd fc

# --- 1. Firecracker needs KVM and nothing else. -----
[ -w /dev/kvm ] || { echo "no /dev/kvm -- see Chapter 4 step 1"; exit 1; }
ARCH=$(uname -m)
curl -sL -o firecracker \
  "https://github.com/firecracker-microvm/firecracker/releases/latest/\
download/firecracker-${ARCH}.tgz" || true
# If that fails, the release page lists a direct tarball; extract the
# firecracker binary beside this script.
chmod +x firecracker 2>/dev/null || true

# --- 2. A kernel and a root filesystem, loaded directly. -----
# No bootloader and no firmware: the VMM places the kernel in memory.
curl -sL -o vmlinux \
  "https://s3.amazonaws.com/spec.ccfc.min/img/quickstart_guide/\
${ARCH}/kernels/vmlinux.bin"
curl -sL -o rootfs.ext4 \
  "https://s3.amazonaws.com/spec.ccfc.min/img/quickstart_guide/\
"
```

```

${ARCH}/rootfs/bionic.rootfs.ext4"

# --- 3. Describe the whole machine. Note how little there is. -----
cat > vm.json <<'JSON'
{
  "boot-source": {
    "kernel_image_path": "vmlinux",
    "boot_args": "console=ttyS0 reboot=k panic=1 pci=off"
  },
  "drives": [{
    "drive_id": "rootfs", "path_on_host": "rootfs.ext4",
    "is_root_device": true, "is_read_only": false
  }],
  "machine-config": { "vcpu_count": 1, "mem_size_mib": 128 }
}
JSON
# pci=off is the chapter: there is no PCI bus to enumerate.

# --- 4. Boot it, and time to the login prompt. -----
echo "=== firecracker ==="
time timeout 20 ./firecracker --no-api --config-file vm.json \
  2>&1 | grep -m1 -E 'login:|Welcome' || true

# --- 5. The same guest under full QEMU, for comparison. -----
echo "=== full qemu guest ==="
time timeout 90 sudo virsh start vss1 --console 2>&1 \
  | grep -m1 'login:' || true
sudo virsh destroy vss1 || true

# --- 6. And a container, for the other end of the axis. -----
echo "=== container ==="
time docker run --rm alpine:3.20 /bin/true

# --- 7. Memory footprint of the VMM process itself. -----
./firecracker --no-api --config-file vm.json &
FCPID=$!
sleep 3
grep -E '^(VmRSS|VmSize)' "/proc/$FCPID/status"
kill $FCPID 2>/dev/null || true
# Compare with the qemu process for vss1: typically 40-80x larger.

# --- 8. Kata, if the host has it: one flag, a whole VM. -----
# Under Kubernetes this is runtimeClassName: kata in the Pod spec.
command -v kata-runtime >/dev/null && kata-runtime check || \
  echo "kata not installed -- see Appendix A"

# --- 9. Tear down. -----
cd ~ && rm -rf fc

```

What to notice. Three boot times spanning roughly two orders of magnitude, and the middle one has the same isolation boundary as the slowest. That single fact is why this chapter exists, and why Lambda is built the way it is.

🔍 Checkpoint — Test Yourself

1. Firecracker boots in 125 ms where a conventional guest takes 25 s. Name the three things that were removed, and say which of them a guest operating system must be willing to give up.
2. A workload runs 18% slower under gVisor than under runc, while another is unaffected. What distinguishes them, and what would you measure to confirm it?

3. A cluster runs 200 internal services and accepts customer-supplied build jobs. Describe the runtime arrangement you would deploy and the single Kubernetes field that expresses it.

✓ Chapter Summary

- Isolation, density and start-up form a three-way trade. This chapter is the set of designs that refuse to pick only two.
- A microVM deletes the emulated 1990s PC. Firecracker is 50,000 lines, boots in 125 ms and costs under 5 MB — the cost was compatibility, not virtualization.
- Firecracker is itself jailed in namespaces, a cgroup and a seccomp filter: containers containing a hypervisor. The layers compose.
- gVisor implements Linux in user space, cutting the host's exposed system calls from about 350 to about 60, at a system-call-rate-dependent performance cost and with imperfect coverage.
- Kata gives every Pod its own virtual machine behind the ordinary OCI interface, selectable per workload with RuntimeClass.
- Unikernels and Wasm go further still; the first has never paid down its operational cost, and the second is young.
- Isolation is now a per-workload setting rather than an architectural commitment. Most clusters should mix.

🔧 Review Questions

1. **(MCQ)** Firecracker achieves its boot time principally by: (a) a faster kernel; (b) removing firmware, PCI enumeration and legacy device emulation; (c) running without KVM; (d) sharing the host kernel.
2. **(MCQ)** gVisor reduces risk by: (a) running each container in a virtual machine; (b) implementing most of the Linux system-call interface in user space; (c) encrypting guest memory; (d) removing the container's network namespace.
3. **(Short)** Explain why Firecracker ships a jailer, given that its own code base is only 50,000 lines.
4. **(Short)** State the trade gVisor makes, and the workload characteristic that decides whether it is acceptable.
5. **(Short)** Why have unikernels not displaced containers, despite better figures on size, start-up and attack surface?
6. **(Applied)** Redo the worked example for functions with an average lifetime of 4 s rather than 400 ms. Give the start-up overhead for all three runtimes and say whether your recommendation changes, and why.
7. **(Applied)** A university wants to let students submit code that is compiled and run automatically on a shared cluster. Set out the isolation design you would propose, what you would measure before going live, and what you would do if the measured overhead were twice what you expected.

PART IV

Storage, Network and Cloud Virtualization

A virtual machine with no disk and no network is a thought experiment. Part IV virtualizes everything that is not a processor — storage, then the network, then the whole arrangement, sold by the hour. It ends at the cloud, which is not a new idea in this book but the previous thirteen chapters with a billing system attached.

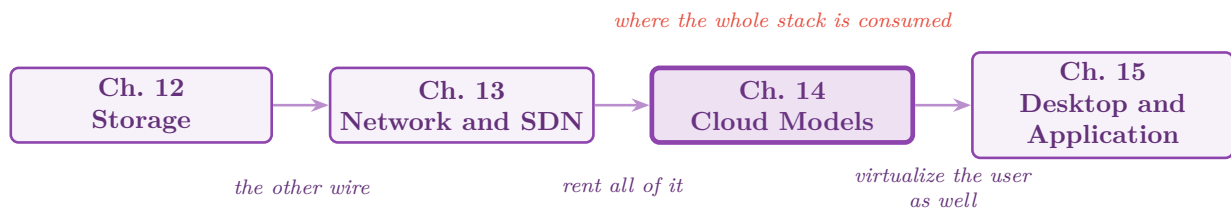


Figure P4.1: Reading path through Part IV. Chapters 12 and 13 complete the virtual machine; Chapter 14 is where the industry sells it, and Chapter 15 is the case where the thing virtualized is somebody's desk.

Storage Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Storage, Network and Cloud Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** block, file and object storage by the interface each presents, and **say** which a virtual machine needs.
- **Explain** what a virtual disk file is, and **compare** raw and qcow2 by what each gives up.
- **Compute** a thin-provisioning overcommit ratio and **predict** when a pool will exhaust.
- **Describe** copy-on-write snapshots and **explain** why a long snapshot chain degrades reads.
- **Explain** how software-defined storage places replicas, and **state** what it replaces.

🔗 Before You Start

Chapter 7 (virtio-blk, and why an emulated disk controller costs 80% of its throughput) and Chapter 1 (encapsulation — a running machine as a file, which this chapter makes literal). From Operating Systems you need filesystems, blocks and the page cache.

📖 Key Terms in This Chapter

block storage • file storage • object storage • LUN • SAN • NAS • iSCSI • datastore • virtual disk • raw • qcow2 • VMDK • VHDX • cluster • L1/L2 table • thick provisioning • thin provisioning • over-subscription • copy-on-write • snapshot chain • backing file • read amplification • software-defined storage • CRUSH • replica • erasure coding • IOPS • discard / TRIM

12.1 Three Interfaces

“Storage” names three different contracts, and a great deal of confusion in this area comes from using one word for all of them.

	What it presents	Spoken over	Used for
Block	An array of fixed-size blocks, with no notion of a file. Whoever mounts it puts a filesystem on it	SAS, Fibre Channel, iSCSI , NVMe-oF	Virtual disks. A guest expects a disk, so it must be block
File	A directory tree with files, locking and permissions, shared by many clients at once	NFS, SMB	Shared datastores, home directories, anything several machines must read together

	What it presents	Spoken over	Used for
Object	Whole immutable objects in a flat namespace, addressed by key over HTTP	S3 API, Swift	Images, backups, logs, anything written once and read many times

💡 Why a Guest Must Have Block Storage

A guest operating system contains a filesystem driver, and a filesystem driver issues block reads and writes. It cannot be given an object store, because it has no idea what an HTTP PUT is, and giving it a file share only works if something underneath turns that into blocks — which is exactly what a virtual disk on NFS does.

So the chain is always: guest filesystem → block device → virtual disk → whatever the host has. The interesting engineering is in the last two steps, and the guest never sees any of it.

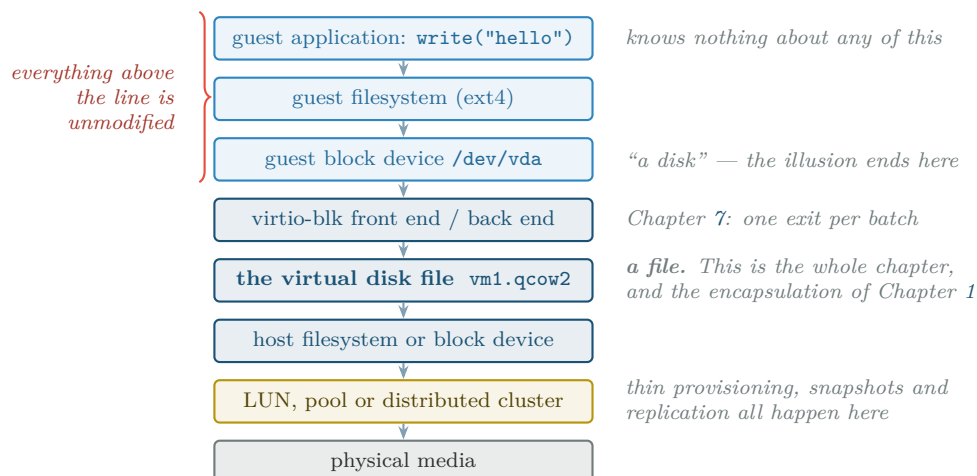


Figure 12.1: The path of one write. Each layer below the virtual disk file can be replaced without the guest noticing — which is what makes migration, snapshots and replication possible at all.

12.2 The Virtual Disk

A virtual disk is a file on the host that the guest sees as a block device. Two formats cover almost everything.

📄 Raw and qcow2

A **raw** disk is a byte-for-byte image: block n of the guest is byte $n \times 512$ of the file. There is no metadata and no translation, so it is as fast as the host filesystem and can be mounted directly with `losetup`.

qcow2 — QEMU copy-on-write, version 2 — adds an indirection table. The file is divided into **clusters** (64 KB by default) and a two-level L1/L2 table maps guest offsets to cluster locations. That indirection buys three things raw cannot do: allocate clusters only when written (thin provisioning), keep internal snapshots, and reference a **backing file** so that several disks share one unwritten base.

VMware's **VMDK** and Microsoft's **VHDX** are the same idea with different metadata, and `qemu-img convert` moves between all of them.

⚠ Raw is faster, and the gap is smaller than people assume

qcow2 must consult its L2 table before reaching data, and must allocate and update metadata on first write to each cluster. On a freshly created image with small random writes the penalty is real — tens of per cent.

Two things shrink it. The L2 tables are cached, so a steady-state workload mostly hits the cache; and `qemu-img create -o preallocation=metadata` builds the tables up front, removing the first-write cost. With a modern qcow2 and a warm cache the difference on typical server workloads is usually in the low single digits.

Use raw when you need every last IOPS and will handle snapshots at the layer below. Use qcow2 when you want the features, which is most of the time.

12.3 Thin Provisioning, and How It Fails

Thick provisioning allocates all of a disk's space when it is created: a 500 GB disk takes 500 GB immediately. **Thin provisioning** allocates only what has been written, so that same disk might occupy 40 GB.

Thin provisioning is the storage equivalent of the memory overcommit in Chapter 6, and it has the same shape: usually a large saving, occasionally a cliff.

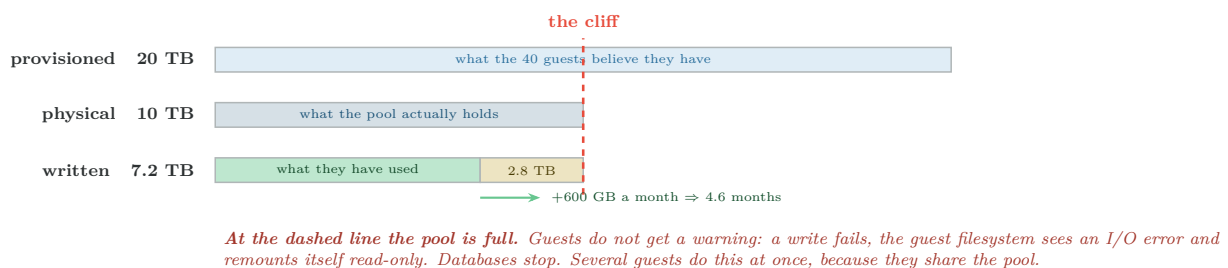


Figure 12.2: Thin provisioning, drawn to scale for the worked example. The gap between the second and third bars is the saving; the gap between the first and second is the exposure.

📊 Worked Example: When Does This Pool Run Out?

A datastore of 10 TB serves 40 guests, each with a 500 GB thin-provisioned disk. Measured: mean 180 GB written per guest, growing 15 GB per guest per month. Decide whether to act, and when.

Step 1 — the three quantities.

$$\text{provisioned} = 40 \times 500 = 20,000 \text{ GB} = 20 \text{ TB}$$

$$\text{physical} = 10 \text{ TB} \quad \text{written} = 40 \times 180 = 7,200 \text{ GB} = 7.2 \text{ TB}$$

Step 2 — the over-subscription ratio.

$$\sigma = \frac{20}{10} = 2.0$$

Every guest could legitimately fill its disk, and the pool can honour half of them.

Step 3 — current headroom.

$$10 - 7.2 = 2.8 \text{ TB free, or } 72\% \text{ of the pool consumed}$$

Step 4 — time to exhaustion at the measured rate.

$$40 \times 15 = 600 \text{ GB per month} \quad \frac{2,800}{600} = 4.6 \text{ months}$$

Step 5 — but that is the optimistic figure. Three things consume the pool faster than the trend line:

- **Snapshots.** Each one holds the blocks its parent has since overwritten. A forgotten snapshot on a busy database can grow by its whole working set.
- **Deleted files do not return space.** A guest deleting 50 GB frees nothing in the pool unless `discard/TRIM` is enabled all the way down. Most estates discover this configuration only here.
- **Correlation.** A patch cycle writes to every guest in the same hour, so consumption arrives in steps, not a slope.

Step 6 — what happens at 100%. Not a slowdown. The pool cannot satisfy a write; the guest receives an I/O error; its filesystem remounts read-only to protect itself. *Many guests hit this within seconds of each other*, because they share the pool — so the failure is simultaneous and estate-wide, and recovery requires space that by definition is not there.

Step 7 — the recommendation. Act now, not in four months.

- Alert at 80% of the pool (2 TB free), which at the measured rate is about six weeks away. Alerting at 95% leaves no time to buy anything.
- Enable `discard` end to end and reclaim; on a four-year-old estate this commonly returns 10–20%.
- Report any snapshot older than seven days, every day.
- Add capacity, or reduce σ by shrinking the 500 GB disks nobody asked for.

12.4 Snapshots and Copy-on-Write

A snapshot freezes the disk as it is now. The implementation is the obvious one: stop writing to the current file and make it read-only, then direct all new writes to a fresh **overlay** file that records only what has changed. A read checks the overlay first and falls back to the frozen file beneath.

This is **copy-on-write**, and it is the same mechanism as the overlays of Chapter 8, one layer down.

💡 Why a Long Chain Is Slow, and Why It Is Dangerous

Each snapshot adds a layer. A read must check each overlay in turn until it finds the block, so a chain of twelve snapshots can require twelve lookups where one would do — **read amplification**. Writes are worse: the first write to a block must copy it up from wherever it currently lives.

The danger is worse than the latency. Every file in the chain is required: lose or corrupt one and every snapshot above it is unreadable. A guest with a six-month chain has six months of fragile dependency, and the usual discovery is that nobody knows which snapshots are still needed.

A snapshot is not a backup. It lives on the same storage as the original, so it does not survive the array failing, the pool filling, or the ransomware that encrypted both. Keep chains short, delete snapshots on a schedule, and keep real backups elsewhere. Chapter 17 covers what a real backup is.

12.5 Software-Defined Storage

The traditional answer to shared storage was a **SAN**: a dedicated array, dual controllers, a Fibre Channel fabric, and a price to match. The software-defined answer is to put ordinary disks in the servers you already have and let software make them one pool.

Ceph is the open example. Data is split into objects; each object's placement is computed by **CRUSH**, a deterministic function of the object name and a map of the cluster's failure domains. There is no lookup table and no metadata server in the data path — any client can compute where a block lives, which is what lets it scale to thousands of nodes. VMware's **vSAN** does the same job inside a vSphere cluster.

💡 Replication or Erasure Coding: a Real Choice

Both survive failures; they trade capacity against CPU and latency.

Three-way replication keeps three copies. Usable capacity is a third of raw; a read is served by one disk; recovery is a copy. Simple and fast.

Erasure coding — say 4 data plus 2 parity — stores 6 chunks for 4 chunks of data. Usable capacity is $4/6 = 67\%$ of raw against replication's 33%, so a petabyte of raw yields 670 TB instead of 330 TB. The cost is that every write computes parity and every degraded read reconstructs it, so latency and CPU both rise.

The usual split: replication for virtual machine disks, where latency matters and the data is active; erasure coding for backups, archives and object storage, where capacity dominates and nothing is latency-sensitive. Putting a busy database on erasure-coded storage is a common and expensive mistake.

❗ Common Pitfalls

- **Monitoring provisioned capacity instead of written capacity.** The first tells you nothing about when you run out. Alert on the pool's free space, at 80%.
- **Treating a snapshot as a backup.** Same storage, same failure, same ransomware.
- **Leaving snapshots behind after an upgrade.** The chain grows, the pool fills and reads slow down. Report anything older than a week.
- **Assuming deleting files inside a guest frees pool space.** It does not without discard enabled end to end, which is off by default in many stacks.
- **Thin-provisioning the pool *and* overcommitting memory *and* oversubscribing CPU on the same host.** Each is defensible alone; together they fail at the same moment, because the event that triggers one triggers the others.
- **Erasure-coding a database.** Capacity efficiency bought with write latency, on the workload least able to pay it.
- **Running a guest's disk on a filesystem that is itself copy-on-write, with qcow2 on top.** Two layers of copy-on-write fragment each other. Pick one.

🔧 Try It Yourself: Thin Pools, Snapshots and the Cliff

Builds a small thin pool with LVM, fills it, and watches a guest filesystem go read-only — the worked example's step 6, at a safe scale. Expect forty minutes.

```
#!/usr/bin/env bash
# ch12_storage.sh -- thin provisioning, snapshots and exhaustion.
set -euo pipefail

# --- 1. Virtual disk formats: what they cost on disk. -----
cd /tmp
qemu-img create -f raw plain.raw 10G
```

```

qemu-img create -f qcow2 thin.qcow2 10G
ls -lh plain.raw thin.qcow2           # apparent size
du -h plain.raw thin.qcow2           # actual size: this is the point
qemu-img info thin.qcow2 | head -6

# --- 2. A backing file: many disks, one shared base. -----
qemu-img create -f qcow2 -b thin.qcow2 -F qcow2 child1.qcow2
qemu-img create -f qcow2 -b thin.qcow2 -F qcow2 child2.qcow2
du -h child1.qcow2 child2.qcow2      # nearly nothing until written
qemu-img info child1.qcow2 | grep -i backing

# --- 3. Build a thin pool, deliberately over-subscribed. -----
sudo truncate -s 2G /tmp/pv.img
LOOP=$(sudo losetup --find --show /tmp/pv.img)
sudo pvcreate "$LOOP" ; sudo vgcreate vsvg "$LOOP"
sudo lvcreate -L 1.5G --thinpool vspool vsvg
# Four 1 GB volumes out of a 1.5 GB pool: sigma = 2.7
for i in 1 2 3 4; do
    sudo lvcreate -V 1G --thin -n "vol$i" vsvg/vspool
done
sudo lvs vsvg -o lv_name,lv_size,data_percent

# --- 4. Put filesystems on them and write a little. -----
for i in 1 2 3 4; do
    sudo mkfs.ext4 -q "/dev/vsvg/vol$i"
    sudo mkdir -p "/mnt/v$i" && sudo mount "/dev/vsvg/vol$i" "/mnt/v$i"
    sudo dd if=/dev/urandom of="/mnt/v$i/data" bs=1M count=200 status=none
done
sudo lvs vsvg -o lv_name,data_percent      # pool filling up

# --- 5. Now walk off the cliff. -----
# Keep writing until the pool cannot satisfy a write.
sudo dd if=/dev/urandom of=/mnt/v1/more bs=1M count=900 || \
    echo "write failed -- the pool is full"
dmesg | tail -5 | grep -iE 'thin|read-only|I/O error' || true
mount | grep -E '/mnt/v[0-9]',             # look for "ro" on several at once

# --- 6. Snapshots and read amplification. -----
cd /tmp
qemu-img create -f qcow2 base.qcow2 1G
sudo modprobe nbd max_part=8 || true
for n in $(seq 1 8); do
    qemu-img create -f qcow2 -b "${ [ $n -eq 1 ] && echo base.qcow2 \
        || echo "snap$((n-1)).qcow2" )" -F qcow2 "snap$n.qcow2"
done
qemu-img info --backing-chain snap8.qcow2 | grep -c '^image:'
# Eight files, every one of which is required to read the newest.

# --- 7. Reclaim: discard is what returns space. -----
sudo fstrim -v /mnt/v2 || echo "discard not supported on this stack"
sudo lvs vsvg -o lv_name,data_percent

# --- 8. Tear down. -----
for i in 1 2 3 4; do sudo umount "/mnt/v$i" || true; done
sudo lvremove -f vsvg || true ; sudo vgremove -f vsvg || true
sudo pvremove -f "$LOOP" || true ; sudo losetup -d "$LOOP" || true
rm -f /tmp/{plain.raw,thin.qcow2,child*.qcow2,base.qcow2,snap*.qcow2,pv.img}

```

What to notice. Step 5 is the part worth sitting with. The failure is not gradual and it is not confined to the volume that caused it: several filesystems go read-only together, because the resource they were quietly sharing ran out. That is the whole argument for alerting at 80%.

? Checkpoint — Test Yourself

1. A pool of 40 TB serves 120 guests with 800 GB disks each, of which a mean 240 GB is written, growing 20 GB per guest per month. Give the over-subscription ratio, the headroom and the time to exhaustion. At what free-space figure would you alert, and why?
2. A guest deletes 300 GB of files and the storage team reports no change in pool usage. Explain why, and give the setting that changes it.
3. Why is a chain of twelve snapshots both slow to read and dangerous to keep?

✓ Chapter Summary

- Block, file and object are three different contracts. A guest needs block, because its filesystem driver issues block requests.
- A virtual disk is a file the guest sees as a device. raw is a byte-for-byte image; qcow2 adds an L1/L2 indirection that buys thin provisioning, snapshots and backing files.
- Thin provisioning is memory overcommit for disks: usually a large saving, occasionally a cliff. Monitor written capacity, never provisioned, and alert at 80%.
- Pool exhaustion is not a slowdown. Writes fail, guest filesystems go read-only, and they do it together because they share the pool.
- Snapshots are copy-on-write overlays. Long chains amplify reads and make every file in the chain load-bearing. A snapshot is not a backup.
- Software-defined storage computes placement rather than looking it up. Replication for virtual disks, erasure coding for archives.

✍ Review Questions

1. **(MCQ)** A guest operating system requires its storage to be presented as: (a) object storage; (b) block storage; (c) an NFS share; (d) any of these.
2. **(MCQ)** When a thin pool reaches 100% of physical capacity: (a) writes slow down; (b) the oldest snapshots are deleted; (c) writes fail and guest filesystems typically remount read-only; (d) the hypervisor migrates guests away.
3. **(Short)** State what qcow2's indirection table buys over raw, and what it costs.
4. **(Short)** Explain read amplification in a snapshot chain, and why every file in the chain matters.
5. **(Short)** Give the rule for choosing between replication and erasure coding, with the reason.
6. **(Applied)** An estate thin-provisions storage at $\sigma = 2.4$, overcommits memory at 1.3 and oversubscribes CPU at 2.0. Explain why these three are more dangerous together than separately, give a scenario that triggers all three at once, and say what you would change first.
7. **(Applied)** You inherit an estate with 400 guests, 1,900 snapshots and no snapshot policy. Write the plan you would follow to get to a safe state, in order, saying what you would measure before deleting anything.

Network Virtualization and SDN

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Storage, Network and Cloud Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** a virtual switch and **explain** what it does that a physical switch cannot.
- **State** the limit that VLANs reach, and **explain** how an overlay removes it.
- **Compute** the encapsulation overhead of VXLAN and **predict** the MTU failure it causes.
- **Explain** the control-plane/data-plane split and **say** what software-defined networking actually moved.
- **Diagnose** a network fault in which small packets succeed and large transfers hang.

🔗 Before You Start

Chapter 7 (virtio-net, SR-IOV, and the fact that VF-to-VF traffic never reaches the host) and Chapter 8 (the network namespace, which is what a container’s “own interface” actually is). From networking you need Ethernet frames, IP, MTU and what a switch does with a MAC address table.

📖 Key Terms in This Chapter

virtual switch • Linux bridge • Open vSwitch • tap device • veth pair • uplink • VLAN • 802.1Q • trunk • overlay • underlay • VXLAN • VNI • VTEP • Geneve • encapsulation • MTU • MSS clamping • path MTU discovery • control plane • data plane • SDN • OpenFlow • NFV • VNF • service chaining • CNI • micro-segmentation

13.1 The Switch That Is Not There

A host with thirty guests has one or two physical network ports. Something must decide which frame belongs to which guest, and that something is a **virtual switch** in software.

Each guest’s virtual NIC terminates in a **tap device** on the host — a virtual interface whose other end is the guest. The switch learns MAC addresses and forwards between taps and the physical **uplink**, exactly as a physical switch forwards between ports. Linux offers two: the simple **bridge**, and **Open vSwitch**, which adds flow tables, tunnelling and a remote control interface.

💡 What a Virtual Switch Can Do That Steel Cannot

It is not merely a cheaper switch. Three things follow from being software inside the hypervisor.

It knows what each port is. A physical switch sees a MAC address. A virtual switch knows that this port is *the web server of tenant 14*, and can apply a rule written in those terms. This is what makes **micro-segmentation** possible: a firewall rule enforced at every guest’s own interface, instead of at a choke point the traffic may never cross.

It moves with the guest. Live-migrate a guest (Chapter 17) and its port, its rules and its counters are recreated on the destination. No cable is patched and no switch port is reconfigured.

It is created and destroyed in milliseconds. A Kubernetes node starting a Pod creates a **veth pair**, attaches one end to a bridge and moves the other into the Pod's network namespace. That is the whole of container networking, and it happens hundreds of times a minute.

The cost: it is a switch made of your host's CPU. At high packet rates it competes with the guests, which is the measurement Chapter 7 made and the reason SR-IOV exists.

13.2 VLANs, and the Number 4094

Separating tenants on a shared network is an old problem with an old answer. A **VLAN** tag — 802.1Q — is four bytes inserted into the Ethernet header, and switches treat differently-tagged frames as if they were on different wires.

The tag contains a 12-bit VLAN identifier. That is $2^{12} = 4096$ values, of which 0 and 4095 are reserved: **4094 usable networks**, per layer-2 domain, for ever.

For one company that is ample. For a cloud provider with fifty thousand customers it is not a limit to be worked around; it is a wall. And there is a second problem that bites even small estates: a VLAN exists only within one layer-2 domain, so a guest cannot keep its address when it migrates to a host in another building.

13.3 Overlays

An overlay solves both by putting the tenant's Ethernet frame *inside* an ordinary IP packet. The physical network — the **underlay** — carries IP between hosts and knows nothing about tenants. The virtual network — the **overlay** — lives entirely in the encapsulation.

VXLAN, VNI and VTEP

VXLAN wraps a guest's complete Ethernet frame in UDP, and sends it to the host where the destination guest lives.

The **VNI** — VXLAN network identifier — is **24 bits**, so $2^{24} = 16,777,216$ virtual networks instead of 4094. A **VTEP** (VXLAN tunnel endpoint) is the component that encapsulates and decapsulates; on a virtualization host it is the virtual switch.

Because the outer packet is ordinary IP, the overlay crosses any routed network — another rack, another building, another data centre — without that network being told anything. **Geneve** is the newer, extensible version and is what most recent stacks use; the arithmetic below is the same.

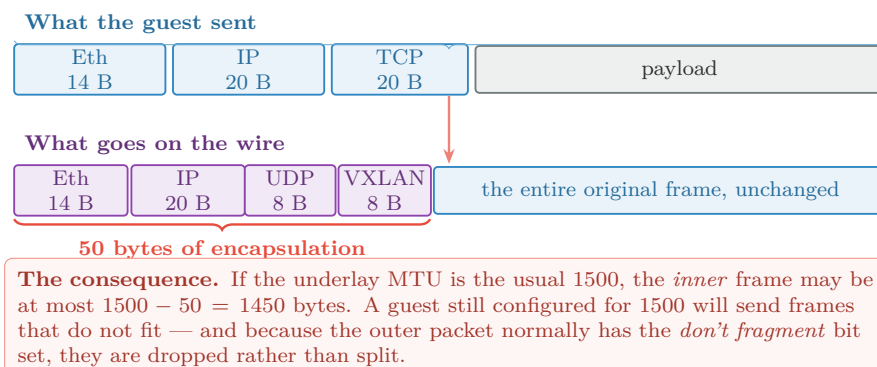


Figure 13.1: VXLAN encapsulation, to scale. The 24-bit VNI in those eight bytes is what replaces the 12-bit VLAN tag; the 50 bytes are what causes the fault in the worked example.

Worked Example: Small Packets Work, Large Transfers Hang

A tenant reports: SSH logs in and then freezes on the first command with any output. `ping` succeeds. Database queries returning one row work; queries returning a thousand rows time out. Backups never complete. The network team reports no errors and no packet loss.

Step 1 — what the two working cases have in common. `ping` sends 64-byte packets. An SSH login exchanges a few hundred bytes. A one-row query is small. Everything that *works* is small, and everything that fails is large. That is an MTU fault, and it is almost the only thing that produces this exact pattern.

Step 2 — the arithmetic. The overlay is VXLAN over a standard network:

$$\underbrace{14}_{\text{outer Eth}} + \underbrace{20}_{\text{outer IP}} + \underbrace{8}_{\text{UDP}} + \underbrace{8}_{\text{VXLAN}} = 50 \text{ bytes}$$

$$\text{largest inner frame} = 1500 - 50 = \mathbf{1450} \text{ bytes}$$

Step 3 — what the guest believes. Its interface is still at the default MTU of 1500. TCP therefore negotiates a maximum segment size of

$$1500 - \underbrace{20}_{\text{IP}} - \underbrace{20}_{\text{TCP}} = 1460 \text{ bytes}$$

so a full segment becomes a 1500-byte inner frame, which becomes a 1550-byte packet on a wire that carries 1500.

Step 4 — why it hangs rather than fails. The VTEP sets the *don't fragment* bit, so the oversized packet is dropped and an ICMP “fragmentation needed” message is returned to tell the sender to use a smaller size. That mechanism — path MTU discovery — is how this is supposed to self-correct.

It does not self-correct, because ICMP is very commonly filtered. The sender never learns, retransmits the same oversized segment, and the connection stalls for ever. This is a **PMTUD black hole**, and the handshake succeeded only because the handshake packets were small.

Step 5 — three fixes, in order of preference.

- **Raise the underlay.** Set the physical network to jumbo frames, 9000 bytes, giving an inner MTU of 8950. Guests keep 1500 and nothing else changes. Best answer, and it requires every switch on the path.
- **Lower the guest.** Set guest interfaces to 1450. Correct, free, and requires touching every guest — and every guest anyone adds later.
- **Clamp the MSS.** Have the virtual switch rewrite the MSS in passing SYN packets to 1410. Works without touching guests, and only helps TCP: UDP and anything tunnelled stays broken.

Step 6 — what to take from it. The encapsulation of this chapter is invisible until it is not, and when it becomes visible the symptom points nowhere near the cause. “Small works, large hangs” should go straight to arithmetic: add up the headers.

13.4 Separating the Planes

Every switch does two different jobs. The **data plane** forwards frames, at line rate, millions per second. The **control plane** decides *how* they should be forwarded — learning addresses, running routing protocols, holding the policy.

Traditionally both live in every switch, so policy is configured box by box and the network's behaviour is an emergent property of several hundred configurations nobody can read at once.

☰ Software-Defined Networking

SDN separates the two: the data plane stays in the switches, and the control plane is lifted out into a **controller** with a view of the whole network. It programmes each device's forwarding tables through an open interface — **OpenFlow** being the first and best known. The change is in where policy lives. “Tenant 14 may not reach tenant 9” stops being a set of access lists on twelve switches and becomes one statement to one controller, which works out the consequences everywhere. Networks become programmable in the way that Chapter 16's infrastructure-as-code makes servers programmable.

💡 What Actually Happened to SDN

It is worth being honest about this, because the textbook account and the industry's experience differ.

Pure OpenFlow — dumb switches, one central controller — did not take over. The controller became a scaling and availability problem, hardware support was uneven, and operators were unwilling to put every forwarding decision behind a single piece of software.

What did take over is the *idea*: a central place that holds intent, and devices programmed from it. It is everywhere in virtualization — NSX, ACI, every cloud's virtual network, every Kubernetes CNI plugin — usually with distributed agents rather than one controller, and usually with the overlay doing the heavy lifting. The vocabulary survived better than the protocol, which is a common pattern and worth recognising when you meet the next one.

13.5 NFV and Container Networking

If a switch can be software, so can a firewall, a load balancer and a router. **Network function virtualization** replaces purpose-built appliances with virtual machines or containers — **VNFs** — running the same function. A chain of them, traffic steered through in order, is **service chaining**, and the economics are the ones from Chapter 1: appliances sized for peak and idle most of the time, consolidated.

In containers the same job is done by **CNI**, the Container Network Interface. When a Pod starts, the kubelet calls a CNI plugin, which creates the veth pair, assigns an address and installs routes. Which plugin you choose decides whether you get an overlay (Flannel's VXLAN), routed addresses without encapsulation (Calico in BGP mode), or an eBPF data path with policy (Cilium).

⚠️ Where the packets you cannot see go

Three places in this book where traffic exists and your monitoring does not see it, each with a different cause:

Guest to guest on one host. The frame goes guest → virtual switch → guest and never touches the uplink. A physical tap or a switch SPAN port sees nothing.

VF to VF on one SR-IOV card (Chapter 7). The card's internal switch forwards it, so even the hypervisor is not involved.

Pod to Pod on one node. The veth pair and the node's bridge handle it entirely.

In each case the fix is the same in principle — the observation point must be the virtual switch, not the wire — and it is the reason Chapter 19's detection chapter cares so much about where sensors are placed.

! Common Pitfalls

- **Deploying an overlay without fixing MTU.** The worked example. Decide between jumbo underlay, lowered guest MTU and MSS clamping *before* the first tenant, not after the first backup fails.
- **Filtering ICMP wholesale.** It breaks path MTU discovery and turns a clean error into a silent hang. Permit type 3 code 4.
- **Assuming the overlay is free.** Encapsulation costs CPU on every packet in both directions. Check whether your NIC offloads VXLAN; the difference is large.
- **Expecting physical network monitoring to see virtual traffic.** Most east–west traffic never reaches a physical port.
- **Treating 4094 as a comfortable number.** It is per layer-2 domain and it also constrains the *underlay* design, not just tenant count.
- **Choosing a CNI plugin by popularity.** Overlay, routed and eBPF plugins differ in MTU behaviour, in policy support and in how they fail. That choice is harder to change than any other in a cluster.

🔧 Try It Yourself: Building an Overlay, and Breaking It

Builds a VXLAN overlay between two network namespaces, then reproduces the black hole of the worked example and fixes it three ways. Needs one Linux host. Expect forty minutes.

```
#!/usr/bin/env bash
# ch13_overlay.sh -- a virtual switch, a VXLAN tunnel and an MTU black hole.
set -euo pipefail

# --- 1. A virtual switch, and two "guests" as namespaces. -----
sudo ip link add br-vss type bridge
sudo ip link set br-vss up
for n in 1 2; do
    sudo ip netns add "g$n"
    sudo ip link add "veth$n" type veth peer name "in$n"
    sudo ip link set "veth$n" master br-vss
    sudo ip link set "veth$n" up
    sudo ip link set "in$n" netns "g$n"
    sudo ip netns exec "g$n" ip addr add "10.10.0.$n/24" dev "in$n"
    sudo ip netns exec "g$n" ip link set "in$n" up
    sudo ip netns exec "g$n" ip link set lo up
done
# This is container networking, in nine lines.
sudo ip netns exec g1 ping -c 2 10.10.0.2
bridge fdb show br br-vss | head -4      # the switch has learned MACs

# --- 2. Now the underlay: two more namespaces playing "hosts". -----
sudo ip netns add h1 ; sudo ip netns add h2
sudo ip link add u1 type veth peer name u2
sudo ip link set u1 netns h1 ; sudo ip link set u2 netns h2
sudo ip netns exec h1 ip addr add 192.168.50.1/24 dev u1
sudo ip netns exec h2 ip addr add 192.168.50.2/24 dev u2
for h in h1 h2; do
    sudo ip netns exec "$h" ip link set lo up
done
sudo ip netns exec h1 ip link set u1 up
sudo ip netns exec h2 ip link set u2 up
sudo ip netns exec h1 ping -c 2 192.168.50.2

# --- 3. A VXLAN tunnel across it: VNI 100. -----
sudo ip netns exec h1 ip link add vx0 type vxlan id 100 \
    remote 192.168.50.2 local 192.168.50.1 dstport 4789 dev u1
```

```

sudo ip netns exec h2 ip link add vx0 type vxlan id 100 \
    remote 192.168.50.1 local 192.168.50.2 dstport 4789 dev u2
sudo ip netns exec h1 ip addr add 172.16.0.1/24 dev vx0
sudo ip netns exec h2 ip addr add 172.16.0.2/24 dev vx0
sudo ip netns exec h1 ip link set vx0 up
sudo ip netns exec h2 ip link set vx0 up
sudo ip netns exec h1 ip -d link show vx0 | grep -o 'vxlan id 100'

# --- 4. Small packets work. -----
sudo ip netns exec h1 ping -c 2 172.16.0.2

# --- 5. Now the black hole: large packets, don't-fragment set. -----
# The underlay is 1500; VXLAN needs 50; so 1450 is the real limit.
sudo ip netns exec h1 ip link set vx0 mtu 1500 # deliberately wrong
echo "---- 1400 bytes: fits ----"
sudo ip netns exec h1 ping -c 2 -M do -s 1400 172.16.0.2 || true
echo "---- 1472 bytes: does not fit, and will not say so ----"
sudo ip netns exec h1 ping -c 2 -W 2 -M do -s 1472 172.16.0.2 || \
    echo "no reply -- exactly the worked example's symptom"

# --- 6. Fix 1: lower the overlay MTU to the arithmetic. -----
sudo ip netns exec h1 ip link set vx0 mtu 1450
sudo ip netns exec h2 ip link set vx0 mtu 1450
sudo ip netns exec h1 ping -c 2 -M do -s 1422 172.16.0.2

# --- 7. Fix 2: raise the underlay instead, and keep 1500 inside. -----
sudo ip netns exec h1 ip link set u1 mtu 9000
sudo ip netns exec h2 ip link set u2 mtu 9000
sudo ip netns exec h1 ip link set vx0 mtu 8950
sudo ip netns exec h2 ip link set vx0 mtu 8950
sudo ip netns exec h1 ping -c 2 -M do -s 8000 172.16.0.2

# --- 8. Watch the encapsulation on the wire. -----
sudo timeout 6 ip netns exec h2 tcpdump -ni u2 -c 4 udp port 4789 &
sleep 1
sudo ip netns exec h1 ping -c 3 172.16.0.2 >/dev/null ; wait || true
# Each capture line is one outer packet carrying one whole inner frame.

# --- 9. Tear down. -----
for n in g1 g2 h1 h2; do sudo ip netns del "$n" 2>/dev/null || true; done
sudo ip link del br-vss 2>/dev/null || true

```

What to notice. Step 5 is the fault that costs the most time in production, reproduced in two commands, and the difference between the working and failing case is 72 bytes. Step 8 shows why: the capture is full-size outer packets, each carrying a frame that was never meant to be carried.

🔍 Checkpoint — Test Yourself

1. An overlay uses Geneve with 16 bytes of options, over IPv6. Outer headers are Ethernet 14, IPv6 40, UDP 8, Geneve 8 + 16. Give the total overhead and the largest inner frame on a 1500-byte underlay.
2. Why does a PMTUD black hole present as a hang rather than an error, and what single firewall change makes it present honestly?
3. A tenant must be isolated from all others, and their guests are spread over three buildings on different IP subnets. Explain why VLANs cannot do this and what you would use.

 Chapter Summary

- A virtual switch forwards between guests in software. It can express rules about identity rather than MAC addresses, it moves with the guest, and it is made of your host's CPU.
- A VLAN tag is 12 bits: 4094 networks, confined to one layer-2 domain.
- VXLAN puts the guest's whole frame inside UDP. The VNI is 24 bits — 16.7 million networks — and the outer packet is ordinary IP, so the overlay crosses any routed network.
- Encapsulation costs 50 bytes, so the inner MTU is 1450 on a 1500-byte underlay. Leave the guest at 1500 and large transfers hang silently.
- Fix it by raising the underlay to jumbo frames, lowering guest MTU, or clamping MSS — in that order of preference.
- SDN separates the control plane from the data plane. Pure OpenFlow did not win; the idea of central intent with programmed devices did.
- Most east–west traffic never reaches a physical port, so monitoring must sit at the virtual switch.

 Review Questions

1. **(MCQ)** The VXLAN network identifier is: (a) 12 bits, giving 4094 networks; (b) 16 bits; (c) 24 bits, giving about 16.7 million; (d) 32 bits.
2. **(MCQ)** VXLAN encapsulation on an Ethernet/IPv4 underlay adds: (a) 4 bytes; (b) 24 bytes; (c) 50 bytes; (d) 100 bytes.
3. **(Short)** Give two things a virtual switch can do that a physical switch cannot, with the reason each is possible.
4. **(Short)** Explain why a VLAN cannot stretch a tenant's network across two routed sites, and how an overlay can.
5. **(Short)** State the three MTU fixes in order of preference, with the drawback of each.
6. **(Applied)** A cluster is being built on a network whose switches cannot do jumbo frames, with guests managed by a team that will not change their images. Choose an approach, state exactly what you would configure and where, and name the traffic your choice will not protect.
7. **(Applied)** A security team asks why their intrusion sensor on the top-of-rack switch sees almost no traffic between application servers that are known to be busy. Explain the three mechanisms that could be responsible and say how you would confirm which it is.

Cloud Service and Deployment Models

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Storage, Network and Cloud Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the five characteristics NIST requires of a cloud, and **test** a given service against them.
- **Distinguish** IaaS, PaaS, SaaS and FaaS by what the provider operates and what you still own.
- **Explain** the shared responsibility model and **identify** which side a given failure falls on.
- **Compute** a three-year total cost of ownership for a workload on premises and in a cloud, and **say** which assumption dominates.
- **Choose** a deployment and pricing model for a stated requirement, and **state** what it commits you to.

🔗 Before You Start

Everything so far. An instance is Chapter 5's hypervisor, Chapter 6's scheduler, Chapter 7's virtio, Chapter 12's thin pool and Chapter 13's overlay, rented by the hour. This chapter adds no mechanism — only the commercial arrangement around it.

📖 Key Terms in This Chapter

NIST definition • on-demand self-service • broad network access • resource pooling • rapid elasticity • measured service • IaaS • PaaS • SaaS • FaaS • serverless • public cloud • private cloud • hybrid • multi-cloud • region • availability zone • instance family • on-demand • reserved instance • savings plan • spot instance • egress • shared responsibility • total cost of ownership • lock-in

14.1 What Makes a Cloud a Cloud

“Cloud” is used loosely enough to be useless as a technical term, so it is worth having a test. The US National Institute of Standards and Technology published one in 2011 and it has aged well: five characteristics, all five required.

Characteristic	What it requires	What fails the test
On-demand self-service	You provision capacity yourself, without talking to anybody	A hosting provider where a new server is a support ticket and two days
Broad network access	Reachable over standard protocols from standard clients	A service requiring a leased line and a vendor client
Resource pooling	Many tenants share one pool; you do not know or care which hardware	A dedicated rack assigned to you permanently

Characteristic	What it requires	What fails the test
Rapid elasticity	Capacity grows and <i>shrinks</i> quickly, apparently without limit	Scaling up in minutes but requiring a contract change to scale down
Measured service	Usage is metered and billed by what you used	A flat annual fee regardless of consumption

💡 The Clause That Catches Most “Private Clouds”

Four of the five are easy to satisfy with a well-run vSphere estate. The one that is not is *rapid elasticity*, and specifically the word *shrinks*.

A department that buys three hosts can provision guests on demand, pool them, meter them and reach them over the network. What it cannot do is return capacity. The hardware is bought; shutting a guest down saves a little electricity and nothing else. Elasticity in the NIST sense requires that unused capacity stops costing money, and that requires somebody else to be able to rent it.

This is the honest difference between a private cloud and a well-managed virtualization estate, and it is worth stating plainly because a great deal of marketing depends on blurring it. What a private cloud genuinely buys is the *interface* — self-service, API-driven, metered — which is valuable on its own. It does not buy the economics.

14.2 The Service Models

The models differ in one thing: where the line falls between what the provider operates and what you do.

	On premises	IaaS	PaaS	SaaS	FaaS
application	you	you	you	your data	your code
data	you	you	you	provider	you
runtime	you	you	provider	provider	provider
middleware	you	you	provider	provider	provider
OS	you	you	provider	provider	provider
hypervisor	you	provider	provider	provider	provider
servers	you	provider	provider	provider	provider
storage and network	you	provider	provider	provider	provider

The red line is the boundary of responsibility, and it only ever moves up. Everything below it is operated by somebody else — and everything below it is also something you can no longer tune, patch on your own schedule, or debug when it misbehaves.

Figure 14.1: The service models as one picture. FaaS sits beside SaaS rather than beyond it, because what you still own there is code rather than configuration.

💡 “Serverless” Means Somebody Else’s Server, Started Fast

FaaS deserves one clarification because the name actively misleads. There are servers. What is removed is any notion of a server you provision, size or keep running: you supply a function, the provider runs it when something calls it, and you pay per invocation and per millisecond. Chapter 11 is how it is implemented. A function invocation on AWS Lambda runs in a Firecracker microVM, which is why the model is economically possible at all: an isolation boundary that costs 125 ms and 5 MB can be created per request, and one that costs 25 s and 400 MB cannot.

So the commercial model in this chapter rests directly on the engineering of the last. That is the clearest instance in this book of a technical property — boot time — determining what can be sold.

14.3 Deployment Models, and What an Instance Is

Public is a provider’s shared pool. **Private** is a pool for one organisation, on its premises or hosted. **Hybrid** connects the two, with workloads placed by policy. **Multi-cloud** spreads across more than one provider — usually for leverage or regulatory reasons, occasionally for availability, and always at the cost of using the least-common-denominator of both.

A provider’s capacity is organised as **regions** (a geographic area, with its own pricing and its own legal jurisdiction) containing **availability zones** (independent data centres, separate power and cooling, linked by low-latency fibre). The rule that follows: spread across zones for availability, across regions for disaster recovery and for data residency.

⚠ An instance type is a promise about scheduling, not just a size

Instance families are the vocabulary of Chapter 6 made commercial. General purpose, compute optimised, memory optimised and storage optimised differ in their core-to-memory ratio — and, less visibly, in whether their vCPUs are oversubscribed.

A *burstable* instance (AWS’s **t** family, and its equivalents) is explicitly oversubscribed: you are granted a baseline fraction of a core and accumulate credits when below it. Exhaust the credits and you are throttled to the baseline, which may be 5% of a core. The ready time of Chapter 6 is here sold as a product feature, and a great many “the server suddenly became slow after three weeks” incidents are exhausted CPU credits.

Read the oversubscription policy of any instance family before benchmarking on it, and never benchmark on a burstable one.

14.4 Paying For It

Three ways to buy the same instance, differing in what you commit to.

Model	You commit to	Typical saving	Suitable for
On-demand	Nothing	—	Unpredictable load, development, anything short-lived
Reserved / savings plan	One or three years of spend	34% for one year, 55% for three (no upfront, EC2 instance plan)	Steady baseline load you are confident will still exist
Spot	Nothing, but can be reclaimed on short notice	70–90%	Batch, CI, rendering, anything that can checkpoint and restart

📊 Worked Example: Three Years, On Premises and in the Cloud

The forty workloads of Chapter 1, each needing 4 vCPUs and 16 GB. Compare three years on premises with three years on AWS. Figures are US list prices for **us-east-1**, checked in October 2026; **re-check before quoting them**, because they move.

On premises. From Chapter 1: three hosts, 13.3:1 consolidation.

- Hardware: $3 \times \$18,000 = \$54,000$

- Shared storage array: \$20,000
- Power and cooling: 2.1 kW at PUE 1.6 is 3.36 kW; $3.36 \times 8,760 \times 3 = 88,300$ kWh at \$0.12 = \$10,600
- Rack space and facility: \$500 a month = \$18,000
- Staff: 0.3 of an engineer at \$45,000 = \$40,500

$$\text{total} = 54,000 + 20,000 + 10,600 + 18,000 + 40,500 = \mathbf{\$143,100}$$

Cloud, on demand. An `m7i.xlarge` is 4 vCPU and 16 GB at \$0.2016 per hour:

$$0.2016 \times 8,760 = \$1,766 \text{ per instance-year}$$

$$1,766 \times 3 \times 40 = \mathbf{\$211,900}$$

Plus 100 GB of gp3 storage each at \$0.08 per GB-month ($40 \times 100 \times 0.08 \times 36 = \$11,520$) and 2 TB a month of egress at \$0.09 per GB (\$6,480 over three years):

$$211,900 + 11,520 + 6,480 = \mathbf{\$229,900}$$

Cloud, three-year reservation. A three-year EC2 instance savings plan with no upfront payment saves 55%:

$$211,900 \times 0.45 = \$95,400$$

$$95,400 + 11,520 + 6,480 = \mathbf{\$113,400}$$

Step 1 — the comparison.

- On premises: \$143,100
- Cloud, on demand: \$229,900 — **61% more**
- Cloud, three-year reserved: \$113,400 — **21% less**

Step 2 — so which is it? Both of the usual slogans are wrong. The same workload on the same provider is either substantially cheaper or substantially more expensive than owning the hardware, and *the only thing that changed was the purchasing decision*. A team that lifts and shifts onto on-demand instances and never revisits it will conclude the cloud is expensive, and will be right about their own bill and wrong about the cloud.

Step 3 — what the arithmetic leaves out, in both directions.

Favouring the cloud: the on-premises figure assumes the three hosts are right. Buy for peak and they are idle; buy for average and you have an outage at peak. It also ignores the refresh in year four, the spare host you should have, and the fact that 0.3 of an engineer is not a thing you can hire.

Favouring on premises: the cloud figure assumes forty instances running continuously for three years, which is the case least suited to the cloud. If half the workloads could run only in office hours — 8×5 instead of 24×7 , a 76% reduction for those — the cloud figure falls by roughly \$36,000 more, and nothing equivalent is available on premises.

Step 4 — the honest conclusion. The decision is not technical. It turns on how variable the load is, how confident you are of a three-year commitment, and whether you have the staff. Steady and predictable favours owning; variable or uncertain favours renting; and whichever you choose, revisiting the purchasing model annually is worth more than the choice itself.

⚠ Egress is the charge that surprises people

Compute, storage and inbound traffic are priced in ways that feel proportionate. Outbound data — **egress** — is not. At around \$0.09 per GB, moving 100 TB out of a cloud costs roughly \$9,000, and moving it *in* is free.

That asymmetry is the economic shape of **lock-in**: nothing prevents you leaving, and the bill for leaving grows with how much you have stored. Price the exit when you design the architecture, not when you need it. Keep the data you may have to move somewhere its egress is cheap, and know the figure.

14.5 Shared Responsibility

Every provider publishes a version of the same diagram, and every provider's version says the same thing: they are responsible for the security *of* the cloud, and you are responsible for security *in* it.

Concretely, under IaaS the provider owns the physical facility, the hardware, the hypervisor and the network fabric. You own the guest operating system and its patching, your application, your data and its encryption, your identity and access configuration, and your firewall rules.

💡 Which Side Is This Failure On?

Nearly every large cloud data breach of the past decade has fallen on the customer's side of that line, and usually on the same two items.

- An object store left publicly readable. The provider's default is private; somebody changed it.
- Credentials committed to a public repository, or an over-broad role attached to an instance.

Neither is a cloud vulnerability. Both are configuration, and both are on your side of the diagram. Meanwhile the provider's side — the hypervisor of Chapter 5, the isolation of Chapter 19 — has a strong record, which is the opposite of what most risk discussions assume.

The practical reading: when evaluating cloud risk, spend your effort on your own configuration, your identity model and your data, because that is where the incidents actually are.

❗ Common Pitfalls

- **Comparing on-demand cloud prices with owned hardware.** It is the most expensive way to buy and the least representative. Compare like with like: a three-year commitment against a three-year asset.
- **Calling a vSphere estate a private cloud.** Check it against the five characteristics, especially whether capacity can be *returned*.
- **Lifting and shifting without resizing.** A 16-vCPU guest that used two becomes a 16-vCPU instance billed for sixteen. The right-sizing of Chapter 6 is worth more in the cloud, because waste is now a monthly invoice.
- **Benchmarking on a burstable instance.** You are measuring credit balance, not performance.
- **Ignoring egress until the exit.** Price it at design time.
- **Assuming multi-cloud buys availability.** It buys the intersection of two feature sets, two operational models and two sets of outages. It buys leverage and sometimes compliance; availability is usually cheaper within one provider across zones.
- **Treating the shared responsibility diagram as marketing.** It is the contract, and it is where the incidents are.

Try It Yourself: One Workload, Three Clouds

Launches the same instance on AWS, Azure and GCP free tiers, compares what each exposes, and tears everything down. **Step 7 is not optional:** a forgotten instance bills until somebody notices. Expect fifty minutes.

```
#!/usr/bin/env bash
# ch14_threeclouds.sh -- one guest on three providers, then delete.
set -euo pipefail

# --- 1. Authenticate to whichever accounts you have. -----
aws sts get-caller-identity 2>/dev/null || echo "no aws credentials"
az account show --query name 2>/dev/null || echo "no azure login"
gcloud config get-value project 2>/dev/null || echo "no gcp project"

# --- 2. AWS: a free-tier instance. -----
AMI=$(aws ssm get-parameters --names \
  /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-6.1-x86_64 \
  --query 'Parameters[0].Value' --output text)
aws ec2 run-instances --image-id "$AMI" --instance-type t3.micro \
  --count 1 --tag-specifications \
  'ResourceType=instance,Tags=[{Key=Name,Value=vss-lab}]' \
  --query 'Instances[0].InstanceId' --output text | tee aws-id.txt

# --- 3. Azure: the same shape. -----
az group create -n vss-lab -l eastus -o none
az vm create -g vss-lab -n vss1 --image Ubuntu2204 --size Standard_B1s \
  --generate-ssh-keys --public-ip-sku Basic -o table

# --- 4. GCP: likewise. -----
gcloud compute instances create vss1 --machine-type=e2-micro \
  --zone=us-central1-a --image-family=debian-12 \
  --image-project=debian-cloud

# --- 5. Ask each guest what it is running on. -----
# Every provider exposes a metadata service on 169.254.169.254. The
# answers show you the hypervisor of Chapter 5 underneath all three.
cat <<'PROBE' > probe.sh
systemd-detect-virt
sudo dmidecode -s system-manufacturer 2>/dev/null
curl -s -H 'Metadata-Flavor: Google' \
  169.254.169.254/computeMetadata/v1/instance/machine-type 2>/dev/null
curl -s 169.254.169.254/latest/meta-data/instance-type 2>/dev/null
PROBE
echo "copy probe.sh to each instance and run it"

# --- 6. What each one costs, from the provider's own API. -----
aws pricing get-products --service-code AmazonEC2 --region us-east-1 \
  --filters 'Type=TERM_MATCH,Field=instanceType,Value=m7i.xlarge' \
  'Type=TERM_MATCH,Field=location,Value=US East (N. Virginia)' \
  --max-results 1 --query 'PriceList[0]' --output text 2>/dev/null \
  | head -c 400 || echo "needs the pricing:GetProducts permission"

# --- 7. TEAR DOWN. Do this even if a step above failed. -----
aws ec2 terminate-instances --instance-ids "$(cat aws-id.txt)" \
  --query 'TerminatingInstances[0].CurrentState.Name' --output text || true
az group delete -n vss-lab --yes --no-wait || true
gcloud compute instances delete vss1 --zone=us-central1-a --quiet || true

# --- 8. Confirm nothing survives. -----
aws ec2 describe-instances --filters 'Name=tag:Name,Values=vss-lab' \
  'Name=instance-state-name,Values=running,pending' \
  --query 'Reservations[].Instances[].InstanceId' --output text
```

```
az group list --query "[?name=='vss-lab'].name" -o tsv
gcloud compute instances list --filter='name=vss1' --format='value(name)'
# All three should print nothing.
```

What to notice. Step 5 is the point of the whole chapter: three providers, three billing models, three APIs — and underneath, the same hypervisor, the same virtio devices and the same overlay network you built by hand in Chapter 13. The cloud is this book, rented.

🔍 Checkpoint — Test Yourself

1. A university runs a self-service vSphere portal where departments provision guests themselves and are charged internally per guest-month. Test it against the five NIST characteristics and say which, if any, it fails.
2. A workload runs 10 hours a day, 5 days a week. Using the worked example's hourly rate, compute its annual on-demand cost, and say why a three-year reservation would probably be a mistake.
3. An object store containing student records was found publicly readable. Which side of the shared responsibility line is this on, and what would you change to stop it recurring?

📋 Chapter Summary

- NIST's five characteristics are a usable test. Most “private clouds” fail on elasticity, because capacity cannot be returned.
- IaaS, PaaS, SaaS and FaaS differ only in where the responsibility line falls. It only ever moves upward, and everything below it is also something you can no longer tune or debug.
- Serverless is somebody else's server started in 125 ms. It is economically possible because of Chapter 11.
- Spread across availability zones for availability, across regions for disaster recovery and data residency.
- On demand, the same workload cost 61% more than owning the hardware; with a three-year commitment it cost 21% less. The purchasing decision mattered more than the platform.
- Egress is charged and ingress is not. That asymmetry is what lock-in actually is; price it at design time.
- You are responsible for security *in* the cloud. Nearly every major breach has been on the customer's side of that line.

🔧 Review Questions

1. (MCQ) The NIST characteristic most private clouds fail is: (a) measured service; (b) rapid elasticity; (c) broad network access; (d) resource pooling.
2. (MCQ) Under IaaS, patching the guest operating system is the responsibility of: (a) the provider; (b) the customer; (c) shared equally; (d) the hypervisor vendor.
3. (Short) Explain why a burstable instance is a poor choice for benchmarking, in terms of Chapter 6's vocabulary.
4. (Short) State what egress pricing has to do with lock-in, with a worked figure.
5. (Short) Why does FaaS depend on the engineering of Chapter 11?
6. (Applied) Repeat the worked example for a workload that runs only during a four-month teaching semester each year. Give the three figures and the recommendation, and state which assumption your answer is most sensitive to.

7. (**Applied**) A department proposes moving its entire estate to a single public cloud. Write the five questions you would require answers to before approving it, and say what answer to each would change the recommendation.

Desktop and Application Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Storage, Network and Cloud Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what VDI moves and what stays, and **contrast** persistent with non-persistent desktops.
- **Describe** what a remote display protocol actually sends, and **explain** why latency matters more than bandwidth.
- **Compute** the bandwidth and storage a VDI deployment needs, and **identify** the resource that fails first.
- **Distinguish** desktop virtualization from application streaming and from remote browser isolation.
- **Decide** whether VDI is the right answer to a stated requirement, and **say** what it costs.

🔗 Before You Start

Chapter 12 (thin provisioning and linked clones — a non-persistent desktop estate is the largest copy-on-write deployment most organisations ever run) and Chapter 6 (overcommit, which VDI pushes harder than any server workload).

📖 Key Terms in This Chapter

VDI • virtual desktop • persistent desktop • non-persistent desktop • golden image • linked clone • instant clone • profile management • remote display protocol • PCoIP • Blast • ICA/HDX • RDP • boot storm • IOPS • session host • published application • application streaming • remote browser isolation

15.1 Moving the Computer, Leaving the Screen

Everything so far has virtualized a server. **Virtual desktop infrastructure** does it to the thing on somebody's desk: the operating system, the applications and the files run in the data centre, and what travels to the user is pixels in one direction and keystrokes and mouse movements in the other.

The motives are the four of Chapter 1, pointed at a different problem:

- **Isolation and control** — the data never leaves the data centre, which is the reason regulated industries buy it;
- **Consolidation** — a hundred desktops on a few hosts rather than a hundred machines under a hundred desks;
- **Encapsulation** — a broken desktop is replaced in ninety seconds, because it is a file;
- **Access** — the same desktop from a laptop, a thin client or a tablet, from anywhere.

☰ Persistent and Non-Persistent Desktops

A **persistent** desktop is one virtual machine per user, kept between sessions. Users install software, change settings and leave files on it, exactly as on a physical PC. Familiar, flexible, and it must be patched, backed up and managed one hundred times over — the server sprawl of Chapter 1, re-created virtually.

A **non-persistent** desktop is created from a **golden image** at logon and destroyed at logoff. Every session starts identical and known. Patching means updating one image. The user's identity has to be reassembled each time from **profile management** — documents on a file share, settings in a profile container — and anything not captured by that is lost at logoff, which users discover the hard way.

Non-persistent is the design that makes the economics work, and profile management is where it succeeds or fails. Almost every unhappy VDI deployment is a non-persistent one whose profile handling was underestimated.

💡 Linked and Instant Clones: Chapter 12, Applied at Scale

A thousand non-persistent desktops from one image are a thousand copy-on-write overlays over one read-only base — exactly the structure of Chapter 12, and of Chapter 8's overlays one layer up.

A **linked clone** shares the base disk and keeps a delta per desktop: a 60 GB image plus perhaps 2 GB each, so a thousand desktops cost 2 TB rather than 60 TB. An **instant clone** goes further and forks a *running* machine's memory as well, so a new desktop is available in one or two seconds rather than ninety.

The read amplification warning of Chapter 12 applies in full. Every one of those thousand desktops reads the same base through the same chain, so the base must be on the fastest storage you have and must stay in cache. VDI is where storage design stops being a background concern.

15.2 What Travels Over the Wire

A remote display protocol is not a video stream. It inspects what changed on the screen and sends the cheapest representation of that change: a rectangle of pixels, a command to move a region, a glyph, or — when the content really is video — an H.264 or H.265 stream with the work done on a server GPU.

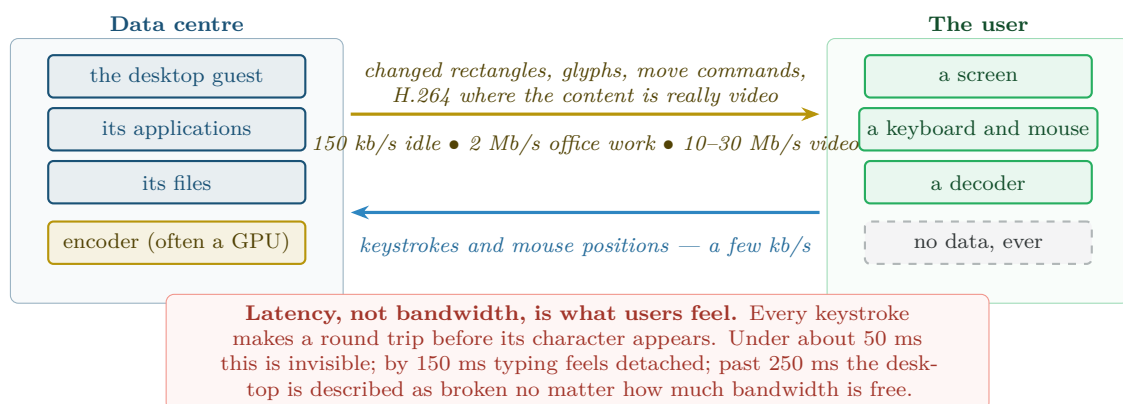


Figure 15.1: A remote desktop session. The asymmetry is extreme — megabits down, kilobits up — and the quality the user perceives is governed by the small return arrow, not the large one.

⚠ The two failure modes nobody provisions for

The boot storm. Nine o'clock, and nine hundred people log in within twenty minutes. Nine hundred desktops boot, read the same base image, load the same profile and start the same antivirus scan. Storage that comfortably serves the estate all afternoon delivers a fraction of what is suddenly demanded, and the queue makes logons take fifteen minutes.

It is a storage *IOPS* problem, not a capacity or a CPU problem, and it is the single commonest reason a VDI pilot succeeds and the rollout fails: fifty users never produce it and nine hundred do.

The antivirus scan. Every desktop scanning its disk at the same hour multiplies the same effect, with no user to blame. Stagger it, or use an agentless scanner that works from outside the guest.

📊 Worked Example: Sizing a 500-Seat Deployment

Five hundred knowledge workers, 2 vCPU and 4 GB each, Windows desktops on an image of 60 GB. Size the hosts, the storage and the network, and find what fails first.

Step 1 — memory, which sizes the hosts. With 10:1 vCPU oversubscription acceptable for desktops — they are idle most of the time — memory is again the constraint. Allow no memory overcommit for desktops, since they are interactive and swapping is immediately felt:

$$500 \times 4 = 2,000 \text{ GB}$$

On hosts of 512 GB, reserving 15% for the hypervisor and headroom:

$$\left\lceil \frac{2,000}{512 \times 0.85} \right\rceil = \left\lceil \frac{2,000}{435} \right\rceil = 5 \text{ hosts}$$

Add one for N+1: **6 hosts.**

Step 2 — check CPU. $500 \times 2 = 1,000$ vCPU on six hosts of 64 cores = 384 cores:

$$\sigma_{\text{cpu}} = \frac{1,000}{384} = 2.6$$

Comfortable for desktop workloads, which sit idle between keystrokes.

Step 3 — storage capacity, with linked clones.

$$\underbrace{60}_{\text{golden image}} + \underbrace{500 \times 2}_{\text{deltas}} = 1,060 \text{ GB}$$

Against 500 full copies at 60 GB, which would be 30 TB. A **28:1** saving, and the reason non-persistent desktops are affordable.

Step 4 — storage IOPS, steady state. A knowledge-worker desktop averages about 10 IOPS, roughly 80% writes:

$$500 \times 10 = 5,000 \text{ IOPS}$$

Any modern NVMe array handles this without noticing.

Step 5 — storage IOPS, at the boot storm. A booting Windows desktop demands around 100 IOPS for perhaps two minutes. Nine hundred — here 500 — logging in over 20 minutes means roughly 50 booting concurrently at any moment if arrivals are even:

$$50 \times 100 = 5,000 \text{ IOPS extra}$$

But arrivals are *not* even. If half the estate logs in during the busiest five minutes:

$$\frac{250 \text{ desktops} \times 2 \text{ min boot}}{5 \text{ min}} = 100 \text{ concurrent} \quad 100 \times 100 = 10,000 \text{ IOPS}$$

plus the 5,000 steady state: **15,000 IOPS, three times the all-day figure, for five minutes.**

Step 6 — network. At 2 Mb/s per session for office work:

$$500 \times 2 = 1,000 \text{ Mb/s} = 1 \text{ Gb/s}$$

If a tenth are watching video at 15 Mb/s:

$$450 \times 2 + 50 \times 15 = 900 + 750 = 1,650 \text{ Mb/s}$$

A 10 Gb/s data-centre uplink is ample; a site's *internet* link may not be, and remote users' home connections certainly vary.

Step 7 — what fails first. Not memory, not CPU, not capacity, not average IOPS. **Peak IOPS at logon**, by a factor of three over everything else, for five minutes a day. Size the storage for step 5 and the rest follows; size it for step 4 — which is what the average suggests — and the estate works beautifully except at nine o'clock, which is when everybody forms their opinion of it.

15.3 Virtualizing the Application Instead

Delivering a whole desktop is often more than the problem requires.

Published applications (Citrix Virtual Apps, Azure Virtual Desktop in multi-session mode) run many users' sessions on one shared **session host** operating system and present individual application windows rather than a desktop. Far denser than VDI — dozens of users per host rather than a handful — because the operating system is shared. The cost is the one from Chapter 8: shared kernel, so one user's badly-behaved application affects the others, and software that assumes it owns the machine misbehaves.

Application streaming goes further and delivers the application to the user's own device, in a container that keeps it from touching the local installation.

Remote browser isolation applies the idea to the one application everybody runs. The browser executes in the data centre or at a provider, and only rendered output reaches the endpoint, so a malicious page compromises a container that is discarded afterwards. It is a security control rather than a productivity one, and Chapter 19 returns to it.

	Isolation	Density	Use it when
Persistent VDI	One guest per user	Lowest	Users need to install software, or have genuinely individual environments
Non-persistent VDI	One guest per session	Moderate	A standard desktop for many similar users; patching one image
Published apps	Shared session host	Highest	A few applications, many users, no desktop needed
Application streaming	Container on the endpoint	N/A	Local performance with central control of versions

	Isolation	Density	Use it when
Browser isolation	Disposable container per session	High	The threat is the web, not the application

💡 When VDI Is the Wrong Answer

VDI is expensive, operationally demanding and unforgiving of under-sizing. It is worth it when at least one of these is true:

- data must not leave the data centre, for regulatory or contractual reasons;
- users are numerous, similar, and need a standard environment;
- access is needed from many places or from untrusted devices;
- the endpoints are thin clients with a long life and little to steal.

It is the wrong answer when users are few, when their needs are individual, when the work is graphically heavy without a GPU budget (Chapter 20), or — most often — when the real problem is patch management and a better endpoint management tool would cost a tenth as much.

The question to ask of any VDI proposal is: *what is it that the data centre must hold?* If there is a crisp answer, VDI is likely right. If the answer is “it would be tidier”, it is not.

❗ Common Pitfalls

- **Sizing storage on average IOPS.** The boot storm is three times the average and it decides the user’s opinion.
- **Piloting with fifty users and extrapolating.** Fifty users never produce a boot storm. Pilot the peak, not the mean.
- **Underestimating profile management.** Non-persistent desktops fail on this more than on anything technical.
- **Optimising bandwidth and ignoring latency.** Users feel the round trip on every keystroke. A fat, distant link is worse than a thin, near one.
- **Overcommitting desktop memory as if they were servers.** A swapping desktop is noticed instantly by the person using it.
- **Running in-guest antivirus on a schedule.** Every desktop scanning at once reproduces the boot storm with no users to show for it.
- **Treating VDI as a cost-saving measure.** It moves cost from endpoints to the data centre and adds licensing. Buy it for control, access or compliance, and be pleased if it also saves money.

🔧 Try It Yourself: A Remote Desktop, and What It Sends

Builds a remote desktop session and measures what crosses the wire under three kinds of work, then adds latency and sees which matters. Expect thirty-five minutes.

```
#!/usr/bin/env bash
# ch15_vdi.sh -- a remote desktop, measured, then made to feel slow.
set -euo pipefail

# --- 1. A desktop guest, from the Chapter 1 image. -----
sudo virsh start vss1 || true ; sleep 25
ip_of () {
    sudo virsh domifaddr "$1" | awk '/ipv4/{split($4,a,"/"); print a[1]}'
}
IP=$(ip_of vss1)
```

```
G () { ssh -o StrictHostKeyChecking=no "ubuntu@$IP" "$1"; }
G "sudo apt-get -qq install -y xfce4 xrdp firefox && sudo systemctl \
  enable --now xrdp"

# --- 2. Connect with an RDP client and leave it idle. -----
# From the host: xfreerdp /v:$IP /u:ubuntu, or any RDP client.
sudo apt-get install -y freerdp2-x11 iftop tcpdump
echo "connect now, then press return"; read -r _

# --- 3. Measure the three cases of the figure. -----
measure () {
  echo "=== $1 ==="
  sudo timeout 20 tcpdump -ni any "host $IP and port 3389" -q 2>/dev/null \
    | wc -l
}
measure "idle -- do nothing for 20 s"
echo "now type continuously in a text editor, then press return"; read -r _
measure "typing"
echo "now play a video full screen, then press return"; read -r _
measure "video"
# Packet counts stand in for bandwidth; the ratios are what matter.

# --- 4. Now the thing users actually feel: latency. -----
# Add 150 ms each way and use the session again. Nothing else changes.
sudo tc qdisc add dev virbr0 root netem delay 150ms
echo "use the session now -- type a sentence. Press return when done."
read -r _
sudo tc qdisc del dev virbr0 root
echo "and again with no added delay. The bandwidth was identical."

# --- 5. The boot storm, at small scale. -----
# Clone four desktops from one base and start them together; watch IOPS.
for n in 1 2 3 4; do
  sudo qemu-img create -f qcow2 -b /var/lib/libvirt/images/vss1.qcow2 \
    -F qcow2 "/var/lib/libvirt/images/d$n.qcow2" 20G
done
ls -lh /var/lib/libvirt/images/d*.qcow2 # linked clones: nearly empty
iostat -x 2 2 > /tmp/before.txt
for n in 1 2 3 4; do
  sudo virt-install --name "d$n" --memory 1024 --vcpus 1 \
    --disk "/var/lib/libvirt/images/d$n.qcow2",format=qcow2 \
    --import --graphics none --noautoconsole --os-variant ubuntu22.04 &
done
wait ; iostat -x 5 3 | tail -12 # the storm, four desktops wide

# --- 6. Tear down. -----
for n in 1 2 3 4; do
  sudo virsh destroy "d$n" 2>/dev/null || true
  sudo virsh undefine "d$n" --remove-all-storage 2>/dev/null || true
done
sudo virsh destroy vss1 || true
```

What to notice. Step 4 is the chapter. The bandwidth in that test was exactly what it was in step 3, and the session became unusable anyway. Then look at step 5's `iostat` output and multiply by a hundred.

🔍 Checkpoint — Test Yourself

1. A 300-seat non-persistent deployment uses a 70 GB golden image with 3 GB deltas. Give the storage capacity needed, and the saving against full clones.

2. At 10 IOPS steady state per desktop, and 120 IOPS for a 90-second boot, estimate peak IOPS if 150 of those 300 users log in within four minutes. Compare with the steady-state figure.
3. A user reports that their remote desktop is “laggy” although the monitoring shows the link is 80% idle. Explain what is probably happening and what you would measure.

✓ Chapter Summary

- VDI runs the desktop in the data centre and sends pixels out, keystrokes back. Buy it for control, access or compliance.
- Persistent desktops are familiar and must be managed individually; non-persistent ones are built from a golden image at logon and succeed or fail on profile management.
- Linked and instant clones are Chapter 12’s copy-on-write at scale — a 28:1 capacity saving in the worked example, and a read-amplified base that must stay fast.
- The display protocol sends changed regions, not video. Bandwidth is asymmetric and modest; *latency* is what users feel, on every keystroke.
- Peak logon IOPS is three times the all-day figure and is what sizing must be based on. It is why pilots succeed and rollouts fail.
- Published applications are denser and share a kernel; browser isolation applies the idea as a security control.

🔧 Review Questions

1. **(MCQ)** The resource that most often limits a VDI deployment is: (a) host memory; (b) peak storage IOPS at logon; (c) network bandwidth; (d) CPU.
2. **(MCQ)** A non-persistent desktop keeps a user’s documents and settings by: (a) storing them in the golden image; (b) profile management on separate storage; (c) the delta disk; (d) it does not keep them.
3. **(Short)** Explain why latency matters more than bandwidth in a remote desktop session, in terms of what happens to a keystroke.
4. **(Short)** Describe a boot storm and give two ways to reduce it.
5. **(Short)** State the trade between published applications and full VDI, naming the chapter whose argument it repeats.
6. **(Applied)** Size a 1,200-seat deployment of 2 vCPU, 8 GB desktops on 768 GB hosts with a 80 GB image and 4 GB deltas. Give host count, capacity, steady and peak IOPS, and bandwidth, and name what fails first.
7. **(Applied)** A faculty proposes VDI so that students can use specialist software from their own laptops. Write the questions you would ask, the two alternatives you would price against it, and the circumstances under which you would recommend each.

PART V

Management, Performance and Security

Everything so far has been how to build the layer. Part V is how to live with one. Fleets are managed, not configured; machines move between hosts while serving traffic; hosts run out of something before they run out of everything, and knowing which is a skill. And the layer that gave you all of this is also a piece of software with an attack surface.

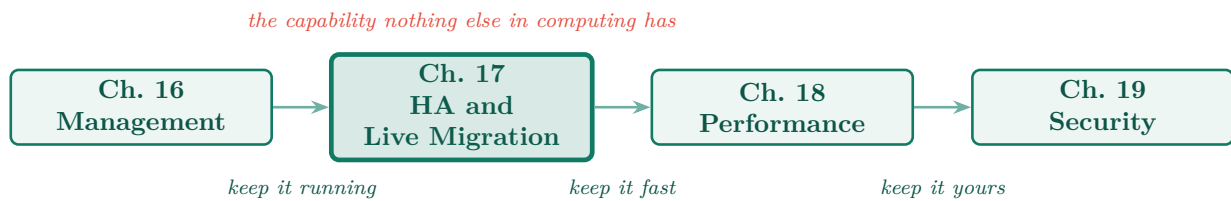


Figure P5.1: Reading path through Part V. Live migration is the one in the middle because it is what the rest is for: a fleet you can manage is a fleet whose machines can move without stopping.

Virtualization Management Platforms

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Management, Performance and Security

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why managing a fleet is a different problem from managing a host, and **name** the capabilities a management layer must add.
- **Compare** vCenter, SCVMM, oVirt, Proxmox and OpenStack by scope and by what each assumes.
- **Write** infrastructure as code that is idempotent, and **explain** what state the tool keeps and why.
- **Describe** the image pipeline — build, publish, instantiate — and **say** what cloud-init does at each boot.
- **Define** configuration drift and **choose** between correcting it and rebuilding.

🔗 Before You Start

Chapter 5 (the four hypervisors, since each management platform assumes one) and Chapter 12 (shared storage, which almost every cluster feature depends on). Chapter 14's self-service characteristic is what this chapter builds on premises.

📖 Key Terms in This Chapter

vCenter • cluster • DRS • resource pool • SCVMM • oVirt • Proxmox VE • OpenStack • Nova • infrastructure as code • declarative • idempotent • Terraform • state file • plan and apply • golden image • Packer • cloud-init • user data • template • configuration drift • immutable infrastructure

16.1 One Host Is Not a Small Fleet

Everything in Parts II and III was about one machine. A hypervisor on its own is a complete, usable product — and it stops being enough at about the third host, for reasons that are all variations on the same one: a decision that was obvious with one host has no obvious answer with twenty.

Question	With one host	With twenty
Where should this guest run?	There is only one answer	A placement decision, against current load, affinity rules and licensing
A host needs patching	Schedule an outage	Evacuate it by live migration and patch it with nobody noticing (Chapter 17)
A host has failed	Everything on it is down	The guests should restart elsewhere, automatically, within minutes

Question	With one host	With twenty
Who may create guests?	Whoever has the root password	Roles, quotas, an API and an audit trail
How was this guest built?	Somebody remembers	Nobody remembers, and it matters
Is the estate consistent?	Trivially	Only if something enforces it

💡 Placement Is the Capability That Pays for the Platform

The feature that justifies a management layer commercially is automated placement — VMware calls it DRS, and the others have equivalents.

The cluster continuously measures load and migrates guests between hosts to balance it. That turns twenty hosts from twenty capacity decisions into one, and it makes maintenance routine: put a host in maintenance mode and its guests leave by themselves.

It also changes the sizing arithmetic of Chapter 6. A single host must be sized for the peak of everything on it. A balanced cluster need only be sized for the peak of the *aggregate*, and peaks that do not coincide stop needing separate headroom — commonly 15–25% less hardware for the same service.

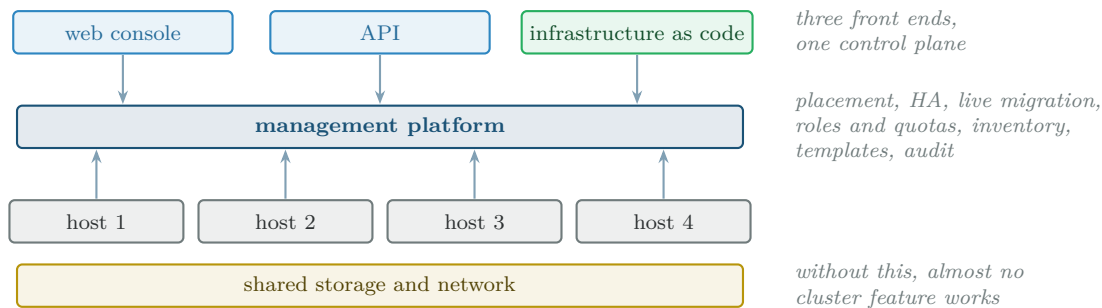


Figure 16.1: The shape every platform in this chapter has. The interesting differences are in scope — how much of the stack the platform claims — not in this diagram.

16.2 The Platforms

	Manages	Scope	Choose it when
vCenter	ESXi	One cluster to a few thousand hosts; the most mature operational tooling there is	You have bought vSphere, and your staff know it
SCVMM	Hyper-V	Fleet management inside a Microsoft estate, with System Center	The estate is already Windows and licensed
oVirt	KVM	The open equivalent of vCenter; Red Hat Virtualization was built on it	You want vCenter’s shape without the licence

	Manages	Scope	Choose it when
Proxmox VE	KVM and LXC	Hypervisor, containers, storage, backup and clustering in one installable product	Small to medium estates; it has taken a visible share of the post-2024 VMware migrations
OpenStack	KVM mostly	An entire <i>cloud</i> : multi-tenancy, quotas, metering, object store, identity, orchestration	You are building a cloud for other people, not managing servers

⚠ OpenStack is not in the same category as the others

It is routinely compared with vCenter and the comparison misleads in both directions. vCenter manages *your* servers for *your* staff. OpenStack provides a *cloud* — the five NIST characteristics of Chapter 14, including tenants, quotas, metering and an API contract you offer to others. It is roughly a dozen interacting services, and that is because the problem is bigger, not because it is badly engineered.

The practical consequence: OpenStack rewards an organisation with a dedicated platform team and punishes one without. Several well-known deployments were abandoned for exactly this reason. If what you need is to run two hundred guests for your own department, it is the wrong tool and Proxmox or oVirt is the right one.

16.3 Infrastructure as Code

A console is fine for one change and hopeless as a system of record. Clicking through a wizard leaves nothing behind that says what was intended, so six months later nobody can say whether a setting was deliberate.

Infrastructure as code makes the declaration the artefact. The definition lives in version control, is reviewed like any change, and the tool makes reality match it — the same reconciliation idea as Chapter 10, applied to infrastructure.

📖 Declarative, Idempotent, and the State File

A declarative tool is given the desired end state, not the steps. Terraform's loop is *plan* — compare desired with actual and show the difference — then *apply*.

Idempotent means applying the same definition twice changes nothing the second time. This is what makes the tool safe to run on a schedule, and it is the property a shell script of `virsh` commands does not have.

To compare desired with actual, the tool must know which real objects it created. That is the **state file**, and it is the part that bites: it is authoritative, it contains secrets, and two people applying at once will corrupt it. Keep it in remote storage with locking, never in the repository.

📋 Worked Example: What Automation Is Worth, and When It Is Not

A team provisions guests by hand: 45 minutes each, including the ticket, the console work and the follow-up corrections. They provision about 12 a month and reconfigure about 30. Decide whether to adopt Terraform.

Step 1 — the current cost.

$$12 \times 45 = 540 \text{ min} = 9 \text{ h a month on new guests}$$

Reconfigurations average 20 minutes:

$$30 \times 20 = 600 \text{ min} = 10 \text{ h a month}$$

$$\text{total} = 19 \text{ h a month} = \mathbf{228} \text{ h a year}$$

Step 2 — the cost after. Writing the module takes 3 days, 24 hours, once. A new guest is then 5 minutes of editing and review; a reconfiguration is 3 minutes:

$$12 \times 5 + 30 \times 3 = 60 + 90 = 150 \text{ min} = 2.5 \text{ h a month}$$

$$= 30 \text{ h a year, plus 24 h to build}$$

Step 3 — first-year saving.

$$228 - (30 + 24) = \mathbf{174} \text{ hours}$$

and 198 hours in every year after. The build pays for itself in about seven weeks.

Step 4 — the benefit that is not in the arithmetic. The 45 minutes included “follow-up corrections” — guests built slightly differently from each other, discovered later. Those corrections do not merely take time; they are the *drift* of the next section, and the saving from eliminating them is larger than the 174 hours and much harder to measure.

Step 5 — and the honest case against. Change the inputs and the answer inverts. A team provisioning *one* guest a month saves $12 \times 40 = 8$ hours a year against a 24-hour build and an ongoing maintenance burden — a clear loss. Automation pays at volume and at repetition, and a tool adopted below that threshold is a liability maintained by someone who uses it too rarely to remember it.

State the threshold explicitly before adopting: *how many of these do we do, and how alike are they?*

16.4 The Image Pipeline

Where does the guest’s disk come from? Three stages, and confusing them is the cause of most “why is this server different” incidents.

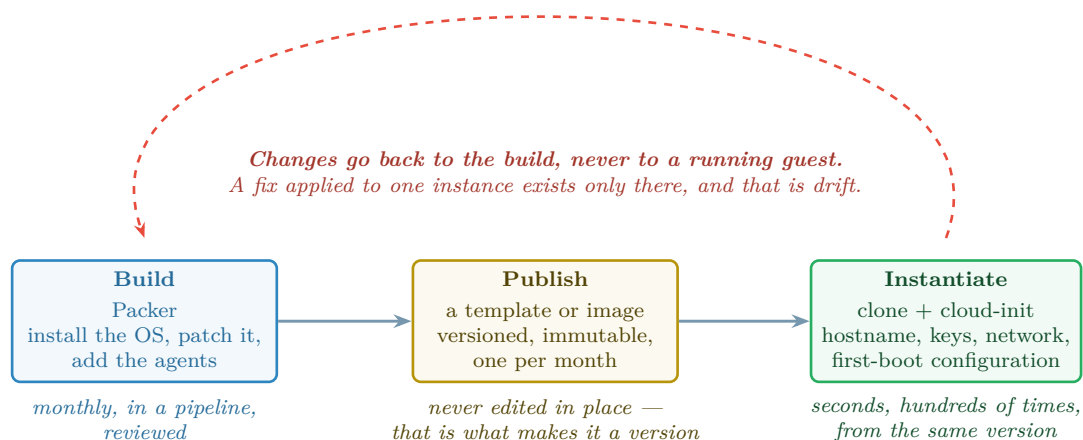


Figure 16.2: The pipeline. The dashed arrow is the discipline: the only supported way to change what a guest contains is to change the image and re-instantiate.

💡 cloud-init: the last mile, every boot

A golden image must be identical for every guest, and every guest must be different — different hostname, address, keys, role. **cloud-init** resolves the contradiction: it runs on first boot, reads **user data** from the platform's metadata service, and applies it.

It is why the Chapter 1 lab could hand a downloaded cloud image a password in a two-line YAML file and have it work. Every cloud in Chapter 14 supports it, and so do libvirt, Proxmox and VMware, which is why one image can be built once and used everywhere.

The rule that follows: *anything that differs between guests belongs in user data, and anything that is the same belongs in the image*. Put a hostname in the image and you will clone a hostname conflict; put a patch in user data and you will apply it at every boot for ever.

16.5 Drift

Configuration drift is the gap between what a system is supposed to be and what it has become: a package installed to debug something at 2 a.m., a limit raised and never recorded, a firewall rule added for a test. Each is small and reasonable. After two years no two servers are alike and nobody can rebuild any of them.

Two responses, and they are genuinely different philosophies.

💡 Converge, or Replace

Convergent management — Ansible, Puppet, Chef — re-applies the desired configuration periodically, correcting what has drifted. It works on long-lived systems, including ones you cannot rebuild, and it is the only option for an estate that already exists. Its weakness is that it corrects only what it is told to check: an unmanaged file stays drifted for ever.

Immutable infrastructure — never change a running system. To patch it, build a new image and replace the instance. Drift becomes structurally impossible, because nothing lives long enough to drift, and every running system is exactly some reviewed version. The cost is that it demands the whole pipeline, fast instantiation and workloads that tolerate replacement — stateless services, or state kept deliberately outside.

Most real estates are both: immutable for the application tier, convergent for the database servers and the appliances that cannot be rebuilt. Knowing which regime a given machine is under — and writing it down — matters more than which one you prefer.

❗ Common Pitfalls

- **Keeping the state file in the repository.** It holds secrets and it must be locked. Remote backend with locking, always.
- **Running apply without reading plan.** The plan is the review. Reading it is the control that stops a one-character change destroying a database.
- **Editing a golden image in place.** Then it is not a version and you cannot say what any guest was built from.
- **Putting per-guest settings in the image.** Clone it and you clone the hostname and the SSH host key.
- **Adopting automation below its threshold.** At one guest a month the tool costs more than it saves and is forgotten between uses.
- **Fixing a drifted server by hand and closing the ticket.** The fix now exists in one place and nowhere else; the next rebuild loses it.
- **Choosing OpenStack to manage a departmental estate.** It is a cloud platform. Match the tool to the problem's size.

Try It Yourself: Declaring a Small Estate

Builds an image with a pipeline, then provisions and reconfigures guests declaratively with Terraform against libvirt. Expect forty-five minutes.

```
#!/usr/bin/env bash
# ch16_iac.sh -- an image pipeline and declarative provisioning.
set -euo pipefail
mkdir -p ~/vsiac && cd ~/vsiac

# --- 1. The build stage: one patched image, made reproducibly. -----
# Packer would do this in a pipeline; here, the same steps by hand.
BASE=https://cloud-images.ubuntu.com/jammy/current
wget -nc -O base.img "$BASE/jammy-server-cloudimg-amd64.img"
cp base.img golden-2026-10.qcow2
sudo virt-customize -a golden-2026-10.qcow2 \
  --update --install qemu-guest-agent,chrony \
  --run-command 'systemctl enable qemu-guest-agent' \
  --truncate /etc/machine-id          # or every clone shares an identity
qemu-img info golden-2026-10.qcow2 | head -4

# --- 2. The publish stage: versioned, and never edited again. -----
sudo mkdir -p /var/lib/libvirt/images/templates
sudo cp golden-2026-10.qcow2 /var/lib/libvirt/images/templates/
sudo chmod 444 /var/lib/libvirt/images/templates/golden-2026-10.qcow2
ls -l /var/lib/libvirt/images/templates/

# --- 3. Declare the estate. -----
cat > main.tf <<'TF'
terraform {
  required_providers {
    libvirt = { source = "dmacvicar/libvirt", version = "~> 0.8" }
  }
}
provider "libvirt" { uri = "qemu:///system" }

variable "guests" {
  type     = map(object({ memory = number, vcpu = number }))
  default = {
    web1 = { memory = 1024, vcpu = 1 }
    web2 = { memory = 1024, vcpu = 1 }
    db1  = { memory = 2048, vcpu = 2 }
  }
}

resource "libvirt_volume" "disk" {
  for_each   = var.guests
  name       = "${each.key}.qcow2"
  base_volume_id = "/var/lib/libvirt/images/templates/golden-2026-10.qcow2"
  size       = 10737418240
}

resource "libvirt_cloudinit_disk" "init" {
  for_each = var.guests
  name     = "${each.key}-init.iso"
  user_data = <<EOT
#cloud-config
hostname: ${each.key}
password: vss2026
chpasswd: { expire: False }
ssh_pauth: True
EOT
}
```

```

resource "libvirt_domain" "vm" {
  for_each = var.guests
  name     = each.key
  memory   = each.value.memory
  vcpu     = each.value.vcpu
  cloudinit = libvirt_cloudinit_disk.init[each.key].id
  disk { volume_id = libvirt_volume.disk[each.key].id }
  network_interface { network_name = "default" }
  console { type = "pty", target_port = "0" }
}
TF

# --- 4. Plan first. The plan IS the review. -----
terraform init -input=false
terraform plan -out=tfplan | tail -20

# --- 5. Apply, and then prove idempotence. -----
terraform apply -auto-approve tfplan
sudo virsh list --all
terraform plan | tail -3          # "No changes" -- this is the property
                                # a shell script of virsh does not have.

# --- 6. Reconfigure declaratively: change one number. -----
sed -i 's/db1 = { memory = 2048/db1 = { memory = 4096/' main.tf
terraform plan | grep -E 'memory|will be'
terraform apply -auto-approve

# --- 7. Create drift by hand, then watch the tool notice. -----
sudo virsh setmaxmem web1 3145728 --config
sudo virsh setmem web1 3145728 --config
terraform plan | tail -8          # the difference, reported

# --- 8. Tear the whole estate down with one command. -----
terraform destroy -auto-approve
sudo virsh list --all

```

What to notice. Step 5's second plan is the whole argument: running it again does nothing, so it is safe to run on a schedule. Step 7 is the other half — the tool can tell you the estate has been changed behind its back, which no console can.

🔍 Checkpoint — Test Yourself

1. A team provisions 3 guests a month at 50 minutes each and reconfigures 8 at 25 minutes. A Terraform module would take 4 days and reduce these to 5 and 3 minutes. Work out the first-year saving and say whether you would adopt it.
2. Explain what belongs in a golden image and what belongs in cloud-init user data, with one example of each being put in the wrong place and the consequence.
3. A database server cannot be rebuilt because its configuration was never captured. Which drift strategy applies to it, and what would you do to change that over the next year?

📋 Chapter Summary

- A management layer exists because placement, evacuation, failure recovery, access control and consistency have no obvious answers at twenty hosts. Automated placement is what pays for it.
- vCenter, SCVMM, oVirt and Proxmox manage *servers*; OpenStack provides a *cloud*. Matching the tool to the size of the problem matters more than the feature comparison.

- Infrastructure as code makes the declaration the artefact: declarative, idempotent, reviewed. The state file is authoritative, holds secrets and must be locked.
- Automation pays at volume and repetition. Below that threshold it costs more than it saves — state the threshold before adopting.
- Build, publish, instantiate. Changes go back to the image; anything differing between guests goes in cloud-init user data.
- Drift is corrected by convergence or made impossible by immutability. Most estates run both, and knowing which applies to a machine matters more than the preference.

Review Questions

1. **(MCQ)** Terraform's state file: (a) is a log of past actions; (b) records which real objects the tool created, and must be locked; (c) is optional; (d) belongs in version control.
2. **(MCQ)** Idempotence means: (a) the tool runs quickly; (b) applying the same definition twice changes nothing the second time; (c) changes can be rolled back; (d) the configuration is encrypted.
3. **(Short)** Explain how automated placement reduces the hardware a cluster needs, referring to Chapter 6.
4. **(Short)** Give the three stages of the image pipeline and state what must never be done between them.
5. **(Short)** Contrast convergent and immutable approaches to drift, and give a workload suited to each.
6. **(Applied)** A department running 60 guests on three standalone ESXi hosts with no vCenter asks what they are missing. Write the answer, in priority order, with the operational consequence of each gap and a free alternative where one exists.
7. **(Applied)** An estate has 200 guests, no image pipeline and no record of how any was built. Design a twelve-month plan to reach a state where any guest can be rebuilt from a reviewed definition, saying what you would do first and why.

High Availability, Live Migration and Disaster Recovery

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Management, Performance and Security

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** pre-copy live migration round by round, and **state** the condition under which it converges.
- **Compute** the number of rounds and the total transfer for a given guest, dirty rate and link speed.
- **Explain** what happens when it does not converge, and **compare** auto-convergence with post-copy.
- **Distinguish** high availability from fault tolerance from disaster recovery, and **express** each as RPO and RTO.
- **Compute** the downtime budget implied by an availability target, and **say** what each additional nine costs.

🔗 Before You Start

Chapter 6 (memory, and the fact that a running guest's pages change under you) and Chapter 12 (shared storage, without which almost nothing here works). Chapter 7's warning that an assigned device cannot be migrated is the exception that proves this chapter's rule.

📖 Key Terms in This Chapter

live migration • pre-copy • post-copy • dirty page • dirty rate • convergence • switchover • stop-and-copy • auto-convergence • page fault over the network • high availability • fault tolerance • cluster • quorum • split brain • fencing • STONITH • RPO • RTO • availability • nines • 3-2-1 rule • immutable backup

17.1 Moving a Running Machine

Nothing else in computing does this. A guest serving traffic moves from one physical host to another, keeps its memory, its open connections and its in-flight work, and pauses for a time its users cannot perceive. It is the capability that makes everything in Chapter 16 possible: hosts can be patched, rebalanced and retired without anyone scheduling an outage.

The difficulty is stated in one sentence: *the memory you are copying is being modified while you copy it.*

📄 Pre-Copy Live Migration

The standard algorithm, used by every platform in Chapter 5:

- **Round 0.** Copy all of the guest's memory to the destination while it keeps running. Track which pages are written during the copy — these are **dirty pages**.
- **Round n .** Copy the pages dirtied during round $n - 1$. The guest is still running, so this dirties more.
- **Repeat** until the remaining dirty set is small enough to copy within the acceptable pause.

- **Switchover.** Pause the guest, copy the last few pages and the processor state, and resume it on the destination. Typically tens of milliseconds.

Each round copies less than the one before, *provided* the guest dirties memory more slowly than the network transfers it. That proviso is the whole engineering problem.

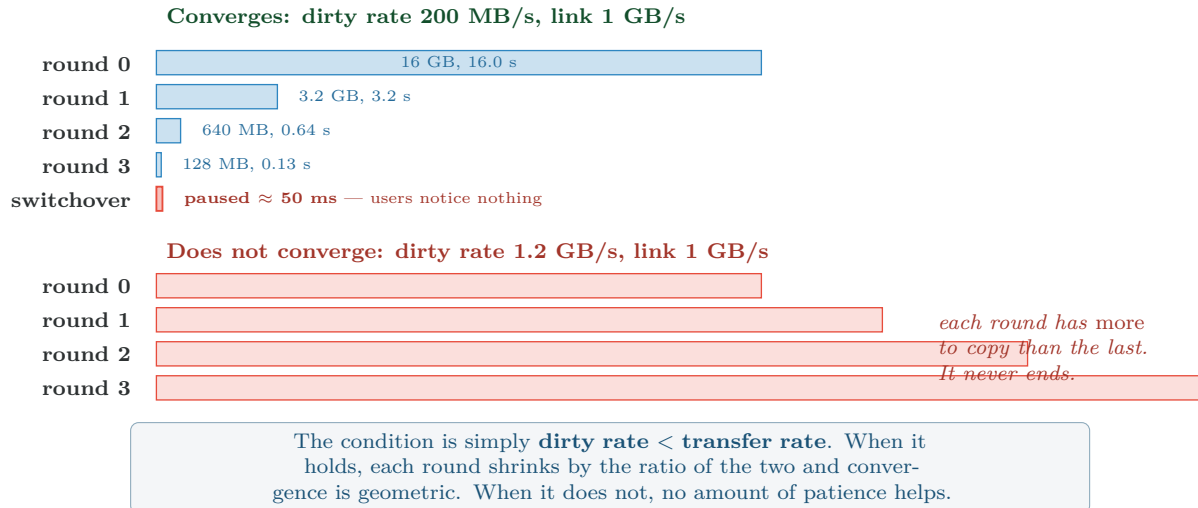


Figure 17.1: Pre-copy migration, drawn to scale for the worked example. The upper case finishes in 20 seconds with a 50 ms pause; the lower one never finishes at all.

Worked Example: Will This Guest Migrate?

A 16 GB guest is to be migrated over a 10 Gb/s link. Measured: the link sustains about 1 GB/s after overheads, and the guest dirties memory at 200 MB/s.

Step 1 — the convergence condition.

$$\text{dirty rate } 200 \text{ MB/s} < \text{transfer rate } 1,000 \text{ MB/s} \quad \checkmark$$

Each round will be $200/1000 = 0.2$ of the one before.

Step 2 — round by round.

$$\text{round 0: } \frac{16,000}{1,000} = 16.0 \text{ s, dirtying } 16.0 \times 200 = 3,200 \text{ MB}$$

$$\text{round 1: } 3.2 \text{ s, dirtying } 640 \text{ MB} \quad \text{round 2: } 0.64 \text{ s, dirtying } 128 \text{ MB}$$

$$\text{round 3: } 0.13 \text{ s, dirtying } 26 \text{ MB} \quad \text{round 4: } 0.026 \text{ s, dirtying } 5 \text{ MB}$$

Step 3 — when to stop. With a 50 ms pause budget the switchover can move $0.05 \times 1,000 = 50$ MB. Round 4 leaves 5 MB, comfortably under it, so the hypervisor stops after round 4.

Step 4 — totals.

$$\text{elapsed} = 16.0 + 3.2 + 0.64 + 0.13 + 0.026 \approx \mathbf{20 \text{ s}}$$

$$\text{transferred} = 16,000 + 3,200 + 640 + 128 + 26 \approx \mathbf{20 \text{ GB}}$$

$$\text{guest paused} = \approx \mathbf{50 \text{ ms}}$$

A TCP connection does not notice 50 ms. Neither does a user.

Step 5 — now a busy database on the same link. Same 16 GB, but dirtying at 1.2 GB/s:

$$1,200 > 1,000 \quad \times$$

Round 0 takes 16 s and dirties 19.2 GB — *more than the guest has*. Round 1 is larger than round 0. The migration runs for ever, consuming the link the whole time.

Step 6 — three ways out.

- **A faster link.** At 25 Gb/s the transfer rate is about 2.5 GB/s and $1.2 < 2.5$ converges. This is why migration networks are separate and fast.
- **Auto-convergence.** The hypervisor deliberately throttles the guest's vCPUs — 10%, then 20%, and so on — which reduces the dirty rate until the condition holds. The guest is slowed on purpose to let it move.
- **Post-copy.** Switch over *first*, then fetch pages on demand. See below.

Step 7 — what to take away. Migration time is governed by memory size; migration *success* is governed by the ratio of dirty rate to link speed. A guest that will not migrate is nearly always a write-heavy one on a link that is too slow, and the fix is usually the network, not the guest.

💡 Post-Copy: the Opposite Risk

Post-copy inverts the order. The guest is paused almost immediately, its processor state is sent, and it *resumes on the destination* with almost no memory. Pages are fetched from the source as the guest faults on them, over the network.

For: total migration time is bounded and predictable — every page is sent exactly once, so there is no convergence problem at all. A guest that cannot migrate any other way will migrate this way.

Against, and it is serious: between switchover and completion, the guest's memory lives on *two* hosts. If the network or the source fails in that window, the guest is unrecoverable — its pages exist nowhere complete. Pre-copy's failure mode is “the migration did not happen”; post-copy's is “the guest is gone”.

The usual arrangement is hybrid: begin with pre-copy, and if it has not converged after a few rounds, switch to post-copy for the remainder. That gets pre-copy's safety for most of the transfer and post-copy's termination guarantee for the tail.

17.2 High Availability, Fault Tolerance and Quorum

Live migration is *planned* movement. The unplanned case is a host failing, and the answer is different.

High availability restarts the failed host's guests on surviving hosts. The guests crash and reboot: everything in memory is lost, and the service is down for as long as a boot takes — a minute or two. It is cheap, it is automatic, and it is what almost every cluster runs.

Fault tolerance runs a shadow copy of the guest on a second host in lockstep, so a host failure is invisible — zero downtime, zero data loss. It costs double the resources, limits the guest's size and vCPU count severely, and is reserved for the few workloads that genuinely justify it.

⚠️ Split brain, and why fencing is not optional

A cluster decides a host has failed because it stopped answering. But “stopped answering” and “stopped running” are different things: a network partition leaves the host alive, still writing to shared storage, and convinced the others failed.

If the surviving side restarts those guests, the same guest now runs twice, both instances writing to the same disks. The result is not downtime; it is *corruption*, and it is worse than the outage that was being avoided.

Two mechanisms prevent it. **Quorum**: only a partition holding more than half the votes may act, so a minority partition stops. This is why clusters have odd numbers of members or a witness. And **fencing**, bluntly named **STONITH** — shoot the other node in the head: before restarting anything, the survivors forcibly power off the suspect host through its management controller or its storage fabric.

A cluster without fencing is not a highly available cluster. It is a system with one additional, correlated way to lose your data.

17.3 RPO, RTO and the Cost of a Nine

Disaster recovery is about the site, not the host, and two numbers specify it.

RPO and RTO

RPO — recovery point objective — is how much *data* you may lose, expressed as time. An RPO of one hour means losing up to an hour of work is acceptable, which means replicating or backing up at least hourly.

RTO — recovery time objective — is how long you may be *down*. An RTO of four hours means service must be restored within four hours of the declaration.

They are independent and they cost differently. Nightly backups to tape give an RPO of 24 hours and an RTO of perhaps a day. Synchronous replication to a second site gives an RPO near zero — and an RTO that still depends entirely on how quickly somebody can bring the second site up, which is a rehearsal question, not a technology one.

Worked Example: What Each Nine Costs

A service is offered at “four nines”. Work out what that means and what the next nine would require.

Step 1 — the downtime budget. A year is $365 \times 24 \times 60 = 525,600$ minutes.

$$99.9\% : 525,600 \times 0.001 = 526 \text{ min} = \mathbf{8 \text{ h } 46 \text{ min}} \text{ a year}$$

$$99.99\% : 52.6 \text{ min a year} = \mathbf{4.4 \text{ min}} \text{ a month}$$

$$99.999\% : 5.3 \text{ min a year} = \mathbf{26 \text{ seconds}} \text{ a month}$$

Step 2 — what fits in each budget.

- **99.9%:** 8 h 46 min allows a monthly patching window of 40 minutes and still leaves room for one unplanned incident. Achievable with HA and careful scheduling.
- **99.99%:** 52 minutes a year. A single HA restart of a guest costs 1–2 minutes, so you may have about thirty of them — *and no planned downtime at all*. Every patch must be applied by live migration.
- **99.999%:** 5 minutes a year. One HA restart is 20–40% of the annual budget. This requires fault tolerance or clustered application software, and it requires that no human decision is in the recovery path, because nobody reads an alert in 26 seconds.

Step 3 — the cost shape. Each nine divides the budget by ten and roughly multiplies the cost by three to five: a second site, a second set of licences, tested automation, rehearsals, and staff who can be woken.

Step 4 — the question to ask instead. Before accepting an availability target, ask what the *outage* actually costs per hour. A service losing \$200 an hour cannot justify the \$150,000 a year that five nines would cost; one losing \$200,000 an hour can justify a great deal. Availability targets chosen without that number are chosen by whoever is most anxious.

Step 5 — and the measurement everyone forgets. An availability figure is meaningless without a stated measurement window and a definition of “down”. 99.99% measured monthly permits 4.4 minutes every month; measured annually it permits 52 minutes in one incident. Those are very different promises, and the difference is where disputes happen.

💡 A Snapshot Is Not a Backup, Restated

Chapter 12 made this point about storage; it is worth making again about disaster recovery, because the failure is so common.

The **3-2-1 rule**: three copies of the data, on two different media, with one off site. A snapshot fails all three — it is on the same storage, the same medium, the same site, and it depends on the original.

Ransomware has added a fourth requirement: at least one copy must be **immutable** — write-once, with a retention lock the backup administrator’s own credentials cannot override. Attackers who gain administrative access delete the backups first, and an estate whose backup server trusts the same directory as everything else has one credential between it and total loss.

And the rule that outranks all of them: **a backup you have not restored is a hypothesis**. Test restores on a schedule, and time them — that measurement is your real RTO, as opposed to the one in the document.

❗ Common Pitfalls

- **Expecting any guest to live-migrate.** A write-heavy guest on a slow link will not converge. Check the dirty rate against the link before promising a maintenance window.
- **Running a cluster without fencing.** Split brain corrupts data, which is worse than the outage HA was bought to prevent.
- **Using an even number of cluster members with no witness.** A clean split leaves neither side with quorum and both stop.
- **Confusing HA with fault tolerance.** HA restarts the guest; the service *is* down and everything in memory is lost.
- **Quoting an availability figure without a window and a definition of down.** 99.99% monthly and 99.99% annually are different promises.
- **Treating replication as backup.** Replication faithfully copies the deletion and the encryption to the second site, usually within seconds.
- **Never testing a restore.** Then the RTO is a guess, and it is always optimistic.
- **Forgetting that a passthrough device prevents migration** (Chapter 7). Plan the exception before you need to patch the host.

🔧 Try It Yourself: Migrating a Running Guest, and Failing To

Live-migrates a guest between two hosts with no dropped pings, then makes the guest dirty memory fast enough that it will not converge. Needs two hosts, or two nested ones. Expect forty-five minutes.

```
#!/usr/bin/env bash
# ch17_migrate.sh -- pre-copy convergence, observed and then broken.
set -euo pipefail
```

```

DEST=${1:?usage: ch17_migrate.sh <destination-host>}

# --- 1. Both hosts must see the same disk. -----
# Shared storage is the precondition for everything in this chapter.
showmount -e "$DEST" 2>/dev/null || \
    echo "set up NFS or iSCSI first -- see Appendix A"
sudo virsh pool-list --all

# --- 2. Start the guest and give it something to serve. -----
sudo virsh start vss1 || true ; sleep 25
IP=$(sudo virsh domifaddr vss1 | awk '/ipv4/{split($4,a,"/");print a[1]}')
G () { ssh -o StrictHostKeyChecking=no "ubuntu@$IP" "$1"; }
G "sudo apt-get -qq install -y stress-ng python3 && \
    nohup python3 -m http.server 8000 >/dev/null 2>&1 &"

# --- 3. Watch continuity: ping and HTTP, both running throughout. ----
ping -i 0.2 "$IP" > /tmp/ping.log &
PINGPID=$!
( while true; do curl -s -o /dev/null -w '%{http_code} ' \
    "http://$IP:8000/" ; sleep 0.2; done ) > /tmp/http.log &
CURLPID=$!

# --- 4. Migrate it, and time the whole thing. -----
time sudo virsh migrate --live --verbose --persistent --undefinesource \
    vss1 "qemu+ssh://$DEST/system"

sleep 2 ; kill $PINGPID $CURLPID 2>/dev/null || true
echo "---- dropped pings ----"
grep -c 'icmp_seq' /tmp/ping.log
tail -2 /tmp/ping.log          # look at the loss percentage
echo "---- non-200 responses ----"
tr ' ' '\n' < /tmp/http.log | grep -vc '^200$' || echo 0
# A correct migration drops nothing. The 50 ms pause is inside one
# ping interval.

# --- 5. Now make it refuse to converge. -----
# stress-ng --vm writes continuously, so the dirty rate exceeds the link.
sudo virsh migrate --live vss1 "qemu+ssh://$DEST/system" || true # back
sleep 10
G "nohup stress-ng --vm 2 --vm-bytes 75% --vm-keep --timeout 300s \
    >/dev/null 2>&1 &"
sleep 5
echo "---- migrating a guest that dirties faster than the link ----"
sudo virsh migrate --live --verbose --timeout 60 \
    --timeout-suspend vss1 "qemu+ssh://$DEST/system" || \
    echo "did not converge within 60 s -- exactly section 17.1"

# --- 6. Make it converge by throttling the guest on purpose. -----
sudo virsh migrate --live --verbose --auto-converge \
    --auto-converge-initial 20 --auto-converge-increment 10 \
    vss1 "qemu+ssh://$DEST/system"
# The guest was deliberately slowed so that it could move.

# --- 7. Measure the dirty rate directly, for the arithmetic. -----
sudo virsh domdirtyrate-calc vss1 --seconds 10 || true
sleep 12 ; sudo virsh domstats vss1 --dirtyrate | grep -i dirty

# --- 8. Unplanned failure: what HA does instead. -----
# No graceful anything: cut the guest off and see what survives.
sudo virsh destroy vss1
echo "the guest is gone; HA would now restart it elsewhere, and"
echo "everything in its memory is lost. That is the difference."

```

```
# --- 9. Tear down. -----
sudo virsh start vss1 || true
```

What to notice. Step 4 should drop no pings at all: the guest moved between physical machines while serving HTTP and nothing noticed. Step 5 is the same command failing for the reason the worked example predicts, and step 6 is the hypervisor buying convergence with the guest's performance.

? Checkpoint — Test Yourself

1. A 32 GB guest dirties memory at 400 MB/s and the migration link sustains 1.2 GB/s. Give the ratio, the size of the first three rounds, and the approximate total elapsed time.
2. A two-host cluster loses the network between the hosts. Both are alive and both can reach the storage. Explain what each host believes, what happens without fencing, and what quorum would do differently.
3. A service promises 99.99% availability measured annually. Give the downtime budget, and say whether a 90-minute planned upgrade once a year is compatible with it.

✓ Chapter Summary

- Pre-copy migration copies memory in rounds while the guest runs, then pauses briefly to send the remainder. It converges only if the dirty rate is below the transfer rate.
- A 16 GB guest at 200 MB/s over 1 GB/s converges in about 20 seconds with a 50 ms pause. At 1.2 GB/s it never converges at all.
- The fixes are a faster link, auto-convergence (throttling the guest on purpose), or post-copy — which always terminates but loses the guest if the network fails mid-transfer.
- High availability restarts guests after a host failure; fault tolerance runs a lockstep shadow. HA loses memory and takes a minute; FT costs double.
- Quorum and fencing prevent split brain. A cluster without fencing has added a way to corrupt data.
- RPO is data lost, RTO is time down, and they cost differently. Each additional nine divides the budget by ten and multiplies the cost by three to five.
- 3-2-1, plus one immutable copy, plus tested restores. A backup you have not restored is a hypothesis.

✍ Review Questions

1. (MCQ) Pre-copy live migration converges when: (a) the guest has fewer than 8 vCPUs; (b) the dirty rate is lower than the transfer rate; (c) shared storage is used; (d) the guest is idle.
2. (MCQ) The characteristic risk of post-copy migration is: (a) it never terminates; (b) the guest's memory is split across two hosts, so a failure in that window loses it; (c) it needs more bandwidth; (d) it cannot be used with shared storage.
3. (Short) Explain auto-convergence and what it trades.
4. (Short) Define RPO and RTO, and give a configuration that produces a low RPO and a high RTO.
5. (Short) State the 3-2-1 rule and explain which parts of it a snapshot fails.
6. (Applied) A 64 GB database guest must be moved for host maintenance. It dirties memory at 2.1 GB/s and the migration network is 10 Gb/s. Show that it will not converge, give three options with their consequences for the database's users, and recommend one.

7. (**Applied**) A department is asked to commit to 99.99% availability for a student-facing system. Write the response: the downtime budget, what it rules out operationally, what it would cost to deliver, and the question you would ask the requester first.

Performance Monitoring and Capacity Planning

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Management, Performance and Security

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why a guest's own metrics are untrustworthy, and **name** the host-side counters that are not.
- **Interpret** ready time, co-stop, balloon, swap and latency together to locate a bottleneck.
- **Explain** why a mean is the wrong statistic for latency, and **use** percentiles instead.
- **Diagnose** a noisy-neighbour problem and **distinguish** it from a workload problem.
- **Produce** a capacity plan from measured percentiles, with a stated headroom and a stated trigger.

🔗 Before You Start

Chapter 6 above all — ready time, co-stop, the reclamation ladder and NUMA are the quantities this chapter measures. Chapter 12 (IOPS and the thin pool) and Chapter 7 (device models) supply the rest.

🔑 Key Terms in This Chapter

%RDY • steal time • co-stop • balloon • swap-in • latency • throughput • saturation • utilisation • percentile • p95 • p99 • tail latency • noisy neighbour • timekeeping • tick drift • USE method • headroom • capacity plan • trigger

18.1 The Guest Is an Unreliable Witness

Every tool a guest has for measuring itself assumes it owns a processor. None of them do.

A guest kernel computes CPU utilisation by sampling what it was doing at each timer tick. If the hypervisor descheduled it for 30 ms, the guest did not experience 30 ms — time simply jumped, and the ticks that should have occurred did not. The guest therefore reports being busy for the time it *ran*, and has no idea about the time it did not.

The consequence is the single most misleading observation in this subject: a guest reporting **40% CPU utilisation while being unable to keep up**. It was given 50% of a core, used 40% of what it was given, and spent the rest of the time not existing.

💡 Measure From Outside, Correlate From Inside

The rule for this whole chapter: **the host knows what happened; the guest knows what it wanted**.

Only the hypervisor can report ready time, co-stop, ballooning and host swap, because only the hypervisor was present for them. Only the guest can report which process was running and which query was slow. A diagnosis needs both, in that order — host first to find the constrained resource, guest second to find out who wanted it.

The one guest-side number that *is* trustworthy is steal time (%st in `top`, `mpstat`), because the hypervisor tells the guest about it explicitly through a paravirtual clock. It is the same

quantity as %RDY, visible from the other side, and it is the first thing to look at when you only have access to the guest.

18.2 The Counters That Matter

Counter	What a high value means	Threshold	What to do
%RDY / steal	The vCPU wanted to run and no core was free	> 5% look, > 10% act	Right-size guests down, or add hosts (Chapter 6)
co-stop	A wide guest waiting for its own siblings to be scheduled together	> 3%	Reduce that guest's vCPU count
balloon	The hypervisor is reclaiming memory from this guest	any sustained value	Reduce overcommit before it reaches swap
host swap-in	The hypervisor is reading guest memory back from disk	any	Emergency. Migrate guests off now
device latency	Storage or network is the constraint, not the host	depends on media	Look below the hypervisor
%SYS on the host	The hypervisor itself is busy — often device emulation	> 10%	Check for emulated devices (Chapter 7)

⚠ Utilisation is not saturation, and the difference is the whole job

A resource at 100% utilisation is *fully used*. A resource that is *saturated* has a queue. They are not the same, and only the second is a problem.

A disk at 100% utilisation with a queue depth of 1 is working perfectly — it has exactly enough work. The same disk at 100% with a queue depth of 40 has forty requests waiting, and every one of them is experiencing latency it did not have an hour ago.

So the useful triple for any resource is **utilisation, saturation, errors** — Brendan Gregg's USE method. Dashboards almost always show the first, sometimes the third and rarely the second, which is why so many estates are described as healthy while the people using them disagree.

18.3 Why the Mean Is the Wrong Number

A dashboard reporting “average response time 45 ms” is reporting almost nothing useful. Latency distributions in virtualized systems are not symmetric: most requests are fast and a few are very slow, and the slow ones are what people experience and remember.

📊 Worked Example: Is This the Neighbours, or Is It Us?

A team reports their application has become slow. Nothing was deployed. The guest's own dashboard shows CPU at 38%, memory at 55% and “no problems”. Decide where the fault is.

Step 1 — the guest's own figures are consistent with both explanations, so they settle nothing. Go to the host.

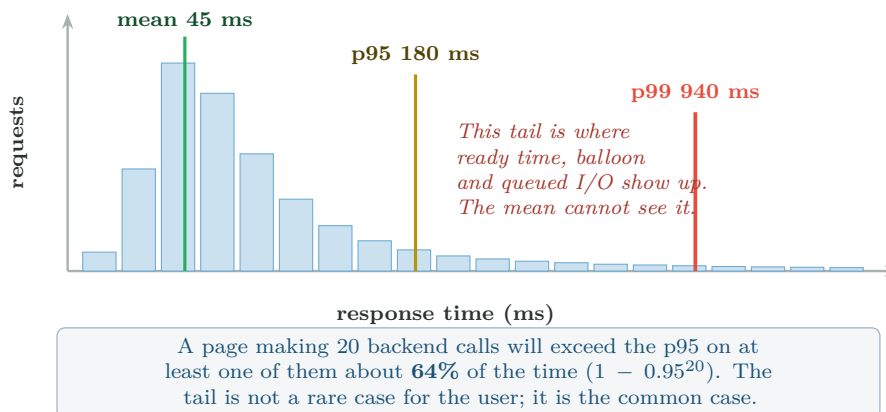


Figure 18.1: A real latency distribution. The mean sits below almost all of the mass, and every pathology in this book lives to the right of it.

Step 2 — host counters for this guest.

- %RDY: 16% (was 3% a month ago)
- co-stop: 1% — not a wide-guest problem
- balloon: 0 — memory is not being reclaimed
- host swap-in: 0 — not an emergency
- storage latency: 2 ms — normal

Ready time has risen fivefold and nothing else has moved. The guest is not being given a processor.

Step 3 — is it this guest's fault? Its own vCPU count has not changed. So either the host gained work, or the other guests did.

Step 4 — the host's arithmetic, then and now. 64 cores.

$$\text{a month ago: } \sigma = \frac{112}{64} = 1.75 \quad \text{now: } \sigma = \frac{168}{64} = \mathbf{2.63}$$

Fourteen guests of 4 vCPUs were added to the host since. **This is a noisy-neighbour problem:** nothing about the complaining application changed, and the resource it depends on was divided more ways.

Step 5 — confirm it, do not assume it. Two checks that distinguish the diagnoses:

- Compare %RDY across *all* guests on the host. If only this one is high, it is the guest. If all are high, it is the host.
- Migrate the guest to a quiet host and measure again. If it recovers with no other change, the question is settled.

Step 6 — what the fix is, and what it is not. Adding vCPUs to the complaining guest would make it worse (Chapter 6). The options are to rebalance the host, to right-size the fourteen new guests, or to add hosts.

Step 7 — the real failure. Nobody checked the host's %RDY before adding the fourteen guests. The capacity decision was made on *allocated* vCPUs, which looked fine, instead of on *contention*, which did not. The next section is how to not do that.

18.4 Benchmarking Honestly

Measurements taken inside guests are easy to get wrong in ways that are not obvious.

💡 Five Ways a Virtualized Benchmark Lies

- **The neighbours.** The host is shared, so your result depends on what everyone else was doing. Run the measurement several times across different hours and report the distribution, not one figure.
- **Timekeeping.** A descheduled guest's clock can drift, so a benchmark measuring its own elapsed time may be measuring the hypervisor's scheduling. Use a paravirtual clock source, and prefer timing from the host where you can.
- **Caching you did not account for.** The host's page cache serves the guest's reads. A “disk benchmark” on a 2 GB file on a host with 256 GB of RAM is a memory benchmark.
- **Burstable instances.** In the cloud (Chapter 14), a short benchmark runs on credits and a long one does not. The two give different answers about the same machine.
- **Nesting.** Anything measured in a nested guest (Chapter 4) is inflated by the outer hypervisor's exits, by a variable amount.

A benchmark that does not state how it handled all five should be read as an anecdote.

18.5 Capacity Planning

A capacity plan is three things: a measured baseline, a growth rate, and a *trigger* — the figure at which you act, chosen so that acting is still possible.

💡 Why the Trigger Comes Before the Target

The common mistake is to plan to a utilisation target: “we will run the cluster at 80%”. That is a destination with no mechanism.

The useful form is a trigger with a lead time attached. Hardware takes — say — ten weeks to specify, approve, order, receive and install. So the trigger must be the figure reached *ten weeks before* the resource would be exhausted, computed from the measured growth rate.

That single reframing is what distinguishes a plan from a dashboard. A dashboard tells you where you are; a plan tells you when the decision was supposed to have been made.

📊 Worked Example: A Capacity Plan From Measurements

A six-host cluster: 64 cores and 512 GB each. Measured over three months at the 95th percentile: 1,150 vCPUs allocated, %RDY 6.2%, memory active 2,060 GB, storage 41,000 IOPS. Growth: 4% a quarter in guests. Lead time for new hardware: 10 weeks. Produce the plan.

Step 1 — where the cluster is now.

$$\sigma_{\text{cpu}} = \frac{1,150}{6 \times 64} = \frac{1,150}{384} = 2.99 \quad \text{memory} = \frac{2,060}{6 \times 512 \times 0.85} = \frac{2,060}{2,611} = 79\%$$

%RDY at 6.2% is already in the band that Chapter 6 says to investigate.

Step 2 — N+1, which is not optional. Six hosts must survive one failing, so plan against five:

$$\sigma_{\text{cpu}}|_{N+1} = \frac{1,150}{5 \times 64} = 3.59 \quad \text{memory}|_{N+1} = \frac{2,060}{5 \times 512 \times 0.85} = 95\%$$

At 95% of memory with one host down, this cluster has no real N+1 today. That is the finding, and it is invisible on any dashboard showing 79%.

Step 3 — the binding resource and the limit. Memory. Take 85% of N+1 capacity as the ceiling — beyond it the reclamation ladder starts working:

$$\text{ceiling} = 5 \times 512 \times 0.85 \times 0.85 = 1,850 \text{ GB}$$

Current 2,060 GB already exceeds it. The cluster is over its own N+1 limit now.

Step 4 — growth. 4% a quarter on 2,060 GB is 82 GB a quarter, about 6.3 GB a week.

Step 5 — the trigger. Working back from the 6-host ceiling of $6 \times 512 \times 0.85 \times 0.85 = 2,220$ GB:

$$\frac{2,220 - 2,060}{6.3} = 25 \text{ weeks until the cluster is full even without N+1}$$

With a 10-week lead time, the order must be placed by week 15 — and the N+1 finding means it should be placed **now**.

Step 6 — the plan, as it would be written.

- **Immediate:** order two hosts. N+1 is already unmet, and %RDY is in the investigate band.
- **In parallel:** right-size. $\sigma_{\text{cpu}} = 2.99$ with 6.2% ready suggests over-allocated vCPUs; the exercise in Chapter 6 commonly recovers 30–40% and may defer the second host.
- **Trigger, standing:** alert when N+1 memory exceeds 85%, when %RDY p95 exceeds 8%, or when storage exceeds 80% of measured peak IOPS. Review quarterly against actual growth.
- **Report:** N+1 figures, not raw utilisation. The difference between 79% and 95% in step 2 is the entire finding.

! Common Pitfalls

- **Trusting guest-reported CPU.** A guest at 40% may be unable to keep up. Only the host knows about the time the guest did not exist.
- **Reporting utilisation without saturation.** A queue, not a busy resource, is what users feel.
- **Planning on means.** The tail is the user experience, and a page making twenty calls meets it most of the time.
- **Reporting capacity without N+1.** 79% and 95% were the same cluster.
- **Benchmarking once, in one hour, on a shared host.** You measured the neighbours.
- **Setting a trigger without a lead time.** A threshold reached too late to act on is a notification, not a plan.
- **Adding capacity before right-sizing.** Inflated allocations are cheaper to fix than hardware is to buy, and buying first preserves the waste permanently.

🔧 Try It Yourself: Finding the Bottleneck

Produces three different bottlenecks on one host and distinguishes them using only the counters in this chapter. Expect forty minutes.

```
#!/usr/bin/env bash
# ch18_bottleneck.sh -- CPU contention, memory pressure and I/O queueing.
set -euo pipefail
sudo apt-get install -y sysstat fio stress-ng
ip_of () {
    sudo virsh domifaddr "$1" | awk '/ipv4/{split($4,a,"/"); print a[1]}'
}
G () { ssh -o StrictHostKeyChecking=no "ubuntu@$(ip_of "$1")" "$2"; }
```

```

# --- 1. Baseline, with the host quiet. -----
sudo virsh start vss1 || true ; sleep 25
G vss1 "sudo apt-get -qq install -y sysstat fio stress-ng"
echo "=== baseline ==="
G vss1 "mpstat 2 3 | tail -2"           # %steal should be near zero
sudo virsh domstats vss1 --vcpu | grep -E 'wait|delay' || true

# --- 2. Bottleneck A: CPU contention from the neighbours. -----
# Other guests take the cores. The complaining guest changes nothing.
for n in 2 3 4; do
    sudo virt-clone --original vss1 --name "n$n" \
        --file "/var/lib/libvirt/images/n$n.qcow2" --auto-clone >/dev/null
    sudo virsh setvcpus "n$n" "$(nproc)" --config --maximum
    sudo virsh setvcpus "n$n" "$(nproc)" --config
    sudo virsh start "n$n"
done
sleep 40
for n in 2 3 4; do
    G "n$n" "nohup stress-ng --cpu \$(nproc) --timeout 120s >/dev/null 2>&1 &"
done
sleep 10
echo "=== A: CPU contention ==="
G vss1 "mpstat 2 3 | tail -2"           # %steal climbs; %usr does NOT
sudo virsh domstats vss1 --vcpu | grep -E 'wait|delay' || true
# The signature: steal high, guest's own utilisation unchanged.

# --- 3. Bottleneck B: memory pressure. -----
for n in 2 3 4; do
    G "n$n" "nohup stress-ng --vm 1 --vm-bytes 85% --vm-keep \
        --timeout 90s >/dev/null 2>&1 &"
done
sleep 20
echo "=== B: memory pressure ==="
sudo virsh dommemstat vss1 | grep -E 'balloon|swap|available'
vmstat 2 3 | tail -2                   # host si/so columns
free -m | head -2
# The signature: balloon moving, or host swap non-zero. Different fix.

# --- 4. Bottleneck C: storage queueing. -----
pkill stress-ng 2>/dev/null || true ; sleep 5
G vss1 "nohup fio --name=load --rw=randwrite --bs=4k --size=512M \
    --numjobs=4 --time_based --runtime=60 --direct=1 \
    >/dev/null 2>&1 &"
sleep 10
echo "=== C: storage queueing ==="
iostat -x 2 3 | awk '/Device|vd|sd|nvme/' | tail -6
# The signature: aqu-sz (queue depth) and await (latency) both high,
# while %util may already have been 100% harmlessly before. Utilisation
# against saturation, section 18.2.

# --- 5. Percentiles, not means. -----
G vss1 "fio --name=lat --rw=randread --bs=4k --size=256M --direct=1 \
    --runtime=30 --time_based --output-format=json" \
    | grep -E '"mean"|"percentile"|"95.000000"|"99.000000"' | head -8
# Compare the mean with p95 and p99 on the same run.

# --- 6. Tear down. -----
for n in 2 3 4; do
    sudo virsh destroy "n$n" 2>/dev/null || true
    sudo virsh undefine "n$n" --remove-all-storage 2>/dev/null || true
done

```

```
sudo virsh destroy vss1 || true
```

What to notice. The three bottlenecks look identical from inside the guest — the application is slow — and have three different signatures and three different fixes on the host. That difference is the whole chapter, and it is why the guest's dashboard settled nothing in the worked example.

? Checkpoint — Test Yourself

1. A guest reports 35% CPU utilisation and its users say it cannot keep up. Give two host-side counters that would distinguish the possible causes, and say what each value would mean.
2. A page issues 12 backend calls. If each has a p95 of 200 ms, what fraction of page loads will contain at least one call slower than 200 ms?
3. A cluster of 8 hosts is at 76% memory. Compute the N+1 figure and say whether you would report the cluster as healthy.

✓ Chapter Summary

- A guest cannot see the time it was not running, so its CPU figures are unreliable. The host knows what happened; the guest knows what it wanted. Steal time is the one trustworthy guest-side number.
- %RDY, co-stop, balloon, host swap-in and device latency each point at a different constraint with a different fix.
- Utilisation is not saturation. A queue is what users feel, and most dashboards do not show it.
- Plan on percentiles. The mean sits below almost all the mass, and a page making twenty calls meets the p95 most of the time.
- Distinguish a noisy neighbour from a workload problem by comparing all guests on the host, and by migrating to a quiet host.
- A virtualized benchmark lies through neighbours, timekeeping, caching, burst credits and nesting. State how you handled all five.
- Report N+1 capacity, not raw utilisation, and set triggers with the procurement lead time built in.

✍ Review Questions

1. (MCQ) The one guest-side counter that reliably indicates host CPU contention is: (a) load average; (b) %usr; (c) %steal; (d) context switches.
2. (MCQ) A disk at 100% utilisation with an average queue depth of 1 is: (a) saturated; (b) fully used but not saturated; (c) failing; (d) idle.
3. (Short) Explain why a guest can report 40% CPU while being unable to keep up.
4. (Short) State the USE method and explain which of its three parts dashboards usually omit.
5. (Short) Give three of the five ways a virtualized benchmark can mislead, with the control for each.
6. (Applied) A four-host cluster of 48 cores and 384 GB each measures 820 vCPUs allocated, %RDY p95 of 9%, and 1,180 GB active memory, growing 5% a quarter. Lead time is 12 weeks. Work through the worked example's six steps and write the plan.

7. (**Applied**) A developer insists their application is slow “because virtualization is slow” and produces a benchmark run inside their guest. Write the response: what the benchmark does not control for, what you would measure instead, and what result would prove them right.

Security in Virtualized Environments

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Management, Performance and Security

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the hypervisor’s attack surface and **explain** why device models are where escapes keep being found.
- **Explain** how a side channel defeats isolation without any software fault, and **name** the mitigations and what they cost.
- **Compare** the blast radius of a guest compromise, a host compromise and a management-plane compromise.
- **List** the container hardening measures and **say** what each one removes.
- **Produce** a hardening plan for a stated estate, in priority order, justified by blast radius rather than by severity score.

🔗 Before You Start

Chapter 3 (what isolation means formally — resource control is the property being attacked), Chapter 7 (device models, which are the attack surface) and Chapter 8 (the shared kernel). Chapter 16 supplies the management plane this chapter argues is the real target.

📖 Key Terms in This Chapter

attack surface • trusted computing base • VM escape • guest-to-host • device model • side channel • covert channel • Spectre • Meltdown • L1TF • MDS • simultaneous multithreading • core scheduling • blast radius • management plane • lateral movement • container escape • privileged container • seccomp • capability • image provenance • secret • micro-segmentation

19.1 What the Layer Added

Virtualization is, on balance, good for security. It gives every workload a hardware-enforced boundary, it makes a compromised machine disposable, and it makes isolation affordable enough to apply everywhere.

It also added two things that did not exist before: a piece of privileged software underneath every workload, and a control plane that can reach every machine at once. This chapter is about those two.

📖 VM Escape

A **VM escape** is code executing in a guest that gains execution on the host. It breaks the resource-control property of Chapter 3 — the one the whole subject rests on — and it is the most severe class of bug a hypervisor can have, because the attacker arrives with every other guest on the host beside them.

Escapes are rare, which is why the architecture works at all. They are not hypothetical: they are found, patched and exploited on a regular schedule.

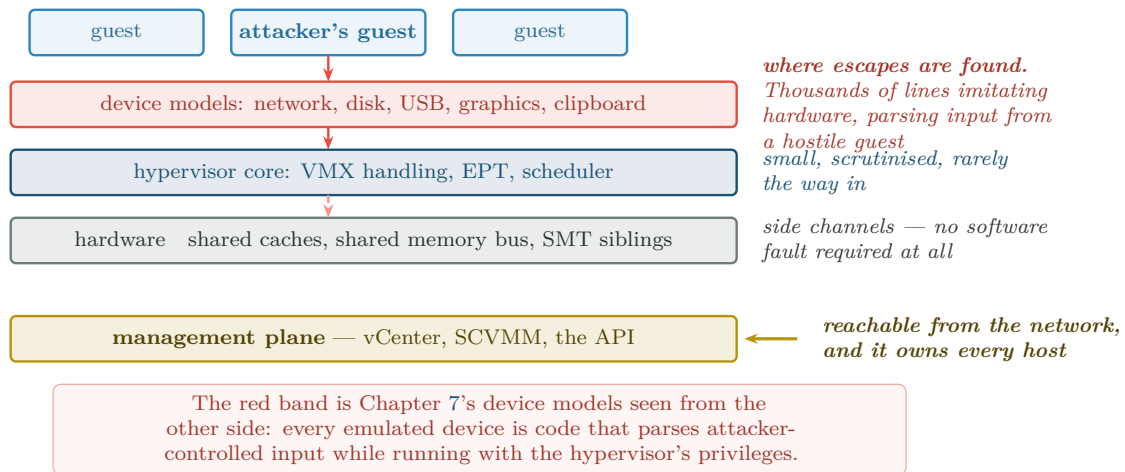


Figure 19.1: The attack surface. The hypervisor core is small and heavily reviewed; the device models around it are neither, and that is where the escapes are.

19.2 Where Escapes Come From

Almost no escape is a flaw in the VMX handling or the page tables. They are overwhelmingly in the *device models* — the software of Chapter 7 that imitates a network card, a disk controller, a graphics adapter or a USB hub.

The reason is structural. That code takes input a hostile guest fully controls — ring descriptors, packet headers, command blocks — parses it in C, and runs with the privileges of the hypervisor. It is large, it imitates decades of hardware quirks, and much of it is rarely exercised.

💡 Three Recent Examples, and What They Have in Common

Dated to October 2026. Check for newer ones before teaching from this; *that* is the lesson rather than the list.

- **CVE-2025-22224, -22225, -22226** (VMware ESXi, March 2025). Three flaws that could be chained from a guest to compromise the hypervisor. Researchers later established that attackers had working exploits well before disclosure, and CISA recorded ransomware groups actively using CVE-2025-22225 to escape guest isolation and encrypt across hypervisors.
- **CVE-2026-47876** (VMware ESXi, July 2026, CVSS 9.3). An out-of-bounds write in the **VMXNET3 virtual network adapter**: an attacker with administrative rights inside a guest executes arbitrary code on the host. A paravirtualized network device — precisely the red band in Figure 19.1.
- **CVE-2026-59309 and -59310** (VMware vCenter, 2026, CVSS 9.8 each). Unauthenticated access and remote code execution against the *management plane* — no guest required.

Two patterns worth more than the individual numbers. The guest-to-host flaws are in device models, every time. And the highest-scoring flaws are not escapes at all — they are in the management plane, and the next section says why that is worse.

19.3 Blast Radius Is the Right Metric

Severity scores rank a vulnerability in isolation. What matters operationally is how much of your estate one successful attack reaches.

Worked Example: Three Compromises, Counted

An estate: 40 hosts, 900 guests, one vCenter, one storage array. Compare what an attacker obtains from each of three footholds.

Case 1 — a guest is compromised (a web application flaw).

$$\text{reach} = 1 \text{ guest} = \frac{1}{900} = \mathbf{0.1\%} \text{ of the estate}$$

Plus whatever that guest can reach over the network, which is why Chapter 13's micro-segmentation matters.

Case 2 — a VM escape (CVE-2026-47876, say). The attacker starts in a guest and reaches the host, and therefore every guest on it:

$$\frac{900}{40} = 22.5 \text{ guests per host} \quad \text{reach} = \frac{22.5}{900} = \mathbf{2.5\%}$$

A 22-fold increase, and the hardest of the three to achieve: it needs an unpatched escape and administrative access inside a guest.

Case 3 — the management plane (CVE-2026-59310, say). Unauthenticated remote code execution on vCenter. vCenter can power off, clone, reconfigure and console into every guest on every host it manages:

$$\text{reach} = \frac{900}{900} = \mathbf{100\%}$$

Step 1 — compare.

- guest compromise: 1 guest, easiest to achieve
- VM escape: 22 guests, hardest to achieve, 22× the reach
- management plane: 900 guests, *no guest access required*, and reachable over the network

Step 2 — which to defend first? Most of the attention in this subject goes to case 2, because escapes are dramatic. The arithmetic says case 3: it gives **forty times** the reach of an escape, requires no foothold at all, and the 9.8-scored flaws of 2026 were in exactly that component.

Step 3 — and the one the arithmetic understates. The storage array holds every guest's disk. An attacker with administrative access to it has all 900 guests' data without touching a hypervisor — and storage management interfaces are routinely less hardened than vCenter, because nobody thinks of them as compute.

Step 4 — the resulting priority. Management plane and storage management first: not reachable from general networks, multi-factor authentication, separate credentials from the ordinary directory, patched fastest. Hypervisor patching second. Guest hardening third — important, but it is the case with the smallest radius.

That ordering is the reverse of how most estates allocate effort, and the arithmetic above is the argument for changing it.

19.4 Side Channels

The attacks above are software faults, and software faults get patched. Side channels are not faults at all.

Two guests on one physical core share caches, branch predictors and memory buses. Sharing is not a bug — it is what makes the hardware fast. But *timing* reveals what the other guest did: if a memory access is fast, the data was in a shared cache, which means somebody else touched it recently.

Spectre and **Meltdown** (2018) made this practical by abusing speculative execution. **L1TF** and the **MDS** family extended it to the exact multi-tenant case — reading data belonging to another guest on the same core.

💡 Why the Mitigations Cost So Much

The fixes attack the sharing, which is also the performance.

- **Flush the L1 cache on every VM entry.** Correct, and it throws away the cache the guest was about to use. Costs most on exit-heavy workloads.
- **Disable simultaneous multithreading.** Two hardware threads on one core share nearly everything, so the only reliable answer for strictly untrusted neighbours is to stop using the second thread — which removes 15–30% of the host’s apparent capacity outright.
- **Core scheduling.** Allow SMT, but only schedule siblings from the *same* guest together. Recovers much of the capacity while keeping tenants off shared cores, at the cost of a harder scheduling problem and some lost efficiency.
- **Microcode and compiler mitigations** — retpolines, IBRS and the rest — each costing a few per cent.

The honest summary: a host running strictly untrusted tenants pays a real and permanent performance tax for these, and a host running only your own trusted workloads may reasonably decline some of it. **That is a documented risk decision, not a default** — and it is one of the few places in this book where the right answer genuinely depends on who the neighbours are.

Chapter 21 is the hardware’s attempt to stop the question arising.

19.5 Containers

Everything in Chapter 8 applies: one kernel, 350 system calls, and an isolation boundary enforced by software.

Measure	What it removes	Cost
Run as non-root (USER)	Most escalation paths that rely on real UID 0	None
User namespaces / rootless	Maps container root to an unprivileged host user	Some storage drivers and features
Drop capabilities	CAP_SYS_ADMIN alone is close to root. Drop all, add back what is needed	Must know what the app needs
seccomp profile	Cuts the reachable system calls from ≈ 350 to ≈ 50	Rare calls may break; test

Measure	What it removes	Cost
Read-only root filesystem	An attacker cannot write a tool to disk	Needs explicit writable volumes
No <code>-privileged</code>	That flag disables essentially all of the above at once	Some monitoring agents claim to need it; most do not
Do not mount the Docker socket	It is equivalent to host root	Use a proper API with scoped credentials
Signed images, pinned digests	Supply-chain substitution (Chapter 9)	Process discipline

⚠ Two mistakes that are each equivalent to giving away the host

`-privileged`. It is not “a few more permissions”. It grants all capabilities, disables seccomp and AppArmor, and exposes host devices. A privileged container is a root shell on the host with extra steps.

Mounting `/var/run/docker.sock`. Anything that can talk to the Docker socket can start a new container with the host’s filesystem mounted and `-privileged` set. CI systems and monitoring agents ask for this constantly, and granting it makes every other control in the table above irrelevant.

If a workload genuinely needs either, it should not be sharing a kernel with anything you care about — which is what Chapter 11 is for.

19.6 Hardening, in Order

💡 A Defensible Priority List

Ordered by blast radius, which is the worked example’s conclusion.

- **Isolate the management plane.** vCenter, SCVMM, the API and the storage array’s management interface on a separate network, reachable only from a bastion, with multi-factor authentication and credentials that are not the general directory’s.
- **Patch the management plane fastest.** The 9.8-scored flaws are here and need no foothold.
- **Patch hypervisors on a schedule you can actually meet.** Live migration (Chapter 17) is what makes this possible without outages — which is why a passthrough guest that cannot migrate is a security problem as well as an operational one.
- **Reduce the device surface.** Remove virtual USB, serial ports, floppy controllers and clipboard sharing from templates. Each is a device model, and device models are where escapes are.
- **Segment east–west** (Chapter 13). It is what bounds case 1 in the worked example.
- **Harden containers** per the table, and use Chapter 11’s runtimes for anything untrusted.
- **Decide the SMT question deliberately** and write down the decision with its reason.
- **Protect the backups** (Chapter 17): immutable, separate credentials. Ransomware deletes backups first.

⚠ Common Pitfalls

- **Defending against escapes and leaving vCenter on the general network.** Forty times the blast radius, no foothold needed.

- **Prioritising by CVSS alone.** A 7.5 in the management plane may matter more to you than a 9.3 that needs guest administrative access you do not hand out.
- **Leaving every virtual device enabled in the template.** Each is attack surface you are not using.
- **Assuming containers isolate like virtual machines.** One kernel. Chapter 11 is the answer when that is not acceptable.
- **Granting `--privileged` or the Docker socket to make something work.** Both are host root.
- **Disabling side-channel mitigations for performance without recording the decision.** It may be right; it must be deliberate and documented.
- **Forgetting the storage array.** It holds every disk, and its management interface is usually the least hardened thing in the room.
- **Treating a snapshot as protection from ransomware.** Same storage, same credentials.

Try It Yourself: Hardening a Container, and Proving It

Demonstrates the escape paths the table closes, then closes them. Run on a disposable machine — step 3 genuinely gives the container the host. Expect forty minutes.

```
#!/usr/bin/env bash
# ch19_harden.sh -- what --privileged means, and what hardening removes.
set -euo pipefail

# --- 1. A default container: how much is still exposed? -----
docker run --rm alpine:3.20 id          # root, by default
docker run --rm alpine:3.20 sh -c \
    'cat /proc/self/status | grep -E "^Cap(Eff|Prm)"'
# Decode those with capsh --decode=<hex>; the default set is ~14 caps.

# --- 2. The kernel is shared, and the container can see it. -----
docker run --rm alpine:3.20 uname -a    # the HOST kernel version
uname -a                               # identical. Chapter 8.

# --- 3. --privileged: the host, with extra steps. DISPOSABLE HOST ONLY.
docker run --rm --privileged alpine:3.20 sh -c \
    'ls /dev/ | head -20; echo "--- host disks visible above ---"'
docker run --rm --privileged --pid=host alpine:3.20 sh -c \
    'ps aux | head -5'                  # the host's process table
# With --privileged you can mount the host root and chroot into it.
# That is the whole "escape": there was never a boundary to cross.

# --- 4. The Docker socket is equivalent. -----
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
    docker:cli sh -c 'docker ps | head -3'
# Anything that can reach the socket can start a privileged container
# with / mounted. Never grant it to something you do not fully trust.

# --- 5. Now harden, measure by measure. -----
docker run --rm --user 10001:10001 alpine:3.20 id
docker run --rm --cap-drop=ALL alpine:3.20 sh -c \
    'cat /proc/self/status | grep ^CapEff' # 0000000000000000
docker run --rm --read-only alpine:3.20 sh -c 'touch /x' 2>&1 | tail -1
docker run --rm --security-opt=no-new-privileges alpine:3.20 \
    sh -c 'echo no-new-privileges set'

# --- 6. seccomp: count the system calls actually reachable. -----
docker run --rm --security-opt seccomp=unconfined alpine:3.20 \
```

```

sh -c 'echo unconfined: every syscall the kernel has'
# The default profile already blocks ~44 of them, including some that
# have historically been escape primitives.
docker info --format '{{.SecurityOptions}}'

# --- 7. All of it together: what a hardened run looks like. -----
docker run --rm \
  --user 10001:10001 \
  --cap-drop=ALL \
  --security-opt=no-new-privileges \
  --read-only --tmpfs /tmp \
  --pids-limit 100 \
  --memory 256m --cpus 0.5 \
  --network none \
  alpine:3.20 sh -c 'id; echo "hardened"; touch /tmp/ok && echo "tmp ok"'

# --- 8. And the alternative, when none of that is enough. -----
# Under Kubernetes this is one field (Chapter 11):
cat <<'YAML'
apiVersion: v1
kind: Pod
metadata:
  name: untrusted
spec:
  runtimeClassName: kata          # a hardware boundary, per workload
  containers:
  - name: job
    image: registry.example/builder@sha256:9b8c...
    securityContext:
      runAsNonRoot: true
      runAsUser: 10001
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: { drop: ["ALL"] }
YAML

# --- 9. Inventory your own estate for the two fatal flags. -----
docker ps -q | while read -r c; do
  priv=$(docker inspect -f '{{.HostConfig.Privileged}}' "$c")
  sock=$(docker inspect -f '{{range .Mounts}}{{.Source}} {{end}}' "$c" \
    | grep -c docker.sock || true)
  [ "$priv" = "true" ] && echo "PRIVILEGED: $c"
  [ "$sock" != "0" ] && echo "DOCKER SOCKET MOUNTED: $c"
done
echo "inventory complete"

```

What to notice. Step 3 is not an exploit — nothing was broken and no vulnerability was used. The flag was given, so the boundary was never there. Most real container “escapes” in the wild are this, and step 9 is how you find out whether your estate has any.

🔍 Checkpoint — Test Yourself

1. An estate has 25 hosts, 600 guests and one management server. Compute the blast radius of a guest compromise, a VM escape and a management-plane compromise, and say which you would defend first.
2. Explain why disabling SMT is sometimes the only sound answer to a side channel, and what it costs.
3. A monitoring vendor’s agent requires `-privileged` and a mount of the Docker socket. State what you are being asked to grant, and give two alternatives.

 Chapter Summary

- Virtualization is good for security on balance, and it added two things: privileged software under every workload, and a control plane that reaches every machine.
- VM escapes are overwhelmingly in *device models* — code parsing hostile input with hypervisor privileges — not in the hypervisor core. CVE-2026-47876 was in a paravirtual network adapter.
- Blast radius beats severity score. In the worked example an escape reached 2.5% of the estate and a management-plane flaw reached 100%, with no guest foothold required.
- Side channels are not software faults. Mitigations attack the sharing that makes hardware fast, and disabling SMT costs 15–30% of capacity. Decide deliberately and write it down.
- Containers share a kernel. `-privileged` and the Docker socket are each equivalent to host root.
- Harden in blast-radius order: management plane, hypervisor patching, device surface, segmentation, containers, SMT decision, backups.

 Review Questions

1. **(MCQ)** Most hypervisor escapes are found in: (a) the VMX instruction handling; (b) the extended page tables; (c) device models; (d) the scheduler.
2. **(MCQ)** Running a container with `-privileged`: (a) adds two capabilities; (b) grants all capabilities and disables seccomp and AppArmor; (c) is required for networking; (d) only affects the container's own filesystem.
3. **(Short)** Define blast radius and explain why it can reorder a priority list derived from CVSS scores.
4. **(Short)** Explain how two guests on one core can leak data between them with no software vulnerability involved.
5. **(Short)** Give three container hardening measures and state precisely what each removes.
6. **(Applied)** A hosting provider runs untrusted customer guests on shared hosts. Write the SMT and side-channel decision you would recommend, the capacity consequence, and how you would justify it to a finance director who sees only the lost capacity.
7. **(Applied)** You are given two weeks to improve the security of a 40-host estate with 900 guests, one vCenter on the general corporate network, templates with every virtual device enabled, and a flat network between guests. Write the plan, in order, with the reason for each position.

PART VI

Emerging Hardware and Practice

The 2017 outline ends with “emerging hardware features and the future of virtualization”. Two things have emerged since. Accelerators became the scarce resource and had to be divided like processors before them; and the trust model inverted — the guest stopped trusting the host, and the hardware had to be taught to take its side.

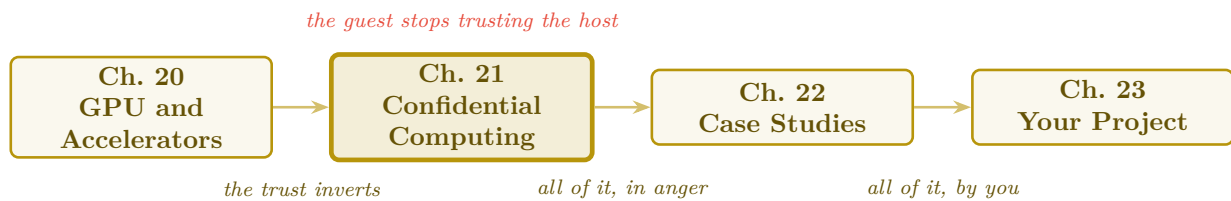


Figure P6.1: Reading path through Part VI. Chapters 20 and 21 are where the hardware is still moving; Chapters 22 and 23 put the whole book to work, first on other people’s decisions and then on your own.

GPU and Accelerator Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Emerging Hardware and Practice

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why a GPU is harder to share than a processor, and **name** the three sharing models.
- **Distinguish** time-sliced vGPU from MIG partitioning by what is divided and what is guaranteed.
- **Compute** the utilisation and cost consequences of each model for a stated set of jobs.
- **Describe** what a DPU moves off the host, and **relate** it to Chapter 7.
- **Choose** a sharing model for a workload and **state** what it gives up.

🔗 Before You Start

Chapter 7 entire — passthrough, VFIO, SR-IOV and the IOMMU are the machinery this chapter applies to a different device — and Chapter 6 (scheduling and overcommit, which work differently here).

📖 Key Terms in This Chapter

GPU passthrough • vGPU • time slicing • MIG • instance profile • SR-IOV for GPUs • framebuffer • context switch • mediated device • VFIO-mdev • DPU • SmartNIC • infrastructure offload • HBM • NVLink • fractional GPU

20.1 Why a GPU Is Not a CPU

Chapter 6 divided a processor among guests by scheduling: a vCPU runs for a slice, is descheduled, and another runs. The state to save is a few hundred bytes of registers, and the switch costs microseconds.

A GPU resists this for three reasons.

- **The state is enormous.** A context includes tens of gigabytes of high-bandwidth memory holding model weights, textures and intermediate results. You cannot save and restore that between time slices; there is nowhere to put it and no time to move it.
- **The work is long and uninterruptible.** A compute kernel runs to completion. Pre-empting it mid-flight is either impossible or very expensive, depending on the generation.
- **The memory is the scarce resource, not the compute.** A training job that needs 40 GB of GPU memory cannot run in 20 GB at half speed. It cannot run at all.

That last point is the one that matters most. Processor overcommit works because time is divisible (Chapter 6); GPU memory is as indivisible as host memory, and there is far less of it.

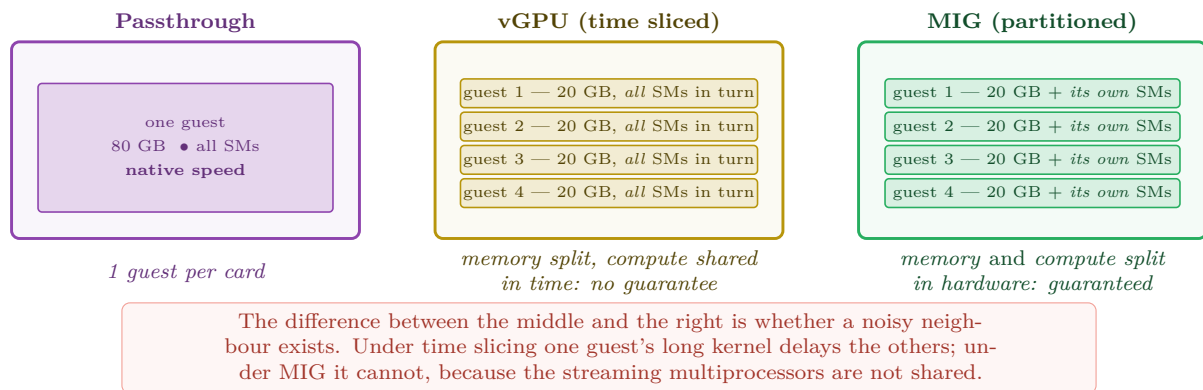


Figure 20.1: Three ways to divide one accelerator. Only the right-hand one gives a guarantee, and only the left-hand one gives the whole card.

20.2 The Three Models

☰ Passthrough, vGPU and MIG

Passthrough assigns the whole physical GPU to one guest with VFIO and the IOMMU, exactly as Chapter 7 assigned a network card. Native performance, the guest's own vendor driver, and all of Chapter 7's costs: no live migration, memory pinned, one guest per card.

vGPU — NVIDIA's term; a **mediated device** in Linux terms — splits the card's memory between several guests and *time slices* the compute between them. Each guest gets a fixed framebuffer and a turn at all the streaming multiprocessors. Density without a performance guarantee.

MIG — multi-instance GPU, from the A100 onward — partitions the hardware itself. Memory, cache, streaming multiprocessors and memory bandwidth are divided into up to seven instances with hardware-enforced boundaries. Each instance behaves like a smaller, independent GPU. Density *with* a guarantee, at the cost of fixed partition sizes.

💡 Which One Matches Which Workload

The deciding question is whether the workload's demand is steady or bursty, and whether it can tolerate interference.

- **Training a large model** — passthrough, or MIG's largest profile. It needs all the memory and it runs for days; sharing buys nothing.
- **Inference serving** — MIG. Many small, latency-sensitive requests that must not be delayed by somebody else's batch job. The guarantee is the point.
- **Virtual desktops with CAD or rendering** (Chapter 15) — vGPU. Demand is bursty and interactive, users are rarely busy simultaneously, and time slicing exploits exactly that.
- **Student or research workloads** — MIG where the hardware supports it, because fairness between users matters more than peak throughput, and one runaway job must not stop the others.

📅 Worked Example: Four Jobs, One Card

A department has one 80 GB accelerator and four research groups. Each needs 18 GB of GPU memory and runs jobs averaging 40 minutes, roughly 6 hours a day each. Compare passthrough, vGPU and MIG.

Step 1 — passthrough. One guest owns the card.

$$\text{concurrent groups} = 1 \quad \text{utilisation} = \frac{6}{24} = \mathbf{25\%}$$

The other three queue. Aggregate demand is $4 \times 6 = 24$ hours a day against 24 hours available, so the card is theoretically just sufficient — but only if the four groups never want it at the same time, which they will, because they work the same hours. Expect long queues at 2 p.m. and an idle card at 3 a.m.

Step 2 — vGPU, four slices of 20 GB. All four run concurrently. Memory fits ($4 \times 20 = 80$). Compute is time sliced, so when all four are busy each gets roughly a quarter of the throughput:

$$\text{job time when all four are active} = 40 \times 4 = \mathbf{160 \text{ minutes}}$$

When only one is active it gets the whole card and finishes in 40. Throughput is good; *predictability is gone*, and a researcher cannot tell whether their job will take 40 minutes or 160.

Step 3 — MIG, four instances. On an 80 GB card the 2g.20gb profile gives four instances of 20 GB with a fixed share of compute — about a quarter each, always.

$$\text{job time, always} = \frac{40}{0.25} = \mathbf{160 \text{ minutes}}$$

The same 160 minutes as the worst vGPU case — but it is *always* 160, whether the neighbours are busy or idle.

Step 4 — compare honestly.

- Passthrough: fastest single job (40 min), 25% utilisation, queueing.
- vGPU: 40–160 min depending on neighbours, high utilisation, unpredictable.
- MIG: always 160 min, high utilisation, predictable, and no job can be harmed by another.

Step 5 — the recommendation, and why it is not obvious. For four research groups sharing one departmental card, **MIG**. The reason is not throughput — vGPU matches or beats it — but that MIG makes the system *explicable*. A researcher who is told “you have a quarter of a card and your job takes 160 minutes” can plan. One whose job takes 40 minutes on Tuesday and 160 on Wednesday for reasons they cannot see will conclude the cluster is broken, and will start asking for their own card.

Step 6 — and when to reverse it. If the four groups’ work were genuinely bursty and rarely simultaneous — interactive visualisation rather than batch training — vGPU would win outright, because time slicing harvests exactly the idleness that MIG’s fixed partitions would waste.

⚠ Three practical constraints that decide this before you do

Licensing. NVIDIA vGPU requires a separate software licence per concurrent user, renewed annually. It is frequently a larger cost than the card over three years, and it is the reason many sites use passthrough or MIG instead.

MIG profiles are fixed. You choose from a published list — one instance of seven-eighths, two of a half, seven of a seventh, and a few combinations. You cannot ask for 30% of a card. Reconfiguring requires stopping every workload on it.

Not every card supports every model. MIG arrived with the A100 and is absent from consumer cards entirely; vGPU needs a supported card and the licence; passthrough works almost everywhere. Check the specific model before designing around a capability.

20.3 DPUs: Moving the Infrastructure Off the Host

The last few chapters have quietly accumulated work the host performs on behalf of guests: the virtual switch of Chapter 13, the encapsulation of VXLAN, the storage path of Chapter 12, the encryption everywhere. On a busy host this is a material fraction of the processor — capacity bought for guests and spent on plumbing.

A **DPU** (data processing unit, or SmartNIC) is a card with its own processors, memory and operating system that takes that work. The virtual switch runs on the card. The storage initiator runs on the card. Encryption runs on the card. The host's cores go back to the guests.

💡 Why This Is a Security Change as Much as a Performance One

The obvious benefit is reclaimed cores — commonly 20–30% of a host that was doing heavy overlay networking.

The structural benefit is that the infrastructure now runs on a processor the host's operating system does not control. A compromised hypervisor cannot rewrite the virtual switch's rules if the switch is not on the hypervisor; the management path to the DPU is separate. This is Chapter 5's disaggregation argument — Xen's driver domains — implemented in silicon rather than in software, and it is why AWS's Nitro architecture is built this way.

It also completes an arc this book has been tracing since Chapter 7: emulate the device, then paravirtualize it, then hand the real one over, and finally move the whole supporting apparatus to a computer of its own.

⚠️ Common Pitfalls

- **Assuming GPU memory can be overcommitted.** It cannot. A job needing 40 GB in a 20 GB instance does not run slowly; it fails.
- **Choosing vGPU for latency-sensitive inference.** Time slicing has no guarantee, and the tail latency of Chapter 18 is exactly what suffers.
- **Forgetting the vGPU licence in the budget.** It can exceed the hardware cost over three years.
- **Designing for MIG on a card that does not have it.** Check the model, not the vendor.
- **Expecting to live-migrate a GPU guest.** All of Chapter 7's passthrough costs apply, and GPU state makes them worse.
- **Measuring GPU utilisation as a percentage and stopping there.** "100% utilisation" from `nvidia-smi` means a kernel was resident, not that the hardware was fully used. Look at achieved occupancy and memory bandwidth.
- **Buying accelerators before measuring queueing.** The worked example's passthrough case was 25% utilised and still had queues, which is a scheduling problem, not a capacity one.

🔧 Try It Yourself: Partitioning an Accelerator

Runs against a real GPU where one is available, and against captured output where one is not — the file is in `code/` and the exercise is the same. Expect thirty minutes.

```
#!/usr/bin/env bash
# ch20_gpu.sh -- passthrough, MIG and what utilisation really reports.
set -euo pipefail

# --- 1. Is there an accelerator here at all? -----
if ! command -v nvidia-smi >/dev/null; then
    echo "no GPU on this host -- read code/ch20_capture.txt instead"
```

```

    sed -n '1,40p' "${dirname "$0"}/ch20_capture.txt" 2>/dev/null || true
    exit 0
fi
nvidia-smi --query-gpu=name,memory.total,mig.mode.current \
    --format=csv

# --- 2. The IOMMU group: what passthrough would have to take. -----
# Chapter 4: the whole group moves together, or nothing does.
for d in $(lspci -D | awk '/NVIDIA/{print $1}'); do
    grp=$(basename "$(dirname "$(readlink -f \
        "/sys/bus/pci/devices/$d/iommu_group")")")
    echo "device $d is in IOMMU group $grp"
    ls "/sys/kernel/iommu_groups/$grp/devices/"
done
# A card and its audio function usually share a group: both must go.

# --- 3. Enable MIG, if this card supports it. -----
sudo nvidia-smi -i 0 -mig 1 || \
    echo "MIG not supported on this card -- see the alertbox"
sudo nvidia-smi mig -i 0 -lgip      # the fixed profiles available

# --- 4. Create four instances, and see them as separate devices. -----
sudo nvidia-smi mig -i 0 -cgi 2g.20gb,2g.20gb,2g.20gb,2g.20gb -C || true
nvidia-smi -L                      # four MIG UUIDs, one card

# --- 5. Run the same job on one instance, then on all four. -----
JOB='python3 -c "
import time
try:
    import torch
except ImportError:
    raise SystemExit("install torch to run this step")
a = torch.randn(8192, 8192, device="cuda")
t = time.time()
for _ in range(50): a = a @ a.T / 1e4
torch.cuda.synchronize(); print(f"{time.time()-t:.1f} s")'

mig_uuids () {
    nvidia-smi -L | awk -F'UUID: ' '/MIG/{print $2}' | tr -d ' '
}
UUID=$(mig_uuids | head -1)
echo "=== alone on one instance ==="
CUDA_VISIBLE_DEVICES="$UUID" bash -c "$JOB"

echo "=== all four instances busy together ==="
mig_uuids | while read -r u; do
    CUDA_VISIBLE_DEVICES="$u" bash -c "$JOB" &
done
wait
# Under MIG the single-instance time and the all-busy time should be
# the SAME. That is the guarantee the worked example is about.

# --- 6. What "utilisation" actually reports. -----
nvidia-smi --query-gpu=utilization.gpu,utilization.memory,\
memory.used,memory.total --format=csv -l 1 -c 5
# utilization.gpu is "a kernel was resident during the sample", not
# "the hardware was fully used". See the pitfalls.

# --- 7. Tear down the partitions. -----
sudo nvidia-smi mig -i 0 -dci || true
sudo nvidia-smi mig -i 0 -dgi || true
sudo nvidia-smi -i 0 -mig 0 || true

```

What to notice. Step 5 is the whole chapter in one measurement. Under MIG the two timings match, because the partitions are enforced in hardware. Repeat the same test under vGPU time slicing and they will not — and that difference, not throughput, is what you are choosing between.

🔍 Checkpoint — Test Yourself

1. Explain why GPU memory cannot be overcommitted the way host memory can, referring to Chapter 6's reclamation ladder.
2. Six groups each need 12 GB and run 5 hours a day on one 80 GB card. Compare passthrough and a six-way MIG partition on utilisation and on predictability.
3. `nvidia-smi` reports 100% GPU utilisation for a job that a colleague says is poorly optimised. Explain how both can be true, and what you would measure instead.

📋 Chapter Summary

- A GPU resists time slicing because its context is tens of gigabytes, its kernels are uninterruptible, and memory — not compute — is the scarce, indivisible resource.
- Passthrough gives one guest the whole card at native speed, with all of Chapter 7's costs.
- vGPU splits memory and time slices compute: density without a guarantee. MIG partitions memory *and* compute in hardware: density with a guarantee, in fixed profiles.
- In the worked example MIG and vGPU gave the same throughput and only MIG gave a predictable answer — which is usually what shared research and inference infrastructure needs.
- Licensing, fixed profiles and card support frequently decide the question before the engineering does.
- A DPU moves the virtual switch, the storage path and encryption to a computer of its own: 20–30% of host cores returned, and the infrastructure placed outside the hypervisor's control.

🔧 Review Questions

1. (MCQ) MIG differs from time-sliced vGPU in that it: (a) supports more guests; (b) partitions compute and memory in hardware, so performance is guaranteed; (c) requires no licence; (d) allows live migration.
2. (MCQ) A job needing 40 GB of GPU memory, given a 20 GB instance, will: (a) run at half speed; (b) swap to host memory; (c) fail; (d) be queued automatically.
3. (Short) Give the three reasons a GPU is harder to share than a CPU, and say which is the most consequential.
4. (Short) State a workload suited to vGPU and one suited to MIG, with the reason for each.
5. (Short) Explain what a DPU moves off the host and why that is a security change as well as a performance one.
6. (Applied) A department will buy one accelerator for eight research students whose jobs need 9 GB each and run in bursts during office hours. Recommend a sharing model, give the expected utilisation, and state the two questions you would answer before purchase.
7. (Applied) An inference service has a p99 latency requirement of 150 ms and currently runs on time-sliced vGPU, missing the target intermittently. Explain the mechanism, propose a change, and say what you would measure to confirm the fix.

Confidential Computing and Emerging Hardware

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Emerging Hardware and Practice

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** how the trust model inverts, and **say** what the guest stops trusting.
- **Explain** memory encryption with per-guest keys, and **say** what it does and does not protect against.
- **Describe** remote attestation and **explain** why encryption without it is nearly worthless.
- **Compare** AMD SEV-SNP, Intel TDX and ARM CCA by maturity and availability as of late 2026.
- **Decide** whether a stated workload justifies a confidential VM, and **state** the cost.

🔗 Before You Start

Chapter 19 (the attack surface, and in particular who is trusted) and Chapter 4 (VMX, the VMCS and extended page tables, which are what this hardware modifies). Chapter 14's shared responsibility model is the commercial arrangement this chapter challenges.

📖 Key Terms in This Chapter

confidential computing • trusted execution environment • TEE • trust model • memory encryption • SEV • SEV-ES • SEV-SNP • TDX • trust domain • ARM CCA • realm • SGX • enclave • attestation • attestation report • measurement • chain of trust • confidential container • CXL • composable infrastructure

21.1 The Trust Model Inverts

Every chapter so far has assumed the hypervisor is trustworthy. It schedules guests, allocates their memory and mediates their devices, and a guest has no choice but to rely on it. Chapter 19 worried about an attacker *reaching* the hypervisor; it never questioned the hypervisor itself.

Confidential computing questions it. The guest is given a way to run on hardware it does not own, operated by people it has not met, *without trusting them with its data*.

This matters commercially rather than philosophically. A hospital will not put patient records in a public cloud if the provider's administrators can read them. A bank will not run a model on hardware whose operator could copy the weights. The cloud's answer has always been contracts and audits; confidential computing offers a technical answer instead.

📖 Trusted Execution Environment

A **TEE** is an execution context whose memory and registers are protected from everything outside it — including the hypervisor, the host operating system, the firmware and anyone with physical access.

For a confidential VM this means:

- memory is **encrypted** with a key the guest's own hardware context holds and the host cannot read;

- register state is protected across every VM exit, so the hypervisor cannot inspect the guest's execution;
- the guest can **prove** what it is — which hardware, which firmware, which image — to a remote party.

The hypervisor still *schedules* the guest, allocates it memory and gives it devices. It simply cannot see inside it.

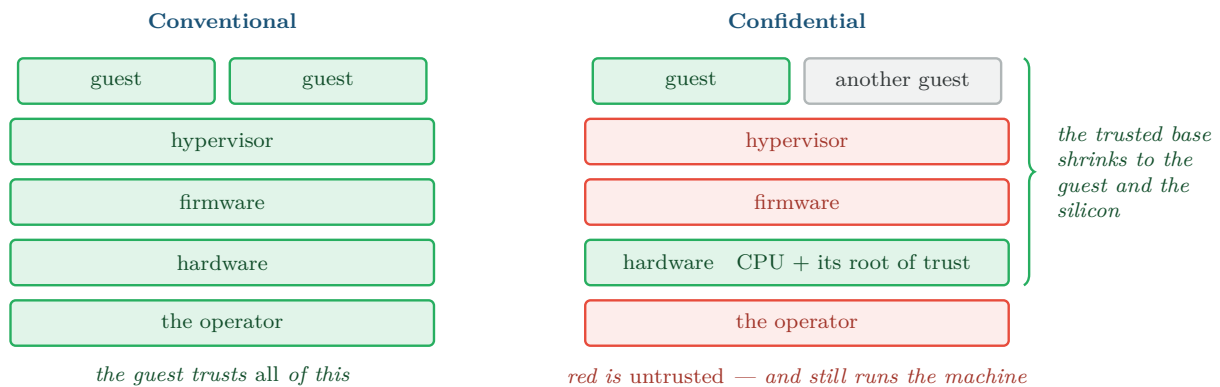


Figure 21.1: The inversion. The hypervisor still schedules the guest and allocates its memory; it simply loses the ability to read it. Everything red is in the attack path and outside the trust boundary.

21.2 What the Hardware Does

💡 Memory Encryption, and Why Encryption Alone Is Not Enough

The memory controller encrypts and decrypts transparently with a key tied to the guest, held in the processor and never exposed to software. Host memory dumped from outside yields ciphertext.

That defeats the straightforward attacks — a hypervisor reading guest pages, a memory image taken from a running host, cold-boot attacks on the DIMMs. AMD's first **SEV** (2017) did this much, and researchers quickly showed it was not enough.

The reason is that a hostile hypervisor does more than read. It can *remap* pages — give the guest a different physical page than it had — *replay* old encrypted pages, or *alias* two guest addresses to one physical page. The data stays encrypted throughout and the guest is still compromised.

SEV-ES (2018) added register-state protection across VM exits. **SEV-SNP** (2020) added the missing piece: *integrity*. A reverse map table records which guest owns each physical page, and the hardware refuses a mapping that contradicts it. Intel's **TDX** was designed with equivalent protections from the start.

The lesson generalises: confidentiality without integrity is a weaker property than it sounds, and the first generation of almost every such system learns this the same way.

📖 Attestation

Attestation is the guest proving what it is to a remote party before that party entrusts it with anything.

The hardware measures the guest’s initial state — firmware, kernel, initial memory image — and produces an **attestation report** containing those measurements, the processor’s security version, and a signature chaining to a key the silicon vendor certified.

The verifier checks the signature, confirms the hardware is genuine and up to date, and compares the measurement against what it expected. Only then does it release the decryption key, the model weights or the database credentials.

Encryption without attestation protects nothing useful. If you cannot verify what is running, a hostile operator simply boots a modified image that asks you politely for the data — and that image’s memory is beautifully encrypted while it exfiltrates. Attestation is the part that makes the encryption meaningful, and it is the part deployments most often skip.

21.3 Where the Platforms Stand

Dated October 2026. This is the fastest-moving material in the book; check before relying on any row.

	AMD SEV-SNP	Intel TDX	ARM CCA
Unit protected	The whole VM	The whole VM (a trust domain)	The whole VM (a realm)
Introduced	SEV 2017, SNP 2020	2023	Announced 2021
Maturity	The most deployed; generally available across major clouds	Production confidential VMs on Azure and Google Cloud, with newer machine families still arriving in preview during 2026	Specification mature, <i>silicon and cloud availability still limited</i> ; previews rather than general availability
Watch for	Attestation report format changed to v4 with a firmware update — parsers written for v3 break	A TDX firmware vulnerability was disclosed and patched in early 2026; keep firmware current	Expect availability to change fastest of the three

⚠ The predecessor that is not this: Intel SGX

SGX protected an *enclave* — a region inside one process — rather than a whole virtual machine. The application had to be rewritten and split, with the sensitive part inside the enclave and everything else outside, across a narrow and awkward boundary. Enclave memory was also small, in the low hundreds of megabytes on most parts.

It saw real use and it did not become general infrastructure, for the obvious reason: nobody wants to re-architect an application to run it somewhere.

TDX, SEV-SNP and CCA all learned from that. They protect an *unmodified virtual machine*, so an existing workload moves in without being rewritten — which is the same bargain Chapter 3 struck when it insisted on running unmodified guests, made again two decades later.

🧑‍💻 Worked Example: Does This Workload Justify a Confidential VM?

A hospital wants to run analytics over patient records in a public cloud. The compliance team objects that the provider’s staff could read the data. Decide whether confidential computing answers the objection, and what it costs.

Step 1 — what is actually being claimed? Not that the provider *would*, but that they *could*, and that the hospital cannot demonstrate otherwise to its regulator. The requirement is therefore evidentiary, not merely technical.

Step 2 — what a confidential VM gives.

- memory encrypted with a key the host cannot read;
- integrity protection against remapping and replay (SNP or TDX);
- an attestation report the hospital can verify before releasing the database key.

The provider's administrators can still stop the machine, and still see how much memory and processor it uses. They cannot read the records.

Step 3 — what it does *not* give. Four gaps worth stating to the compliance team before they believe too much:

- **Availability.** The operator can still destroy or detain the machine. Confidentiality and integrity; not availability.
- **Side channels.** Timing and cache attacks (Chapter 19) are not fully addressed, and some published attacks target exactly these systems.
- **Your own bugs.** A flaw in the application inside the enclave leaks the data the normal way.
- **Data in use elsewhere.** The moment records leave for an unprotected service, the guarantee ends. The boundary must cover the whole path.

Step 4 — the cost. Performance overhead is workload-dependent and generally modest — commonly in the region of 2–8% for ordinary server workloads, more where I/O is heavy, because encrypted guests use bounce buffers for device transfers. Instance choice is restricted to supported families, and some features — live migration (Chapter 17), certain memory management — may be unavailable or restricted.

The larger cost is operational: attestation must be built, verified and maintained. Somebody must decide what measurement is expected, store it, check it, and update it with every image change. **A confidential VM whose attestation nobody verifies is a normal VM with a performance penalty.**

Step 5 — the recommendation. Yes, with a condition. Use a confidential VM *and* build the attestation flow, so the database key is released only to a verified measurement. Record the expected measurement under change control. If the project cannot commit to maintaining that, the honest advice is to keep the data on premises, because the alternative is a control that exists only on a slide.

21.4 Composable Infrastructure and CXL

One more direction the hardware is moving, included because the 2017 outline asked about the future of virtualization and this is a plausible part of it.

Every host in this book is a fixed box: so many cores, so much memory, bought together and discarded together. A host with spare memory cannot lend it to the host next to it, so every machine carries headroom nobody else can use — stranded capacity, at scale.

CXL — Compute Express Link — allows memory to be attached to a fabric rather than to a socket, so a pool of memory can be allocated to whichever host needs it. The ambition, **composable infrastructure**, is to assemble a machine from pooled components for a workload and return them afterwards.

💡 Why This Is the Same Idea as Chapter 1

Pooled memory, allocated on demand and returned, is *virtualization of the memory bus* — multiplexing, by the definition of Chapter 1. The motive is the one from the opening pages: capacity bought and left idle because it could not be shared.

The obstacle is the one the whole book keeps meeting: latency. Fabric-attached memory is slower than socket-attached memory, as NUMA's remote node is slower than its local one (Chapter 6) — only more so. Early deployments are therefore tiered, with hot pages local and cold pages pooled, which is exactly the memory-reclamation ladder of Chapter 6 wearing different clothes.

Treat this section as a direction, not a product. The useful thing to carry away is that you already have the framework to evaluate it when a vendor arrives: ask what is multiplexed, what the lie costs, and who maintains it.

❗ Common Pitfalls

- **Deploying encryption without attestation.** You have hidden the data from an attacker who was not going to read it, and not from one who boots a modified image.
- **Believing the first generation is enough.** Plain SEV lacked integrity and was broken by remapping. Use SEV-SNP or TDX.
- **Assuming availability is protected.** It is not. The operator can still stop your machine.
- **Assuming side channels are solved.** They are not, and published attacks target these systems specifically.
- **Hard-coding an attestation report format.** AMD's move to v4 broke parsers written for v3.
- **Forgetting the rest of the path.** The guarantee ends where the data leaves the TEE.
- **Planning on ARM CCA availability.** As of late 2026 it is previews, not general availability. Check before designing around it.

🔧 Try It Yourself: Attestation, Verified

Inspects a confidential VM and verifies an attestation report. Where no suitable processor is available, the captured reports in `code/` make the verification exercise identical. Expect thirty-five minutes.

```
#!/usr/bin/env bash
# ch21_attest.sh -- is this guest confidential, and can it prove it?
set -euo pipefail

# --- 1. Does this processor support any of the three? -----
grep -o -m1 -E '\<(sev|sev_es|sev_snp|tdx_guest)\>' /proc/cpuinfo || \
  echo "no TEE features on this host"
ls /sys/firmware/ 2>/dev/null | grep -iE 'tdx|sev' || true
dmesg | grep -iE 'SEV-SNP|TDX|Memory Encryption' | head -5 || true

# --- 2. From inside a guest: am I confidential? -----
# Each platform exposes itself differently; check all three.
[ -e /dev/sev-guest ] && echo "AMD SEV-SNP guest device present"
[ -e /dev/tdx-guest ] && echo "Intel TDX guest device present"
systemd-detect-virt
# On a normal guest, none of these exist. That is the point of step 4.

# --- 3. Ask the hardware for an attestation report. -----
# snpguest (AMD) or trustee/tdx tools (Intel). A 64-byte nonce from the
# verifier goes in; a signed report comes out.
```

```

NONCE=$(openssl rand -hex 32)
if [ -e /dev/sev-guest ] && command -v snpguest >/dev/null; then
    snpguest report report.bin request.txt --random
    snpguest display report report.bin | head -30
else
    echo "no TEE here -- using the captured report in code/"
    cp "$(dirname "$0")/ch21_report.txt" report.txt 2>/dev/null || true
    sed -n '1,30p' report.txt 2>/dev/null || true
fi

# --- 4. The three fields that matter, and why. -----
cat <<'NOTES'
    measurement      the hash of the guest's initial state. Compare this
                      with what you built; if it differs, something else
                      is running and you must NOT release the key.
    report_data       your nonce, echoed back. Proves the report is fresh
                      and not a recording of an earlier genuine boot.
    signature + cert chain chains to a key the silicon vendor certified. Proves
                      the report came from real hardware, not from software
                      pretending very convincingly.
NOTES

# --- 5. Verify the chain. This is the step people skip. -----
if command -v snpguest >/dev/null; then
    snpguest fetch ca-der milan ./certs          # vendor certificates
    snpguest fetch vcek-der milan ./certs report.bin
    snpguest verify certs ./certs
    snpguest verify attestation ./certs report.bin && \
        echo "VERIFIED: genuine hardware, measurement as reported"
else
    echo "verification exercise: compare the measurement in report.txt"
    echo "with code/ch21_expected_measurement.txt and decide."
fi

# --- 6. Now the attack this defends against. -----
# A hostile operator boots a DIFFERENT image that asks for your key.
# Its memory is perfectly encrypted while it exfiltrates. The only
# thing that stops it is step 5 refusing the measurement.
diff <(grep -i measurement report.txt 2>/dev/null || echo a) \
    <(cat "$(dirname "$0")/ch21_expected_measurement.txt" 2>/dev/null \
        || echo b) >/dev/null \
    && echo "measurement matches -- release the key" \
    || echo "measurement DIFFERS -- refuse. This is the whole mechanism."

# --- 7. What it costs: the same benchmark, confidential or not. -----
# On a cloud that offers both, run this on a confidential and a normal
# instance of the same family and compare.
sysbench cpu --threads=4 --time=20 run 2>/dev/null | grep -E 'events/s' \
    || echo "install sysbench to measure the overhead"
dd if=/dev/zero of=/tmp/t bs=1M count=2048 oflag=direct 2>&1 | tail -1
rm -f /tmp/t
# I/O usually shows the larger penalty: encrypted guests bounce device
# transfers through shared buffers.

```

What to notice. Step 6 is the chapter. Encryption was working perfectly in both the honest and the hostile case; the only thing that distinguished them was a hash comparison that somebody had to perform. Skip step 5 and you have bought a performance penalty.

? Checkpoint — Test Yourself

1. Explain why memory encryption without integrity protection was insufficient, naming one concrete attack it failed to stop.
2. A team deploys confidential VMs but releases the database key from a configuration file at boot, without checking any report. State exactly what they are protected against and what they are not.
3. Name two properties confidential computing does *not* provide, and say why each matters to a compliance argument.

✓ Chapter Summary

- Confidential computing inverts the trust model: the guest stops trusting the hypervisor, the firmware and the operator, and trusts only itself and the silicon.
- A TEE encrypts guest memory with a key the host cannot read, protects register state across exits, and lets the guest prove what it is.
- Encryption alone was not enough: a hostile hypervisor can remap, replay and alias pages. SEV-SNP and TDX add integrity.
- Attestation is what makes the encryption meaningful. A confidential VM whose attestation nobody verifies is a normal VM with a performance penalty.
- As of late 2026 SEV-SNP is the most widely deployed, TDX is in production on major clouds with newer families still arriving, and ARM CCA remains limited in availability.
- It does not protect availability, does not solve side channels, and does not help with your own bugs. Overhead is commonly 2–8%, more for I/O.
- CXL and composable infrastructure are the same multiplexing idea applied to the memory bus, with latency as the familiar obstacle.

✍ Review Questions

1. **(MCQ)** SEV-SNP added, over SEV-ES: (a) larger enclaves; (b) integrity protection against page remapping and replay; (c) live migration; (d) support for containers.
2. **(MCQ)** Without attestation, memory encryption fails to stop: (a) a cold-boot attack; (b) a hypervisor reading guest pages; (c) an operator booting a modified image that requests the data; (d) a physical memory dump.
3. **(Short)** Explain the three fields of an attestation report and what each one proves.
4. **(Short)** Why did Intel SGX not become general infrastructure, and what did the VM-scoped designs do differently?
5. **(Short)** State three things confidential computing does not protect.
6. **(Applied)** A bank wants to run a proprietary model on a public cloud without the provider being able to copy the weights. Set out the design, the verification step that makes it real, what it costs, and the residual risks you would record.
7. **(Applied)** Using the framework of Chapter 1 — what is multiplexed, what does the lie cost, who maintains it — evaluate CXL pooled memory as a vendor would present it to you, and write the three questions you would ask.

Case Studies in Enterprise and Cloud Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — *Emerging Hardware and Practice*

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Analyse** a virtualization decision using the arithmetic of the preceding chapters rather than the vendor's framing.
- **Identify** the measurement that would have changed each decision, and **say** when it should have been taken.
- **Explain** why a technically correct migration can still be a commercial failure.
- **Apply** the consolidation, overcommit, migration and cost models to a case you have not seen before.
- **Write** a recommendation that states its assumptions and the evidence that would overturn it.

🔗 Before You Start

Everything. Each case below uses the arithmetic of a specific earlier chapter, named where it arises, and the point of the chapter is that the arithmetic is the same whatever the vendor calls the product.

📖 Key Terms in This Chapter

consolidation ratio • right-sizing • lift and shift • refactoring • egress • reserved capacity • licence model • blast radius • edge site • intermittent connectivity • post-mortem • counterfactual

22.1 How to Read These

Four cases. The figures are representative of real deployments rather than drawn from one named organisation, and they are constructed so that the arithmetic can be checked — that is what they are for. Each follows the same shape: what was decided, what happened, what the numbers said, and the single measurement that would have changed the outcome.

⚠️ Every number in this chapter is invented, and says so

The cases are composites. They are built to be arithmetically consistent and to exercise the models of earlier chapters, not to report on any particular institution. Where a real, dated fact is used — a licence change, a published vulnerability — it is cited as such in the chapter that introduced it.

Treat the figures as you would a worked example: reproduce them, change the inputs, and see where the recommendation flips. The skill being taught is the flipping point, not the answer.

22.2 Case 1: The Consolidation That Nearly Failed

💡 The decision

A university's computing department had 64 physical servers, bought one at a time over nine years. It consolidated onto 5 hosts and a shared array. The business case cited the arithmetic of Chapter 1: a 12.8:1 ratio, 85% less power, one estate to patch instead of 64. Eleven weeks after the migration the estate was described by its own users as “slower than the old servers”.

What the numbers said. Sizing had been done on *allocated* capacity, as Chapter 6 warns.

$$\sigma_{\text{cpu}} = \frac{64 \text{ guests} \times 4 \text{ vCPU}}{5 \times 48 \text{ cores}} = \frac{256}{240} = 1.07$$

which looks comfortable. But N+1 was not applied, and one host was taken for maintenance in week 11:

$$\sigma_{\text{cpu}}|_{N+1} = \frac{256}{4 \times 48} = \frac{256}{192} = 1.33$$

That alone is survivable. What was not survivable was memory. The 64 old servers had 32 GB each; the guests were given 32 GB each because that is what they had before, and nobody measured what they used:

$$\text{allocated} = 64 \times 32 = 2,048 \text{ GB} \quad \text{installed} = 5 \times 384 = 1,920 \text{ GB}$$

$$\sigma_{\text{mem}} = \frac{2,048}{1,920 \times 0.85} = 1.26$$

With all five hosts this was absorbed by page sharing and ballooning (Chapter 6). With four hosts it was not:

$$\sigma_{\text{mem}}|_{N+1} = \frac{2,048}{4 \times 384 \times 0.85} = \frac{2,048}{1,306} = 1.57$$

The hosts reached the bottom of the reclamation ladder and began swapping. %RDY was fine throughout, which is why the first week of investigation looked at the wrong resource.

The measurement that would have changed it. Active memory, before migration. Measured afterwards it was 9.4 GB per workload:

$$64 \times 9.4 = 602 \text{ GB}$$

against 1,920 GB installed — the estate needed a third of what was bought, and the guests should have been given 12 GB, not 32.

What was done. Right-sizing, over three weeks, guest by guest. σ_{mem} fell to 0.59 and $\sigma_{\text{mem}}|_{N+1}$ to 0.74. No hardware was added.

💡 The lesson, stated generally

Migrating a server is not sizing a guest. The old machine's specification was a purchasing decision from up to nine years earlier, made against a different workload, with headroom for growth that never arrived. Carrying it across preserves every bad guess ever made and adds the hypervisor's overhead on top.

Measure for a month before migrating; size to the 95th percentile with a stated margin; and apply N+1 to *every* ratio you quote, because the failure happens on the day one host is away.

22.3 Case 2: The Cloud Migration That Cost More

💡 The decision

A software house of 40 engineers moved its entire estate — 120 guests, development and production — to a public cloud over four months. The projection showed a 15% saving. The first full month's invoice was 2.3 times the projection, and the second was higher.

Where the money went. Four causes, in descending order, each traceable to a chapter.

Cause	Share of overrun	What happened	Chapter
Lift and shift without right-sizing	44%	120 guests moved at their existing vCPU and memory, which had never been measured	Chapter 6, Chapter 14
On-demand pricing throughout	31%	No reservations, because nobody was confident enough to commit. Chapter 14's worked example: on demand was 61% more than owning	Chapter 14
Egress	17%	Nightly backups written <i>out</i> of the cloud to the old on-premises array, at \$0.09 per GB	Chapter 14
Development guests running continuously	8%	44 non-production guests billed 24×7 for work done 8×5	Chapter 14

The arithmetic of the fix. All four were addressed in six weeks, with no architectural change.

- **Right-sizing.** Measured p95 across 120 guests; mean allocation fell from 6.2 vCPU and 21 GB to 2.1 vCPU and 7 GB. Compute cost $\times 0.34$.
- **Reservations** on the 61 guests that were genuinely steady. Chapter 14's figure for a three-year commitment is 55%; a one-year commitment was taken instead at 34%, because the business would not commit for three.
- **Backups kept in the cloud**, with only a weekly copy leaving. Egress fell by about 85%.
- **Development guests stopped out of hours**, a scheduled task: $24 \times 7 = 168$ hours becomes $10 \times 5 = 50$, a 70% reduction on those 44 guests.

The invoice settled at about 0.78 of the on-premises baseline — close to the original projection, reached eight months late.

💡 The lesson: lift and shift is a migration strategy, not a cost strategy

Lifting and shifting is a perfectly reasonable way to *move*: it is fast, it is low-risk, and it decouples the move from the redesign.

What it is not is a way to save money, and the reason is structural. On premises, waste is invisible — a guest with four times the memory it needs costs nothing extra once the host is bought. In the cloud, every unit of waste is a line on a monthly invoice. The cloud did not make this estate expensive; it made it *legible*, and what became legible was eleven years of unmeasured allocation.

If a migration business case claims savings, ask which of the four items above it has budgeted for. If the answer is none, the projection is wrong by roughly the factor this case saw.

22.4 Case 3: Edge Virtualization at Twenty-Three Sites

💡 The decision

A logistics company ran a small application at each of 23 depots: stock, weighing, dispatch. Each site had two aging PCs under a desk. Connectivity to head office was a consumer broadband line, and it failed somewhere in the estate most weeks.

The proposal was to centralise everything into the data centre and give each depot a thin client (Chapter 15).

Why centralisation was rejected. The arithmetic was not about cost but about Chapter 17's availability vocabulary.

Depot operations must continue when the line is down. Measured over a year, each site's link had about 99.2% availability:

$$(1 - 0.992) \times 8,760 = \mathbf{70} \text{ hours a year}$$

Seventy hours a year per site of *no dispatch at all*, against the current arrangement where a line failure meant only that head office reporting was delayed. Centralisation would have improved manageability and made the business worse.

What was built instead. One small two-node cluster per depot, running three guests: the application, a local database and a management agent. Specifically:

- **Immutable images** (Chapter 16). Depots have no technical staff, so nothing is configured on site. A new version is a new image; rollback is the previous image.
- **Containers, not VMs, for the application** (Chapter 8), because the hardware budget per site was small and the start-up speed made unattended recovery practical.
- **Local persistence with asynchronous replication** to head office. RPO of 15 minutes, accepted explicitly: a depot losing 15 minutes of dispatch records in a total site failure was judged tolerable, and synchronous replication over a failing consumer line was not possible anyway.
- **No live migration** (Chapter 17). Two nodes, so quorum is impossible; a witness ran at head office, and when the line was down the site operated in a documented degraded mode with HA disabled rather than risking the split brain of Chapter 17.

💡 The lesson: the constraint is rarely the one in the proposal

The proposal was framed as a management problem — 46 unmanaged PCs — and the answer was framed as centralisation. The actual constraint was a 99.2% network link, which nobody had measured until someone asked for the number.

Edge deployments invert several assumptions this book has relied on: bandwidth is scarce and unreliable, there is no one on site, physical security is weak, and the hardware is whatever fits under a desk. The techniques still apply — images, containers, replication, quorum — but the *weightings* change completely.

The transferable question: *what does this site have to keep doing when the link is down?* If the answer is “nothing”, centralise. If it is “everything”, the compute follows the work.

22.5 Case 4: A Licence Change, and What It Revealed

💡 The decision

A hospital trust ran 340 guests on 14 hosts under vSphere. At renewal, a change in the vendor’s packaging — bundled subscription rather than perpetual per-socket licensing, as the industry saw after 2024 (Chapter 5) — raised the quoted renewal by a factor of roughly four.

Three options were considered: pay, migrate to another platform, or move to a public cloud.

What the comparison looked like. Over three years, approximately:

Option	Licence	Migration effort	Risk
Stay on vSphere	High	None	Exposed to the next repricing
Migrate to Proxmox or oVirt	Near zero	340 guests, 9–14 months, two engineers	Staff retraining; tooling rebuilt; clinical systems vendors may not support it
Public cloud	N/A	Largest; also a data-residency review	Egress; ongoing cost; regulatory

The finding that decided it. Not cost. When the trust catalogued what would have to be rebuilt, it discovered that **nobody could enumerate what the estate did**. There was no inventory mapping guests to clinical services, no record of which of the 340 were still needed, and 61 guests whose owner could not be identified at all.

The migration could not be planned because the thing to be migrated was not known.

What was done. A twelve-month discovery and rationalisation programme first: inventory, ownership, dependency mapping, decommissioning. It retired 78 guests — 23% of the estate — that nothing used. Only then was the platform decision taken, and by then the licence quantity had fallen with the guest count.

💡 The lesson: lock-in is not technical

Every chapter in this book would say the migration is feasible. The hypervisors are comparable (Chapter 5), the disk formats convert (Chapter 12), the networking is reproducible (Chapter 13).

What made it hard was institutional knowledge that had never been written down. That is the real lock-in, and it is not specific to any vendor: it accumulates in any estate that grows without an inventory, and it is what makes every platform change cost three times its estimate.

The cheapest insurance against a repricing you cannot predict is an inventory you maintain. It costs little, it is useful every day for other reasons, and it converts a crisis into a project.

22.6 What the Four Have in Common

Case	Framed as	Actually about	Missing measurement
1	Consolidation ratio	N+1 memory under maintenance	Active memory, before migrating
2	Cloud versus on premises	Eleven years of unmeasured allocation becoming visible	p95 utilisation per guest
3	Managing 46 unmanaged PCs	What must keep working when the link fails	Link availability per site
4	A licence renewal	Not knowing what the estate does	An inventory with owners

💡 The pattern

In all four, the decision was framed as a technology choice and settled by a measurement nobody had taken. In none of them was the chosen technology wrong.

That is worth carrying into Chapter 23 and into professional practice. The engineering in this book is the easy part: hypervisors work, containers work, migration works. The difficulty is almost always in knowing what you have, what it uses and what it must keep doing — and those are questions of measurement and record-keeping, not of architecture.

When you are handed a proposal, the most useful question is rarely “is this the right technology?”. It is “*what number would change this decision, and has anyone taken it?*”

❗ Common Pitfalls

- **Quoting a consolidation ratio without N+1.** Case 1 failed on the day one host went away for maintenance.
- **Carrying old specifications into new guests.** You preserve every purchasing guess ever made and add overhead.
- **Expecting lift and shift to save money.** It moves the estate; it does not fix it, and the cloud bills for what on premises concealed.
- **Centralising without measuring the link.** Availability at the edge is a property of the worst site, not the average one.
- **Treating a platform migration as a technical project.** The cost is in the knowledge that was never written down.
- **Reading a case study as a recommendation.** Change one input and the answer changes; the skill is knowing which input.

🔧 Try It Yourself: Analysing a Case You Have Not Seen

No commands. Work in pairs, 45 minutes, and write a one-page recommendation — this is deliberately the shape of the examination’s applied questions and of Chapter 23’s report.

The brief. A college runs 90 guests on 4 hosts (32 cores, 256 GB each), bought three years ago, under a platform whose support expires in seven months. Measured over the last term:

- allocated: 410 vCPU, 1,740 GB
- active: 198 GB total, p95; %RDY p95 11%
- storage: 24 TB provisioned on an 11 TB thin pool, 7.9 TB written, growing 180 GB a month
- 14 guests have no identified owner
- the finance office requires three-year costings

Produce:

1. σ_{cpu} and σ_{mem} , with and without N+1.
2. The storage over-subscription ratio and the time to pool exhaustion (Chapter 12).
3. The single most urgent finding, with the arithmetic that supports it.
4. Three options with three-year costings, using Chapter 14's method for the cloud one.
5. A recommendation, the assumptions it rests on, and **the one measurement that would overturn it.**

Marking yourselves. A good answer identifies that %RDY of 11% and σ_{cpu} of 4.0 mean the estate is already contended; that memory is wildly over-allocated at 1,740 GB against 198 GB active; that the thin pool has about 17 months at the current rate but no N+1 storage headroom; and that the 14 unowned guests are case 4 in miniature. A weak answer compares products.

? Checkpoint — Test Yourself

1. In case 1, σ_{mem} was 1.26 with five hosts and 1.57 with four. Explain why the first was survivable and the second was not, naming the mechanism.
2. Case 2's estate was not made more expensive by the cloud. State in one sentence what the cloud actually did, and name the two items that together accounted for 75% of the overrun.
3. In case 3, why did a two-node cluster at each depot need a witness elsewhere, and what was done when the link to it failed?

✓ Chapter Summary

- Case 1: a consolidation sized on allocated rather than active memory failed the day a host was taken for maintenance. Quote every ratio with N+1.
- Case 2: a cloud migration cost 2.3 times its projection through lift and shift, on-demand pricing, egress and always-on development guests. The cloud made existing waste legible rather than creating it.
- Case 3: centralising 23 depots would have improved management and cost 70 hours a year of operations per site. The constraint was link availability, which nobody had measured.
- Case 4: a licence repricing was blocked not by technology but by the absence of an inventory. Lock-in is institutional, not technical.
- In all four the decision was framed as a technology choice and settled by a missing measurement.
- The question to ask of any proposal: what number would change this, and has anyone taken it?

 Review Questions

1. **(MCQ)** In case 1, the resource that failed was: (a) CPU; (b) memory under N+1; (c) storage capacity; (d) network bandwidth.
2. **(MCQ)** The largest single contributor to case 2's overrun was: (a) egress; (b) on-demand pricing; (c) lift and shift without right-sizing; (d) development guests running continuously.
3. **(Short)** Explain why the cloud “made waste legible” and what that implies for a migration business case.
4. **(Short)** State the question that decides whether an edge site should keep local compute, and apply it to a supermarket till system.
5. **(Short)** Explain what case 4 means by lock-in being institutional rather than technical.
6. **(Applied)** Work the lab's brief and submit the one-page recommendation.
7. **(Applied)** Choose any one of the four cases and write the counterfactual: what would have happened had the missing measurement been taken six months earlier? Be specific about what would have been decided differently and what it would have cost.

The Semester Project

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — *Emerging Hardware and Practice*

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Choose** a project that can be built, measured and defended in the time available.
- **State** a hypothesis your deployment will test, with the measurement that settles it.
- **Build** a virtualized service and **measure** it honestly, controlling for the five ways a virtualized benchmark lies.
- **Write** a report whose conclusions are supported by your own figures.
- **Present** and **defend** the work, including what did not work.

🔗 Before You Start

The whole book. The project is assessed on whether you can apply it, and the strongest projects are the ones that use the arithmetic of Chapter 6, Chapter 12 and Chapter 18 on something the student built themselves.

📖 Key Terms in This Chapter

hypothesis • baseline • control • confounder • repeatability • percentile • teardown • reproducibility • scope • deliverable • rubric • peer review

23.1 What the Project Is

Build a virtualized or containerized service, measure something about it that you did not know before you started, and report what you found.

Three deliverables, listed in the order they are worth marks:

- a **working deployment**, demonstrated live in Week 16;
- a **report** of 2,500–3,500 words containing your own measurements;
- a **ten-minute presentation** with five minutes of questions.

The project is set in Week 9 and due in Week 16 (Appendix G). Teams of two or three; individual work is permitted but not encouraged, because the defence is easier when somebody has already argued with you about it.

⚠️ The commonest way to lose marks is scope

The strongest projects in this course are narrow. “Measure the throughput cost of three virtio queue configurations” earns more than “build a private cloud”, because the first can be finished, measured and defended in seven weeks and the second cannot.

A project that builds a great deal and measures nothing is the single most common failure mode. The deployment is the apparatus; the measurement is the work.

23.2 Choosing One

A good project has a **hypothesis** — a statement that could turn out to be false — and a measurement that settles it. Here are examples at the right scale; your own, if it has the same shape, is better.

Project	The hypothesis it tests	Chapters
Device model throughput	Paravirtualized devices cost less host CPU per unit of work than emulated ones, and the ratio depends on request size	Chapter 7, Chapter 18
Container density ceiling	Memory, not CPU, limits container density, and the ratio to VMs depends on the workload's own footprint	Chapter 8
Live migration convergence	Migration succeeds iff the dirty rate is below the transfer rate, and auto-convergence buys success with guest performance	Chapter 17
Thin pool behaviour at exhaustion	Guest filesystems fail together, not individually, and <code>discard</code> measurably changes the time to exhaustion	Chapter 12
Overlay MTU and throughput	The cost of each of the three MTU fixes, measured, including the one that only helps TCP	Chapter 13
Isolation overhead	gVisor's cost is proportional to system-call rate, not to CPU work	Chapter 11
Right-sizing an estate	A measured estate needs materially less than it was allocated, and <code>%RDY</code> falls when vCPUs are removed	Chapter 6, Chapter 18
Cloud cost model	The three-year TCO of a real workload, on premises and in two clouds, with the assumption that dominates identified	Chapter 14

💡 The test for a hypothesis

Write your hypothesis down and then ask: *what result would prove this wrong?*

If you cannot answer, it is not a hypothesis — it is a plan to build something. “Containers are lighter than VMs” fails the test, because no measurement could contradict it. “On this workload, the container-to-VM memory density ratio is under 4:1 because the application's own footprint dominates” passes: it names a number, and the number could come out at 20:1 and refute you.

A refuted hypothesis, honestly reported, earns full marks. A confirmed one you could not have refuted earns few.

23.3 Measuring Honestly

This is where most of the marks are, and Chapter 18 is the chapter to re-read. Five controls, and your report must state how you handled each.

Threat	What it does to your result	What to do about it
Neighbours	The host is shared, so your figure includes everyone else's work	Run at several times of day; report the distribution; state what else was running
Timekeeping	A descheduled guest's own clock drifts	Time from the host where possible; use a paravirtual clock source
Caching	The host page cache serves your "disk" reads	Use files larger than host RAM, or <code>direct=1</code> ; state which
Burst credits	A short cloud benchmark runs on credits	Never use burstable instances; say which family you used
Nesting	The outer hypervisor inflates everything, variably	Measure on bare metal, or state clearly that figures are nested and comparative only

💡 Three rules that separate a good report from a weak one

Report percentiles, not means. Chapter 18 showed why: the mean sits below almost all the mass. Give p50, p95 and p99, and say how many samples.

Report the baseline. A number with nothing to compare it to is not a measurement. Every result should be *against native*, *against the default configuration*, or *against the other arm*.

Repeat, and show the spread. Three runs minimum. If your three runs differ by more than the effect you are claiming, you have not measured the effect — you have measured the noise, and saying so is a legitimate finding.

23.4 The Report

Roughly this shape. Marks follow the rubric below, not the word count.

1. **The question** (200 words). The hypothesis, and why it is worth testing. Name the chapter it comes from.
2. **Background** (400). What the reader needs from this book to follow you. Cite chapters rather than re-explaining them.
3. **Method** (600). The apparatus, in enough detail that somebody else could rebuild it. Hardware, versions, configuration. The five controls above, each addressed explicitly.
4. **Results** (700). Your figures. Tables and plots with units and sample counts. No interpretation in this section.
5. **Discussion** (800). What the figures mean. Whether the hypothesis survived. What surprised you. What you would do differently.
6. **Threats to validity** (300). What could make your conclusion wrong. Every honest report has this section and it costs no marks to write — omitting it costs several.
7. **Conclusion and further work** (200).

8. **Appendix:** your scripts, your raw data, your teardown procedure.

⚠ Negative and inconclusive results are full-mark results

If your measurement shows no difference where you expected one, report that. If your three runs disagreed and you ran out of time to find out why, report that too, with what you would do next.

What loses marks is a conclusion the figures do not support — claiming a 12% improvement from runs that varied by 30%, or reporting a mean that hides the tail. The examiners can read your appendix.

23.5 The Presentation

Ten minutes, five for questions. Expect to be asked: *how do you know?*, *what else could explain that?*, and *what would change your conclusion?*

Peer review is part of Week 16: each team reviews another's deployment against the rubric and submits half a page. It is marked on the quality of the review, not on how generous it is.

💡 Demonstrate live, and have a recording

The deployment is demonstrated running. Things fail in front of audiences, so record a working run beforehand and have it ready — using it costs nothing, and having nothing to show costs a great deal.

If something breaks live, say what you think happened. Diagnosing a failure in front of the room, out loud, using the counters from Chapter 18, is the single most convincing thing that can happen in a project presentation.

23.6 Marking

Component	Weight	What earns the marks
Deployment	25%	It works, it is reproducible from your own instructions, and it is torn down cleanly. Complexity earns nothing by itself
Measurement	30%	A real hypothesis; controls addressed; percentiles; baseline; repetition. The largest single component
Report	25%	Clear, structured, conclusions supported by the figures, honest threats-to-validity section
Presentation and defence	15%	Explained clearly; questions answered; failures discussed rather than hidden
Peer review	5%	A specific, useful, fair review of another team

💡 What distinguishes a first from a pass

A passing project builds something that works and reports what it did.

A first-class project does three further things. It states a hypothesis that could have been refuted. It controls for at least three of the five threats and says so. And it reaches a conclusion the author did not expect at the start — which is only possible if the measurement was capable of surprising them.

Notice that none of the three requires more building. They require deciding, at the outset, what you are trying to find out.

! Common Pitfalls

- **Building for six weeks and measuring in the last one.** Build for two, measure for four. The measurement is where the marks are.
- **A scope that cannot be finished.** “Build a private cloud” is not a project; one measured property of one is.
- **Measuring once.** One run is an anecdote.
- **Reporting means.** Percentiles, always.
- **No baseline.** A figure with nothing to compare it to says nothing.
- **Benchmarking in a nested guest without saying so.** State it, and treat the figures as comparative only.
- **Hiding what failed.** The failure is usually the most interesting part, and the examiners will ask.
- **Leaving cloud resources running.** Every lab in this book ends with a teardown step for a reason. A forgotten instance bills until somebody notices.

🔧 Try It Yourself: Week 9: Your Proposal

One page, submitted in Week 9, and the thing most likely to determine your final mark. Expect ninety minutes, and do it with your team.

```
# ch23_proposal.md -- one page, submitted in Week 9.
# Fill in every section. If you cannot fill one in, that is the
# section to work on, not the one to delete.

# --- 1. The hypothesis -----
# One sentence, containing a number or a comparison, that could turn
# out to be FALSE.
#   Bad : "Containers are more efficient than virtual machines."
#   Good: "On a 256 MB Flask service, container-to-VM memory density
#         is below 4:1, because the application footprint dominates
#         the per-instance overhead."

# --- 2. What would refute it -----
# State the result that would prove you wrong. If you cannot, go
# back to 1.

# --- 3. The apparatus -----
# Hardware you will use. Hypervisor and versions. How many guests.
# What you will install. Say explicitly whether it is nested.

# --- 4. The measurement -----
# What exactly you will measure, with which tool, how many times,
# and which percentiles you will report.
#   metric:      ...
#   tool:        ...
#   repetitions: at least 3
#   report:      p50, p95, p99 and the spread

# --- 5. The controls -----
# One line each on the five threats of section 23.3:
#   neighbours:  ...
#   timekeeping: ...
#   caching:     ...
#   burst credits: ...
#   nesting:     ...

# --- 6. The baseline -----
# What are you comparing against? Native? Default configuration?
```

```
# The other arm?

# --- 7. Weekly plan -----
# wk 10 proposal agreed; apparatus specified
# wk 11 deployment built; first measurement attempted
# wk 12 measurement working; baseline captured <-- gate
# wk 13 full runs; experiments FROZEN at the end of this week
# wk 14 analysis; report drafted
# wk 15 report finished; presentation rehearsed
# wk 16 demonstrate, present, peer review
# The week 13 freeze is not a suggestion. Teams that are still
# changing the apparatus in week 15 have no time to analyse it.

# --- 8. What could go wrong -----
# Name the two most likely failures and what you will do instead.
# "The GPU is unavailable" -> "use the captured output in code/".

# --- 9. Teardown -----
# How every resource you create will be destroyed, and when.
```

What to notice. Sections 1, 2 and 5 are the ones that decide the mark, and they are the ones with no building in them. A proposal that is strong on 3 and weak on 1 describes a deployment, not a project.

🔍 Checkpoint — Test Yourself

1. Rewrite this as a testable hypothesis: “We will investigate whether Kubernetes is good for our department.”
2. A team reports a 9% throughput improvement from a configuration change. Their three runs gave 7%, 24% and −2%. What should the report conclude, and what should they do next?
3. Your project needs a GPU and none is available in Week 12. Using the proposal’s section 8, write what you would do instead, without changing the hypothesis.

📋 Chapter Summary

- Build a service, measure something you did not know, report it. The measurement is 30% of the mark and the deployment 25%.
- A hypothesis is a statement that could be refuted. If no result would prove you wrong, you have a build plan, not a project.
- Narrow scope beats ambition. Build for two weeks, measure for four.
- Control for neighbours, timekeeping, caching, burst credits and nesting, and say in the report how you did.
- Percentiles, a baseline, and at least three repetitions. If the runs vary more than the effect, you measured noise — and saying so is a result.
- Negative and inconclusive results earn full marks. Conclusions the figures do not support do not.
- Freeze the apparatus in Week 13. Teams still changing it in Week 15 have no time to analyse anything.

🔧 Review Questions

1. (MCQ) The largest single component of the project mark is: (a) the deployment; (b) the measurement; (c) the report; (d) the presentation.

2. **(MCQ)** A hypothesis is testable if: (a) it is about a popular technology; (b) some possible result would prove it false; (c) it can be built in seven weeks; (d) it cites three chapters.
3. **(Short)** State the five threats to a virtualized measurement and one control for each.
4. **(Short)** Explain why a refuted hypothesis can earn full marks and a confirmed one may not.
5. **(Short)** Why is the Week 13 experiment freeze in the plan?
6. **(Applied)** Take any one project from the table in Section 23.2 and write its proposal sections 1, 2, 4 and 6 in full.
7. **(Applied)** You are peer-reviewing a team whose report claims containers start 300 times faster than virtual machines, from one measurement of each, taken on a shared laboratory machine during a lecture. Write the half-page review: what is sound, what is not, and the two changes that would most improve it.

Appendices

The Laboratory Toolkit

Everything the labs in this book need, installed and verified in one sitting. Work through it once in Week 1; every later chapter assumes it.

A.1 What You Need

One machine, with:

- a 64-bit x86 processor with VT-x or AMD-V *enabled in firmware*;
- 16 GB of memory — 8 GB works for most labs and not for Chapter 6’s or Chapter 15’s;
- 100 GB of free disk;
- Linux, or Windows with WSL2.

Where a lab needs more — nested virtualization, two hosts, a GPU — it says so in its first line and ships captured output in code/.

! Check this before anything else

A large share of “the hypervisor will not start” is a cleared firmware bit.

```
# Should print a number greater than zero. Zero means the feature is
# absent or disabled in BIOS/UEFI -- look for "Intel VT-x", "AMD-V",
# "SVM Mode" or "Virtualization Technology" and enable it.
grep -c -E '\(<vmx|svm\)>' /proc/cpuinfo
```

On Windows, Task Manager’s Performance tab reports “Virtualization: Enabled”.

A.2 The Stack, in One Pass

```
#!/usr/bin/env bash
# appA_setup.sh -- install and verify everything the labs need.
set -euo pipefail

# --- 1. Hypervisor: KVM, libvirt and the tools around them -----
sudo apt-get update
sudo apt-get install -y \
  qemu-kvm libvirt-daemon-system libvirt-clients virtinst \
  libguestfs-tools cloud-image-utils virt-manager bridge-utils \
  qemu-utils ovmf
sudo usermod -aG libvirt,kvm "$USER"      # log out and in for this
sudo systemctl enable --now libvirtd
sudo virsh net-start default 2>/dev/null || true
sudo virsh net-autostart default

# --- 2. Verify the hypervisor before going further -----
sudo virt-host-validate qemu              # every line should be PASS or WARN
sudo virsh list --all
sudo virsh net-list

# --- 3. Nested virtualization: Chapters 4, 11 and 17 need it -----
MOD=$(lsmod | grep -o -m1 -E 'kvm_intel|kvm_amd')
cat "/sys/module/$MOD/parameters/nested"
# If that prints N or 0:
echo "options $MOD nested=1" | sudo tee "/etc/modprobe.d/$MOD.conf"
sudo modprobe -r "$MOD" && sudo modprobe "$MOD"
```

```

cat "/sys/module/$MOD/parameters/nested"      # now Y or 1

# --- 4. Containers: Docker and kind -----
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"
sudo systemctl enable --now docker
curl -fsSL -o /tmp/kind \
    https://kind.sigs.k8s.io/dl/latest/kind-linux-amd64
sudo install -m 755 /tmp/kind /usr/local/bin/kind
curl -fsSL -o /tmp/kubect1 \
    "https://dl.k8s.io/release/$(curl -fsSL \
        https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1"
sudo install -m 755 /tmp/kubect1 /usr/local/bin/kubect1

# --- 5. Measurement tools, used from Chapter 2 onwards -----
sudo apt-get install -y \
    sysstat fio iperf3 stress-ng htop iotop htop \
    linux-tools-common "linux-tools-$(uname -r)" \
    tcpdump iproute2 bridge-utils net-tools ethtool

# --- 6. Infrastructure as code: Chapter 16 -----
curl -fsSL https://apt.releases.hashicorp.com/gpg \
    | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg] \
    https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
    | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt-get update && sudo apt-get install -y terraform packer

# --- 7. A base image, used by nearly every lab -----
BASE=https://cloud-images.ubuntu.com/jammy/current
wget -nc -O ~/ubuntu.img "$BASE/jammy-server-cloudimg-amd64.img"
cat > ~/user-data <<'YAML'
#cloud-config
password: vss2026
chpasswd: { expire: False }
ssh_pwauth: True
YAML
cloud-localds ~/seed.iso ~/user-data

# --- 8. Build the guest the labs call vss1 -----
sudo cp ~/ubuntu.img /var/lib/libvirt/images/vss1.qcow2
sudo qemu-img resize /var/lib/libvirt/images/vss1.qcow2 10G
sudo virt-install --name vss1 --memory 2048 --vcpus 2 \
    --disk /var/lib/libvirt/images/vss1.qcow2,format=qcow2 \
    --disk ~/seed.iso,device=cdrom --os-variant ubuntu22.04 \
    --import --graphics none --noautoconsole

# --- 9. Verify end to end -----
sleep 30
sudo virsh list
IP=$(sudo virsh domifaddr vss1 | awk '/ipv4/{split($4,a,"/"); print a[1]}')
echo "guest address: $IP"
ssh -o StrictHostKeyChecking=no "ubuntu@$IP" 'uname -a; systemd-detect-virt'
echo "toolkit ready"

```

A.3 Windows Hosts

Windows Pro and above have Hyper-V, which is a genuine Type-1 hypervisor (Chapter 5) and is sufficient for most labs.

- Enable Hyper-V and WSL2 from “Turn Windows features on or off”.

- Install a Linux distribution under WSL2 and run the script above inside it — nested KVM under WSL2 works for the container and Kubernetes labs, and for some of the hypervisor ones.
- For the virtual-machine labs proper, use Hyper-V Manager directly, or VirtualBox *with Hyper-V disabled*.

⚠ Hyper-V and VirtualBox on one machine

As Chapter 5 explains, enabling Hyper-V — which Docker Desktop, WSL2, Windows Sandbox and memory integrity all do — makes Windows a guest, so VirtualBox and VMware Workstation become nested and much slower.

Choose one. To turn Hyper-V off:

```
# In an elevated PowerShell. Reboot afterwards.
bcdedit /set hypervisorlaunchtype off
# To turn it back on:
bcdedit /set hypervisorlaunchtype auto
```

Benchmarks taken without checking this are measuring a nested guest.

A.4 Cloud Accounts

Chapter 14's lab uses three free tiers. Create the accounts in Week 1; card verification takes time and the free tiers have validity windows.

	Free allowance	CLI	Watch for
AWS	12 months of a small instance, monthly hours capped	<code>aws</code>	Elastic IPs and EBS volumes bill after the instance is gone
Azure	Credit for 30 days, then limited always-free services	<code>az</code>	Delete the whole resource group, not the VM
GCP	Credit for 90 days, plus a small always-free instance	<code>gcloud</code>	Disks and reserved addresses persist after deletion

⚠ Set a budget alert before you launch anything

Every lab in this book ends with a teardown step. Do them. Then add a hard safety net, because a forgotten instance bills until somebody notices.

```
# AWS: alert at $5 of forecast spend
aws budgets create-budget --account-id "$(aws sts get-caller-identity \
  --query Account --output text)" --budget \
  '{"BudgetName":"vss-lab","BudgetLimit":{"Amount":"5","Unit":"USD"},\
    "TimeUnit":"MONTHLY","BudgetType":"COST"}'
# Azure and GCP have equivalents in the portal under Cost Management.
```

A.5 When a Lab Cannot Run Here

Three labs need hardware not every reader has. Each ships captured output in code/, and the exercise — the analysis — is unchanged.

Lab	Needs	Substitute
Chapter 4	nested virtualization	Section A.2 step 3 enables it on most laptops; where firmware forbids it, the exit counts in <code>code/ch04_exits.txt</code>
Chapter 17	two hosts and shared storage	Two nested guests acting as hosts, with NFS between them; the setup script is <code>code/appA_twohost.sh</code>
Chapter 20	an NVIDIA GPU with MIG	<code>code/ch20_capture.txt</code> — real <code>nvidia-smi</code> output from a partitioned card
Chapter 21	an SEV-SNP or TDX processor	<code>code/ch21_report.txt</code> and <code>code/ch21_expected_measurement.txt</code>

A.6 Tearing It All Down

At the end of the semester:

```
# --- 1. Guests and their storage -----
for d in $(sudo virsh list --all --name); do
    sudo virsh destroy "$d" 2>/dev/null || true
    sudo virsh undefine "$d" --remove-all-storage --snapshots-metadata || true
done

# --- 2. Containers, images and clusters -----
kind delete clusters --all 2>/dev/null || true
docker system prune -af --volumes

# --- 3. Cloud. Check, do not assume. -----
aws ec2 describe-instances --filters 'Name=instance-state-name,Values=running' \
    --query 'Reservations[].Instances[].InstanceId' --output text
az group list --query '[] .name' -o tsv
gcloud compute instances list --format='value(name,zone)'
# All three should print nothing. If they do not, delete what they list.
```

Command Reference

Organised by task rather than by tool, because that is how the question arrives. Keep this open beside the labs.

B.1 Is This Machine Capable?

```
grep -c -E '\<(vmx|svm)\>' /proc/cpuinfo # VT-x / AMD-V present
grep -o -m1 -E '\<(ept|npt)\>' /proc/cpuinfo # two-dimensional paging
ls /sys/kernel/iommu_groups | wc -l # IOMMU on (Chapter 4)
cat /sys/module/kvm_intel/parameters/nested # nesting (or kvm_amd)
sudo virt-host-validate qemu # everything, with verdicts
systemd-detect-virt # am I already a guest?
sudo dmidecode -s system-manufacturer # which hypervisor, if so
lscpu | grep -E 'Socket|Core|NUMA|Thread' # topology (Chapter 6)
```

B.2 Virtual Machines: virsh

```
# --- lifecycle -----
virsh list --all # every guest and its state
virsh start NAME # power on
virsh shutdown NAME # ask the guest to stop (ACPI)
virsh destroy NAME # pull the power. Not the same thing
virsh undefine NAME --remove-all-storage --snapshots-metadata
virsh console NAME # serial console; Ctrl-] to leave
virsh autostart NAME # start with the host

# --- inspecting -----
virsh dominfo NAME # the allocation
virsh dumpxml NAME # the whole definition
virsh domstats NAME --vcpu --balloon --block --interface
virsh dommemstat NAME # balloon, swap_in, swap_out (Ch 6)
virsh domdirtyrate-calc NAME --seconds 10 # then domstats --dirtyrate
virsh domblklist NAME # which files back which disks

# --- changing -----
virsh setvcpus NAME 4 --config --maximum
virsh setvcpus NAME 4 --config
virsh setmaxmem NAME 8G --config
virsh setmem NAME 8G --config
virt-xml NAME --edit --network model=virtio # Chapter 7
virt-xml NAME --edit target=vda --disk bus=virtio

# --- snapshots (Chapter 12) -----
virsh snapshot-create-as NAME tag "why I took it"
virsh snapshot-list NAME
virsh snapshot-revert NAME tag --running
virsh snapshot-delete NAME tag

# --- migration (Chapter 17) -----
virsh migrate --live --verbose --persistent --undefinesource \
NAME qemu+ssh://DEST/system
virsh migrate --live --auto-converge NAME qemu+ssh://DEST/system
virsh migrate-setspeed NAME 1000 # MiB/s cap
virsh domjobinfo NAME # progress of a running migration
```

B.3 Virtual Disks: qemu-img

```
qemu-img create -f qcow2 disk.qcow2 20G
qemu-img create -f qcow2 -b base.qcow2 -F qcow2 child.qcow2  # Ch 12
qemu-img info disk.qcow2
qemu-img info --backing-chain disk.qcow2  # the whole chain
qemu-img convert -O raw disk.qcow2 disk.raw  # also vmdk, vhdx, vpc
qemu-img resize disk.qcow2 +10G
qemu-img check disk.qcow2
qemu-img snapshot -l disk.qcow2  # internal snapshots
du -h disk.qcow2 ; ls -lh disk.qcow2  # actual vs apparent size
```

B.4 Containers: docker

```
# --- images (Chapter 9) -----
docker build -t name:tag .
docker history name:tag --format '{{.Size}}\t{{.CreatedBy}}'
docker image inspect name:tag --format '{{json .RootFS.Layers}}'
docker save name:tag | tar -t | head  # the layers, as files
docker pull repo@sha256:...  # pin a digest, not a tag

# --- running -----
docker run --rm -it image sh
docker run --init image  # a real PID 1 (Chapter 8)
docker stats --no-stream
docker inspect -f '{{.HostConfig.Privileged}}' CONTAINER

# --- hardening (Chapter 19) -----
docker run --user 10001:10001 --cap-drop=ALL \
  --security-opt=no-new-privileges --read-only --tmpfs /tmp \
  --pids-limit 100 --memory 256m --cpus 0.5 image

# --- the kernel primitives underneath (Chapter 8) -----
ls -l /proc/self/ns/  # this process's namespaces
unshare --pid --mount --uts --ipc --net --fork --mount-proc CMD
systemd-cgls  # the cgroup tree
cat /sys/fs/cgroup/CGROUP/cpu.stat  # throttled_usec
cat /sys/fs/cgroup/CGROUP/memory.max
```

B.5 Kubernetes: kubectl

```
kubectl get pods -o wide --all-namespaces
kubectl describe pod NAME | sed -n '/Events/, $p'  # why it will not run
kubectl get events --sort-by=.lastTimestamp | tail -20
kubectl top nodes ; kubectl top pods  # needs metrics-server
kubectl get pod NAME -o custom-columns=\
  NAME:.metadata.name,QOS:.status.qosClass  # Chapter 10
kubectl describe node NAME | sed -n '/Allocated/, $p'  # requests vs capacity
kubectl set resources deployment NAME --requests=cpu=500m,memory=2Gi
kubectl rollout status deployment NAME
kubectl rollout undo deployment NAME
kubectl logs -f POD --previous  # the run that crashed
```

B.6 Networking

```
# --- namespaces and virtual interfaces (Chapter 13) -----
ip netns add NAME ; ip netns exec NAME CMD
ip link add veth0 type veth peer name veth1
ip link add br0 type bridge ; ip link set veth0 master br0
bridge fdb show br0          # the switch's MAC table
bridge link show

# --- overlays -----
ip link add vx0 type vxlan id 100 remote IP local IP dstport 4789 dev eth0
ip -d link show vx0          # confirm the VNI
ip link set vx0 mtu 1450     # the arithmetic of Chapter 13

# --- diagnosis -----
ping -M do -s 1472 HOST      # find the real path MTU
tracpath HOST                # reports the MTU at each hop
tcpdump -ni IFACE udp port 4789 # watch the encapsulation
ethtool -k IFACE | grep -i offload # is VXLAN offloaded?
ss -tuln                     # what is listening
tc qdisc add dev IFACE root netem delay 150ms # simulate latency
tc qdisc del dev IFACE root
```

B.7 Storage

```
# --- thin pools (Chapter 12) -----
pvccreate /dev/sdX ; vgcreate vg /dev/sdX
lvcreate -L 100G --thinpool pool vg
lvcreate -V 50G --thin -n vol vg/pool
lvs vg -o lv_name,lv_size,data_percent,metadata_percent
fstrim -v /mountpoint        # return freed blocks to the pool

# --- measurement -----
iostat -x 2                  # aqu-sz and await are the ones
fio --name=t --rw=randread --bs=4k --size=1G --direct=1 \
    --runtime=30 --time_based --output-format=json
dd if=/dev/zero of=/tmp/t bs=1M count=2048 oflag=direct
lsblk -t                     # queue depths and scheduler
```

B.8 Measurement

```
# --- from inside a guest (Chapter 18) -----
mpstat 2 5                   # %steal is ready time, from inside
vmstat 2 5                   # si/so are swap; r is run queue
top                           # %st in the CPU line

# --- from the host -----
perf stat -e 'kvm:kvm_exit' -p PID -- sleep 10 # VM exits (Ch 4)
perf record -e 'kvm:kvm_exit' -p PID -- sleep 10 ; perf script
virsh domstats --vcpu        # wait and delay per vCPU
sar -u -r -b 2 5             # cpu, memory, block, together

# --- generating load -----
stress-ng --cpu 4 --timeout 60s --metrics-brief
stress-ng --vm 2 --vm-bytes 80% --vm-keep --timeout 60s
iperf3 -s -D ; iperf3 -c HOST -t 20 -P 4
```

B.9 Shell Idioms Used in the Labs

If any of these are unfamiliar, this is the section the prerequisite self-check was pointing at.

Idiom	What it does
<code>set -euo pipefail</code>	Stop on the first error, on an unset variable, and on a failure anywhere in a pipeline. Every lab script begins with it
<code>VAR=\$(command)</code>	Capture a command's output into a variable
<code>cmd awk '/pat/{print \$2}'</code>	Print the second field of lines matching a pattern — how the labs extract an address
<code>cmd true</code>	Do not let a failure here stop the script; used around teardown
<code>name () { ... }</code>	Define a function. The labs use these to keep lines under 76 columns
<code>cmd «'EOF' ... EOF</code>	A here-document. Quoted 'EOF' means nothing inside is expanded — important for YAML and for anything containing \$
<code>& then wait</code>	Run in the background, then wait for all background jobs. Used to make several guests busy at once
<code>for x in \$(seq 1 4); do ... done</code>	Repeat with a counter

Answers to the Checkpoints

Three answers per chapter, in order. Attempt the checkpoint from memory before reading these, and argue with them where you disagree — several are judgement calls rather than facts, and they say so.

Chapter 1 — What Virtualization Is

1. Active memory is $24 \times 19 = 456$ GB. With 30% headroom, $456 \times 1.3 = 593$ GB, so $\lceil 593/384 \rceil = 2$ hosts, plus one for $N+1 = 3$ hosts. The consolidation ratio is $24/3 = 8 : 1$. Note what was *not* used: the 128 GB installed per server. Sizing on that would have given $24 \times 128 \times 1.3/384 = 11$ hosts for the same work.
2. The second half is wrong. There *is* a virtualization layer — namespaces and control groups — it is simply inserted inside the kernel rather than beneath it. The correct statement is that containers start faster and cost less memory because they share the host's kernel instead of each running their own, not because no layer exists.
3. **Encapsulation.** A running machine becomes a file, so it can be snapshotted, copied, versioned and restored. The operation with no physical equivalent is the snapshot: you can return a machine to its state four minutes ago in a few seconds. A physical server has no way to record its entire state — memory, disk and devices — as something that can be put back.

Chapter 2 — Virtualization Architectures

1. By Goldberg's test it is **Type-1**: the hypervisor reaches the hardware without another operating system's permission, and the stripped Linux exists only to boot it. The more useful questions are *how large is the trusted computing base?* — here, a kernel plus every driver it loads — and *what else runs on this machine?* Those decide the security and performance arguments; the Roman numeral decides neither.
2. CPU overhead is $(412 - 400)/400 = 3\%$, which is normal. Disk throughput is $1 - 210/1050 = 80\%$ **lost**, which is not. The pattern — small CPU penalty, enormous I/O penalty — is the signature of an **emulated disk controller**. Change the disk bus to virtio; expect the loss to fall to a few per cent.
3. Paravirtualization requires modifying the guest kernel, which requires its source and the ability to rebuild it. With the vendor gone, neither exists. Use **hardware-assisted virtualization**, which needs no guest changes at all, and install paravirtualized *drivers* if any exist for that operating system — a far weaker requirement than recompiling the kernel.

Chapter 3 — The Virtual Machine Monitor

1. RDCFG is **behaviour-sensitive**: its result depends on the privilege configuration, which the monitor must control. It is **not privileged**: it does not fault in user mode. So there is a sensitive instruction outside the privileged set, the condition $sensitive \subseteq privileged$ fails, and **the architecture is not virtualizable by trap-and-emulate alone**. A guest would read the deprivileged level and could detect, or misbehave in, virtualization.
2. Because a crash is a *signal*. The monitor author would learn immediately, at the offending instruction, with a stack trace. What x86 does instead is accept the instruction, update the arithmetic flags and silently discard the write to IF: the guest believes it disabled interrupts

and did not. The resulting corruption is intermittent, unreproducible and traceable to nothing. Silent incorrect behaviour is far harder to engineer around than a loud failure.

3. When you control the guest and want the lowest overhead — for example a cloud provider shipping its own Linux images, or an appliance vendor building both halves. A hypercall costs tens of cycles where a trap costs hundreds, the monitor is far simpler, and the guest's source is available by construction. The requirement that makes paravirtualization unusable in general is precisely the one that does not bind here.

Chapter 4 — Hardware Support for Virtualization

1. Backward compatibility. Making POPF and the others privileged would change the behaviour of existing, correct user programs that rely on the write to IF being silently ignored — they would begin faulting. x86's commercial value rests on not breaking old software, so both vendors added a new processor *mode* in which existing instructions behave as before and the *hardware* decides when to hand control to the hypervisor.
2. For each of the 5 guest levels: 5 EPT accesses to translate that level's address, plus 1 read of the guest entry = 6, so $5 \times 6 = 30$; plus 5 to translate the final guest-physical address: **35** memory accesses. In general, for n levels: $n(n + 1) + n = \mathbf{n^2 + 2n}$. Checking against the chapter, $n = 4$ gives $16 + 8 = 24$.
3. **Device passthrough** is lost — assigning a physical device directly to a guest, including SR-IOV virtual functions. It must be refused rather than warned about because a device performs DMA to physical addresses: without an IOMMU to translate and confine them, the guest's driver can direct the device to write anywhere in host memory, including the hypervisor and other guests. That is not a degraded configuration; it is the complete loss of the resource-control property of Chapter 3.

Chapter 5 — Major Hypervisor Platforms

1. **ESXi**: in the VMkernel itself — a small, purpose-built trusted base of about 150 MB, at the cost of a short hardware compatibility list. **Hyper-V**: in the parent partition, a full Windows installation — thin hypervisor, large privileged guest. **KVM**: in the Linux kernel — every driver Linux has, and all of them in the trusted base. **Xen**: in dom0, or in a separate driver domain — a one-megabyte hypervisor, and the only design that can put a driver's blast radius below the whole host.
2. Docker Desktop enables Hyper-V (or the WSL2 platform, which uses it). That slides the Hyper-V hypervisor underneath the running Windows, demoting it to a parent partition. VirtualBox then no longer runs on the hardware; it runs as a *nested* guest above Hyper-V, and nested virtualization multiplies every exit the inner guest takes. Nothing about VirtualBox changed — the machine underneath it did.
3. Using the chapter's figures, vSphere cost about \$200,000 more over three years for three hosts, which is roughly 1.5 full-time engineers. A licence becomes defensible once the estate is large enough that vCenter genuinely saves that much effort — in practice somewhere in the low hundreds of hosts, or sooner where availability has contractual penalties. **The assumption doing the most work** is that a self-managed KVM estate would need those 1.5 engineers; if the team already runs Linux at scale and has the automation, the saving is much smaller and the threshold much higher.

Chapter 6 — CPU and Memory Virtualization

1. $\sigma_{\text{cpu}} = (20 \times 6)/48 = 120/48 = \mathbf{2.5}$. A mean %RDY of 14% is well above the 10% action threshold, so the host is oversubscribed. **Measure first:** the actual vCPU utilisation of each guest — 6 vCPUs is a large allocation and is very likely more than most of them use. **Then act:** right-size downward before buying anything. Halving the allocations would bring σ to 1.25 and would almost certainly resolve the ready time without new hardware.
2. Because the hypervisor cannot tell which pages matter. From outside, every guest page looks alike; it cannot distinguish a hot database buffer from a page of cache the guest would discard without noticing, so taking pages itself means guessing, and a wrong guess costs the guest a disk read. The guest kernel has a page-replacement algorithm that is good at exactly this question. The balloon driver asks in the one language the guest understands — it allocates memory — and the guest decides what to give up.
3. **The guest is wider than a NUMA node.** It has 40 vCPUs against 32 cores per socket and 400 GB against 256 GB per socket, so it must span both sockets and roughly half its memory accesses cross the interconnect at 1.5–2× the latency. No hypervisor setting fixes it because the memory is physically attached to a socket; locality cannot be created after the fact. The change that would work is **resizing the guest to fit inside one node** — 32 vCPU and 256 GB or less. Failing that, enable vNUMA so the guest's own scheduler can at least keep each thread's memory on its own node.

Chapter 7 — Paravirtualization and Device Virtualization

1. $1.6/0.2 = \mathbf{8.0}$ host cores per Mpps, roughly twenty-five times what vhost costs. That figure, together with the very low packet rate, is the signature of an **emulated network adapter** — an e1000 or similar, where every device register access is a VM exit. Change the interface model to **virtio**, and enable vhost; expect the rate to rise by an order of magnitude and the host CPU to fall.
2. Because the exit is caused by the *doorbell*, not by the request. A guest places buffers in the shared virtqueue with ordinary memory writes, which cost nothing, and rings the doorbell once to say work is waiting — and one doorbell can cover a hundred queued requests. Under load the back-end is already polling the ring and finds new work without being told, so the guest can suppress notification entirely. The mechanism is **batching with notification suppression**, and it makes the exits-per-request ratio fall as the rate rises.
3. **The conflict:** SR-IOV virtual functions are assigned devices, so those twelve guests cannot be live-migrated, and live migration is how a host is patched without downtime. Two resolutions: (i) *bonded failover* — give each guest a VF and a virtio interface in a bond, detach the VF before migrating and reattach it afterwards, accepting reduced throughput during the move; (ii) *scheduled downtime for those twelve guests only*, documented as an exception at design time rather than discovered at the first patch window.

Chapter 8 — From Virtual Machines to Containers

1. 137 is 128+9: the process was terminated by signal 9, SIGKILL, which here means the **memory limit** and the kernel's OOM killer. The symptom is a kill rather than a slowdown because memory is not divisible in time: a page is either available or it is not, so there is nothing to degrade. A CPU limit, by contrast, throttles — the container simply runs more slowly and nothing fails.
2. A container shares the host's kernel, so there is no guest kernel to load a module into; loading one would mean loading it into the *host*, which is both a privilege escalation and shared with

every other container. The two options are (i) load the module on the host and expose the resulting device to the container, accepting that the module is now host-wide, or (ii) use a **virtual machine**, or a microVM from Chapter 11, which has a kernel of its own.

3. The image is stored once and shared: $400 + 3 \times 50 = \mathbf{550}$ MB. The logs are in each container's writable layer, so they count three times and are lost when the containers are replaced. The change is to **write logs to stdout** and let the runtime collect them, or mount a volume — either removes them from the copy-on-write layer entirely.

Chapter 9 — Docker and the Container Ecosystem

1. **node:20** is a *tag*, which is a mutable pointer. The publisher republished it during the week — a patch release, a new base layer — so Friday's build used different bytes. The change that prevents it is to **pin the digest**: `FROM node:20@sha256:...`, which identifies exact content for ever. Pair it with an automated job that proposes digest updates as reviewable changes, so security fixes are still applied deliberately.
2. **Same RUN: no.** Only the layer's final state is committed, and the key was deleted before the layer closed. **Separate RUN: yes.** The first layer contains the key, the second records its deletion, and both layers ship. Anyone who pulls the image can extract the earlier layer and read it. This is why secrets belong in build secrets or a mount, never in a layer at all.
3. Stating assumptions: say the dependency layer is 1.0 GB, the application layer 20 MB, and there are 20 builds a day. *Wrong order*: each build rebuilds and pushes about 1.02 GB, and 40 nodes pull it: $20 \times 1.02 + 40 \times 20 \times 1.02 \approx \mathbf{836}$ GB a day. *Right order*: only the 20 MB layer changes: $20 \times 0.02 + 40 \times 20 \times 0.02 = \mathbf{16.4}$ GB a day. A saving of roughly **820 GB a day** in transfer alone, before counting the build time.

Chapter 10 — Container Orchestration with Kubernetes

1. Cluster totals are $6 \times 12 = 72$ cores and $6 \times 48 = 288$ GB; after existing requests, 32 cores and 98 GB remain. In aggregate the new Deployment's 15 cores and 90 GB would fit — but Pods are placed on individual nodes. Each node has $12 - 40/6 = 5.3$ cores and $48 - 190/6 = 16.3$ GB free, giving $\min(\lfloor 5.3/1 \rfloor, \lfloor 16.3/6 \rfloor) = 2$ Pods per node. So $2 \times 6 = \mathbf{12}$ schedule and 3 stay **Pending**. The binding resource is **memory**.
2. **BestEffort**, which is evicted first whenever a node comes under memory pressure. The symptom is that the workloads run perfectly for weeks and then are killed — apparently at random, but in fact *only when the cluster is busy*, which is exactly when they are most needed. Declaring nothing is not neutral; it is choosing to be first in the eviction queue.
3. A liveness probe *kills* the container when it fails. If the probe checks a shared database and the database slows, every replica fails the probe at the same moment, so Kubernetes kills them all. They restart, reconnect simultaneously in a thundering herd, overload the already-slow database and fail again. Use a **readiness** probe: it removes the Pod from the Service's endpoints without killing it, so the replicas stop taking traffic, the database recovers, and they return.

Chapter 11 — Lightweight and Secure Virtualization

1. Removed: **firmware initialisation**, **PCI bus enumeration** and **legacy device emulation**; the kernel is loaded directly into memory rather than booted through a bootloader. What the guest must be willing to give up is **looking like a PC** — it cannot expect a BIOS, a PCI bus or any legacy device, so it must be built to use virtio and to be entered directly. The cost was compatibility, not virtualization.

2. **System-call rate.** gVisor handles system calls in a user-space Go program instead of in the kernel, so a workload making many small calls — lots of short reads and writes, many file operations — pays heavily, while a CPU-bound workload that rarely calls the kernel is barely affected. Confirm by measuring the calls per second of each, with `strace -c` or `perf`, and correlating that against the measured slowdown.
3. Run the 200 internal services on the default runtime (runc) at full density, and the customer build jobs on a hardware-isolated runtime — Kata, or a microVM-backed equivalent. The manifests are otherwise identical. The single field that expresses it is `runtimeClassName` in the Pod spec.

Chapter 12 — Storage Virtualization

1. Provisioned = $120 \times 800 = 96$ TB, so $\sigma = 96/40 = \mathbf{2.4}$. Written = $120 \times 240 = 28.8$ TB, so headroom = $40 - 28.8 = \mathbf{11.2}$ TB. Growth is $120 \times 20 = 2.4$ TB a month, giving $11.2/2.4 = \mathbf{4.7}$ months to exhaustion. Alert at **80% of the pool** — 32 TB used, 8 TB free — because that is reached in about six weeks, which is the last point at which there is still time to order capacity or reclaim space. Alerting at 95% leaves no time to do anything.
2. Deleting a file inside a guest only marks blocks free in the *guest's* filesystem. The pool underneath was never told, so it still holds those blocks as written. The setting is **discard / TRIM, enabled end to end** — in the guest mount options or a periodic `fstrim`, in the virtual disk controller, and in the storage layer. On an estate that has never had it, enabling it commonly returns 10–20% of the pool.
3. **Slow** because reads are amplified: a block must be looked for in each overlay in turn until it is found, so twelve layers can mean twelve lookups where one would do, and a first write to a block must copy it up from wherever it currently lives. **Dangerous** because every file in the chain is load-bearing: lose or corrupt any one of them and every snapshot above it is unreadable — and after six months nobody knows which snapshots are still needed.

Chapter 13 — Network Virtualization and SDN

1. $14+40+8+(8+16) = \mathbf{86}$ bytes of overhead, so the largest inner frame is $1500 - 86 = \mathbf{1414}$ bytes. Note that IPv6 and Geneve options together cost 36 bytes more than the VXLAN-over-IPv4 case in the chapter — the arithmetic must be redone for each encapsulation, never assumed at 50.
2. The oversized packet is dropped and an ICMP “fragmentation needed” message is returned to tell the sender to use a smaller size. Path MTU discovery depends on that message arriving — and ICMP is very commonly filtered, so the sender never learns, retransmits the same segment and stalls for ever. The handshake succeeded because its packets were small. The firewall change is to **permit ICMP type 3 code 4** (destination unreachable, fragmentation needed), which turns a silent hang into a connection that corrects itself.
3. A VLAN is a tag within a single layer-2 domain: it does not survive being routed, so it cannot span three buildings on different IP subnets. It is also limited to 4094 identifiers. Use an **overlay** — VXLAN or Geneve — which puts the tenant's Ethernet frame inside an ordinary IP packet, so it crosses any routed network, and whose 24-bit identifier gives about 16.7 million isolated networks.

Chapter 14 — Cloud Service and Deployment Models

1. It satisfies on-demand self-service (departments provision themselves), broad network access, resource pooling and measured service (charged per guest-month). It fails **rapid elasticity** — specifically the ability to *shrink*. The hosts are bought; a department returning a guest frees a little electricity and nothing else, and the university cannot rent the spare capacity to anyone. It is a well-run virtualization estate with a cloud-like interface, which is valuable, but it does not have the economics.
2. $10 \times 5 \times 52 = 2,600$ hours a year at \$0.2016 is **\$524** a year. A three-year reservation discounts the rate by 55% but commits you to paying continuously: $0.45 \times 0.2016 \times 8,760 = \795 a year. The reservation would therefore cost **more than paying on demand**, because the workload uses only $2,600/8,760 = 30\%$ of the hours it would be committing to. Reservations pay for steady load, not for intermittent load at any discount.
3. **The customer's side** — under IaaS and under every provider's shared responsibility model, data classification and access configuration belong to the customer. The provider's default for object storage is private; somebody changed it. To stop it recurring: enable account-level public-access blocking so it cannot be set at all, scan for public buckets continuously, and require infrastructure as code (Chapter 16) so that a permission change is a reviewed commit rather than a click.

Chapter 15 — Desktop and Application Virtualization

1. $70 + 300 \times 3 = 970$ GB. Full clones would be $300 \times 70 = 21,000$ GB, so the saving is $21,000/970 \approx 21.6 : 1$. The base image is read by all 300 desktops, so it must sit on the fastest storage available and stay in cache — the read amplification of Chapter 12 applies at full force.
2. Steady state is $300 \times 10 = 3,000$ IOPS. For the storm: 150 users over 4 minutes with a 90-second boot gives $150 \times (1.5/4) \approx 56$ booting at any instant, so $56 \times 120 = 6,750$ IOPS, plus the 150 already-logged-in users' steady 1,500: about **8,250** IOPS, roughly **2.75 times** the all-day figure — for four minutes. Size for this, not for the average.
3. **Latency, not bandwidth.** Every keystroke makes a round trip before its character appears, so a high-latency path feels broken no matter how much capacity is spare; under about 50 ms it is invisible, past 150 ms typing feels detached. Measure the round-trip time from the client to the session host, and the protocol's own reported frame latency — not throughput, which the monitoring has already told you is fine.

Chapter 16 — Virtualization Management Platforms

1. Now: $3 \times 50 + 8 \times 25 = 350$ min a month = 70 hours a year. After: $3 \times 5 + 8 \times 3 = 39$ min a month = 7.8 hours a year, plus a 32-hour build. First-year saving = $70 - (7.8 + 32) = 30.2$ hours; thereafter about 62 hours a year. **Adopt** — but note it is marginal in year one, so the case rests on the estate still existing in year two and on the drift and review benefits, which are real but not in this arithmetic.
2. **In the image:** everything identical across guests — the patched operating system, the agents, the standard configuration. **In user data:** everything that differs — hostname, network address, SSH keys, role. Wrong way round: a hostname baked into the image means every clone has the same name and the same SSH host key, so they conflict and tooling cannot distinguish them; a security patch applied through user data is re-applied at every boot for ever, slowing each start and never updating the image everybody else builds from.
3. **Convergent** management — it is the only strategy available for a system that cannot be rebuilt. Over the year: write a configuration definition that describes it as it currently is,

apply it in check-only mode until it reports no differences, then bring it under enforcement; in parallel, build and test a replacement from that definition, and when a rebuild succeeds the machine can move to the immutable regime. The intermediate goal is not automation but *a written description that is known to be accurate*.

Chapter 17 — High Availability, Live Migration and Disaster Recovery

1. The ratio is $400/1,200 = \mathbf{0.33}$, so it converges. Round 0: $32,000/1,200 = 26.7$ s, dirtying 10,667 MB. Round 1: $10,667/1,200 = 8.9$ s, dirtying 3,556 MB. Round 2: $3,556/1,200 = 3.0$ s, dirtying 1,185 MB. Subsequent rounds are 1.0 s, 0.3 s and 0.1 s, so the total elapsed time is about **40** seconds with a switchover pause of tens of milliseconds.
2. Each host believes the *other* has failed, because from its own position the symptom is identical. **Without fencing**, both restart the other's guests, so the same guest runs twice and both instances write to the same disks: not downtime but corruption, which is worse than the outage HA was bought to prevent. **Quorum** requires a majority before acting; with two hosts neither side has one, so both stop — which is why a two-node cluster needs a third vote, a witness, and why fencing (powering the suspect host off through its management controller) must happen before anything is restarted.
3. $525,600 \times 0.0001 = \mathbf{52.6}$ minutes a year. A 90-minute planned upgrade **exceeds the entire annual budget on its own**, so it is not compatible. Either the upgrade must be performed without downtime — live migration, rolling updates — or the availability target must be renegotiated to 99.9%, which allows 8 h 46 min.

Chapter 18 — Performance Monitoring and Capacity Planning

1. **%RDY (or steal time) and balloon / host swap-in**. High %RDY with no memory activity means the guest is not being given a processor — host contention, and the fix is right-sizing or rebalancing. Low %RDY with ballooning or host swap means memory pressure, and the fix is reducing overcommit. Both appear from inside the guest as “the application is slow”, and the guest's own 35% is consistent with either.
2. $1 - 0.95^{12} = 1 - 0.540 = \mathbf{0.46}$, so about **46%** of page loads contain at least one call slower than the p95. The tail is not a rare case for the user; it is close to half of their experience.
3. With one host down, seven remain, so the figure is $76\% \times 8/7 = \mathbf{87\%}$. That is above the 85% ceiling at which reclamation begins, so **no** — the cluster should not be reported as healthy. It has no real N+1 today, and the 76% on the dashboard conceals exactly that.

Chapter 19 — Security in Virtualized Environments

1. Guest compromise: $1/600 = \mathbf{0.17\%}$. VM escape: a host carries $600/25 = 24$ guests, so $24/600 = \mathbf{4\%}$. Management plane: **100%**. **Defend the management plane first** — it is 25 times the reach of an escape, it needs no foothold inside any guest, and it is reachable over the network. Do not forget the storage array, which holds all 600 guests' disks and is usually less hardened.
2. Two hardware threads on one core share caches, branch predictors and execution resources almost completely, so a side channel between them cannot be closed by flushing at VM entry — the two threads are running *simultaneously*, not alternately. Where the neighbours are strictly untrusted, the only reliable answer is to stop using the second thread. It costs **15–30% of the host's apparent capacity**. Core scheduling recovers much of that by allowing SMT only between siblings of the same guest.

3. You are being asked to grant **host root, twice over**. `-privileged` grants all capabilities and disables seccomp and AppArmor; the Docker socket allows the agent to start a new container with the host filesystem mounted and privileges set, which is the same thing by another route. Alternatives: (i) run the agent as a host process under systemd with specific, scoped permissions rather than as a container; (ii) give it a read-only, scoped API credential — most monitoring needs metrics, not the socket — and if the vendor cannot work without either, run it where it shares a kernel with nothing you care about.

Chapter 20 — GPU and Accelerator Virtualization

1. The reclamation ladder works because host memory can be recovered gradually: pages are shared, then ballooned, then compressed, then swapped, and each rung degrades performance rather than causing failure. None of that exists for GPU memory. There is no balloon driver, no page sharing between contexts and nowhere cheap to swap to, and a compute kernel that needs 40 GB of working set cannot run in 20 GB more slowly — the allocation simply fails. GPU memory is a hard allocation, not an elastic one.
2. Demand is $6 \times 5 = 30$ GPU-hours a day against 24 available, so **the card is undersized whatever is done**. *Passthrough*: one group at a time, so five groups queue at any moment and the utilisation is 100% with permanently unmet demand. *A six-way MIG partition at 12 GB each does not exist* — profiles are fixed, and on an 80 GB card the choices are seven instances of 10 GB (too small for 12 GB), three of 20 GB, two of 40 GB or one of 80 GB. The realistic option is three 20 GB instances, giving three groups concurrent, predictable access and a queue for the other three. Predictability is better than passthrough; capacity is still short.
3. `utilization.gpu` reports the fraction of sampled intervals in which *at least one kernel was resident* — not how much of the hardware was doing work. A job using a single streaming multiprocessor badly will show 100% while leaving most of the card idle. Measure **achieved occupancy, memory bandwidth utilisation** and the SM activity counters — through Nsight or the DCGM profiling metrics — which distinguish a busy card from a resident kernel.

Chapter 21 — Confidential Computing and Emerging Hardware

1. Because a hostile hypervisor does more than read memory. Encryption stops it *reading*, and leaves it able to manipulate the mapping: the concrete attack is **page remapping** — giving the guest a different physical page than it had, or aliasing two guest addresses to one physical page — and **replay**, re-presenting an old encrypted page. The data stays encrypted throughout and the guest is compromised anyway. SEV-SNP's reverse map table and TDX's equivalent add the integrity check that refuses a contradictory mapping.
2. They are protected against an operator or an attacker **reading the guest's memory** — a memory dump, a cold-boot attack, a hypervisor inspecting pages. They are **not** protected against the attack the technology exists to stop: an operator who boots a *different* image which requests the key at startup. That image's memory is perfectly encrypted while it exfiltrates the database. Without verifying a measurement, they have bought a performance penalty.
3. **Availability** — the operator can still stop, detain or destroy the machine, so a compliance argument resting on continuity is not helped at all. **Side channels** — timing and cache attacks are not fully addressed, and published attacks target these systems specifically, so a claim of absolute confidentiality is overstated. (Equally acceptable: protection against your own application bugs, which leak data the ordinary way, and protection of data once it leaves the TEE.)

Chapter 22 — Case Studies in Enterprise and Cloud Virtualization

1. At 1.26 the shortfall was small enough to be absorbed by the **upper rungs of the reclamation ladder** — transparent page sharing across 64 similar guests, and ballooning, both of which degrade performance imperceptibly. At 1.57 the shortfall exceeded what the guests could give back, so the hypervisor reached **host swapping**, which is not a slope but a cliff: guest pages went to disk and latency rose by orders of magnitude. The mechanism is the same ladder; the difference is which rung it reached.
2. The cloud **made existing waste legible** — on premises an over-allocated guest costs nothing once the host is bought, whereas in the cloud every unit of waste is a line on a monthly invoice. The two items accounting for 75% were **lift and shift without right-sizing** (44%) and **on-demand pricing** (31%).
3. Because quorum requires a majority, and two nodes can never produce one: a clean split leaves each node with one vote out of two, so neither may act and both stop. A third vote at head office breaks the tie. When the link to the witness failed, the site **operated in a documented degraded mode with HA disabled** — accepting that a node failure during that window would not be recovered automatically, rather than risking the split brain that automatic recovery without quorum would invite.

Chapter 23 — The Semester Project

1. Any hypothesis containing a number and capable of being false, for example: “*Migrating our six internal services from virtual machines to Kubernetes reduces mean deployment time from 25 minutes to under 5, without increasing p99 request latency by more than 10%.*” It names two measurable quantities, states thresholds, and could come out either way — which the original could not, because no result would have contradicted “is good for our department”.
2. That **the measurement does not support any claim**: the spread of the three runs (7%, 24%, −2%) is far larger than the 9% mean effect, and one run has the opposite sign. Reporting “a 9% improvement” would be reporting noise. The report should say so. Next: find the source of the variance — neighbours, caching, burst credits, nesting — control it, and repeat with enough runs to separate the effect from the spread; and if the variance cannot be controlled, report that as the finding.
3. Switch to the captured output shipped in `code/`. The hypothesis is about *partitioning policy and its effect on predictability*, which can be tested against recorded measurements from a partitioned card: the analysis, the percentiles and the comparison with the unpartitioned baseline are all still yours to do. State in the method section that the figures are from captured runs rather than your own hardware, and treat them as comparative. What you must *not* do is quietly change the hypothesis to fit what is available.

Glossary

 Not written yet

This appendix is a placeholder.

Resources and Further Reading

 Not written yet

This appendix is a placeholder.

The Boundary with the Neighbouring Courses

 Not written yet

This appendix is a placeholder.

The Sixteen-Week Schedule

 Not written yet

This appendix is a placeholder.